Noise and Uncertainty in Molecular Property Prediction:

Robust Models and Adaptive Representations

Adelaide Punt

[Your College]

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

[Term] 2026

Statement of Originality

I, the undersigned, declare that this is my own work unless where otherwise acknowledged and referenced.

This thesis includes work carried out in collaboration. In Chapter 3, A.P. implemented the NoiseInject framework and conducted all experiments; T.H., S.W., and G.M. supervised the project and contributed to the experimental design. The work in Chapters 4 to 6, including the adaptive hybrid representation and all of its experiments, was designed, implemented, run and written up by A.P. alone.

Candidate: Adelaide Punt

Signed: _____

Date: _____

Declaration of AI Use

During this doctorate I used Claude, a large language model developed by Anthropic (https://claude.ai), [model versions and dates of use]. I used it for three purposes. First, I used it to help write, test and debug software, including parts of the code that ran the experiments reported in this thesis. Second, I used it to draft and edit parts of the text and to typeset it in LaTeX. Third, I used it to search for literature and to check citations against their sources. I reviewed, edited and verified all of its output before using it. [No data, results or figures were generated by the tool; every reported number comes from code that I ran and checked.] The research questions, experimental design, analysis and conclusions are my own. [My supervisors were aware of this use.] I take full responsibility for the content of this thesis, including any errors.

Acknowledgements

This DPhil was funded by the University of Oxford's UK EPSRC Centre for Doctoral Training in Sustainable Approaches to Biomedical Science: Responsible and Reproducible Research, grant EP/S024093/1. It was also supported by Lhasa Limited, a not-for-profit organization.

Abstract

Contents

Statement of Originality i

Declaration of AI Use ii

Acknowledgements iii

Abstract iv

List of Abbreviations x

1 Introduction 1
- 1.1 Outline of the thesis and contributions . . . 4
- 1.2 Collaborations . . . 5

2 Molecular Representations, Models and Label Noise 7
- 2.1 Overview . . . 7
- 2.2 Molecular Representations in QSAR . . . 7
- 2.3 Models for molecular property prediction . . . 12
  - 2.3.1 Tree ensembles . . . 15
  - 2.3.2 Support vector machines . . . 15
  - 2.3.3 Gaussian processes . . . 16
  - 2.3.4 Bayesian neural networks . . . 16
- 2.4 Molecular similarity and activity cliffs . . . 17
- 2.5 Noise in molecular data . . . 19
  - 2.5.1 Artificial noise injection . . . 21
  - 2.5.2 Learning with noisy labels . . . 23
- 2.6 NoiseInject: a benchmarking framework . . . 24
  - 2.6.1 NoiseInject . . . 24
- 2.7 Evaluation methodology . . . 27
- 2.8 Conclusions . . . 28

3 Noise Robustness Across Molecular Representations and Model Architectures 29
- 3.1 Introduction . . . 29
- 3.2 Methods . . . 30
  - 3.2.1 Datasets . . . 30
  - 3.2.2 Representations . . . 31
  - 3.2.3 Models . . . 32
  - 3.2.4 Noise conditions and levels . . . 33

3.2.5 The QM9 precompute pipeline . . . 34
3.2.6 Uncertainty . . . 36
3.2.7 Evaluation . . . 37
3.3 Representation governs clean performance, architecture governs robustness . . . 39
3.4 Probabilistic models are no more noise-robust than deterministic ones 44
3.5 Relative model robustness is largely strategy-independent . . . 46
3.6 The pattern holds across prediction targets . . . 48
3.7 Uncertainty under label noise . . . 48
3.7.1 Models with an explicit noise term track prediction error under noise . . . 48
3.7.2 NGBoost and the GP stay near target coverage while Bayesian variants miscalibrate . . . 51
3.7.3 Uncertainty attribution is itself representation-dependent . . . 52
3.7.4 Fingerprint representations carry the usable uncertainty signal 55
3.8 Conclusions . . . 55

4 The Flaws of the Similarity Assumption 58
4.1 Introduction . . . 58
4.2 Flaws in standard base representations . . . 60
4.2.1 Most features do not vary on a given dataset . . . 60
4.2.2 Molecules the representation cannot tell apart . . . 63
4.3 Flaws in similarity and other pairwise computations . . . 68
4.4 Consequences for methods that rely on similarity . . . 74
4.4.1 The exploratory learning-with-noisy-labels study . . . 75
4.4.2 Noise detection by neighbors . . . 77
4.5 Conclusions: what a representation would need . . . 80

5 Building Adaptive Hybrid Representations 83
5.1 Introduction . . . 83
5.1.1 Combining representations and the alternatives . . . 84
5.1.2 The adaptive hybrid representation . . . 90
5.1.3 Blueprints and what they tell a chemist . . . 93
5.1.4 Beyond small molecules . . . 94
5.2 Methods: the hybrid search . . . 95
5.2.1 From base representations to a hybrid . . . 96
5.2.2 The search space and why each axis is swept . . . 102
5.2.3 Augmentation . . . 117
5.2.4 Fast tier and adaptive pruning . . . 120
5.3 Methods: experimental setup . . . 125
5.3.1 Datasets and splitting . . . 126
5.3.2 Representations by domain . . . 131
5.3.3 Models and metrics . . . 145
5.3.4 Comparators, controls and ablations . . . 148
5.3.5 Transfer across datasets, antibody mutational scans and data held at separate sites . . . 154
5.4 Results: when the hybrid helps . . . 159
5.5 Results: why the hybrid helps . . . 159
5.6 Results: blueprints on specific datasets . . . 159

5.7 Results: transfer across datasets and antibody mutational scans . . . 159
5.8 Results: data held at separate sites . . . 159
5.9 Discussion . . . 159
5.10 Limitations . . . 160
5.11 Conclusions . . . 160

6 Hybrid Representations for Similarity and Label Noise 161
6.1 Introduction . . . 161
6.2 Methods . . . 162
6.2.1 Similarity in a hybrid . . . 163
6.2.2 Noise conditions . . . 165
6.2.3 Noise mitigation methods . . . 166
6.3 Results: similarity . . . 170
6.4 Results: noise detection and estimation . . . 170
6.5 Results: noise mitigation by noise type . . . 170
6.5.1 Censoring: correcting labels clipped at a detection limit . . . 170
6.5.2 Gaussian noise: learning-with-noisy-labels methods . . . 170
6.5.3 Real labels, nothing injected: cleaning . . . 170
6.5.4 By noise type . . . 170
6.6 Conclusions . . . 170

7 Discussion and Conclusions 171
7.1 Summary . . . 171
7.2 No single method fits every dataset . . . 173
7.2.1 Simple is often better . . . 174
7.3 Limitations . . . 176
7.4 Future directions . . . 177
7.5 Conclusions . . . 178

Summary of Research Contributions 180

A Datasets and curation 181
A.1 Small molecules . . . 181
A.2 The Chapter 3 versions of LogD, Caco-2, hERG K_i and QM9 . . . 185
A.3 Chemical reactions . . . 186
A.4 Antibodies . . . 187
A.5 Nucleotide sequences . . . 189
A.6 Cancer genomics (TCGA) . . . 189

B Data and code availability 191

List of Figures

2.1 Three representations of one molecule: ECFP4, one-hot SMILES and a molecular graph. 8
2.2 Deterministic and Bayesian neural networks. 17
2.3 The seven noise conditions applied to one set of labels. 23

3.1 Memory-mapped loading compared with in-memory loading. 35
3.2 ANOVA variance decomposition of performance and robustness on QM9. 41
3.3 Representation–model interaction effects on QM9. 42
3.4 Overview of noise robustness on QM9. 43
3.5 Neural network families under Gaussian noise on QM9. 45
3.6 Noise degradation slope on LogD, Caco-2 and hERG K_i. 49
3.7 Noise robustness on QM9 and three assay datasets. 50
3.8 Uncertainty estimates under label noise on QM9. 53

4.1 Share of constant and near-constant features in each representation. . 64
4.2 Two hERG K_i molecules that differ at one stereocenter. 68
4.3 Absolute pK_i difference against distance for hERG K_i pairs. 73

List of Tables

3.1 ANOVA variance decomposition by noise strategy on QM9. 40
3.2 Wilcoxon signed-rank tests for the probabilistic transformations on QM9. 44
3.3 Noise degradation slope by model on PDV, QM9. 46
3.4 Strongest and weakest combinations for uncertainty–noise correlation on QM9. 51

4.1 Constant and near-constant features, part 1. 62
4.2 Constant and near-constant features, part 2. 63
4.3 Molecules that share a feature vector with another molecule. 67
4.4 All-pairs correlation, five-nearest-neighbor prediction and activity cliffs per dataset. 74
4.5 Agreement between representations per dataset. 75
4.6 Recovery in R^2 with neighbor-based sample weights, by representation. 79
4.7 Flaws of standard representations and similarity, and the requirement each sets. 82

5.1 Ways of combining or shrinking representations. 91
5.2 The three importance methods searched. 106
5.3 Importance method of the best configuration in a preliminary run. . . 108
5.4 The five allocation methods searched. 111
5.5 Worked example of the four fixed-share allocation methods. 111
5.6 The three augmentation conditions. 120
5.7 Values tried for each search setting. 122
5.8 The adaptive pruning rules. 124
5.9 The 16 datasets used in the hybrid experiments. 128
5.10 The 16 datasets and the best single representation on each. 129
5.11 Representations used for small molecules. 134
5.12 Representations used for chemical reactions. 136
5.13 Representations used for antibodies. 137
5.14 Representations used for nucleotide sequences. 139
5.15 Representations used for cancer patients (TCGA). 140
5.16 Datasets and the representations each one uses. 142

A.1 Sample counts before and after curation. 182
A.2 Curation of the three assay datasets. 186

List of Abbreviations

AC Activity cliff
ADME Absorption, distribution, metabolism, and excretion
ANOVA Analysis of variance
$\mathrm{AUC}_{\mathrm{norm}}$ Normalized area under the retention curve
AUROC Area under the receiver operating characteristic curve
BNN Bayesian neural network
CDR Complementarity-determining region
CP Conformal prediction
DFT Density functional theory
DNA Deoxyribonucleic acid
DNN Deep neural network
ECE Expected calibration error
ECFP Extended-connectivity fingerprint
ECFP4 Extended-connectivity fingerprint with radius 2, diameter 4
ESM Evolutionary Scale Modeling (protein language models)
F1 F1 score (harmonic mean of precision and recall)
GCN Graph convolutional network
GIN Graph isomorphism network
GNN Graph neural network
GP Gaussian process
HOMO Highest occupied molecular orbital
ICC Intraclass correlation coefficient
KL Kullback–Leibler
kNN k-nearest neighbors
LightGBM Light Gradient Boosting Machine
LNL Learning with noisy labels
LOF Local outlier factor
LUMO Lowest unoccupied molecular orbital
MACCS Molecular ACCess System
MAE Mean absolute error
MCC Matthews correlation coefficient
MHG-GNN Molecular Hypergraph Grammar Graph Neural Network
ML Machine learning
MLP Multi-layer perceptron
mRMR Minimum redundancy, maximum relevance
NDS Noise degradation slope
NGBoost Natural gradient boosting

NN Neural network
NN-α Neural network architecture α ([128, 64], dropout after each layer)
NN-β Neural network architecture β ([128, 128], dropout before output)
OHE One-hot encoding
OPIG Oxford Protein Informatics Group
PCA Principal component analysis
PDV Physicochemical descriptor vector
QRF Quantile regression forest
QSAR Quantitative structure–activity relationship
QSPR Quantitative structure–property relationship
RBF Radial basis function
RF Random forest
RFE Recursive feature elimination
RMSE Root mean squared error
RPPA Reverse-phase protein array
SHAP SHapley Additive exPlanations
SMILES Simplified molecular input line entry system
SNS Sort & Slice
SVM Support vector machine
TCGA The Cancer Genome Atlas
VBLL Variational Bayesian Last Layer
XGBoost eXtreme Gradient Boosting

Chapter 1

Introduction

New avenues in the drug discovery pipeline have opened up thanks to advances in large dataset collection, analysis, and computation. Therapeutic targets are identified, potential drug candidates are screened and optimized, and the most promising of these go on to pre-clinical and clinical trials [1]. There are various methods by which potential drug candidates can be screened. Virtual screening allows for the rapid and large-scale analysis of libraries of compounds, and is a particularly useful approach used to speed up the drug discovery process [2]. One standard screening method is the prediction of bioactivities and pharmacological properties of such candidates. There is a long history of using Quantitative Structure-Activity Relationship (QSAR) models to make these predictions, to act as a filter for compounds with undesirable properties, and thus reduce the time and cost associated with experimental screening [3]. QSAR models aim to predict the relationship between a molecule's molecular features as encoded within a particular molecular representation and a desired molecular property [3] of interest. A QSAR model typically consists of a molecular representation paired with a machine learning (ML) model. An ideal molecular representation will allow a model to learn only the relevant information about each molecule with respect to the target property. Representations are either 'static' hand-crafted descriptors or circular fingerprints that encode predefined structural and physicochemical features [4–7]; or learned representations, where the encoding is derived from a predictive model, often taken from the activations of the final layer of a neural network, or a latent space embedding [3, 8].

QSAR models are of course, sensitive to the quality of the input data on which they are trained. Experimental noise can arise from measurement errors, variability under experimental conditions, varying provenance of the data, and intrinsic biolog-

ical and chemical variability. Even the very measures of model performance may be flawed if test and validation sets are laden with noise. Kolmar and Grulke [9] show that the variance associated with experimental data can contribute more to prediction error than the model's own error. There is also a limit to how much noise the model can tolerate; Jorner et al. [10] found that once the model error approaches the magnitude of the experimental measurement error on labels, adding more training data yields diminishing returns in predictive performance. In the context of activity prediction, the inherent variability within experimental setups and assays, batch effects, measurement errors, and biological variability introduce additional layers of complexity [9, 11]. All of this increases the likelihood of encountering noise in the data [12]. Even the molecular representations in the training data themselves may be incorrect or misleading [13], and chemical datasets contain too few repeat measurements of each compound to characterize the distribution the label is drawn from. Thus, models treat a single label as the population mean [9]. Repeat measurements of the same compound-target pair disagree by about 0.68 log units for pIC_{50}, and by about 0.54 log units for pK_i [14, 15]. Measurements from a single assay lowers the mean disagreement from 0.50 to 0.27 log units [16].

This thesis addresses the challenge of conducting molecular property prediction when the underlying experimental data contain noisy labels. Most existing work on noise in QSAR modeling assumes that label noise can be modeled as random error drawn from a Gaussian distribution, and evaluates either molecular representations or predictive model architectures in isolation. Cortes-Ciriano et al. [17] examined 12 algorithms across multiple noise levels but used fixed representations. Studies that compare representations are typically done on clean data. No work has systematically compared the relative importance of representation versus architecture choice for noise robustness, assessed whether probabilistic methods justify their computational overhead in noisy settings, or tested whether findings generalize across different corruption patterns and molecular property types. The first half of this thesis therefore asks four questions about the behavior of QSAR models under label noise. First, how does the choice of molecular representation and model architecture explain the variance in predictive performance, and does that division change when we instead consider how much performance a QSAR model retains under increasing label noise? Second, what does the choice of model provide under label noise? Within this question we compare deterministic and probabilistic modeling. Third, does the type of label noise matter, or just the quantity of it? We generate and add

label noise in several ways, both to clean data and to data with inherent experimental noise. Fourth, what is the impact of artificial label noise on a model's uncertainty estimates? Does the estimated uncertainty rise with increased label noise, and can it point to which labels are corrupted? Together, these questions aim to identify what, if anything, makes a QSAR model robust to noise. To answer them we introduce NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty quantification in machine learning models.

Learning with noisy labels (LNL) is the broad family. Noise mitigation, which removes, reweights or corrects labels, is one part of it.

We hypothesized that the distance itself was at fault. On a given dataset, most features of a general-purpose representation either do not vary or do not relate to the property. Every such feature adds a term to the distance, so together these irrelevant features dominate it. Their contribution grows linearly with the number of features, not exponentially. Chapter 4 measures this on base representations and on the distances computed from them. The fix we propose is to select a fixed number of task-relevant features from several representations, and to compute distances in that compact space. A first version showed that these selected feature sets also work as ordinary model inputs, with no distance involved. The question then became how to choose the features, which is the subject of Chapter 5. Chapter 6 returns to distances, noise detection and LNL, including noise mitigation, with the hybrid representation as the feature space.

The second half asks whether the representation itself can be adapted to the task:

1. How do you select the best set of features for a given prediction task?
2. Do hybrid representations achieve higher noise robustness (retention percentage) than their base representations?
3. Do the selected hybrid features help stabilize distance-based noise mitigation?
4. Does feature importance weighting improve effectiveness of hybrid representation creation, model training, and noise mitigation tasks?
5. When sample removal is suboptimal, can sample weighting or label smoothing provide superior noise handling under Gaussian corruption?

1.1 Outline of the thesis and contributions

Chapter 2 introduces, once, the ideas that the later chapters rely on: molecular representations, model families, molecular similarity and activity cliffs, noise in molecular data, and the evaluation protocol. It defines the seven noise conditions used in Chapter 3. It ends by introducing NoiseInject, the open-source benchmarking framework released with this work.

Chapter 3 studies noise robustness. The main task is the HOMO–LUMO gap on QM9. Its labels are computed, so they carry negligible measurement noise. We repeat the experiments on three assay datasets with experimental labels: LogD, Caco-2 and hERG K_i. The chapter splits the variance in performance, and in how much performance is retained under noise, between representation and model architecture. It compares seven noise conditions at a matched amount of noise. It also compares each model with its probabilistic counterpart, and asks what label noise does to predicted uncertainty. Model architecture matters more than representation for noise robustness. For R^2 under noise, the two matter about equally. The shape of the noise distribution makes little difference. Noise shared within a scaffold family, and labels clipped at an assay limit (censoring), do the most damage. NGBoost and random forests were the most robust models on QM9, and random forests were the most consistent across all four datasets. Predicted uncertainty rises with noise on average, but it does not pick out which labels were corrupted.

Chapter 4 is a motivation chapter. It introduces no method, and its measurements are smaller than those of Chapter 3. It makes three arguments. First, on a given dataset, standard representations carry many features that do not vary, and some map different molecules to the same vector. Second, similarity computed over those representations inherits both problems. Third, methods that rely on that similarity fail as a result. One such method cleans labels by comparing each molecule's label with the labels of its nearest neighbors. It is a noise mitigation method tested in an exploratory study of learning with noisy labels carried out earlier in this project. The chapter ends by stating what a representation would need in order to fix these problems. That is what Chapter 5 builds.

Chapter 5 is planned as the first of two method papers, intended for submission to a venue still to be confirmed. From several base representations, the method selects each one's most important features for the task. It splits a fixed budget of features between the representations and combines the selected features into one

representation. The chapter also describes the datasets and the experimental setup. It will test when the hybrid helps, against the best single representation and against an ensemble of one model per representation. It will test why it helps, against PCA, mRMR, recursive feature elimination and random features at the same number of features. We will also ask whether the selected features can be read against the chemistry, and how stable the selection is across folds. A further test asks whether search settings found on one dataset carry over to another. The chapter will test whether the method extends beyond small molecules. Antibodies are covered in most depth, with reactions, nucleotide sequences and cancer genomics as proofs of concept. Finally, it will test whether the method extends to data held at separate sites that cannot be pooled. **[RESULT NEEDED: one sentence of findings for Chapter 5.]**

Chapter 6 is planned as the second method paper, also intended for submission to a venue still to be confirmed. It returns to the failures described in Chapter 4 and asks whether a hybrid representation repairs them. For similarity, it compares standard representations with hybrids selected for predicting each molecule's activity from its five nearest neighbors. For noise detection and estimation, it asks whether neighbors in a hybrid pick out corrupted labels better than neighbors in a standard representation. It then tests learning with noisy labels, including noise mitigation, one noise type at a time. **[RESULT NEEDED: one sentence of findings for Chapter 6.]**

Chapter 7 summarizes each chapter. Its closing argument is that the noise should be modeled before a method is chosen. No single method fits every dataset, and simple methods are often as good as complex ones. The chapter then sets out the limitations of this work and directions for future work, and ends with the conclusions of the thesis.

This work contributes software. It is NoiseInject, which is open source, and the KIRBy software repository, which implements the hybrid search and its experiments. Repositories, licenses and data are described in Appendix B.

1.2 Collaborations

Thierry Hanser and Stephane Werner at Lhasa Limited co-supervised this work, with Garrett Morris as supervisor. Lhasa runs Effiris, a federated-learning platform [18]. Lhasa also supplied the hERG data used in Chapter 5: 7,772 training molecules

and 4,383 test molecules. We use these as data held at separate sites, the setting that federated learning addresses [18, 19].

The Oxford Protein Informatics Group (OPIG) pointed us to the antibody and protein datasets used in Chapters 5 and 6. The full list of datasets is in Appendix A. The collaboration also produced five posts on OPIG's blog, Blopig, listed under Blogposts in the Summary of Research Contributions.

The adaptive hybrid representation, its software and the experiments in Chapters 4 to 6 are my own work.

Chapter 2

Molecular Representations, Models and Label Noise

2.1 Overview

This chapter introduces the ideas that every later chapter refers back to. We first describe molecular representations, which turn a molecule into a fixed-length vector of features, and the model families that learn from them. We then set out the assumption that similar molecules have similar properties, and where it breaks down. Next we describe where label noise in real assay data comes from, the seven artificial noise conditions used in this thesis, and the methods for learning with noisy labels. The chapter ends with NoiseInject, the package that injects the noise and reports robustness and uncertainty metrics, and with the group-aware splits and metrics used throughout.

2.2 Molecular Representations in QSAR

The success of machine learning hinges on the quality of input data. Although the quality can refer to noise, which most of this work centers on, here we mean the type of input labels. Molecular representations provide different features with which QSAR models and noise detection techniques can be learned. However, a potential bottleneck on larger data sets should be noted; the more complex the data structure, the greater the computational cost. For smaller data sets, the number of parameters is limited by the size of the training set, to prevent overfitting.

One of the most common and intuitive molecular representations are Simplified

MISSING FIGURE: molecular_representations.png

Figure 2.1: Visualization of the molecule O=C1C=CN(C1)C=O using three different methods: a) ECFP4 showing the molecule broken into substructures with circular neighborhoods for each atom. Each substructure is hashed to produce unique identifiers, which are then compiled into a binary bit vector representing the presence or absence of specific substructures. b) SMILES tokenization and one-hot encoding of the molecule. The SMILES string is tokenized, and using a vocabulary of the most common tokens, these tokens are converted into a one-hot encoded matrix. c) Molecular graph representation where nodes represent atoms and edges represent bonds.

Molecular-Input Line-Entry System (SMILES) strings which consist of a sequence of ASCII characters generated by a fixed set of rules to specify a given chemical identity [20]. A common way to use SMILES strings as inputs to QSAR models is to tokenize and one-hot-encode them rather than to use the strings themselves [21]. The strings are tokenized into atomic branches and symbols and mapped to indices of a vocabulary, and finally converted into binary vectors using one-hot encodings [21]. This representation, referred to as one-hot-encoded SMILES, allows models to process molecular structures as bit-vector-based fingerprints or as sequences of vectors. Sequence-based models such as recurrent neural networks and transformers have proven to be particularly successful in capturing text semantics within a sequence. When applied to raw or one hot encoded (OHE) SMILES strings, these models can effectively capture the structure of the molecule in its textual representation [21].

Another common representation is the molecular fingerprint, a vector containing information about the properties of the molecule in question. In a typical bit vector-based fingerprint, each bit accounts for the presence (1) or absence (0) of a given chemical feature or substructure [22]. One common fingerprint is the Molecular ACCess System (MACCS) key, a substructure key-based fingerprint that encodes predefined atom symbols and properties, as well as bond types [23]. Another is the Extended Connectivity Fingerprint (ECFP), which encodes local neighborhoods around each atom and bond connectivity in molecules [23]. ECFPs are typically constructed with a radius of 2, a length of 2048 bits, and active chirality flags. These

existing molecular representations are well established and have been quite successful when coupled with QSAR models [3, 24]. They are relatively simple, and as the bits can be mapped back to one or more substructures, they are human-interpretable, and furthermore easy-to-compute. However, they are subject to information loss due to bit collisions, which happens when two different substructures are encoded at the same bit position [22]. They are also based on a fixed procedure and are not trainable or adaptable to a particular dataset. Alternative representations that capture different structural, chemical, or physical features are able to overcome these limitations.

Recent advances have led to the development of new molecular fingerprints. One such method is the Sort & Slice (SNS) technique, an alternative to hash-based folding for pooling ECFP substructures [6]. Sort & Slice begins by sorting all circular substructures in the training set according to their frequency of occurrence. It then "slices" away all but the N most frequent substructures, which are subsequently used to generate an N-dimensional, bit-collision-free binary fingerprint. This approach solves the ambiguity problem of hashed fingerprints, where multiple substructures might be encoded at the same bit position, making it very interpretable [6]. Sort & Slice fingerprints outperform ECFP4 with standard hash-based folding methods across various molecular property prediction tasks. These experiments consisted of different data splitting techniques, machine learning models, fingerprint diameters, fingerprint dimensions, and atomic invariants [6]. Furthermore, Sort & Slice outperformed advanced substructure selection schemes such as filtered fingerprints and mutual information maximization [6].

Neural fingerprints are a category of learned representations that overcome the fixed, hard-coded nature of traditional fingerprints. Unlike ECFP or MACCS keys, which rely on predefined hashing schemes or substructure catalogs, neural fingerprints train on molecular data to capture chemical information [25, 26]. These representations are the result of deep learning architectures that have learned dense, continuous embeddings which can be used for downstream tasks.

Mol2vec, which adapts the Word2vec algorithm from NLP to molecular substructures [25], is common neural fingerprint. Mol2vec treats molecules like "sentences" and circular substructures derived from Morgan fingerprints like "words". The model learns vector embeddings for each substructure through unsupervised training, such that chemically similar substructures are mapped to nearby points in the embedding space [25]. The embeddings of all present substructures are summed

to obtain a full representation. This approach produces 300-dimensional dense vectors [25]. Mol2vec has performed strongly on various QSAR benchmarks, matching or exceeding Morgan fingerprints on solubility and bioactivity prediction tasks [25].

Transformer-based architectures have also been applied to molecular representation learning, similarly stemming from NLPs ideas. MolBERT applies a bidirectional transformer to learn molecular representations from SMILES strings through self-supervised pre-training [27]. The model has domain-relevant auxiliary adaptations, such as the prediction of calculated physicochemical properties alongside the masked token prediction, to improve the chemical fidelity of the learned embeddings [27]. MolBERT demonstrated improvements over traditional fingerprints and graph-based methods on Virtual Screening and QSAR benchmarks [27]. ChemBERTa is another transformer-based approach which pre-trains masked language models on SMILES data from PubChem [26]. ChemBERTa uses a tokenization strategy adapted for the SMILES notation, which allows the model to learn representations of atoms and functional groups through self-supervised learning [26]. Attention visualization reveals that ChemBERTa learns chemically meaningful patterns, with different attention heads specializing in functional groups, ring structures, or SMILES syntax [26]. ChemBERTa-2, an optimized variant pre-trained in 77 million compounds, achieved even stronger performance than the original ChemBERTa in MoleculeNet benchmarks [28].

Neural fingerprints provide several advantages over traditional fingerprints. They produce dense continuous vectors that avoid sparseness and information loss from bit collisions [25]. They can also be pre-trained once on large unlabeled datasets and then fine-tuned for specific downstream tasks, opening the door to transfer learning [26, 27]. However, these methods are often computationally expensive [25] and due to their black-box nature, they are not as interpretable. [26]. However, attention mechanisms can have some degree of interpretability through visualization [26].

Beyond fingerprints, physicochemical-descriptor vectors (PDVs) are a class of continuous-value molecular descriptors that encode structural and chemical properties of molecules [29]. Unlike binary fingerprints or graph-based representations, PDVs aggregate atom and bond-level quantities into fixed-length feature vectors that capture a wide variety of information about a molecule and its structure [30]. PDVs typically consist of hundreds of properties including constitutional descriptors such as molecular weight, atom and bond counts, and heteroatom ratios; topological indices encoding molecular connectivity patterns; physicochemical properties

such as logP (lipophilicity), molar refractivity, and topological polar surface area; and electronic descriptors derived from partial charges and polarizabilities [29, 31]. The `RDKit` cheminformatics toolkit provides a comprehensive suite of more than 200 molecular descriptors that cover these categories [32]. PDVs are very interpretable; each descriptor has a clear physicochemical meaning [31]. They are also less computationally expensive than some 3D methods or neural fingerprints, with many descriptors simply requiring a 2D molecular structure without conformer generation or extensive pre-training. This efficiency makes PDVs particularly appropriate for high-throughput applications. However, it is also possible to use descriptors based on 3D components [30].

However, PDVs have limitations. The dimensionality of PDV feature spaces can lead to problems with multicollinearity [29]. This can be mitigated by feature selection or dimensionality reduction techniques [31]. Additionally, PDVs encode molecular properties as aggregated global features, which may not capture fine-grained structural details such as specific substructure arrangements or local chemical environments, information important for particular endpoints [30].

One of the more intuitive representations are molecular graphs. These graphs can be categorized into 2D and 3D graphs. 2D graphs rely on adjacency matrices that encode topological atom- and bond-level features but leave out geometric properties such as bond angles or torsion, meaning they may fail to capture stereochemistry. 3D graphs incorporate these spatial features, but are often limited by remaining in a single conformation [33]. A standard 3D molecular graph is constructed such that each molecule is represented as $G = (V, E)$ with node features $\mathbf{x}_i$ and edge features $\mathbf{e}_{ij}$. Nodes encode atom type, degree, hybridization, aromaticity, and formal charge; edges encode bond order and conjugation [33].

Another common molecular graph is the standard distance compatibility molecular graph, in which atoms and bonds are mapped to sets of nodes and edges [5, 34]. These molecular graphs are typically 2D objects that represent 3D information such as atomic coordinate, bond angles, chirality, etc. [5]. However, there are alternative molecular graph representations such as graphs with quantum 3D embeddings, weight alignment graphs, association graphs containing pairs of molecules, and geometric interaction graph neural network that incorporate 3D structures and physical interactions [34–36]. Du et al. [37] developed hybrid 2D and 3D graphs, combining 3D data, guaranteeing precision and completeness by providing unabridged information, and 2D data, providing chemical intuitions for guidance, in a zipper-like

arrangement. These graphs pose various challenges, such as the atoms being localized at voxels creating issues with the points being too sparse, leading to poor performance. Kuzminykh et al. [38] developed a novel approach in which atoms are extended to fill nearby voxels with a wave transformation.

These graph representations do not highlight specific substructures or specific arrangements of atoms and bonds within each molecule. The substructures contain functional groups that are related to a range of chemical properties and are potentially involved in downstream tasks related to the function of molecules. Bouritsas et al. [39] developed graph substructure networks, a topologically aware message passing scheme based on substructure encoding. The method involves encoding structural roles based on the appearance of specific substructures and using them in message passing updates. Since message-passing GNNs are at most as powerful as the Weissfeiler Leman test, they cannot isolate sufficient features from the graph structure [39]. Graph substructure networks, on the other hand, capture these substructure-related features. And recent research has highlighted the abilities of pretrained GNN encoders produce embeddings on unlabeled datasets that capture structural features [8, 40].

Kolmar and Grulke [9] examined the predictive limits of QSAR in the presence of noise, showing that by learning the underlying trends rather than fitting to noise, the models can achieve predictions more accurate than those of their training data. They assumed experimental noise could be modeled as Gaussian-distributed, using the Central Limit Theorem to argue that aggregated experimental measurements from multiple independent error sources exhibit a Gaussian distribution. However, Heid et al. [41] used mean-variance estimation and bias-variance decomposition to show that noise can be tied to specific modalities, for example structure dependence. As an example, nitrogen-containing functional groups introduce measurement deviations, implying the presence of systematic noise.

2.3 Models for molecular property prediction

One key issue with QSAR models is the standard assumption that labels represent true values. Most ML models rely on the assumption that the training data is, in fact, an accurate reflection of the population. Therefore, they treat these potentially noisy labels as discrete quantities rather than distributions, which opens the door to overfitting to noise and giving unduly optimistic assessments of the models.

There are a wide variety of model architectures that can be used for QSAR modeling. Starting off with simpler and more explainable models, we have tree-based methods. Decision trees are non-parametric supervised learning algorithms that perform recursive binary partitioning of the feature space [42]. A decision tree builds a predictive model by learning a series of if-then rules that recursively split the data based on given feature values, creating a tree where internal nodes represent decision rules, branches represent the results of those decisions, and leaf nodes contain predicted values or class labels [42]. Each recursive split is chosen to maximize homogeneity within the resulting partitions. For classification, Gini impurity is commonly used and for regression, mean squared error reduction [42].

There are several properties that make decision trees a good foundation for QSAR modeling. First, they handle non-linear relationships between molecular descriptors and target properties without requiring explicit feature engineering or transformation [42]. They also do not make assumptions about the underlying distribution of the data, making them robust to violations of parametric assumptions that can limit linear regression or other classical methods [43].

Decision trees handle data in a "contributory" rather than "consensus" manner. In consensus methods such as linear regression or neural networks, every data point influences the prediction at every other point in the feature space. Tree-based methods make predictions based on the subset of training samples that fall into the same leaf node as the test point [42]. This mechanism essentially quarantines outliers in their respective leaf nodes, limiting their impact on predictions for samples in other regions or chemical domains.

Individual decision trees tend to have high variance and often overfit, particularly at large depths [42]. Small modifications made to the training data can result in large differences in the structure of the tree, resulting in unstable predictions [44]. Ensemble methods are used to counteract this instability, aggregating predictions from many trees. Random forests (RFs) aggregate predictions from trees trained on bootstrap samples of data with random subsets of features at each split [44]. Gradient boosting methods such as XGBoost sequentially fit trees to the residuals of previous iterations, progressively reducing the prediction error [45]. Bootstrapping ensures that each tree is trained on a slightly different subset of data, which effectively reduce the influence of any individual sample [44]. This is especially beneficial with noisy samples. The selection of random features at every node decorrelates the trees, preventing systematic biases from propagating through the ensemble [44]. The

aggregation of trees further prevents overfitting to noise by smoothing out prediction errors from individual noisy labels.

Neural networks (NNs), a very popular family of models, can also be used for QSAR. A NN is a system of interconnected nodes which learn by passing information through it's layers. There are weights and biases attached to the nodes and various other parameters which the model optimizes to minimize the difference between the predicted output and the actual target value. During training, the network parameters are updated using backpropagation, in which gradients of the loss function with respect to each weight are propagated backward through the network [46]. The structure of these networks and the hyperparameters used depend on the type of NN. In QSAR, one of the most commonly used architectures is the multilayer perceptron (MLP), a NN containing fully-connected layers with non-linear activation functions which transforms molecular feature vectors (i.e., fingerprints, descriptors) into a property or activity prediction [47].

For molecular property prediction, it is recommended that architectures use 2-3 hidden layers with 128-512 units each, ReLU activations, and dropout regularization to prevent overfitting. Training is performed using backpropagation with gradient descent optimizers such as Adam, to minimize a loss function (MSE for regression, cross-entropy for classification) on batches of training data [48].

However, NN performance on QSAR-related tasks has been very mixed. They are often outperformed by more traditional methods: decision trees and related ensemble models tend to perform stronger on smaller datasets, overfit less, and don't rely on a static 3D molecular structure like NNs do [49, 50]. Their complexity and use of gradients make them prone to label memorization [51], which in the context of noise is problematic, as it means that models may memorize the noise themselves. Although models that are memorizing labels may initially appear to have strong predictive performance, they cannot generalize out of distribution samples properly and often provide inaccurate uncertainty estimates. Various approaches have been proposed to improve neural network robustness to label noise, including noise-robust loss functions [51], confident learning to identify mislabeled examples [52], and regularization techniques such as mixup or label smoothing [51].

All the models up to this point have assumed a vector-based input. Molecular graphs, on the other hand, require different types of predict modeling. Two standard ML methods on molecular graphs include Convolutional Networks (GCNs) and Graph Isomorphism Networks (GINs). Graph Convolutional Networks (GCNs)

apply the concept of convolution to graph-structured data. Instead of operating on grid-based data, such as images, GCNs aggregate information from neighbors of a node to update its representation [21, 53]. In contrast, Graph Isomorphism Networks (GINs) address the limitations in GCNs' ability to differentiate between non-isomorphic graphs. Unlike GCNs, which normalize and weight the sum of neighboring node features, GINs use a simple sum followed by a trainable MLP, giving the model flexibility to learn complex transformations of node features. The GIN model's design is informed by the Weisfeiler-Lehman test for graph isomorphism, enabling it to distinguish between molecular graphs that are structurally similar but differ in subtle ways [21, 53]. This makes GINs particularly suited to Activity Cliffs (ACs), where small variations in molecular structure can lead to significant changes in chemical or biological activity [21].

2.3.1 Tree ensembles

RFs are one of the most common choices for QSAR modeling thanks to their robustness and interpretability [43]. Quantile Regression Forests (QRFs) hold the distribution of training labels in each leaf and compute quantiles over all trees [54].

Gradient-boosted tree ensembles include eXtreme Gradient Boosting (XGBoost) [55, 56], Light Gradient-Boosting Machine (LightGBM) [57], and Natural Gradient Boosting (NGBoost) [58]. NGBoost extends deterministic gradient boosting by treating the parameters of a chosen parametric distribution as regression targets and learns them through boosting with a natural gradient update rule [58].

2.3.2 Support vector machines

Support vector machines (SVMs) are a well-established baseline in QSAR modeling [43, 59].

Support vector regression fits a function that is as flat as possible while keeping training errors small [59, 60]. It uses the ϵ-insensitive loss: errors smaller than ϵ cost nothing, and larger errors cost their size minus ϵ. A constant C trades flatness against tolerance of large errors, and classification uses the same machinery with a maximum-margin boundary [61]. The features enter only through the kernel. With the RBF kernel this is a function of Euclidean distance, so every feature enters every distance, and we standardize features before fitting. Three properties suggest robustness to label noise: errors inside the ϵ tube change nothing, the loss outside

it grows linearly rather than quadratically, and each molecule's weight is capped at C [60]. Under simulated experimental error, the RBF-kernel SVM degraded smoothly and linearly [17]; Chapter 3 reports how it holds up under each of our noise conditions. The main limitation is cost: training works on a kernel matrix that grows with the square of the number of training molecules [62].

2.3.3 Gaussian processes

Gaussian processes (GPs) a non-parametric class of models that use a kernel to produce a Gaussian predictive distribution over every data point [63, 64]. One of the major limitations of using GPs in practice is computational scaling; inference has $O(N^3)$ time complexity and $O(N^2)$ memory, where N is the number of training points [64]. This can be mitigated with sparse GP approximations, but these introduce approximation error and can be more difficult to tune [65].

The `Gauche` framework [66] defines kernels for strings, fingerprints, and graphs that operate on a range of widely used molecular representations. The Tanimoto kernel [67, 68] is defined for binary vectors $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$ for $d \geq 1$ as:

$$k_{\text{Tanimoto}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \frac{\langle \mathbf{x}, \mathbf{x}' \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - \langle \mathbf{x}, \mathbf{x}' \rangle}, \tag{2.1}$$

where $\|\cdot\|$ represents the Euclidean norm [66].

The radial basis function (RBF) kernel is defined for $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ as:

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \tag{2.2}$$

where ℓ is the lengthscale.

2.3.4 Bayesian neural networks

NNs are often deterministic; however, they can be transformed into probabilistic models.

Unlike standard deterministic ML methods, Bayesian methods model distributions over predictions using Bayes' rule, allowing for the quantification of uncertainty within parameters and labels. Bayesian methods treat the model parameters as random variables with distributions. Uncertainty can also be estimated from other non-deterministic methods, including deep ensembles [69], bootstrap modeling [70], and federated learning [19].

MISSING FIGURE: bayesian_transformation.png

Figure 2.2: Deterministic versus Bayesian neural network architectures. Deterministic networks use fixed weights W to produce point predictions, while Bayesian networks treat weights as distributions $\mathcal{N}(\mu, \sigma^2)$, generating prediction histograms through multiple stochastic forward passes to quantify epistemic uncertainty.

Epistemic uncertainty is model-driven and can be reduced by collecting either more or higher quality training data. Aleatoric is data-driven and reflects the intrinsic noise in the labels.

2.4 Molecular similarity and activity cliffs

Distance-based reasoning rely on the assumption that the data embeddings contain quality information. When the labels are corrupted or the important features are scarce, similarity scores over-rely on compounding false or useless information, and the assumption that similar molecules have similar properties breaks down. Bajusz et al. [71] found that while the Tanimoto coefficient can be effective with binary fingerprints, changing the similarity formula yields minimal improvement if the underlying representation is weak. This problem is further complicated by activity cliffs (ACs), which are defined as structurally similar molecules with large property differences, often due to a subtle chemical change [72, 73]. These molecules, which potentially contain the most important information for model prediction, can be confused with noise. Another barrier to distance-based reasoning is persistent noise across labels. The task of trying to determine if a molecule's value matches that of its neighbors becomes a lot harder when all of the neighbors have some form of noise as well.

Much of cheminformatics rests on the similar property principle: molecules that are structurally similar tend to have similar properties [74, 75]. One of the clearest applications of this principle is the use of nearest-neighbor prediction which shows up across cheminformatics workflows and in the case of this research, in the kernels used by GPs and at the basis of many noise identification algorithms.

The difficulty with this assumption is that similarity is defined only relative

to a similarity measure, and that measure is fixed by the choice of representation and coefficient [76]. There is no standard definition of similarity between molecules, especially when handling different molecular embeddings in a computational setting. Two molecules that are near-identical under a substructure fingerprint may be far apart under a physicochemical descriptor vector. Maggiora et al. [76] caution that quantified similarity can be over-interpreted in this manner.

The most common measure for binary fingerprints is the Tanimoto coefficient [71]. For two molecules with fingerprints $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$,

$$T(\mathbf{x}, \mathbf{x}') = \frac{\sum_{j=1}^{d} x_j x'_j}{\sum_{j=1}^{d} x_j + \sum_{j=1}^{d} x'_j - \sum_{j=1}^{d} x_j x'_j}, \tag{2.3}$$

the number of bits the two molecules share, divided by the number of bits set in either. Continuous representations are compared with a distance instead, most often the Euclidean distance $\sqrt{\sum_{j=1}^{d}(x_j - x'_j)^2}$. Both are sums over the features, and neither weighs a feature by whether it relates to the property. In the Tanimoto coefficient, a bit that is off in both molecules adds nothing, but every bit that is set in either molecule counts. In the Euclidean distance, every feature that differs between the two molecules counts. Chapter 4 returns to this (Equation 4.3).

There are many downstream tasks which rely on the concept of similarity. The second half of this thesis relies on many such tasks, and as such focuses on identifying representations that agree with this principle.

The sharpest failure of the similar property principle is the activity cliff: a pair of structurally similar molecules with a large difference in potency, produced by a small chemical change [72, 77]. Activity cliffs inherit the measure-dependence of the principle they violate, and there is no widely accepted definition of one [75]. The MoleculeACE benchmark used in this thesis makes the choice explicit: a pair is "highly similar" if it exceeds 90% similarity under at least one of three measures — Tanimoto on extended connectivity fingerprints, Tanimoto on scaffold fingerprints, or a scaled Levenshtein distance on SMILES strings — and forms a cliff if it also differs by tenfold or more in K_i or EC_{50} [75]. The soft consensus over three measures is used precisely because each captures only part of the underlying chemistry. Cliff compounds account for between 7% and 52% of a dataset across the thirty targets.

A further difficulty is dimensionality itself. Aggarwal et al. [78] show that as the number of dimensions grows, the distance to the nearest point and the distance to the

furthest point become increasingly similar, leaving pairwise distances progressively less discriminative. Molecular representations are routinely high-dimensional and sparse, and when a large proportion of their dimensions carry no information relevant to the property being modeled, distances computed over them are dominated by those uninformative dimensions.

2.5 Noise in molecular data

Real-world cheminformatics datasets contain label noise arising from multiple sources across the drug discovery pipeline. To properly model experimental noise, we must first understand where the noise is coming from.

Studies involving noise, and specifically the quantification of noise robustness, typically approach label noise by adding independent, zero-mean, Gaussian error to the labels. However, there's evidence to suggest that noise from experimental assays takes on different forms. Krüger and Overington [79] rejected normality of bioactivity differences by Anderson-Darling test and fitted a Laplace distribution instead. Kalliokoski et al. [14] examined the widest disagreements among 16,844 pairs of repeat pIC_{50} measurements and found that in almost all cases, the wide gap between repeat values was due to annotation error rather than to an imprecise assay.

Beyond just the shape, the assumptions of independence and zero mean may fail as well. Bentz et al. [80] examined P-glycoprotein transport measurements across 23 laboratories and found that 62% of the variance in log efflux ratio was due to differences between laboratories. When you pool data from different sources, each source has the potential to carry a shared offset. These sources usually contain families of related compounds, breaking the independence assumption. The zero-mean assumption is not assured either, especially when true values fall above or below an assay's detection limit, called censoring. Svensson et al. [81] count censored labels in fifteen industrial assays, of which thirteen carry censored labels and eight sit between a quarter and two thirds censored. In this study, we tested noise conditions that vary the shape, the correlation, and the bias of injected error separately at a matched quantity.

Assay-related variability represents a primary source of label noise in bioactivity data. High-throughput screening assays are often laden with errors from physical phenomena such as edge effects, where compounds in wells exhibit different activ-

ities at plate boundaries due to evaporation or temperature gradients [82]. Other examples include compound carryover between wells, incomplete liquid transfers, and air bubbles [83]. Detection reagents or unstable compounds can degrade over time, which leads to drift within a single plate run. This turns into heteroscedastic noise correlated with plate position or read time [84].

Instrument calibration and detection limits can impact measurement accuracy. Spectrophotometric assays near detection limits exhibit higher relative error, such that signal-to-noise ratios degrade for weak binders [85]. Fluorescence quenching by test compounds introduces a systematic bia in fluorescence-based assays, resulting in false negatives for potent inhibitors that accidentally get absorbed at emission wavelengths [86]. Radiometric assays require background subtraction and decay corrections that accumulate error, particularly for low-activity compounds where the signal barely exceeds background [85].

Another source of noise is biological variability. Even minor variations in cell culture conditions such as seeding density, plating efficiency, growth rate, and cell cycle distribution have the potential to affect downstream tasks such as cell metabolism and drug responsiveness. Even serum properties may introduce variability [87]. The speed of cell culture passages is even found to be a significant factor in behavior [88].

Data aggregation across multiple sources compounds these individual error sources. Common databases like ChEMBL, BindingDB, and PubChem aggregate measurements from hundreds of laboratories using a diverse set of protocols, compound purities, and quality control standards [89, 90]. The K_i or IC_{50} values from the same compound-target pair with between 10 and 20 reported values, often end up spanning two orders of magnitude apart, making it challenging to identify the underlying ground truth [15]. All the sources of error mentioned above regarding inconsistent experimental protocols cannot be distinguished easily from the genuine underlying value without extensive, and often unrealistic amounts of metadata [91].

Federated datasets, a method of data aggregation centered around privacy protection, presents a particularly challenging situation for noise. In pharmaceutical research and beyond, companies often want to merge collective information without exposing proprietary data to competitors. One example of such platforms is Lhasa Limited's federated learning platform Effiris. Teacher models are first trained separately on confidental data, and then used to label a shared public chemical data set. Those results are consolidated into a federated student model [18]. While this

is an effective approach of protecting intellectual property, it introduces additional label noise through the prediction errors of all the teacher models. There also exists aleatoric uncertainty from the underlying experimental measurements, many of which likely come from differing protocols, resulting in compounding label noise.

Beyond the experimental setups, annotation errors in the data itself can arise from mis-registered structures, incorrect stereochemistry assignments, and salt form ambiguity [92]. Public databases are estimated to contain 1-5% structure errors from incorrect SMILES parsing, tautomer misassignment, or manual transcription mistakes during data entry [93]. Ambiguous protonation states and counterions create representation-level noise where chemically distinct samples are classified as the same [94].

These noise sources can interact with each other and present in more ways than a simple Gaussian. Unlike artificial noise injection, real-world noise is neither independent, identically distributed, nor symmetric [41]. We address this complexity by examining a variety of ways of both modeling and adding noise to data sets.

Although the general assumption in many of the noise analysis and mitigation literature is that noise, specifically label noise, will have a negative effect on the model, this is not necessarily the case. There are multiple instances of using noise as a method of regularization. Although not the optimal approach to counteracting overfitting in linear regression, adding artificial noise to labels is considered a form of regularization [95]. Furthermore, adding artificial noise drawn from a Gaussian distribution to the activation layers of a neural network is considered another form of regularization [96]. These studies indicate that the presence of noise may act as a form of regularization in some contexts. It is important to address this potential of regularization from label noise in QSAR and QSPR models.

2.5.1 Artificial noise injection

Our objective is to evaluate the robustness of QSAR models against label noise. To do so, we "inject" artificial noise into the labels during training while validation and test data remain free of noise. Validation labels are given noise of the same type and quantity as the training labels, drawn independently. This artificial noise simulates real-world experimental noise. The clean target label y_i is replaced by a noisy label $\tilde{y}_i = y_i + \epsilon_i$, where ϵ_i is the artificial noise injected at the index i, the value of which is determined by a specified noise strategy. As we increased the quantity of noise,

we measured how much predictive performance fell.

We explored seven methodologies of adding noise, referred to as noise conditions (Figure 2.3). Three vary the shape of the distribution from which the error is drawn. These shapes include Gaussian, Student-t with $\nu = 5$, and Laplace distributions. Two conditions concentrate error on specific subset of molecules; outlier isolates a random tenth of them and grouped-wider concentrates on Bemis–Murcko scaffold families. Group shifted also applies noise to scaffolds, but instead gives each scaffold its own offset rather than a wider error. The seventh, censoring, sets a limit such that every label extends past that limit is reassigned to the limit itself.

For a given label, every condition except censoring delivers the same expected quantity of noise at a set noise level. These levels are defined as a fraction of the standard deviation of clean training labels. The noise added to the label of molecule i is

$$\epsilon_i = \frac{\tau}{G}\, s_i z_i, \qquad G = \sqrt{\frac{1}{n}\sum_{j=1}^{n} s_j^2}\;\; \sigma_z, \tag{2.4}$$

Here τ is the target standard deviation of the noise, in the units of the label, and z_i is drawn from the condition's distribution at unit scale. The multiplier s_i is 1 for most molecules and $\lambda = 3$ for those the condition affects, which are a random 10% of molecules under outlier and whole scaffold families covering 20% of the split under grouped-wider. The constant σ_z is the standard deviation of z, equal to 1 for the Gaussian, $\sqrt{\nu/(\nu-2)}$ for the Student-t and $\sqrt{2}$ for the Laplace. The factor G divides both of these out, so that ϵ has root-mean-square τ in expectation whatever the distribution and whichever molecules are affected.

Grouped-shifted widens no molecule's error, and instead gives every scaffold family g an offset of its own,

$$\epsilon_i = \frac{\tau}{G}\left(\sqrt{\rho}\, b_{g(i)} + \sqrt{1-\rho}\, w_i\right), \tag{2.5}$$

with one draw b per family and one draw w per molecule. The two variances sum to τ^2, and the family offset carries a share ρ of that total.

We selected parameters from published measurement errors, wherever possible. As inter-laboratory error is estimated to be approximately three times intra-laboratory error, we set $\lambda = 3$ [97]. Hampel [98] reports that $1 - 10\%$ of routine scientific data contains gross errors; we selected the upper bound of 10%. Laboratory identity accounts for 62% of the variance in Caco-2 across 23 laboratories,

prompting us to set $\rho = 0.62$ [80]. We took the Student-t and Laplace shapes from the distributions fitted to repeated public bioactivity measurements [79], and contaminated a fraction of the samples with a wider error according to Huber's contamination model [99]. As we found no published estimate of how large a share of a given dataset inter-laboratory differences affect, we chose to randomly select scaffolds until 20% of the molecules had been covered.

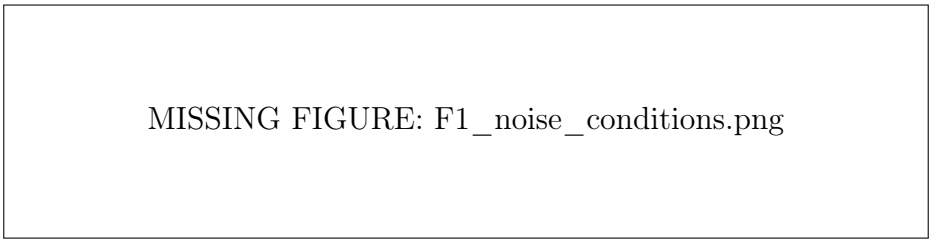


Figure 2.3: The seven noise conditions applied to one set of labels. The labels are 2,000 synthetic values rather than a study dataset, split at random into 40 groups that stand in for scaffold families. a)–g) Clean labels in gray and noised labels in color, at a noise level of 0.5 of the clean label spread. Censoring is drawn with 25% of labels clipped, since its level is the fraction clipped. The bottom axis is the label value and the side axis is the share of labels in each bin. h) For each condition except censoring, the standard deviation of the mean error within a group, divided by the standard deviation of all errors. A value near 1 means a group's labels move together. With about 50 labels per group, noise that ignores the groups gives about 0.14.

2.5.2 Learning with noisy labels

Learning with noisy labels (LNL) covers every method for training a model when some of its labels are wrong. We divide it into two parts. Noise mitigation changes, drops or reweights the training labels once, before any model is trained, and any model can then be fitted to the result. Its methods include removal, down-weighting, smoothing, relabeling and correction. Noise-robust training leaves the labels alone and changes how the model trains. Its methods include robust losses, noise-modeling layers, regularization, and reweighting inside the training loop, as in co-teaching and Meta-Weight-Net. A sample-selection step taken out of its training loop, and used only to delete molecules before a refit, counts as noise mitigation. Noise detection and noise estimation find noisy labels or estimate how many there are. They feed noise mitigation, but are not LNL methods themselves. Song et al. [100] sort 62 deep-learning methods into five groups: robust architecture, robust regularization, robust loss function, loss adjustment and sample selection. No single method is

expected to win. Averaged over all problems, no learning algorithm beats another [101, 102]. The chemistry analog is a study of twelve algorithms on twelve bioactivity datasets with simulated experimental errors [17]. Neighbor-based cleaning goes back to editing a training set by its nearest neighbors [103]. It relies on similar molecules having similar labels, and activity cliffs break that assumption. QSAR models also frequently fail to predict activity cliffs [73]. Chapter 6 tests this.

2.6 NoiseInject: a benchmarking framework

The noise conditions described above are implemented twice. The three experimental datasets and every uncertainty result use NoiseInject, an open-source Python package. QM9 uses a separate Python–Rust precompute pipeline, which carries its own Rust implementation of the same conditions. An executable cross-check compares the two implementations before any cluster run. Rust's and NumPy's random number generators produce different streams, so the check compares statistics rather than individual draws: on the same labels, groups and seeds, it compares the delivered noise, the unit dose G, the median error, the number of badly corrupted labels and the number of molecules affected.

2.6.1 NoiseInject

NoiseInject (version 1.0.0) evaluates model robustness and uncertainty calibration under artificial label noise, for both regression and classification. It operates on label arrays and on the predictions of any externally trained model, so it is model-agnostic. The core package depends only on NumPy, pandas, SciPy and scikit-learn. Results are returned as `pandas` DataFrames, one row per noise level and one aggregate summary row. The software is available under an MIT license and archived on Zenodo [104].

Regression noise. The regression injector takes a noise shape (Gaussian, Student-t or Laplace) and a targeting rule (uniform, grouped-wider, grouped-shifted, outlier or censoring) separately. Each named condition in the package's registry is one combination of the two with its parameters, and the same name is written on every results row. The registry holds more settings than this study runs: Student-t at $\nu \in \{3, 5, 10\}$, and outlier contamination at 1%, 5% and 10%. It also holds an

outlier variant in which the unselected molecules receive no noise at all, rather than the base-scale noise of Equation 2.4.

The requested level is the root-mean-square noise to deliver, τ, in the label's own units. The injector computes the per-molecule multipliers s_i and solves for the scale τ/G in closed form (Equation 2.4); there is no calibration search. A level of zero returns the labels exactly unchanged. Which molecules are affected is drawn from a second random generator, seeded separately from the noise draws and re-seeded on every call. The affected molecules are therefore the same at every noise level, including zero, while the draws themselves change. Under grouped-wider, whole scaffold families are added in random order until the fraction of affected molecules is as close as possible to the target of 20%, because a fixed fraction of families gives a very different fraction of molecules when family sizes are uneven.

Censoring is parameterized by the fraction f of labels clipped. The limit is the $(1-f)$ quantile of the reference labels, which are the clean training labels. On the training split, the censored set is exactly the $k = \mathrm{round}(fn)$ largest labels. On any other split it is whichever molecules lie past the training limit.

Each injection returns its provenance alongside the noisy labels: the target dose, the unit dose G, the solved scale, the delivered root-mean-square noise (in label units and as a fraction of the clean label standard deviation), the mean of ϵ, the fraction of molecules affected, the number and largest share of groups, and the seeds. The injector also checks the delivered amount. The delivered dose is an estimate from a finite sample, so it is compared against a band of three standard errors,

$$\left|\frac{\hat{\tau}}{\tau} - 1\right| \le \max\left(3\sqrt{\frac{\kappa - 1}{4\, n_{\mathrm{eff}}}},\ 0.005\right), \qquad n_{\mathrm{eff}} = \frac{\left(\sum_i s_i^2\right)^2}{\sum_i s_i^4}, \tag{2.6}$$

where $\hat{\tau}$ is the delivered root-mean-square noise and κ is the sample kurtosis of the unscaled draws z, bounded between 3 and 60. The effective count n_{eff} is the number of independent contributions the delivered dose averages over. It falls below n when the noise is concentrated on a few molecules. Under grouped-shifted it is $n_{\mathrm{eff}} = \left(\rho^2/g_{\mathrm{eff}} + (1-\rho)^2/n\right)^{-1}$, with $g_{\mathrm{eff}} = \left(\sum_g n_g\right)^2 / \sum_g n_g^2$ for family sizes n_g. At $\nu \le 4$ the Student-t has no fourth moment, so the standard error is instead $1.3\, n_{\mathrm{eff}}^{2/\nu - 1}$, with the constant calibrated on the measured spread. A draw outside the band raises a warning rather than an error. At three standard errors a correct injector falls outside the band on about 1% of draws, and the delivered amount is recorded on the row either way.

A second method returns the scale each molecule's region would receive without drawing any noise. We use it to score held-out molecules against the pattern the training labels were exposed to. For uniform and grouped-shifted noise this scale is the same for every molecule, so the package flags it as degenerate. A rank correlation against a constant is undefined, and must not be reported as zero.

A separate injector sizes noise against a unit the caller supplies rather than the label standard deviation. In the KIRBy experiments the unit is the standard deviation of a model's out-of-fold residual on the clean training labels, so the noise is sized against the error the model already makes. Its conditions add Gaussian noise with variance equal to the level times the unit squared; move 2%, 5% or 10% of labels by the level times the unit, with a random sign or always upward; or add Gaussian noise whose standard deviation follows either a per-molecule covariate or a hidden log-normal draw. Both of the last two are scaled to the same mean variance as the first.

Classification noise. Six label-flipping strategies are provided: uniform flips; flip rates set by class frequency; asymmetric flip rates for binary tasks; a random per-sample flip rate; a separate flip rate for each class; and flips directed by a confusion matrix. Because the realized flip rate differs from the nominal flip probability under most of these strategies, a binary search finds the flip probability that produces a target fraction of changed labels.

Robustness metrics. For regression the package computes R^2, RMSE and mean absolute error at every noise level. For classification it computes accuracy, and precision, recall and F1 score, both macro- and weighted-averaged, with a per-class breakdown. Robustness is read off the retention curve $\mathrm{ret}(x) = m(x)/m(0)$, where m is a bounded, higher-is-better metric (R^2 for regression; accuracy, F1, precision or recall for classification) and x is the noise level. The primary summary is the normalized area under that curve,

$$\mathrm{AUC}_{\mathrm{norm}} = \frac{1}{x_{\max} - x_{\min}} \int_{x_{\min}}^{x_{\max}} \mathrm{ret}(x)\,\mathrm{d}x, \tag{2.7}$$

evaluated by the trapezoid rule. It is the mean fraction of clean performance retained across the sweep, lies near $[0, 1]$, and does not depend on how good the model was on clean labels. A secondary summary fits $\mathrm{ret}(x) \approx \exp\bigl(-(x/\tau_W)^{\beta}\bigr)$. The shape β

separates curves that hold and then collapse ($\beta > 1$) from curves that fall early and then plateau ($\beta < 1$), and τ_W sets the scale of the decay. The fit needs at least four noise levels. These two summaries replace the linear slope of performance against noise used in earlier versions, which misdescribed a nonlinear curve and was coupled to clean performance. The package also reports the clean baseline and the retention at the highest level. A flag marks any curve that falls below zero retention, and an optional threshold withholds the curve summaries for models whose clean baseline is too low to degrade meaningfully.

Uncertainty metrics. For a model that returns a per-sample predicted standard deviation u_i, the package computes, at each noise level: the Spearman correlation between u_i and the absolute error $|y_i - \hat{y}_i|$; the empirical coverage $\frac{1}{n}\sum_i \mathbb{1}[|y_i - \hat{y}_i| \le k u_i]$ at $k = 1$ and $k = 2$, against Gaussian targets of 68.3% and 95.4%; the mean interval width $\frac{1}{n}\sum_i 2u_i$; and the expected calibration error,

$$\mathrm{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{n} \left| \bar{u}_b - \overline{|y - \hat{y}|}_b \right|, \tag{2.8}$$

over ten equal-count bins B_b of u. These are computed on the clean test labels. The Spearman correlation between u_i and the injected noise $|\epsilon_i|$ can only be computed where the noise is known, which is the corrupted molecules, so it takes a separate pair of inputs. It is reported within each noise level. A pooled correlation across all levels is also returned, under a separate name. The pooled value mostly reflects that uncertainty and noise both rise with the level, so it does not measure whether a model can pick out which molecules are corrupted.

Three optional wrappers produce u_i from a point predictor: split-conformal prediction for any scikit-learn-compatible model, optionally scaled by the label spread of each molecule's nearest training neighbors; an exact Gaussian process with a Tanimoto or RBF kernel, via `Gauche` [66]; and Monte Carlo dropout for any `PyTorch` network [105]. Example notebooks and scripts cover regression and classification end to end.

2.7 Evaluation methodology

Root mean squared error (RMSE) and the coefficient of determination (R^2) were primarily used to compare QSAR predictive performance.

When evaluating uncertainty, we assessed the relationship between predicted uncertainty and prediction quality. Spearman's rank correlation coefficient (ρ) was used to quantify the relationship between predicted uncertainty (u_i) and absolute error ($|y_i - \hat{y}_i|$), as well as between predicted uncertainty and injected noise magnitude. Higher correlations indicate that the model's uncertainty estimates reliably identify predictions that are likely to be incorrect.

For probabilistic models, empirical coverage at 1σ and 2σ and the expected calibration error (Equation 2.8) are defined with the NoiseInject uncertainty metrics above.

2.8 Conclusions

This chapter has introduced both halves of a predictor: the representation and the model. Real assay noise is not independent, identically distributed or symmetric, so our artificial noise model has seven conditions. We have set out the methods for learning with noisy labels, with noise mitigation as the subset that acts on the labels, and no single method wins on every dataset. NoiseInject injects these conditions and reports the robustness and uncertainty metrics, and the evaluation section fixes the splits and metrics. Chapter 3 asks whether the representation or the model architecture governs how badly a predictor degrades under label noise, through the four questions set out in Chapter 1. The representations, similarity definitions and noise conditions introduced here are reused in Chapters 4 to 6.

Chapter 3

Noise Robustness Across Molecular Representations and Model Architectures

3.1 Introduction

While there have been many investigations into the effect of noise on molecular property prediction, direction comparisons between how different molecular representations responds to label noise remains limited. The choice of molecular representation certainly affects predictive performance. Deng et al. [106] evaluated 62,820 models and found that extended-connectivity fingerprints (ECFP) fingerprints frequently outperform learned graph embeddings across models. Whether these conclusions translate to noise robustness is our question. Between different types of label noise, Heid et al. [41] found no difference in model performance between error distributions with the same mean and standard deviation. We expand on this by considering alternative patterns of noise, namely noise concentrated in structural groups or on a sparse subset of samples.

This chapter answers the first four questions set out in Chapter 1: how representation and model architecture divide the variance in predictive performance and in noise robustness, what probabilistic models add, whether the type of label noise matters, and whether uncertainty estimates track label noise.

3.2 Methods

We used four datasets: QM9, with the HOMO–LUMO gap as the target, and three assay datasets, logD, Caco-2 and hERG K_i. On each we fitted six representations and 20 model configurations. The 20 configurations are thirteen base models and seven variant models, both defined under Models below. Every pairing was exposed to seven noise conditions, defined in Section 2.5.1, at seven noise levels. Hyperparameters for every model are given in Additional file 1.

3.2.1 Datasets

Although we are investigating the effects of label noise stemming from experimental settings, for the purposes of testing we inject controlled amounts of artificial noise into clean labels. Data were extracted from the QM9 molecular property dataset distributed by PyTorch Geometric [107], which contains 130,831 small organic molecules with pre-computed quantum-mechanical properties obtained from density functional theory (DFT) calculations at the B3LYP/6-31G(2df,p) level of theory. We selected the gap between the highest occupied and the lowest unoccupied molecular orbital (HOMO–LUMO gap) as the prediction target in QM9, as it captures a molecule's electronic excitability, charge transfer capability, and chemical stability [108]. We assumed that QM9 labels were free of noise, notwithstanding approximations in the level of theory. Each experiment was replicated ten times with a distinct random seed and a randomly selected subset of $N = 10{,}000$ molecules, split 80/10/10 into 8,000 training, 1,000 validation and 1,000 test molecules. All models were fitted on those 8,000 training molecules, except the Gaussian processes, which were fitted on a random subset of 5,000 of them; the validation and test molecules were unchanged.

We repeated these experiments on three molecular property datasets with experimentally determined endpoints. From the OpenADMET initiative [109], we used LogD and Caco-2 efflux permeability, with $N = 5{,}039$ and $N = 2{,}161$ respectively. We also selected hERG K_i data from ChEMBL [110], following a protocol inspired by Landrum and Riniker [16]: filter for binding assays, deduplicate by median pChEMBL value, and remove compounds with an inter-assay standard deviation greater than 1.0 log unit, resulting in $N = 1{,}415$ compounds. Each of the three was partitioned into five folds by grouped k-fold cross-validation, and each fold's training block gave up a further grouped fifth as a validation set for early stopping.

A model therefore fits between 3,194 and 3,364 of the 5,039 LogD molecules, depending on the fold. Unlike QM9's ten replicates, which are ten independent draws from one pool, each assay dataset is a single fixed set in which every molecule is tested exactly once and nothing is redrawn.

All splits and cross-validation folds were grouped by the chirality-blind Bemis–Murcko scaffold, implemented such that each distinct acyclic molecule forms its own group, and groups are filled in random order. Prior to modeling, all molecular structures were sanitized and canonicalized using `RDKit` to remove invalid valence states and standardize atom and bond typing [32]. Prediction targets were standardized to zero mean and unit variance. Experiments that track uncertainty were run once rather than in replicates.

3.2.2 Representations

In this study, we evaluated six molecular representations: extended-connectivity fingerprints of diameter four (ECFP4), Avalon, Sort & Slice (SNS), physicochemical descriptors (PDV), ChemBERTa and Molecular Hypergraph Grammar GNNs (MHG-GNNs). The same six were used on QM9 and on the three assay datasets. ECFP4 is a Morgan fingerprint with radius $r = 2$ with $d = 2048$ bits [4, 32]. Sort & Slice takes the same substructures as ECFP4, but it ranks them by prevalence and selects the top L ($L = 1024$) [6]. We fitted the Sort & Slice vocabulary on the training molecules alone, refitting it for each QM9 replicate and for each cross-validation fold. Avalon is a 2,048-bit fingerprint, generated with the Avalon toolkit through RDKit [7, 32]. The PDV representation is a vector of 200 physicochemical descriptors computed with RDKit's `MolecularDescriptorCalculator` [31, 32]; the 200 are listed in Additional file 1, and the list is fixed in our code rather than taken from RDKit's default descriptor set, which grows between releases. ChemBERTa embeddings are the mean over the non-padding token embeddings of the `DeepChem/ChemBERTa-77M-MTR` checkpoint, producing 384 features [28]. MHG-GNNs are a GIN-based autoencoder pre-trained on 1.34 million PubChem molecules using β-VAE loss, producing 1024-dimensional embeddings through iterative message passing and found in the published `mhggnn_pretrained_model_0724_2023` checkpoint [8]. We z-score normalized PDV, ChemBERTa and MHG-GNN per feature, using the training molecules' mean and standard deviation; ECFP4, Avalon and Sort & Slice reach the model as built.

3.2.3 Models

To understand model behavior in the presence of label noise, we explored a range of model families. We fitted tree ensembles (random forests, quantile regression forests, XGBoost, LightGBM and NGBoost), a support vector machine, Gaussian processes and neural networks. Several are variants of one another, differing in the probabilistic machinery they add. All hyperparameters for the models below are included in Additional file 1.

While we used 100 trees for RF, we observed a noisy quantile estimate with 100 trees for QRF, prompting us to use 300 instead. To observe the effect of the quantile mechanism, we then fit a second RF at 300 trees so the two models were idential in all other settings.

NGBoost relies on scoring off validation data, which differs by dataset. On QM9 the validation split is already held out of every fit. However, on the three assay datasets the validation set is a scaffold-grouped fifth of the rows NGBoost is handed, which already exclude the validation molecules.

We used a radial basis function (RBF) with $C = 1.0$ and $\gamma =$ `scale` as the kernel, as it accomodates continuous vectors.

We fitted three variants of GPs, each with a constant mean and a scaled kernel. The first uses an RBF kernel and runs on all six representations. The second uses the Tanimoto kernel [67, 68] and was run on ECFP4 alone. The third is a heteroscedastic variant in which the RBF process is given a second network that predicts the observation noise of each molecule. As such, we used a standard GP and only fit on at most 5,000 training molecules, drawn by a seeded sample. Due to the size of the datasets, this cap only affected QM9. We implemented these GP models in `GPyTorch` [111], with the Tanimoto kernel taken from the `Gauche` framework [66].

While preliminary tests indicated that the Tanimoto kernel provides GP with substantial benefits over other kernels, for SVM, the RBF kernel was sufficient for all types of data.

We tested two deterministic feed-forward architectures, both fully connected and both implemented in PyTorch [112]: (i) NN-α with two hidden layers of sizes 128 and 64, and dropout after each hidden layer and (ii) NN-β with two hidden layers, each of size 128, with dropout applied before the output layer. Each model was trained using the Adam optimizer with early stopping on validation loss. We used Bayesian transformations to convert NNs into Bayesian neural networks (BNNs),

with ReLU activations and dropout regularization ($p = 0.2$). Additional file 1 gives their full widths, dropout fractions, learning rates and batch size.

To get these networks producing uncertainty estimates, we applied four probabilistic transformations to each architecture, producing eight probabilistic networks. The first replaces every linear layer with a Bayesian layer carrying a Gaussian prior $\mathcal{N}(0, 0.1^2)$ on its weights, producing BNN-α and BNN-β. They are trained on an evidence lower bound whose Kullback–Leibler (KL) divergence term is divided by the number of training molecules. While this is a straightforward approach, it is computationally expensive. We explored alternative techniques like our second variant, where we add an output head to the standard NN and fit the negative log likelihood of a Gaussian with that variance [113]. The third, Variational Bayesian Last Layer (VBLL), replaces every linear layer with a layer that produces mean-field variational posteriors $q(\mathbf{W}) = \mathcal{N}(\boldsymbol{\mu}_W, \mathrm{diag}(\boldsymbol{\sigma}_W^2))$ over the last-layer weights and trains by maximizing the evidence lower bound. This can be thought of as minimizing the reconstruction loss plus KL divergence $D_{\mathrm{KL}}(q(\mathbf{W})\|p(\mathbf{W}))$ from a standard normal prior, scaled by $1/N$ [114]. Because only its output layer keeps a learned observation-noise parameter, VBLL networks report only epistemic uncertainty. The fourth makes observation noise a function of the input. All eight probabilistic networks estimate their predictive distributions from 100 stochastic forward passes. Each network and its Bayesian counterpart were tuned to the same set of parameters so that NN-α against BNN-α and NN-β against BNN-β differ by the Bayesian layers alone.

3.2.4 Noise conditions and levels

The seven noise conditions are defined in Section 2.5.1.

We ran experiments across seven noise levels, 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5. We added the noise to the raw label and standardized the target afterward, using the clean training mean and spread. For censoring, the level is the fraction of labels clipped, at 0, 0.1, 0.2, 0.25, 0.3, 0.4 and 0.5 [81].

We ran Gaussian, grouped-wider and grouped-shifted noise on every model and representation pairing. The remaining four conditions ran on selected pairs.

3.2.5 The QM9 precompute pipeline

QM9 [107] runs through a separate pipeline, loaded via PyTorch Geometric. Python loads the data, splits it, computes every representation, and trains and scores the models. A Rust program sits between featurization and training. It injects the noise, standardizes the labels and writes the files the models are trained from.

Each replicate draws a fresh scaffold split, as described under Datasets. Each acyclic molecule's group is keyed on its canonical SMILES, and a group is never divided between two splits. The same grouping defines the scaffold families used by the grouped noise conditions.

Python computes the six representations described above. Binary fingerprints are bit-packed, Sort & Slice counts are stored as 16-bit integers, and the descriptors and learned embeddings are stored as 32-bit floats. A molecule whose ECFP4 or Avalon fingerprint is entirely zero stops the run, so it cannot enter training as if it had been featurized. One-hot encoded SMILES can still be built but are not used in this thesis.

Traditional approaches load entire datasets into memory from CSV or Parquet files, creating RAM bottlenecks at scale and redundant deserialization overhead across experimental replicates. Our pipeline instead serializes preprocessed data into custom memory-mapped binary files (`.mmap`) that enable on-demand access with constant low memory footprint regardless of dataset size (Figure 3.1). Python writes one file per split, the Rust program reads each one and writes a new file with the processed labels in its place, and Python then reads the new files back for training.

Each mmap file stores a sequence of fixed-format entries. Entry structure includes: length-prefixed UTF-8 SMILES strings (isomeric and canonical), the raw target value (float32, 4 bytes), the requested molecular representations as fixed-width blocks (Sort & Slice: uint16[1024] = 2,048 bytes, PDV: float32[200] = 800 bytes, ChemBERTa: float32[384] = 1,536 bytes, MHG-GNN: float32[1024] = 4,096 bytes, Avalon: 256 bytes, ECFP4: 256 bytes), and the processed target value incorporating noise and normalization (float32, 4 bytes). Only the requested representations are written, so the byte offset of every field depends on the list of representations. This format enables zero-copy memory mapping—Python and Rust access the same disk-backed memory pages without deserialization, maintaining constant RAM usage regardless of dataset size.

The Rust program builds a noise plan for each split separately, in that split's own record order. The training plan defines the condition. The dose, the censoring limit and the affected scaffold families are all computed from the clean training labels. Validation labels receive an independent draw, from a seed derived from the training seed, at the same dose in label units, the same limit and the same affected families. The program checks that validation received the same amount of noise as training. Test labels are never noised. Every split still records the per-molecule scale its region would have received, so held-out molecules can be scored against the training pattern.

For every molecule in every split, the Rust program writes a provenance row: its canonical SMILES, clean label, injected noise, noise scale, the pattern its region was exposed to, noisy label and the standardized value written. It also writes a run-level manifest of the condition's parameters and the delivered dose. The record at each row index must hold the same molecule on the way out as when the noise was drawn, or the run stops. A self-test mode runs the same dose and targeting checks on a label file without the rest of the pipeline.

Models. The pipeline trains the models described under Models above. Hyperparameters are either package defaults, values tuned by Optuna [115], or supplied from a JSON file. For models that return uncertainty, the pipeline can also score the training molecules out of fold, so that each training molecule's uncertainty comes from a model that never saw its corrupted label.

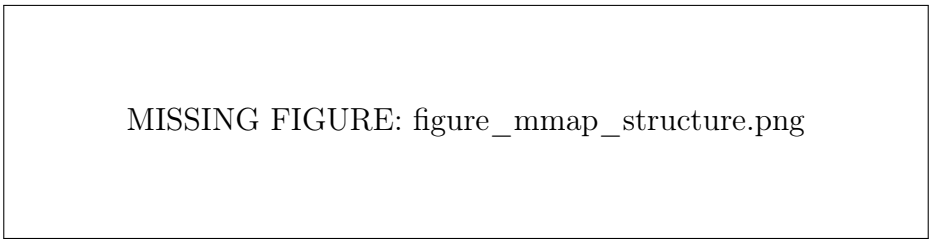


Figure 3.1: Memory-mapped file architecture contrasted with traditional in-memory loading. Top: CSV/Parquet approaches load entire datasets into RAM, creating bottlenecks at scale. Bottom: Memory-mapped binary files enable on-demand line-by-line access with constant low RAM usage. Right: Entry-level structure showing byte-aligned fields with fixed-size representations (target values, fingerprints, domain labels) and length-prefixed variable-length fields (SMILES strings, one-hot encodings).

3.2.6 Uncertainty

Up to this point we have been concerned with how noise affects the predictive accuracy of QSAR models. However, label noise should also affect the uncertainty these models report. Some models return only a point prediction, while others also return an estimate of uncertainty, either as a predicted distribution or as the spread across an ensemble. Furthermore, some report a single value of uncertainty while others report one that can be decomposed into an aleatoric and an epistemic component [113]. The aleatoric component describes underlying noise the model associates with the data, while the epistemic component represents the uncertainty within the model's fit. We sought to determine if, as expected, the aleatoric component rises with injected label noise while the epistemic component does not.

For the BNN and VBLL transformations, we took the two components over $T = 100$ stochastic forward passes [113]:

$$\hat{u}^2_{\text{ale}}(x) = \frac{1}{T}\sum_{t=1}^{T}\hat{v}_t(x), \qquad \hat{u}^2_{\text{epi}}(x) = \frac{1}{T}\sum_{t=1}^{T}\big(\hat{\mu}_t(x) - \bar{\mu}(x)\big)^2, \tag{3.1}$$

where $\hat{\mu}_t(x)$ and $\hat{v}_t(x)$ are the mean and the variance returned for molecule x on pass t, and $\bar{\mu}(x)$ is the mean over passes. Here $\hat{v}_t(x)$ is the predicted observation noise variance. It varies per molecule in the variance-head and heteroscedastic variants, and is one number for the whole fit in the VBLL transformation. A full-BNN without a variance head returns no variance, so its uncertainty is epistemic alone.

The other models follow the same split. For GPs, the epistemic component is the posterior variance of the latent function and the aleatoric component is the learned observation noise [64]. All variants of the GP produce one number for the whole fit except heteroscedastic. For QRF models, each tree holds multiple training molecules. So for a given molecule, the aleatoric component is the variance the labels the molecule's given leaf. The total aleatoric uncertainty are those variances averaged over the trees. The epistemic component is the variance of the leaf means across the trees. NGBoost predicts one variance for each molecule from a single fit, which is aleatoric alone [58]. In every model, the total uncertainty is the sum of the the aleatoric and epistemic components.

No held-out label carries injected noise, so every comparison against the injected noise was made on training molecules. Each training set was divided into five folds grouped on Murcko scaffolds, and each molecule was scored by a refit that excluded

its own fold. For NGBoost on QM9, three of the five folds were scored to limit its cost. No uncertainty was calibrated after fitting.

3.2.7 Evaluation

We compute statistics on the held-out molecules, for QM9 the test split and for three assay datasets the held-out fold of the scaffold cross-validation. Different combinations of dataset, model, representation, noise condition, and noise level are examined in each scenario. We used the coefficient of determination (R^2) throughout. For molecule i, let y_i be the clean label, $\hat{y}_i$ the prediction, ϵ_i the noise injected into its label and u_i the predicted uncertainty.

To evaluate the effect of label noise, we tracked predictive performance retention under increasing artificial noise. For each QSAR pairing we recorded $R^2(\tau)$ at each noise level τ, and normalized it to obtain the retention ratio $R^2(\tau)/R^2(0)$. To quantify robustness, we used the normalized area under this retention curve, $\mathrm{AUC}_{\mathrm{norm}}$(Equation 2.7), evaluated using the trapezoidal rule over the seven noise levels. While we tested other metrics of robustness, we found $\mathrm{AUC}_{\mathrm{norm}}$to be less directly correlated to clean-label R^2 predictions. For censoring, we sweep over a set of fractions to clip the labels prior to normalization.

An $\mathrm{AUC}_{\mathrm{norm}}$value near 1 indicates that the QSAR model in question has retained predictive performance with artificial noise, meaning it is more robust to noise. Since we normalize the curve by its own clean-label predictive performance, a model with a low clean R^2 has less to lose, so $\mathrm{AUC}_{\mathrm{norm}}$is reported beside clean R^2. We excluded replicates with a clean $R^2 < 0.3$ from the robustness analysis (Additional file 2), as performance retention ratios become unstable when the clean-label denominator approaches zero. On QM9 one $\mathrm{AUC}_{\mathrm{norm}}$is computed per replicate, and we report the median over the ten replicates. On the three assay datasets one value is computed per scaffold fold, and the median over the five folds is reported.

To compare the relative effect of the thirteen base models across datasets and noise conditions, we compared their rankings using Spearman's rank correlation coefficient (ρ) between clean R^2 and $\mathrm{AUC}_{\mathrm{norm}}$. We compared probabilistic models with its deterministic counterpart using a two-sided Wilcoxon signed-rank test paired on the replicate, at $\alpha = 0.05$, separately for each representation and noise condition. This test was run only on QM9, as with five scaffold folds the smallest attainable p is 0.0625.

We conducted a separate two-way analysis of variance (ANOVA) for each noise condition to identify the relative contributions of molecular representation and model architecture to both predictive performance and noise robustness. For a given metric y, either R^2 at a noise level of 1.0 or $\mathrm{AUC}_{\mathrm{norm}}$:

$$y_{ijr} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + e_{ijr}, \tag{3.2}$$

where α_i represents the effect of model architecture i, β_j represents the effect of molecular representation j, $(\alpha\beta)_{ij}$ is the interaction term, and e_{ijr} is the residual for replicate r. The proportion of variance explained by each term was calculated as the effect size

$$\eta^2_{\mathrm{term}} = \frac{SS_{\mathrm{term}}}{SS_{\mathrm{total}}}, \tag{3.3}$$

using Type I (sequential) sums of squares, entered in the order model architecture, molecular representation, interaction. The ten QM9 replicates provide the residual. The uncertainty on each share is the range of η^2 when the decomposition is repeated with each replicate left out in turn.

ANOVA relies on the assumption that observations both within and across groups are independence. Across the whole set of models and representations in this study, some pairs produced similar predictions that are not independent. This causes the residual degrees of freedom to increase, while the residual sum of squares barely moves. To handle this issue, we computed pairwise Spearman rank correlations between all models and between all representations across noise strategies. Models whose profiles correlated at $\rho > 0.99$ with another model were excluded, including quantile regression forests (QRF), which were very similar to RF, as well as different BNN variants.

Across representations, SNS correlated at $\rho > 0.90$ with the ECFP4 fingerprint and was excluded to avoid inflating representation degrees of freedom, as both encode overlapping circular substructure information. We also computed intraclass correlation coefficients ICC(1,1) for all model pairs. ICC(1,1) measures the proportion of total variance attributable to between-subject differences, with values near 1.0 indicating that two models rank configurations almost identically. The full redundancy and ICC tables are reported in Additional files 3–5.

To assess the interaction between uncertainty estimation and label noise, we posed three questions. First, whether uncertainty rise with label noise, as measured

by the mean u_i over held-out molecules at the highest noise level divided by its value with no added noise, in each replicate or fold. The second pertains to models who report separate aleatoric and epistemic components. We divided the increase of mean epistemic uncertainty between the highest level of noise and clean labels by the mean of the aleatoric component to determine how each responds to label noise. A ratio near 0 means the added noise was attributed to the labels while a ratio of 1 or above means the epistemic component grew as much or more. We also compared the mean aleatoric component with the standard deviation of the injected noise. The third question we had was whether per-sample uncertainty values track injected label noise, as measured by the Spearman correlation between u_i and $|\epsilon_i|$ over out-of-fold training molecules, per fold and per noise level. We also report the Spearman correlation between u_i and the absolute error of the prediction. Finally, we asked whether dividing a molecule's out-of-fold absolute error by u_i ranks the corrupted labels better than the error alone, and report the difference between the two Spearman correlations with $|\epsilon_i|$.

3.3 Representation governs clean performance, architecture governs robustness

Considering the two main components of a QSAR model, the molecular representation and the model architecture, we sought to understand how each contributes to predictive performance and noise robustness. ANOVA decomposition across six noise strategies on models to predict HOMO–LUMO gap from QM9 demonstrates that for predictive performance (R^2 at $\sigma = 0.3$), the model–representation interaction term is the largest source of variance across all six strategies, followed by model architecture and representation (Table 3.1; Figure 3.2a). However, for noise robustness as measured by the noise degradation slope (NDS), this ordering shifts. The choice of model architecture is instead the largest source of variance, while the interaction term drops to second and representation third. This observed inversion makes sense because the molecular representation dictates the chemical information the model receives, while the architecture determines how efficiently that information is used. The interaction term reflects the idea that these two are not independent. In the context of label noise robustness, representation has less of an impact since the features are not corrupted by noise. Meanwhile, the model's regularization,

ensembling, and uncertainty handling have a stronger impact on how it handles noise.

The impact varied by type of noise, represented by noise strategy (Figure 3.2b). The strategies which degraded in performance the most, threshold and value-proportional, both corrupted label regions, forcing models to rely on their inductive biases. This translates to the model architecture explaining a significant portion of the variance. On the other hand, outlier noise only affects statistical outliers. For outlier noise, model and interaction effects were both smaller.

Table 3.1: ANOVA variance decomposition by noise strategy on the QM9 HOMO–LUMO gap. Each cell reports η^2 (%), the share of variance explained by that factor. Columns: **Model** = model architecture effect; **Rep** = molecular representation effect; **Inter.** = model×representation interaction; **Resid.** = residual unexplained variance. "Performance" uses R^2 at $\sigma = 0.3$ as the outcome; "Robustness" uses the noise degradation slope (NDS). Bold entries highlight the dominant factor in each row.

	Performance (η^2, %)				**Robustness (η^2, %)**			
Strategy	**Model**	**Rep**	**Inter.**	**Resid.**	**Model**	**Rep**	**Inter.**	**Resid.**
Gaussian	24.6	22.8	**48.1**	4.5	**48.7**	8.2	11.9	31.2
Quantile	25.1	22.4	**46.9**	5.6	**47.9**	6.8	14.9	30.4
Threshold	25.0	21.2	**48.0**	5.8	**48.0**	10.1	28.0	13.9
Heteroscedastic	27.9	23.5	**46.2**	2.5	**37.0**	4.5	17.5	41.0
Value-prop.	27.8	22.0	**46.5**	3.8	**52.8**	6.9	22.2	18.2
Outlier	25.3	22.0	**49.2**	3.5	12.7	3.2	4.7	**79.3**

The interaction between model architecture and representation is seen in Figure 3.3. While some model architectures, like SVM and full BNNs maintain consistent noise robustness scores across all representations, others like NN-β display greater variation. Model robustness on one representation strongly predicts robustness on another (Spearman $\rho = 0.73$, $p = 0.007$ for ECFP4 vs PDV; 11 models). However, the scatter around the diagonal reflects the interaction term identified by the ANOVA.

We filtered out model–representation configurations with baseline predictive performance $R^2 < 0.6$, excluding 66 configurations where poor clean-data performance would produce misleadingly shallow degradation slopes (Additional file 2). We evaluated all model architectures under increasing label noise on the PDV representation across six strategies, considering both predictive performance as measured by R^2 as well as the noise degradation slope (NDS). We observe that NDS varies substantially across architectures and noise strategies, as seen in Figure 3.4.

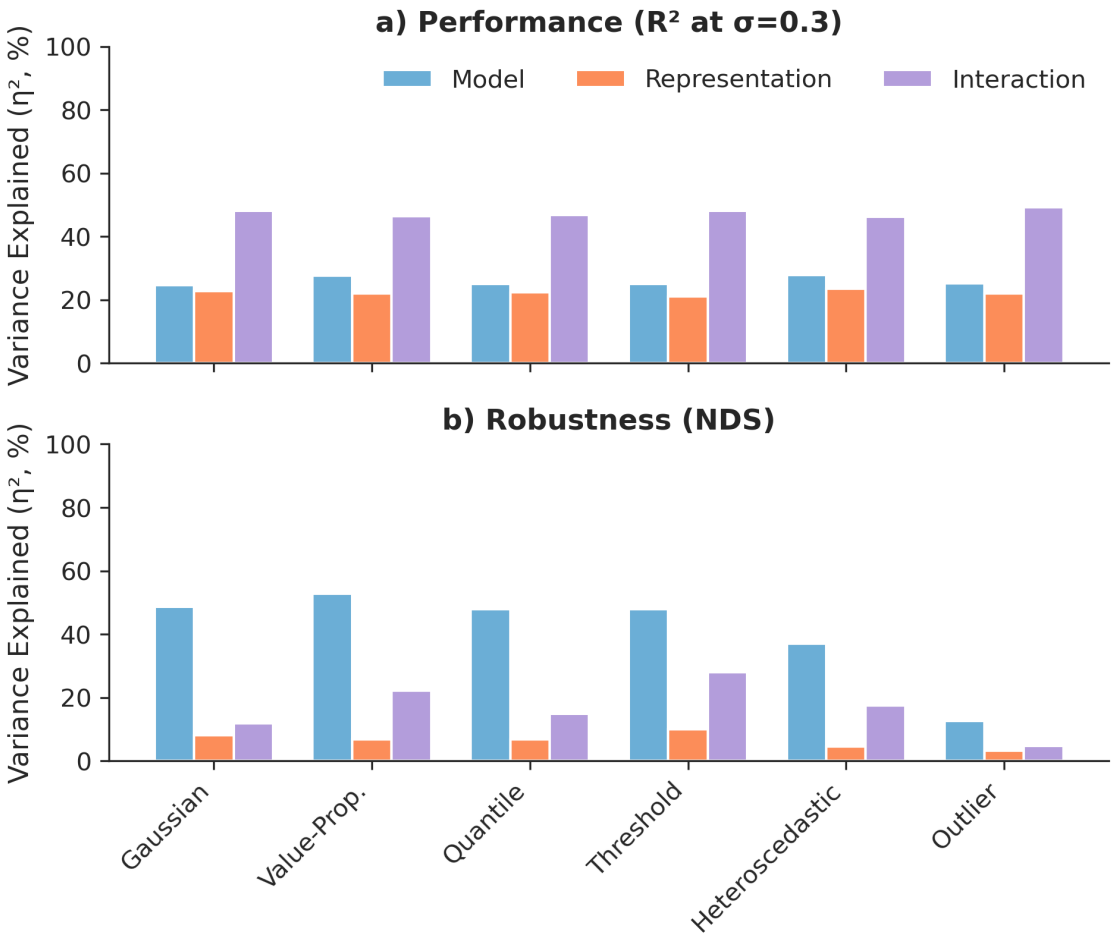


Figure 3.2: ANOVA variance decomposition (η^2, %) for a) predictive performance (R^2 at $\sigma = 0.3$) and b) noise robustness, quantified by the noise degradation slope (NDS), by noise strategy. Lower (more negative) NDS indicates faster performance loss under noise.

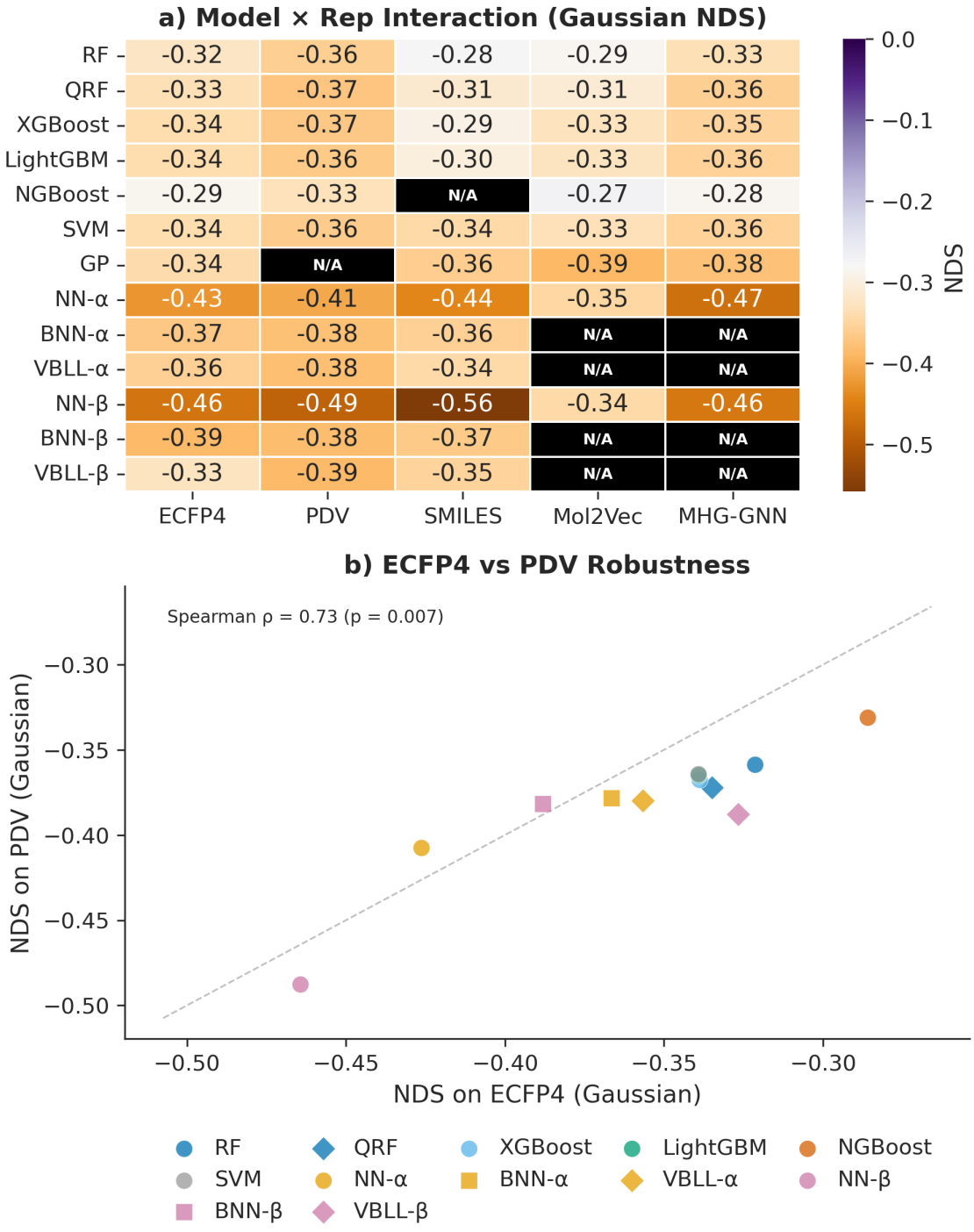


Figure 3.3: Representation–model interaction effects on the QM9 HOMO–LUMO gap. a) Noise degradation slope (NDS) heatmap showing model robustness across representations under Gaussian noise; lower (more negative) NDS indicates faster degradation. "N/A" indicates configurations excluded due to baseline $R^2 < 0.6$ or kernel incompatibility (GP uses a Tanimoto kernel defined on binary vectors, which is incompatible with PDV). b) NDS on PDV versus ECFP4 fingerprint for each model under Gaussian noise, with Spearman correlation.

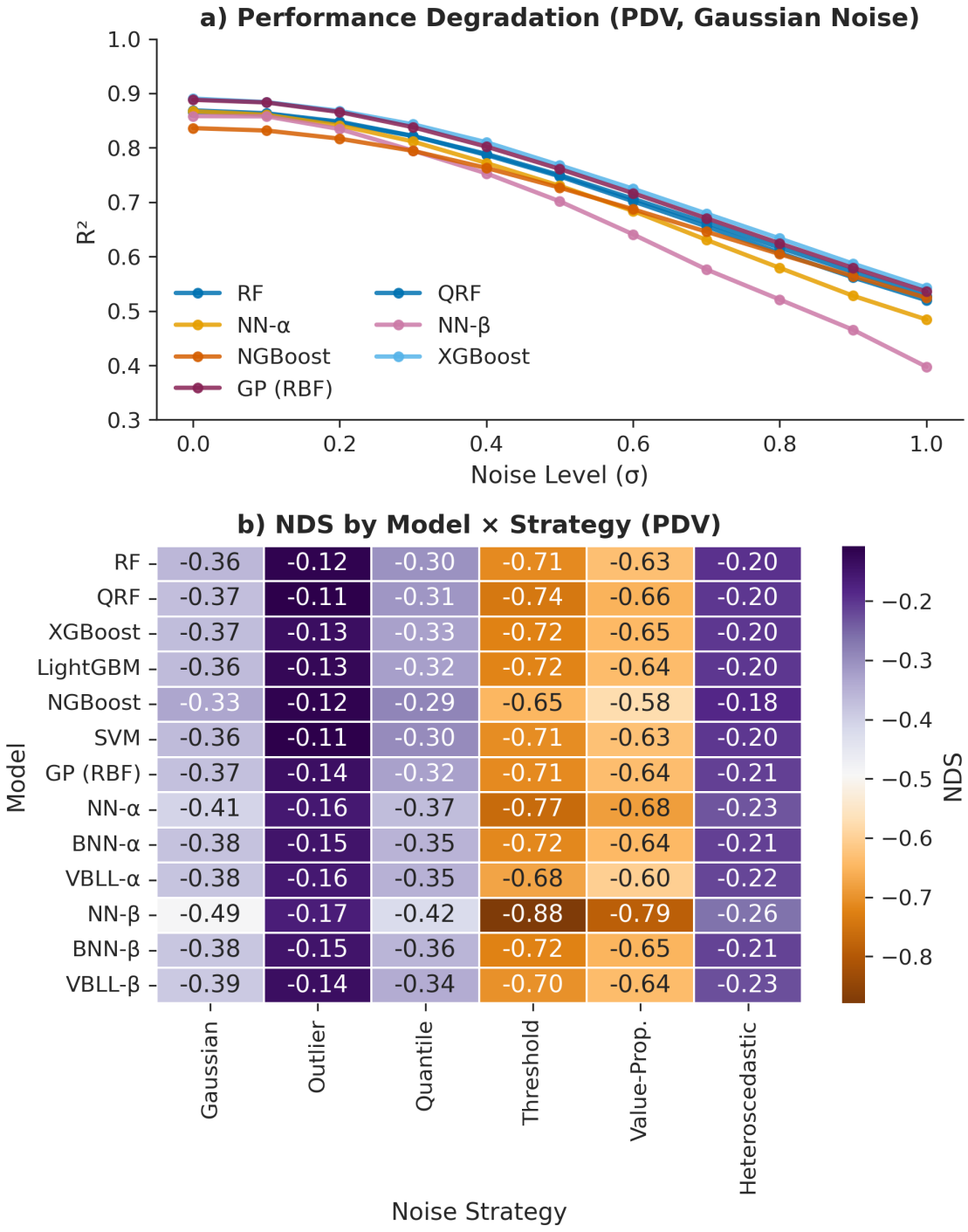


Figure 3.4: Global overview of noise robustness on the QM9 HOMO–LUMO gap. a) Performance degradation curves (PDV, Gaussian noise) showing R^2 versus noise level σ for representative models. b) Noise degradation slope (NDS) heatmap across all model architectures and noise strategies on PDV; lower (more negative) NDS indicates faster degradation under noise. The Tanimoto GP is absent because its kernel is incompatible with PDV (see Section 2); a separate GP with an RBF kernel is included for PDV-specific analyzes.

3.4 Probabilistic models are no more noise-robust than deterministic ones

To further explore the impact of model architecture on noise robustness, we examined variants of the same core architecture. Specifically, Bayesian transformations were used to explore the impact of the Bayesian framework on noise. By comparing models with the same core architecture, we observe that with full Bayesian transformations, representation explains less than 11% of NDS variance ($p > 0.25$) while variational Bayesian inference on the same NNs is representation-dependent. On MHG-GNN and mol2vec representations, VBLL models were unstable, such that all 24 VBLL $\times$ {MHG-GNN, mol2vec} configurations were excluded due to baseline $R^2 < 0.6$. Further work could involve more thoroughly-tuning these models and exploring different core architectures to adapt to these embedding-style inputs.

In general, the full Bayesian transformations reduced the degradation in performance with Gaussian noise the most, as seen in Figure 3.5. Both full BNN and VBLL transformations significantly improved robustness for both NN-α and NN-β (Table 3.2). Full transformations are more computationally expensive, making VBLL a potentially stronger approach to handling noise with NNs when working with larger data sets or modeling architectures. The choice of transformation varies by representation, as seen in Additional file 8, with SMILES receiving the largest benefits from Bayesian transformations. QRF was significantly less robust than RF (Table 3.2), suggesting that quantile regression adds complexity and flexibility which may result in overfitting.

Table 3.2: Wilcoxon signed-rank tests for Bayesian and probabilistic transformations on the QM9 HOMO–LUMO gap. **Base** is the deterministic reference model; **Variant** is the probabilistic/Bayesian transformation tested against it. **Δ NDS** = mean change in noise degradation slope (NDS) from base to variant; positive values mean the variant is more robust (slope closer to zero). **Sig.** marks pairs that are statistically significant at $p < 0.05$ (asterisk).

Base	**Variant**	**Δ NDS**	***p*-value**	**Sig.**
NN-α	BNN-α	+0.056	2.9×10^{-10}	*
NN-α	VBLL-α	+0.061	1.2×10^{-6}	*
NN-β	BNN-β	+0.096	2.9×10^{-11}	*
NN-β	VBLL-β	**+0.124**	1.2×10^{-7}	*
RF	QRF	**−0.022**	1.6×10^{-10}	*

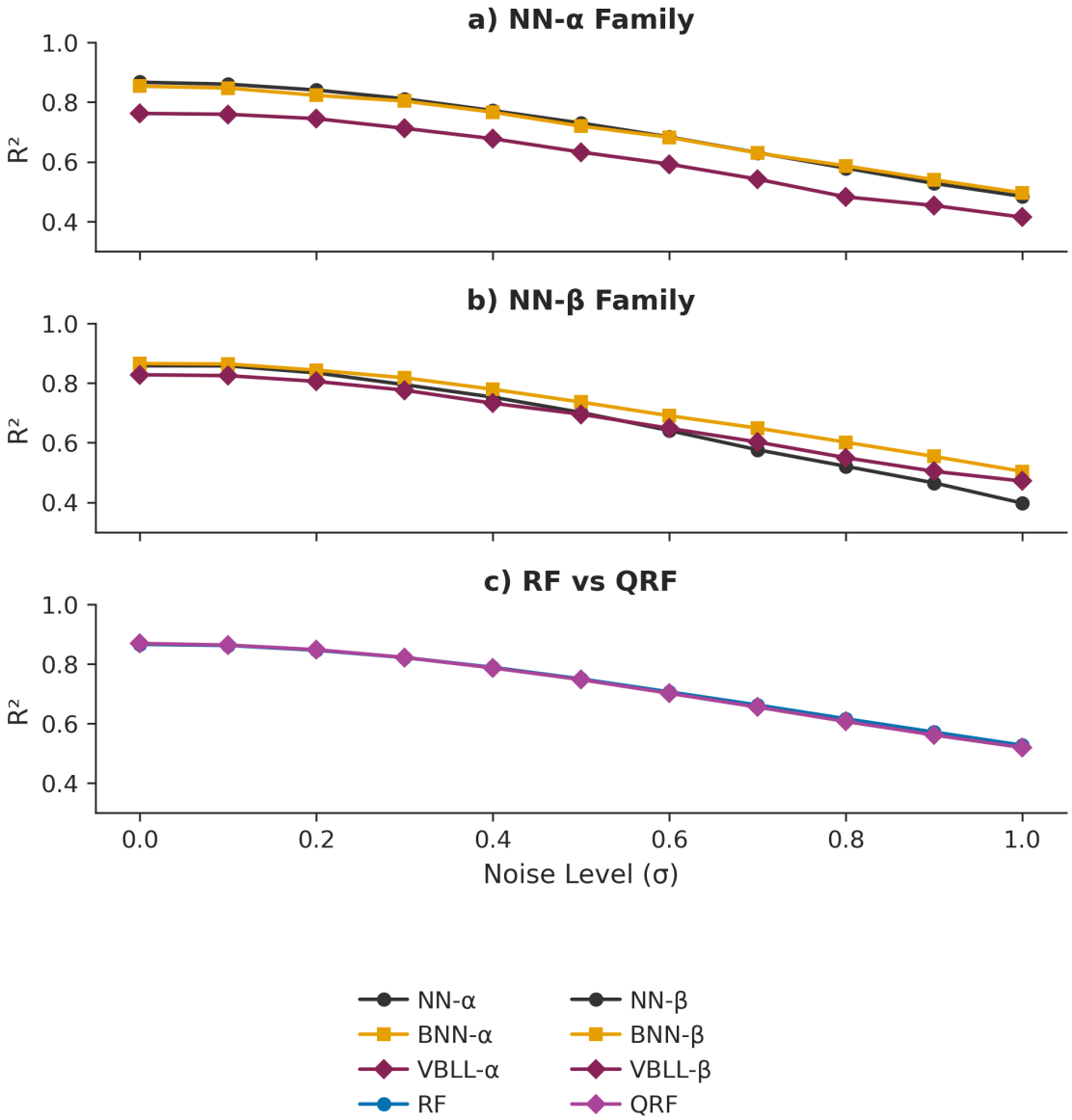


Figure 3.5: Neural network family comparison on the QM9 HOMO–LUMO gap with N=10k: R^2 versus σ under Gaussian noise on the physicochemical descriptor vector (PDV) representation for a) NN-α family, b) NN-β family, and c) RF versus QRF. Heteroscedastic results (not shown) are qualitatively consistent.

3.5 Relative model robustness is largely strategy-independent

NGBoost and SVM showed the smallest degradation in predictive performance across all strategies, while NN-β showed the steepest (Table 3.3). When sorting models by NDS, their ranks across noise strategies were consistent: Kendall's $W = 0.92$ ($p = 2.7 \times 10^{-8}$; 11 models, 6 strategies), indicating that a model's relative robustness is largely strategy-independent. The same ranking pattern holds on the ECFP4 fingerprint (Additional file 6). Robustness is also largely decoupled from baseline predictive performance: across models on PDV under Gaussian noise, NDS clusters near -0.38 regardless of baseline R^2 (Additional file 7), so a high-performing model is not automatically a more robust one.

Table 3.3: Noise degradation slope (NDS) by model on PDV (QM9 HOMO–LUMO gap), ranked by mean across six strategies. Lower |NDS| (slope closer to zero) indicates greater robustness. Strategy abbreviations: **Gauss.** = Gaussian, **Quant.** = Quantile, **Thresh.** = Threshold, **Hetero.** = Heteroscedastic, **Val.-P.** = Value-Proportional. Bold rows mark the two most robust models; bold cell entries mark each strategy's most robust configuration.

Model	Gauss.	Outlier	Quant.	Thresh.	Hetero.	Val.-P.	Mean
NGBoost	**−0.33**	−0.12	**−0.29**	**−0.65**	**−0.18**	**−0.58**	**−0.36**
SVM	−0.36	**−0.11**	−0.30	−0.71	−0.20	−0.63	−0.38
RF	−0.36	−0.12	−0.30	−0.71	−0.20	−0.63	−0.39
LightGBM	−0.36	−0.13	−0.32	−0.72	−0.20	−0.64	−0.40
VBLL-α	−0.38	−0.16	−0.35	−0.68	−0.22	−0.60	−0.40
XGBoost	−0.37	−0.13	−0.33	−0.72	−0.20	−0.65	−0.40
VBLL-β	−0.39	−0.14	−0.34	−0.70	−0.23	−0.64	−0.41
BNN-α	−0.38	−0.15	−0.35	−0.72	−0.21	−0.64	−0.41
BNN-β	−0.38	−0.15	−0.36	−0.72	−0.21	−0.65	−0.41
NN-α	−0.41	−0.16	−0.37	−0.77	−0.23	−0.68	−0.44
NN-β	−0.49	−0.17	−0.42	−0.88	−0.26	−0.79	−0.50

The strategies differed not only in predictive performance severity, but also in their ability to discriminate between architectures. The spread of NDS across models differed by noise strategy, with outlier noise having a spread of 0.06 and threshold noise having 0.23, presumably exposing architectural differences that are masked by other types of noise. As seen in Table 3.3, NGBoost was the most resistant to noise across strategies. With respect to the Variational Bayesian Last Layer (VBLL) models, VBLL-α showed a similar pattern, outperforming all tree-based models except NGBoost under threshold and value-proportional noise despite ranking below

them under Gaussian noise. This suggests that its learned noise variance is most beneficial when label corruption is systematic rather than uniform. This observation with VBLL is not alone; predictive performance on clean data is not well-correlated with noise robustness. In fact, some of the models with the lowest NDS across strategies such as NGBoost and SVM did not perform particularly well on clean data. These models may be more prone to picking up on noise rather than underlying signals in the data.

As established by the ANOVA, model architecture and the model–representation interaction term dominate NDS variance. A handful of models stood out as being noise-robust regardless of representation, namely SVM and full BNNs. Both methods rely on inductive biases, SVM on margin maximization and BNN on weight priors. However, other models like RF and NN-β show the opposite pattern. These models can be robust to noise, but only when paired with particular representations. The pairings differ by model.

While representation may not be a dominant determinant in NDS variance, the model–representation interaction term does have an impact. For SMILES, model choice explains over 91% of robustness variance, while on PDV, model choice explains 72%. Across all six noise strategies, PDV configurations produced the shallowest mean NDSs across representations, with its best performance coming with models with strong inductive biases like SVM and full BNN. This suggests that PDVs are preferable when dealing with noisy data. It appears that PDV's compact feature space complements margin-based and Bayesian regularization. Embeddings struggled, particularly when paired with NNs. These patterns are aligned with the importance of the model–representation interaction term, indicating that representation choice modulates noise robustness within a given architecture.

Taking a closer look at model choice for noise robustness, we ranked all model–representation configurations on QM9 by their Gaussian NDS. Tree-based ensembles dominate the top 10: NGBoost appears five times, XGBoost and RF twice each, and LightGBM once. No neural-network configuration appears the top 10. Rankings are consistent across strategies and the top QM9 configurations maintain their advantage on ADME datasets, though SVM configurations rank higher on these ADME datasets than NGBoost.

3.6 The pattern holds across prediction targets

In addition to working with "clean" quantum molecular properties, we tested our findings on experimentally-obtained data, replicating our process of adding artificial noise. Unlike QM9, where $\sigma = 0$ corresponds to a noise-free baseline, the experimental endpoints carry their own measurement noise in both training and test labels; the injected σ therefore stacks on top of an unknown noise floor. Per-dataset NDS heatmaps across all model architectures and noise strategies are shown in Figure 3.6. The ANOVA on these additional datasets confirms that model architecture dominates robustness variance on all three datasets, as seen in Additional file 10, and that the trends with respect to noise robustness generalize. NGBoost ranks first on both QM9 and external datasets, SVM also transfer quite well. Some models' predictive capabilities are limited on these external datasets, likely due to a combination of smaller training sets, narrower chemical coverage, and the unknown experimental noise in the labels. XGBoost and BNN variants suffer the most here, as seen in Figure 3.7a.

QRF was consistently less robust than RF on every external data set (Additional file 11). As seen with QM9, choice of molecular representation has a minimal effect on noise robustness across all three external datasets, reinforcing the finding that model architecture, not representation, is the main driver of noise robustness.

3.7 Uncertainty under label noise

The sections above showcased the importance of model selection when working with noisy labels. We take this one step further and ask, for the models which produce uncertainty estimates, how those estimates are affected by noise and if they can be used to track noise.

3.7.1 Models with an explicit noise term track prediction error under noise

Kolmar and Grulke [9] found that the mean predicted uncertainty derived from the GPs increases with the amount of label noise in the training data. We go beyond the population level and instead ask if a model's per-sample uncertainty tracks label noise. One key sign that a model is able to handle noise is its ability to track it,

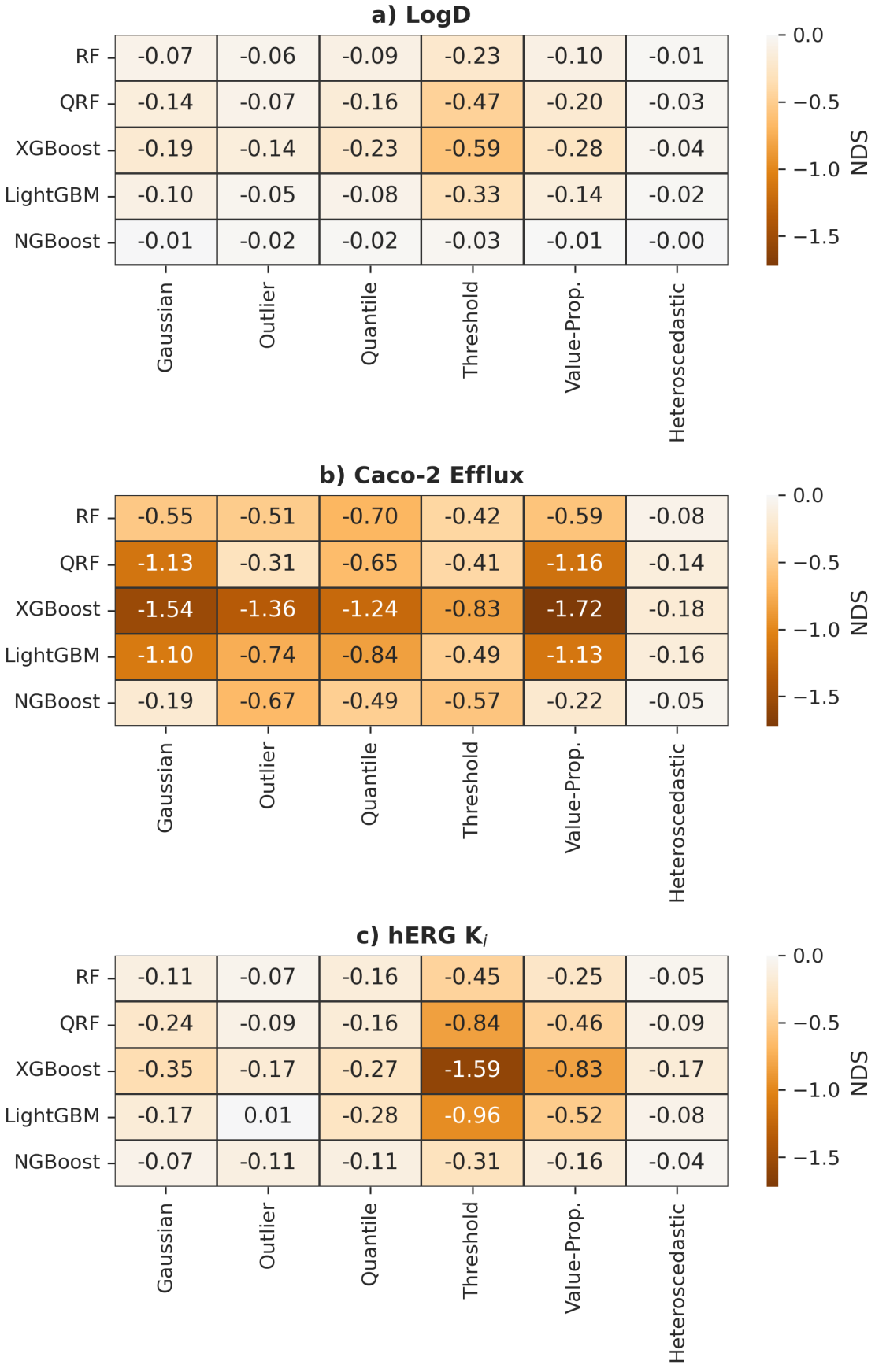


Figure 3.6: Noise degradation slope (NDS) heatmaps for three external validation datasets: a) LogD, b) Caco-2 Efflux, and c) hERG K_i. Lower (more negative) NDS indicates faster degradation under noise. Black cells indicate baseline $R^2 < 0.3$ for that (model, strategy) pair; "N/A" indicates filtered extreme values ($|NDS| > 2$). Models whose baseline R^2 fell below 0.3 across all strategies are omitted entirely, accounting for the smaller set of rows on hERG K_i.

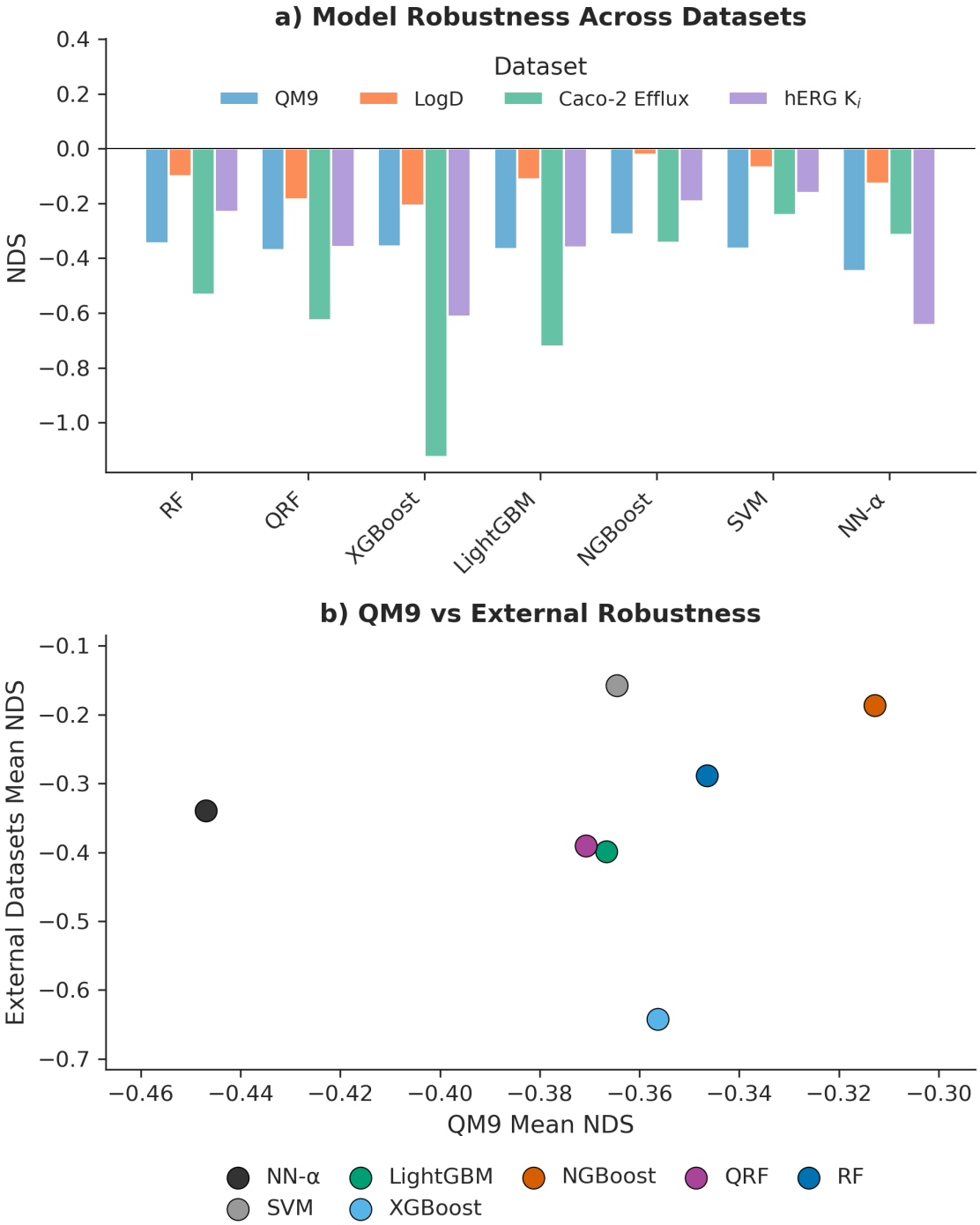


Figure 3.7: Comparison of noise robustness on QM9 (HOMO–LUMO gap, N = 10,000) and three external validation datasets: LogD (N = 5,039), Caco-2 Efflux (N = 2,161), and hERG K$_i$ (N = 1,482). a) Model robustness measured by the noise degradation slope (NDS) across all four datasets; lower (more negative) NDS indicates faster degradation under noise. b) QM9 vs external robustness: models with inherent robustness mechanisms (NGBoost, SVM) maintain their relative standing, while ensemble-dependent models (XGBoost) degrade on the external datasets.

represented by the per-sample Spearman correlation between predicted uncertainty and noise magnitude (Table 3.4). While GPs and NGBoost gave the strongest correlations, BNNs were moderate. Learned embeddings (MHG-GNN and mol2vec) struggled, so much so that their correlations dropped to near zero or below on every model tested.

Table 3.4: Strongest and weakest model–representation combinations for uncertainty–noise correlation on the QM9 HOMO–LUMO gap (Gaussian noise strategy). Columns: **Unc-Noise** ρ = Spearman correlation between predicted uncertainty and injected noise magnitude $|\epsilon|$ (higher = better noise detection); **Unc-Error** ρ = Spearman correlation between predicted uncertainty and absolute prediction error; **ECE** = expected calibration error (lower = better-calibrated); **Cov.** 1σ / 2σ = empirical coverage at the 1σ (68% target) and 2σ (95% target) intervals.

Model	Rep.	Unc-Noise ρ	Unc-Error ρ	ECE	Cov. 1σ	Cov. 2σ
GP	**SNS**	**0.56**	0.23	0.14	71.2%	94.6%
GP	Morgan	0.53	0.22	0.14	71.4%	95.4%
NGBoost	**SNS**	**0.47**	0.16	0.13	69.3%	95.4%
NGBoost	**PDV**	**0.47**	**0.29**	**0.11**	69.0%	95.2%
NGBoost	PDV (binary)	0.44	0.25	0.13	69.8%	95.4%
NGBoost	Morgan	0.44	0.20	0.14	69.8%	95.3%
BNN-α	SNS	0.42	0.21	0.17	73.4%	95.1%
BNN-β	SNS	0.40	0.21	0.18	73.5%	95.1%
BNN-β	Morgan	0.38	0.20	0.18	72.1%	95.0%
BNN-β	PDV	0.38	0.24	0.19	72.1%	94.0%
Contrast: lowest correlations (graph/embedding representations)						
BNN-β	mol2vec	−0.05	0.02	1.33	50.6%	76.4%
VBLL-α	MHG-GNN	−0.05	0.09	13.41	26.6%	38.6%
VBLL-β	MHG-GNN	−0.03	0.10	15.34	31.1%	47.1%
VBLL-β	mol2vec	−0.03	0.15	23.86	44.8%	58.0%

3.7.2 NGBoost and the GP stay near target coverage while Bayesian variants miscalibrate

Samples containing label noise tend to produce large residuals in training, and every model has its own way of handling them. NGBoost contains predicted scales that increase to absorb residual [58]. The GP contains has a single global observation noise term. It does not absorb individual residuals, so its per-sample uncertainty derives primarily from the posterior variance rather than label noise [63, 64]. BNNs treat these large residuals differently; they broaden their weight posteriors, increasing predicted uncertainty without explicitly modeling observation noise [105, 113].

VBLL adds a learned noise variance to the BNN loss [114], but this value is a global scalar, independent of the input. As seen in Table 3.4 and Additional file 9, both BNNs and VBLLs achieve moderate correlations which improve when paired with fingerprint or descriptor-based representations. Among models that learn separate distributional parameters during training, GP and NGBoost produced the strongest correlations between uncertainty and label noise, though QRF, which separates parameters using quantile distributions, produced both poor label predictions and uncertainty-noise correlations. At the population level, mean predicted uncertainty increases with artificial noise. Although we expect to see aleatoric uncertainty increase with injected noise, for VBLL both the aleatoric and epistemic components increased (Figure 3.8b).

3.7.3 Uncertainty attribution is itself representation-dependent

Mean predicted uncertainty rises with label noise, but that alone does not say what the model attributes the rise to. We expected the aleatoric part to rise with injected label noise, and the epistemic part to stay roughly flat. Three kinds of model split their uncertainty per molecule: the full BNNs with a variance head, the heteroscedastic GP, and QRF [54, 64, 113]. We excluded the other GPs. Their observation noise is one number for the whole fit, so it cannot follow individual molecules [64]. For each model we took the rise in mean epistemic uncertainty from no added noise to noise level 1.5, over held-out molecules. We divided it by the rise in mean aleatoric uncertainty over the same range. A ratio near 0 means the added noise went to the aleatoric part. A ratio of 1 or more means the epistemic part grew as much or more (**[FIGURE NEEDED: table of the ratio of epistemic rise to aleatoric rise, one row per model (the two full BNNs with a variance head, heteroscedastic GP, QRF) and noise condition (Gaussian, grouped-shifted), one column per representation, each entry the range over datasets on held-out molecules]**).

The full BNNs with a variance head split the added noise as expected. Their ratio is at most 0.43 in every model, dataset, representation and noise condition. QRF does not. Both of its parts rise with noise, and on some representations the epistemic part rises faster (**[FIGURE NEEDED: aleatoric and epistemic parts against noise level, one panel per model with a per-molecule split, QM9 HOMO–LUMO gap, ECFP4, Gaussian noise]**). This holds although QRF is

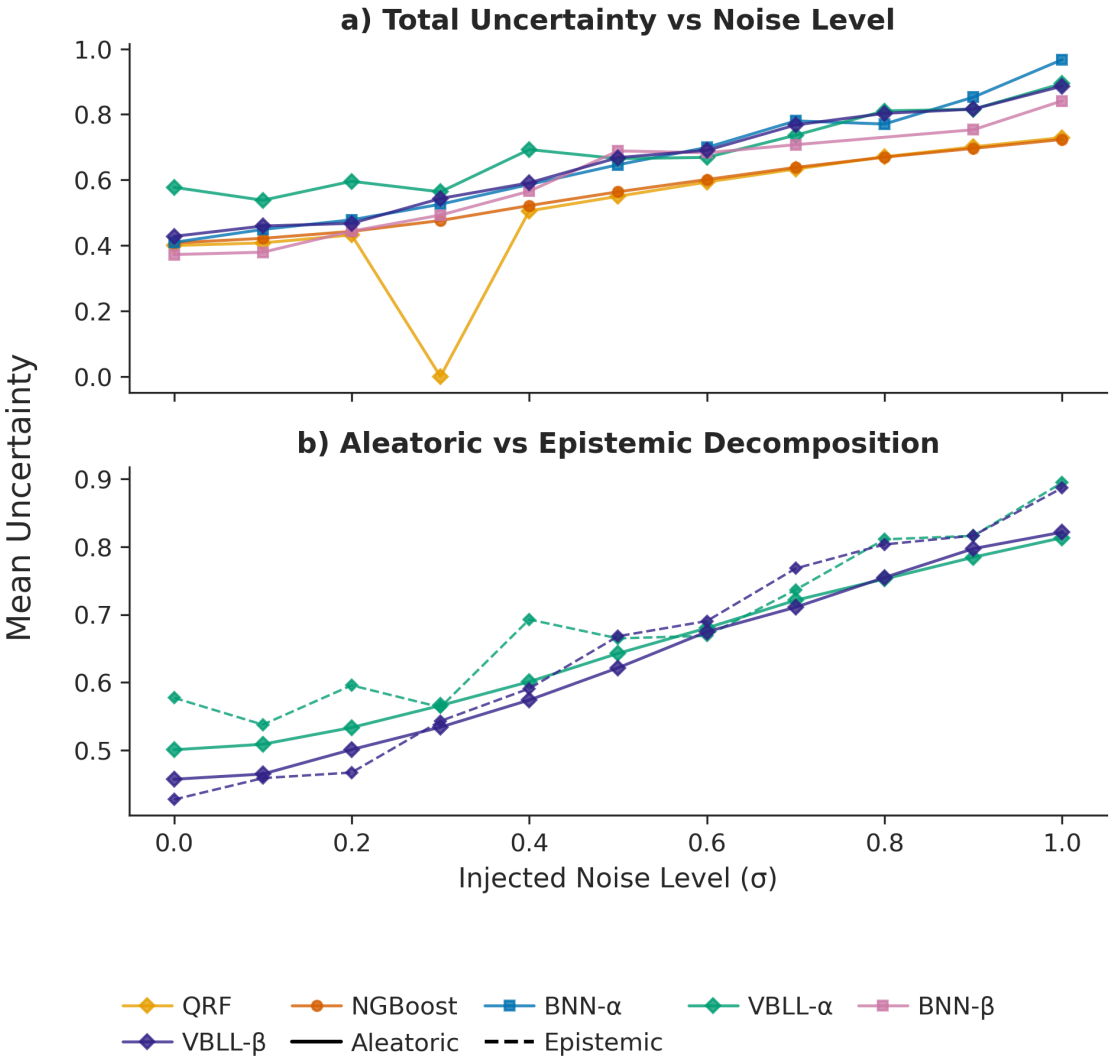


Figure 3.8: Uncertainty estimates under label noise on the QM9 HOMO–LUMO gap. a) Mean predicted uncertainty versus σ for probabilistic models (PDV, Gaussian strategy). b) Aleatoric and epistemic uncertainty components versus σ for VBLL models.

among the more noise-robust models. On QM9 under Gaussian noise, it ranks 4th of the 19 models and variants on $\mathrm{AUC}_{\mathrm{norm}}$on four of the six representations. Its clean R^2 on the same data ranks between 6th and 13th of the 19, depending on the representation.

Representation explained little of the variance in noise robustness, but for QRF it decides how the added noise is split. Over datasets and every noise condition except censoring, the QRF ratio on held-out molecules ranges from 0.63 to 1.09 on the three fingerprints, ECFP4, Avalon and Sort & Slice. On PDV the same range is 1.03 to 1.42. On ChemBERTa and MHG-GNN it is 1.28 to 2.05. Under Gaussian noise alone, the held-out ranges are 0.65–0.75 on ECFP4, 0.69–0.76 on Sort & Slice and 0.81–0.89 on Avalon. They are 1.17–1.32 on PDV, 1.54–1.72 on MHG-GNN and 1.60–1.88 on ChemBERTa. On the learned embeddings, QRF therefore puts more of the added noise into its epistemic part than into its aleatoric part.

The type of noise also moves the split, most clearly for the heteroscedastic GP. Under Gaussian noise its ratio on held-out molecules lies between -0.05 and 0.36 on every representation. Under grouped-shifted noise the range over datasets is 0.73–1.00 on ECFP4, 0.44–0.61 on Sort & Slice and 0.32–0.46 on Avalon. On PDV, ChemBERTa and MHG-GNN it is 0.13–0.22, 0.14–0.23 and 0.21–0.25. Laplace, Student-t and outlier noise give splits like those under Gaussian noise. Grouped-shifted noise raises the epistemic share of both the heteroscedastic GP and QRF, for every dataset and representation. One possible reading is that a shared offset per scaffold family looks to the model like structure in the data, not label noise.

We next asked how closely the aleatoric part follows the amount of added noise. Held-out labels carry no injected noise, so this comparison uses out-of-fold training molecules. We fitted the slope of mean aleatoric uncertainty against the standard deviation of the added noise, both in the label's units, under Gaussian noise. We report the median over folds (**[FIGURE NEEDED: aleatoric part against the standard deviation of the added noise, per model, ECFP4 and PDV, out-of-fold training molecules, dashed line where reported noise equals added noise]**). On ECFP4 and PDV the slope runs from 0.36, for QRF on PDV on Caco-2, to 1.09, for a full BNN with a variance head on PDV on hERG K_i. Including ChemBERTa widens the range to 0.28, for QRF on Caco-2, and 1.62, for a full BNN with a variance head on hERG K_i. The aleatoric part can therefore compare noise between datasets or between noise levels. It does not estimate a dataset's own measurement error.

Censoring behaves differently from the other conditions. Clipping labels at a limit narrows their spread. NGBoost and QRF shrink their total uncertainty with that spread on every dataset, and the GPs do the same on hERG K_i and Caco-2. None of these models treats a clipped label as a noisy one. They grow more confident while the labels lose information.

3.7.4 Fingerprint representations carry the usable uncertainty signal

The choice of representation has a much stronger impact on per-sample uncertainty tracking than on noise robustness. Fingerprints and physicochemical descriptors provided a clear signal with which models such as GP and NGBoost could separate noise-induced residuals from structural variation. However, learned embeddings such as MHG-GNN and mol2vec did not produce viable uncertainty-noise correlations (Table 3.4). Although these learned embeddings can be effective representations for many tasks, this research suggests that they provide challenges for per-sample noise tracking. However, there is room for further investigation, as only two embeddings were tested.

3.8 Conclusions

These results lead to two main conclusions. The first is that noise robustness, defined as the rate at which predictive performance degrades with increasing label noise, is primarily determined by the model's training mechanism. The margin maximization of SVM, the weight priors of BNNs, and the ensembling in tree methods all appear to keep those models from fitting to noise. The choice of representation has a stronger influence on predictive performance than noise robustness. The interaction term between representation and model has a stronger impact; some models like SVM and full BNNs remain robust across all representations, while RF and NNs are robust only with certain ones. While the pattern of noise has a strong impact on predictive performance, it does not influence noise robustness rankings between models or representations much.

We investigated how a wide variety of molecular representations and model architectures perform under increasing label noise in QSAR modeling. We used ANOVA variance decomposition across three noise conditions on the QM9 HOMO–LUMO

gap task to show that with regards to predictive performance, model architecture and molecular representation account for comparable shares of the variance. However, for robustness to label noise, measured as $\mathrm{AUC}_{\mathrm{norm}}$, the normalized area under the R^2 retention curve, model architecture is the dominant factor, while representation explains less than 10% of the variance.

Threshold and value-proportional noise produce the widest spread of NDS across models; outlier noise barely separates them. Model rankings are highly concordant across strategies (Kendall's $W = 0.92$), suggesting that a model's robustness to noise is independent of type of noise. Predictive performance on clean data is not correlated with noise robustness; NGBoost and SVM, the most noise-robust models, did not perform particularly well on clean data. Embeddings (MHG-GNN, mol2vec) struggled with regards to noise robustness, especially with neural network architectures. On the flip side, QRF was significantly less noise-robust than RF across datasets, suggesting that quantile regression may be overfitting to noisy labels rather than absorbing them.

The second conclusion concerns whether a model's uncertainty estimates track per-sample label noise. Some models struggle, particularly those like BNNs whose uncertainty comes from a posterior over weights or QRF which does produce a full quantile, but is unable to successfully track noise. Models that fit a separate, per-sample scale or observation-noise parameter during training are able to track that label noise more effectively. NGBoost identifies noisy labels through a per-sample predicted scale that absorbs large residuals. The GP does not: it has a single global observation noise term, so its per-sample uncertainty comes from the posterior variance, which depends on where a molecule sits relative to the training molecules and not on its label. Among representations, uncertainty tracking collapses with the learned embeddings MHG-GNN and mol2vec, while fingerprints and descriptors produce a viable signal. Neither strong predictive performance nor noise robustness implies noise tracking; again it is up to the model's architecture and how it handles loss.

Although the choice of representation carries less weight than that of models, we observed an interesting phenomenon. PDVs produce the strongest predictive performance on clean data, but are the least noise-resistant. Embeddings, particularly mol2vec, are the most robust to noise yet perform the worst noise tracking. At the end of the day, the choice of both model and representation comes down to your problem, including the amount of noise in your data, your objectives, and your

compute limits.

QM9 served as a relatively clean baseline; it is computationally-derived with negligible measurement noise. The validation datasets, LogD, Caco-2 [109], and hERG Ki [110], tested are smaller and cover a narrow range of the chemical and biological space, and have an unknown inherent amount of noise from experimentally-determined values, on top of the added artificial noise. These validation datasets supported the QM9 findings, with XGBoost the notable exception, degrading on the external datasets. Future work should extend this type of noise robustness benchmarking to classification tasks and larger experimental datasets with better-known uncertainty estimates.

When working with noisy experimental data in QSAR settings, it is important to keep in mind that noise robustness and noise-aware uncertainty estimation are primarily driven by the model's mechanism of training, and to a certain extent the choice of molecular representation, usually with respect to the model. While Bayesian transformations on neural networks improve noise robustness, they do not produce uncertainty estimates that correlate particularly strongly with noise. Models like NGBoost and GPs, which include a learned noise or scale parameter in the training objective, are the most robust to noise across both various types of noise and datasets and often produce uncertainty estimates which track per-sample label noise. Pairing these models with fingerprint-based representations provides additional benefits in restricting, and potentially detecting and mitigating label noise.

Chapter 4

The Flaws of the Similarity Assumption

This chapter asks whether the standard ways of comparing molecules suit the dataset and task at hand. General-purpose representations are built for all of chemistry, so we count how many of their features vary on one dataset. On hERG K_i, 1,815 of ECFP4's 2,048 features take one value in at least 95% of the 1,412 molecules. Some representations also give different molecules the same vector, mostly stereoisomers, and no model can tell those molecules apart. A distance compares every feature, so features irrelevant to the task dilute it, and different representations disagree about which molecules are close. In an exploratory study of learning with noisy labels, all 20 configurations with the lowest retention used a distance, and noise detection by neighbors usually made the model worse.

4.1 Introduction

One of the fundamental assumptions in cheminformatics is that two structurally similar compounds display similar chemical properties [74]. While there are important exceptions, namely activity cliffs [72, 75], this is a well-established concept. However, there is no single way to define similarity between compounds, and this is where the assumption begins to break down in practice. When defining similarity between two molecules, we typically rely on some pairwise mathematical function, or coefficient, that takes in two molecular representations as input. There's two varying factors here, the representation and the coefficient. Bajusz et al. [71] found that among standard coefficient, the choice between them matters little. This question

questions the underlying representation.

General-purpose molecular representations are designed to work across chemical domains. Consequently, when considering data from a specific domain, many of their features show little to no variance. Some representations assign separate molecules identical feature vectors. Since common fingerprints do not encode stereochemistry [116], they are often unable to differentiate stereoisomers, which can have substantially different measured activity [117]. As these coefficients compare every feature, irrelevant features dilute the result. Different representations also disagree about which molecules are close based on the information they carry.

The problem of whether neighboring molecules can predict chemical activity has been measure before. The MODelability Index (MODI) of Golbraikh et al. [118] scores a binary dataset by the proportion of nearest-neighbor pairs that share an activity class, producing a single value for how strongly the similarity assumption holds on that particular combination of representation and data. It is not clear whether a low value is due to the data being intrinsically hard to model or to the representation not capturing adequate features for the task. Other work in this area relates to activity cliffs. Sheridan et al. [119] compared experimental error, activity distribution and activity-cliff metrics as predictors of how well a dataset can be modeled, and found the cliff metrics the strongest of the three. However, they found that these metrics cannot separate real cliffs from those produced by uncertain activity measurements. Medina-Franco [120] point out that some of these activity cliffs are artifacts of the chosen representation, similarity threshold, or noise in the data. Activity cliffs are distrinct from noise, as they carry some of the most valuable information about the interaction between structure and activity. None of these measures ask specifically about whether the specific features within these representations matter for the property at hand.

There has been work that looks into how much the choice of representation changes similarity measure. Riniker and Landrum [121] benchmarked 12 fingerprints for similarity across 3 public datasets and found high correlations between the, only two simple baseline fingerprints differed signficantly from the rest. The same was found for learned embeddings as neural embeddings gave negligible or no improvement over ECFP across 25 pretrained models and 25 datasets [122]. Huber and Pollmann [123] did find a difference between the structure of the fingerprints. Measured using a graph-based structural reference, fingerprints that store raw counts improve agreement with the reference over folded fingerprints. The process of folding

introduces bit collisions that distort similarity for molecules with many bit sets [123]. Activity cliffs follow the same pattern, across 15 combinations of representation and similarity meausre on 3 datasets, the representation disagreed about which pairs form cliffs [124]. This supports the point Medina-Franco [120] make about cliffs as artifacts of their representation. Any method built on similarity like activity cliffs inherits this choice Wilson [103]. For the purposes of this thesis, these similarity measures are used across noise estimation and mitigation tasks.

The purpose of this chapter is to establish the problem that Chapter 5 sets out to solve. Section 4.2 asks for a given dataset, how much the features of standard representations vary and how many molecules map to identical feature vectors.

Section 4.3 looks at what happens when you used these features to quantify similarity, while Section 4.4 further follows the consequences of using these similarity measures in downstream methods. This is supported by a sweep of LNL methods reported in Section 4.4.1.

4.2 Flaws in standard base representations

To quantify similarity between two compounds, we must first choose a representation to encode each compound. If the features within these representations are flawed, the similarity measure inherits them. Many standard representations are built to represent compounds across a wide array of chemical domains and be task-independent. However, chemical datasets often only cover a small region of the chemical space, leaving these general-purpose representations to describe chemistry which is absent from the dataset at hand.

4.2.1 Most features do not vary on a given dataset

ECFP4 folded to 2,048 bits is built to describe any molecule, before any dataset or task is known [4]. It is defined in Chapter 2. One dataset often comes from a few chemical series. On such a dataset most bits are off in almost every molecule. The features that matter for a task are a small subset of the ones that vary, and which subset depends on the task. This subsection counts only how many features vary at all. Whether the varying ones relate to the task is the question of Chapter 5.

We count two things per representation and dataset. A constant feature has one value in every molecule of the dataset. A near-constant feature has one value in at

least 95% of the molecules. The near-constant count includes the constant features. We count on all molecules of each dataset, using the feature matrices the full runs use. The datasets are hERG K_i, Caco-2, LogD and QM9, plus MDR1–MDCK efflux, human and rat liver microsome clearance, and the Lhasa hERG blocker training set. For each dataset we count its base representations, plus MACCS keys wherever they were computed.

On these datasets, the hashed substructure fingerprints have the most near-constant features. The learned embeddings have none, with one exception described below. ECFP4 on hERG K_i (1,412 molecules) has 1,815 near-constant features out of 2,048. On the seven datasets where ECFP4 is a base representation, this count runs from 1,815 (hERG K_i) to 1,997 (QM9, 9,978 molecules). SMI-TED (768 features) and ChemBERT (256 features) on hERG K_i have 0 near-constant features. GROVER, GraphMVP, MoLFormer, ChemBERTa-2, SELFormer and Mol2vec also have 0 on every dataset where they are base representations. The exception is MHG-GNN. Of its 1,024 features, 292 are zero in every molecule. They are the same 292 features on all seven datasets that use it, so they come from the saved MHG-GNN output, not from the data. We have not checked why the encoder gives zero there. The WL graph kernel also has 0 near-constant features. It keeps only labels present in at least 5% of a dataset's molecules, so it is filtered on each dataset before it is counted. Every other count is in Tables 4.1 and 4.2, and Figure 4.1 shows them as shares of each representation's features.

Which of the two counts matters depends on the distance. A bit that is off in every molecule does not change Tanimoto similarity, because bits off in both molecules are ignored. A bit that is on in every molecule is not harmless. It adds one to both the shared and the total bit counts, so it raises every Tanimoto similarity slightly. On hERG K_i all 105 constant ECFP4 bits are off. On Caco-2, 1 of its 194 constant ECFP4 bits is on, and on LogD 1 of 39. On hERG K_i, 2 of the 24 constant MACCS features are on. After standardization, a constant feature adds zero to a Euclidean or Manhattan distance. Near-constant features do distort these distances. After standardization they take large values on the few molecules that have them. A bit that is on in 5% of molecules becomes 4.36 on those molecules. A bit on in one of hERG K_i's 1,412 molecules becomes 37.56 on that molecule. So the argument rests on the near-constant count, not the constant count. Section 4.3 shows the effect on distances.

Dablander [125, p. 112] states, without a count, that the least frequent substruc-

tures usually appear in only a few compounds. The same point recurs on p. 121. His statement is about substructures before hashing. Our count is of bits after folding to 2,048. Bit collisions (Chapter 2) also make a selected bit harder to read, because one bit can stand for several substructures. So a selected fingerprint bit is harder for a chemist to read than a named descriptor. This returns in Chapter 5, in the discussion of blueprints and what they tell a chemist.

If too few bits vary, different molecules can get identical vectors.

Tables 4.1 and 4.2 give, for each representation and dataset, the number of constant features and the number of near-constant features. The Features column gives the total number of features. The WL graph kernel's number of features differs between datasets, so its entries give the total as a third number.

Table 4.1: Constant and near-constant features, part 1. Each entry is "constant / near-constant" features of that representation on that dataset. The number of molecules is under each dataset name. – means the representation is not in that dataset's base representations. n/a means no saved MACCS features exist for QM9.

Representation	Features	hERG K_i (1,412)	Caco-2 (2,158)	LogD (5,028)	QM9 (9,978)
ECFP4	2,048	105 / 1,815	194 / 1,826	39 / 1,834	0 / 1,997
Atom pairs	2,048	201 / 977	563 / 1,239	–	–
Topological torsions	2,048	–	1,179 / 1,822	1,059 / 1,818	–
Avalon	2,048	50 / 985	98 / 1,128	30 / 1,076	4 / 1,380
MACCS keys	167	24 / 60	36 / 86	31 / 87	n/a
WL graph kernel	140–317	0 / 0 / 276	0 / 0 / 317	–	–
PDV	200	21 / 55	35 / 66	–	–
Mordred	1,613	242 / 336	–	247 / 340	307 / 394
GROVER	4,800	0 / 0	0 / 0	0 / 0	–
MHG-GNN	1,024	292 / 292	292 / 310	292 / 304	–
GraphMVP	300	–	–	0 / 0	0 / 0
MoLFormer	768	–	0 / 0	0 / 0	0 / 0
SMI-TED	768	0 / 0	0 / 0	0 / 0	0 / 0
ChemBERT	256	0 / 0	–	0 / 0	0 / 0
ChemBERTa-2	384	–	–	–	0 / 0
SELFormer	768	–	–	–	0 / 0
Mol2vec	300	–	–	–	0 / 0

MACCS keys have 167 features here. RDKit stores the 166 MACCS keys plus an unused bit 0, which is off in every molecule. That bit is one of the constant MACCS features counted above. On every dataset but QM9, all constant Mordred features are zero in every molecule. On QM9, one of the 307 is 1 in every molecule and the rest are zero. Missing or infinite descriptor values are set to 0 (Chapter 5, "Representations by domain"). So some of these may be descriptors that failed to

Table 4.2: Constant and near-constant features, part 2. Entries read as in Table 4.1. hERG blockers is the 7,772-molecule training set from Lhasa Limited.

Representation	Features	MDR1–MDCK (2,641)	HLM clearance (3,086)	RLM clearance (3,053)	hERG blockers (7,772)
ECFP4	2,048	1 / 1,890	–	1 / 1,887	0 / 1,840
Atom pairs	2,048	308 / 1,233	260 / 1,240	262 / 1,238	28 / 929
Topological torsions	2,048	–	–	–	625 / 1,754
Avalon	2,048	5 / 1,032	8 / 1,044	8 / 1,045	0 / 866
MACCS keys	167	22 / 58	20 / 59	20 / 58	9 / 50
WL graph kernel	140–317	0 / 0 / 143	0 / 0 / 141	0 / 0 / 140	0 / 0 / 162
PDV	200	21 / 59	14 / 59	16 / 59	–
Mordred	1,613	226 / 331	222 / 336	222 / 331	197 / 304
GROVER	4,800	0 / 0	0 / 0	0 / 0	0 / 0
MHG-GNN	1,024	292 / 292	292 / 293	292 / 293	292 / 292
MoLFormer	768	–	0 / 0	0 / 0	0 / 0
SMI-TED	768	0 / 0	0 / 0	0 / 0	–
ChemBERT	256	–	0 / 0	–	–
Mol2vec	300	–	–	–	0 / 0

compute. We have not checked which.

4.2.2 Molecules the representation cannot tell apart

A standard fingerprint- or descriptor-based representation can produce the same feature vector for two separate molecules. In turn, models using this representation will produce a single prediction for the molecules with identical features. It should be noted that identical feature vectors within a dataset do not necessarily mean that they derive from the same underlying molecule. The dataset could just hold two different measurements of the same molecule twice, as is possible with data pooled across different sources.

Many fingerprints are computed without stereochemistry [116]. In this research, we build ECFP4 fingerprints with stereochemistry off while the 2D descriptor sets (PDV, Mordred) also ignore it such that for every dataset, each molecule they label a tie is a stereoisomer of another. Any representation read from the 2D graph merges stereoisomers, since the two share an identical graph. Separately, methods using the Weisfeiler–Lehman relabeling scheme are bounded by the 1-Weisfeiler–Lehman test explained in Chapter 2, so some non-isomorphic graphs also receive the same vector; the graph-kernel features tie 105 of 1,412 hERG K_i molecules, 56 of which differ only in stereochemistry and 49 in more than that. Whether a learned embedding differentiates stereoisomers is up to its individual implementation. SMI-TED, a

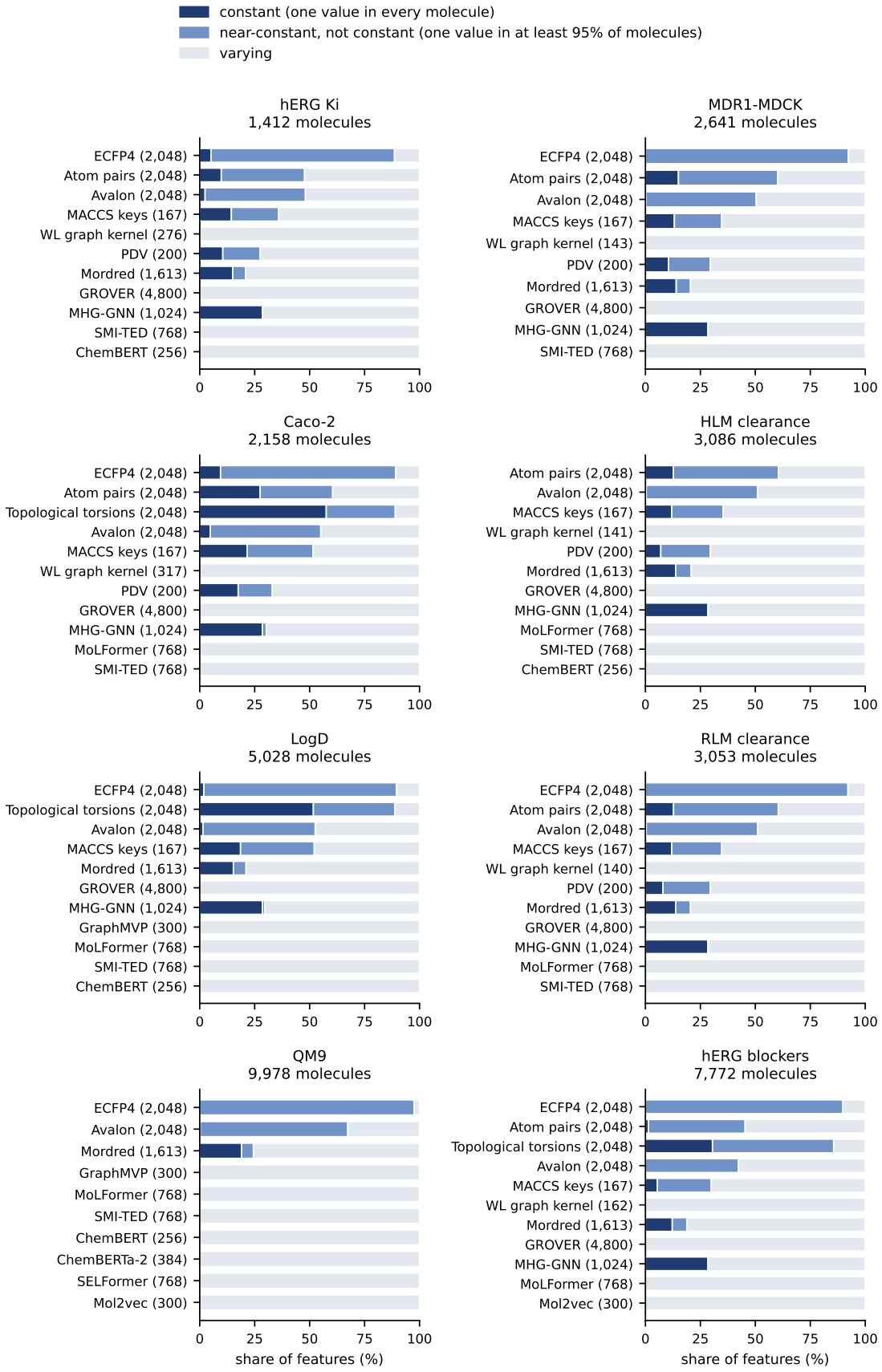


Figure 4.1: Share of each representation's features that are constant (one value in every molecule), near-constant but not constant (one value in at least 95% of molecules), and varying. One panel per dataset, one bar per representation. The number of features is in brackets after each name.

pretrained SMILES encoder, ties 52 hERG K_i, 633 LogD and 177 Caco-2 molecules, all stereoisomers. A tokenizer can drop stereochemistry before the model even sees the input string. The ChemBERTa-77M-MTR tokenizer used in Chapter 3 gives 72 of 1,415 hERG K_i molecules the same token sequence as another molecule. If the difference between stereoisomers was negligible, this wouldn't matter. However, stereoisomers can differ in measured activity [117], meaning these representations are all lacking potentially important chemical information.

For each dataset and representation, we grouped the molecules whose stored feature vectors are exactly identical. We then gave every molecule in such a group one of the three reasons above. A molecule counts as a repeat if another molecule in its group has the same InChIKey. It counts as a featurization failure if its feature vector is all zeros. Otherwise it is a distinct molecule. We call a distinct molecule stereo-only when every molecule it shares a vector with has the same InChIKey connectivity block, so the two differ only in stereochemistry. No model is fitted.

Every molecule in a tied group must receive one prediction, and the prediction with the lowest squared error is the group's mean label. The lowest error any model can reach is therefore

$$\mathrm{RMSE}_{\min} = \sqrt{\frac{1}{n}\sum_{G}\sum_{i\in G}(y_i - \bar{y}_G)^2}, \tag{4.1}$$

where n is the number of molecules, G runs over the tied groups and $\bar{y}_G$ is the mean label of group G. A molecule in no tied group adds nothing to the sum. $\mathrm{RMSE}_{\min}$ is in the dataset's label units. The matching best R^2 is $1 - \mathrm{RMSE}^2_{\min}/\mathrm{Var}(y)$. We also report the label spread inside each tied group, its highest label minus its lowest. For scale, repeat pK_i measurements of one compound disagree by about 0.54 log units [15]. A tied group whose pK_i values differ by more than that is not explained by measurement noise alone. We counted on hERG K_i (1,412 molecules after deduplication by InChIKey), LogD (5,028 molecules) and Caco-2 (2,158 molecules). Each dataset was counted on its final pool of representations and on its augmentation blocks.

No tie on any of the three datasets came from a repeated molecule or from a row of zeros. Every tie is a merge made by the representation. Table 4.3 lists every representation that ties at least one molecule. Atom pair, PDV, Mordred and SMI-TED tie only stereoisomers, on every dataset whose pool contains them. ECFP4, Avalon and topological torsion tie mostly stereoisomers, plus a few molecules that

are not stereo-only. ECFP4 on hERG K_i ties none of the latter. The most is topological torsion on LogD, with 44 such molecules. On hERG K_i, ECFP4, atom pair, PDV, Mordred and SMI-TED all split the 1,412 molecules into the same 26 groups, holding the same 56 molecules. So joining any of these five leaves all 56 tied. The graph kernel ties 49 hERG K_i and 26 Caco-2 molecules that are not stereo-only. MACCS ties the most molecules on every dataset, and most of its tied molecules are not stereo-only. On hERG K_i only 30 of its 377 tied molecules are. The 3D representations tie no molecules. Neither do the learned embeddings ChemBERT, GROVER, MHG-GNN, MoLFormer and GraphMVP, on any dataset whose pool contains them. This includes ChemBERT on hERG K_i and on LogD. Joining every representation in a dataset's pool side by side leaves no molecule tied on any of the three datasets.

The lowest reachable error stays small next to the spread of the labels. On hERG K_i it is at most 0.134 pK_i (MACCS), against a label standard deviation of 0.915 pK_i. The spread inside one tied group can still be large. The largest ECFP4 group spread on hERG K_i is 1.14 pK_i, more than twice the repeat-measurement scale. Figure 4.2 shows that pair.

The structure–activity landscape index of a pair of molecules i and j is

$$\mathrm{SALI}_{ij} = \frac{|y_i - y_j|}{1 - \mathrm{sim}(i, j)} \tag{4.2}$$

[126]. A tied pair has similarity 1, so its index is infinite whenever the two labels differ. A tie with different labels is therefore the most extreme activity cliff there can be. Some cliffs are made by the representation, not by the chemistry. A method that uses nearest neighbors sees a tied neighbor at distance 0 with a different label. It may then flag a correct label as noise (Section 4.4.2).

Each row of Table 4.3 is one representation on one dataset. Only representations that give at least two molecules the same vector are listed.

On hERG K_i, the mean label spread inside a tied group is 0.28 pK_i under ECFP4 and 0.39 pK_i under MACCS. Of the 26 ECFP4 groups, 24 hold molecules with different labels, and 6 differ by more than 0.54 pK_i. Of the 153 MACCS groups, 138 hold different labels, and 38 differ by more than 0.54 pK_i.

It is worth noting that some of these representations are more straightforward than others in terms of the underlying chemistry they reference. For example, the PDV carries very clear physicochemical properties such that if you examined one of

Table 4.3: Molecules that share a feature vector with another molecule. *Tied*: molecules whose vector is identical to at least one other molecule's (share of the dataset in brackets). *Stereo-only*: tied molecules that share their vector only with their own stereoisomers. *Best RMSE* and *best* R^2: the lowest error any model can reach when every tied group gets one prediction, its mean label. *Largest spread*: the highest minus the lowest label inside one tied group. RMSE and spread are in the dataset's label units, given after its name. (aug.) marks an augmentation block. Representations in a dataset's pool that are not listed tie no molecules.

Dataset	Representation	Molecules	Tied	Stereo-only	Best RMSE	Best R^2	Largest spread
hERG K_i (pK_i)	ECFP4	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Atom pair	1,412	56 (4.0%)	56	0.041	0.998	1.14
	PDV	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Mordred	1,412	56 (4.0%)	56	0.041	0.998	1.14
	SMI-TED	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Avalon	1,412	66 (4.7%)	56	0.049	0.997	1.20
	Graph kernel	1,412	105 (7.4%)	56	0.071	0.994	1.49
	MACCS (aug.)	1,412	377 (26.7%)	30	0.134	0.978	1.80
LogD (log units)	ECFP4	5,028	641 (12.7%)	633	0.044	0.999	1.00
	Mordred	5,028	633 (12.6%)	633	0.043	0.999	1.00
	SMI-TED	5,028	633 (12.6%)	633	0.043	0.999	1.00
	Avalon	5,028	647 (12.9%)	629	0.047	0.998	1.40
	Topological torsion	5,028	675 (13.4%)	631	0.047	0.998	1.10
	MACCS (aug.)	5,028	2,343 (46.6%)	437	0.220	0.965	2.80
Caco-2 (log units)	ECFP4	2,158	179 (8.3%)	177	0.027	0.996	0.74
	Atom pair	2,158	177 (8.2%)	177	0.027	0.996	0.74
	PDV	2,158	177 (8.2%)	177	0.027	0.996	0.74
	SMI-TED	2,158	177 (8.2%)	177	0.027	0.996	0.74
	Avalon	2,158	181 (8.4%)	177	0.027	0.996	0.74
	Topological torsion	2,158	186 (8.6%)	177	0.029	0.996	0.74
	Graph kernel	2,158	203 (9.4%)	177	0.028	0.996	0.74
	MACCS (aug.)	2,158	804 (37.3%)	139	0.111	0.935	1.72

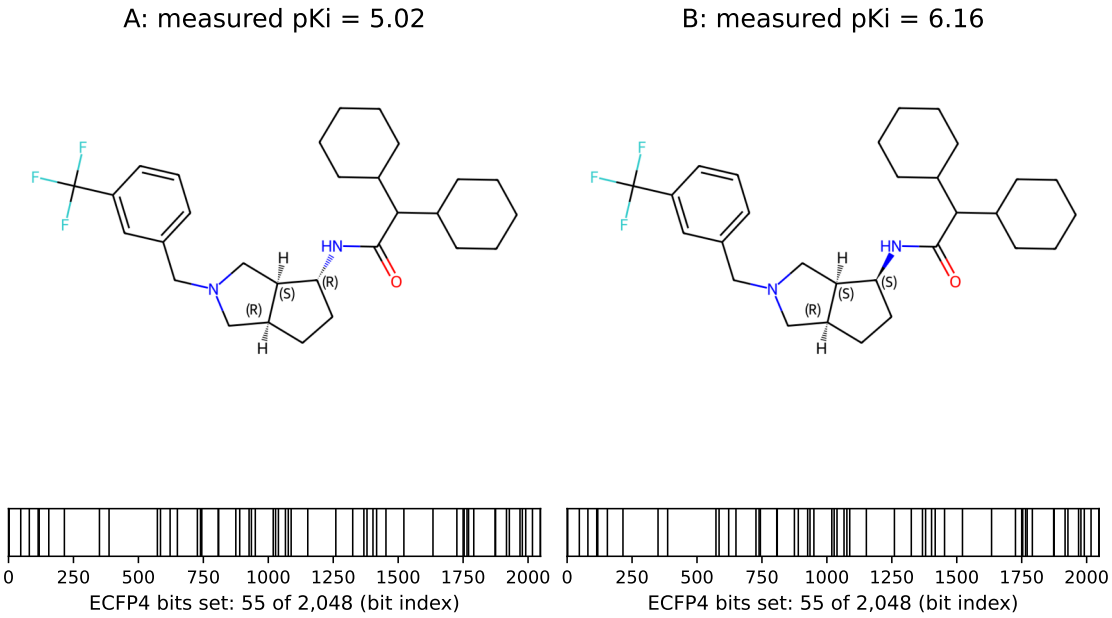


Figure 4.2: Two hERG K_i molecules that differ only at one stereocenter, the carbon bearing the amide nitrogen (R in A, S in B). Below each molecule are the ECFP4 bits it sets: the same 55 of 2,048 bits in both. The measured pK_i values are 5.02 and 6.16. Any model on ECFP4 must give both the same prediction, and the one with the lowest squared error is their mean, 5.59 pK_i. Each molecule is then 0.57 pK_i off.

it's features, you'd know exactly the type of chemical information it contains. Other representations like frozen embeddings, representing the last layer of a network, do not tell us what type of chemical information, if any, they relate to.

4.3 Flaws in similarity and other pairwise computations

We've already discussed some of the flaws of individual representations, but what happens when you compare two instanes of one of these representations? These pairwise comparisons, as described in Chapter 2, have the potential to violate the similarty assumption. When considering similarity and other pairwise comparisons, we discuss both the representation and the coefficient.

The standard way of comparing two small molecules is the Tanimoto coefficient, which operates on any bit vector, applied to Morgan fingerprints [127]. Independently, the Tanimoto coefficient is described as the "coefficient of choice" for molecular similarity [71], while Morgan fingerprint, represented as ECFP4 in this research, is likewise one of the strongest fingerprints for the task, ranking among the

strongest fingerprint for virtual screening and target prediction [128]. ECFP4 and ECFP6 specifically, alongside topological torsion, rank among the best of 28 fingerprints tested for ordering diverse structures by similarity [129]. However, it's not a one-size-fits-all solution. Across 96,456 human metabolites, 72.9% had an ECFP4 nearest neighbor at Tanimoto distance 0 [128], again running into the issue of the representation lacking stereochemical information and more generally being unable to distinguish molecules.

The Tanimoto coefficient is written out in Chapter 2 as the ratio inside the Tanimoto kernel of Equation 2.1, specifically the number of bits set in both molecules divided by the number set in either. The corresponding distance is $1 - T$ [71]. Bits that are always off or always on across a dataset are counted in Section 4.2.1.

All a similarity coefficient can do is compare every feature, yet it cannot tell which features matter to the prediction task at hand. And the similarity assumption holds only relative to such a measure, which is fixed by the choice of representation. To estimate what this costs, we categorize a representation's features into those relevant to the task, R, and those that are irrelevant, I. We take the squared Euclidean distance as a sum over both:

$$d^2(\mathbf{x}, \mathbf{y}) = \sum_{j \in R} (x_j - y_j)^2 + \sum_{j \in I} (x_j - y_j)^2. \tag{4.3}$$

Every irrelevant feature that varies adds one further term, so the irrelevant part grows linearly with $|I|$. After standardization each feature contributes roughly the same amount, so the share of the distance carrying signal is about $|R|/(|R| + |I|)$. When we use a general-purpose representation like ECFP4 on a dataset spanning a small portion of the chemiccal space, $|I|$ can be a lot larger than $|R|$. Features that remain constant across the dataset contribute nothing. Rather, the damage comes from features that vary but bear no relation to property in question. The worst case scenario involves a single rare bit set in only a handful of molecules. After standarization, uncommon bits take on larger values, outweighing common features that may be more relevant. This is why the near-constant count in Section 4.2.1 matters more than the constant count.

Larger representations, or higher dimensionality, compounds the problem. As the number of dimensions grow, the nearest and farthest molecules end up at nearly the same distance, meaning that these pairwise distances are less able to discriminate, as explained in Chapter 2. We can measure this effect by measuring the

relative contrast, defined as the difference between the largest and smallest measured distance divided by the smallest, and hubness, a measure of how unevenly the molecules appear in other molecule's nearest neighbors [130]. These errors propogate along larger data pipelines, starting from the representation into similarity, from similarity into thes set of neighbors a molecule is compared against, and from there into a prediction of some kind.

We count activity cliffs with the rule of Chapter 2. A pair is similar at ECFP4 Tanimoto of at least 0.55. The activity gap is set per dataset so that 15% of molecules are cliffs. The share moves in steps as the gap changes, so LogD reaches 14.5%. The number of cliff molecules is therefore close to fixed by the rule, and the gap is what varies (Table 4.4). On hERG K_i, 211 of 1,412 molecules are cliffs at a gap of 1.76 pK_i. On QM9, only 1,093 pairs among 9,978 molecules reach a Tanimoto of 0.55. The gap that flags 15% of QM9 molecules is then 0.05 eV, 0.06 times the median absolute deviation of the labels. Three datasets come from the Biogen set: rat liver microsome (RLM) clearance, MDR1–MDCK efflux and human liver microsome (HLM) clearance. On these the gap is 0.20 to 0.34 log units, 0.50 to 0.60 times the median absolute deviation. On QM9 and these three, the gap is smaller than the median absolute deviation of the labels. A gap of twice the median absolute deviation, the rule our code used before, flags 31 of 9,978 QM9 molecules and 3,529 of 5,028 LogD molecules.

The count also depends on the representation. We kept each dataset's gap and replaced ECFP4 with each other representation in the pool. For each, the similar pairs were its own closest pairs, as many as ECFP4 calls similar. On hERG K_i the number of cliff molecules then ranges from 138 (atom pairs) to 267 (WL graph kernel). Only 57% to 74% of the pairs ECFP4 calls similar are among another representation's closest pairs. On QM9 that share falls to 1% to 24%. Every count is in Table S.11.

Two different questions can be asked of a distance. The first is about all pairs: does distance track the difference in property across the whole dataset? Plotting descriptor distance against activity difference for every pair is the neighborhood-behavior plot of Patterson et al. [131]. We summarize that plot with one number, the Spearman correlation between pair distance and absolute label difference. The second question is about nearest neighbors only: can each molecule's label be predicted from its five nearest neighbors? The prediction is the mean label of the five. The neighbors can carry signal even when the all-pairs correlation is near zero. In

what follows, each claim says which of the two it is about.

We computed both for every representation in the pool of each of the seven regression datasets. Six are the small-molecule and QM9 datasets of Table 5.16. The seventh is HLM clearance, which is not in that table. Representations with only 0/1 features were compared by Tanimoto distance, $1 - T$. All others were compared by Manhattan distance on standardized features, leaving out features that are constant across the dataset. Manhattan distance is defined in the next subsection. For the all-pairs correlation, each dataset over 2,000 molecules was cut to a fixed random sample of 2,000 molecules (seed 0). That gives 1,999,000 pairs per dataset. hERG K_i used all 996,166 pairs of its 1,412 molecules. For the neighbor prediction, all molecules were used, and each molecule's five neighbors came from other scaffold groups. No molecule is therefore predicted from its own scaffold.

Across the 70 combinations of dataset and representation, the all-pairs correlation ran from −0.04 (MACCS keys on MDR1–MDCK efflux) to 0.23 (PDV on the same dataset). It was below 0.10 in 44 of the 70. The five-neighbor prediction reached a Spearman correlation between 0.29 (Avalon on RLM clearance) and 0.89 (ChemBERTa-2 on QM9). In all 70 it was the larger of the two numbers. On hERG K_i, ECFP4 gives an all-pairs correlation of 0.06. Its five-neighbor prediction reaches 0.72, with an RMSE of 0.63 pK_i against a label standard deviation of 0.92 pK_i. Table 4.4 gives both numbers for three representations per dataset.

Figure 4.3 shows why the two numbers differ on hERG K_i. Of the 996,166 pairs, 93% have an ECFP4 Tanimoto below 0.20, and 1.1% reach 0.55. Across all pairs the median difference is 0.81 pK_i. Above a Tanimoto of about 0.4, the median falls to between 0.20 and 0.30 pK_i. Yet among the 774 pairs at 0.8 or above, 8.3% still differ by more than 1 pK_i. Under Mordred, the closest 5% of pairs have a median difference of 0.44 pK_i. Every other 5% of pairs has a median between 0.78 and 1.11 pK_i. The signal sits in a thin tail of close pairs. The all-pairs correlation is set mostly by the other 95% of pairs. A five-nearest-neighbor prediction looks only at each molecule's closest few.

A third view asks how often two representations agree on each molecule's five nearest neighbors. We compared ECFP4, one descriptor set and SMI-TED, a learned embedding that is in every pool. The descriptor set is Mordred, or PDV on Caco-2, where Mordred is not in the pool. HLM clearance's pool does not include ECFP4, so its ECFP4 was read from the same cache for this comparison only. The neighbor sets are the ones used for prediction. On hERG K_i any two of the three share 0.47 to

0.52 of their five neighbors per molecule, on average. On the three Biogen datasets and QM9 they share 0.13 to 0.21, under one neighbor in five (Table 4.5).

A fourth view asks whether fingerprint distance agrees with a structural distance. We sampled 500 random pairs per dataset (seed 0) and found their maximum common substructure with RDKit [32]. Ring atoms matched only ring atoms, only complete rings were kept, and each pair had a 2-second limit. The distance is $1 - n/(a + b - n)$, where n is the number of atoms in the common substructure and a and b are the two molecules' heavy atoms. Pairs that hit the time limit were left out: 5 on hERG K_i, 8 on Caco-2 and 6 on LogD. The Spearman correlation between ECFP4 Tanimoto distance and this distance ran from 0.20 (hERG K_i and HLM clearance) to 0.51 (Caco-2 and LogD).

A similarity can also be learned. Coupry and Pogány [132] learn a molecular similarity from graph embeddings, but separately from any endpoint. It is still not chosen for the property being predicted. Earlier work did choose the similarity for the task. Zheng and Tropsha [133] predict each molecule's activity as the average of its k most similar molecules, using Euclidean distance over descriptors. They select the subset of descriptors by simulated annealing to maximize the leave-one-out q^2. Itskowitz and Tropsha [134] extend this to optimize the descriptor subset, the number of neighbors and the neighbor weighting function together. Both select features for a nearest-neighbor model of one endpoint. The method in this thesis selects features across several representations. Every method in Section 4.4 takes a similarity as input and inherits the dilution described here. "Similarity in a hybrid" (Chapter 6) tests whether a feature set chosen for the task gives a better similarity.

One row of Table 4.4 is one dataset. The all-pairs columns give the Spearman correlation between pair distance and absolute label difference. The five-neighbor columns give the Spearman correlation between the five-neighbor prediction and the label.

One row of Table 4.5 is one dataset. The overlap columns give the fraction of a molecule's five nearest neighbors that two representations share, averaged over all molecules.

Distance metrics for continuous molecular descriptors

We now consider 8 commonly used distance metrics and how their formulation impacts down-the-line applications to distance-based noise detection. While binary

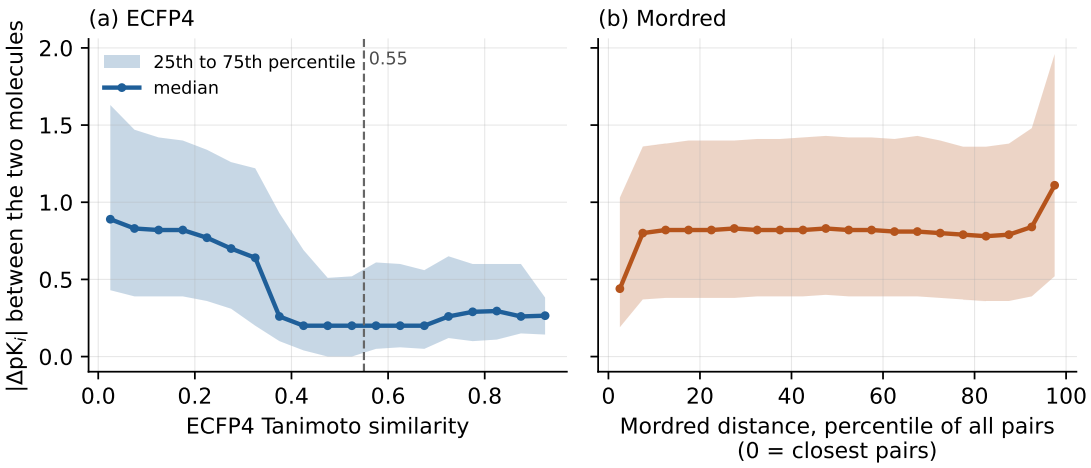


Figure 4.3: Absolute pK_i difference against distance for all 996,166 pairs of the 1,412 hERG K_i molecules. The line is the median in each bin and the band runs from the 25th to the 75th percentile. (a) ECFP4 Tanimoto similarity, in bins of 0.05; bins with fewer than 20 pairs are not drawn. The dashed line marks 0.55, the cliff rule's similarity. (b) Manhattan distance on standardized Mordred features, as a percentile of all pairs, in bins of 5% of pairs. The left end holds the closest pairs, so the two panels run in opposite directions.

fingerprints are compared with the Tanimoto coefficient, as described in Chapter 2, continuous features need a different measure. The performance of these 8 metrics in noise detection algorithms is compared in Section 4.4.2.

- Euclidean (L_2): standard geometric distance, $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$
- Manhattan (L_1): sum of absolute differences, $d(x, y) = \sum_i |x_i - y_i|$, more robust to outlier features
- Chebyshev (L_∞): maximum absolute difference, $d(x, y) = \max_i |x_i - y_i|$
- Cosine: angle-based, $d(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$, invariant to magnitude
- Correlation: $d(x, y) = 1 - \rho(x, y)$, where ρ is the Pearson correlation taken across the features of one pair of molecules, not across molecules
- Canberra: weighted L_1, $d(x, y) = \sum_i \frac{|x_i - y_i|}{|x_i| + |y_i|}$
- Bray–Curtis: ecological dissimilarity, $d(x, y) = \frac{\sum_i |x_i - y_i|}{\sum_i |x_i + y_i|}$ [135]
- Mahalanobis: covariance-adjusted distance, $d(x, y) = \sqrt{(x - y)^\top S^{-1} (x - y)}$, where S is the feature covariance estimated on the training molecules [136]

Table 4.4: All-pairs correlation and five-nearest-neighbor prediction for three representations, and activity cliffs, per dataset. Desc. is Mordred, except on Caco-2 efflux where it is PDV. Datasets over 2,000 molecules use a fixed sample of 2,000 molecules for the all-pairs columns. Neighbors come from other scaffold groups. Cliffs: ECFP4 Tanimoto ≥ 0.55 and a label difference above the gap, which is set so that 15% of molecules are cliffs. †ECFP4 is not in this dataset's pool. Every representation is in Table S.11.

		All pairs			Five neighbors			Cliffs	
Dataset	Molecules	ECFP4	Desc.	SMI-TED	ECFP4	Desc.	SMI-TED	Molecules	Gap
hERG K_i	1,412	0.06	0.07	0.09	0.72	0.72	0.69	211	1.76 pK
LogD	5,028	0.12	0.06	0.10	0.76	0.80	0.76	731	2.90 log
Caco-2 efflux	2,158	0.08	0.18	0.14	0.59	0.62	0.57	324	1.21 log
RLM clearance	3,053	0.02	0.07	0.04	0.36	0.49	0.46	458	0.34 log
MDR1–MDCK efflux	2,641	0.02	0.16	0.10	0.51	0.58	0.59	397	0.20 log
HLM clearance	3,086	$-0.00^{\dagger}$	0.07	0.03	$0.37^{\dagger}$	0.49	0.45	462	0.27 log
QM9	9,978	0.19	0.18	0.13	0.88	0.84	0.85	1,495	0.05 eV

All features are standardized to zero mean and unit variance prior to distance computation to ensure equal contribution across features with different relative scales. Bray–Curtis takes non-negative input, so for it the standardized matrix is shifted once to make its minimum zero. Canberra is computed on the standardized values directly.

Chapters 5 and 6 use four of these: Manhattan by default, Euclidean, cosine, and Minkowski with $p = 0.5$, $d(x, y) = (\sum_i |x_i - y_i|^{0.5})^2$, which breaks the triangle inequality and is therefore a dissimilarity rather than a metric. Feature weights from the hybrid are applied after standardization, as defined in Chapter 6.

4.4 Consequences for methods that rely on similarity

We have identified flaws in the representations themselves and in the similarities computed over them. For a given molecule, its set of neighbors, built from one representation and one distance, is the input to several other processes such as predicting a molecule's activity from its nearest neighbors, detecting noisy labels by comparing a label against its neighbors' labels, defining an applicability domain by distance to the training set [137], grouping molecules into clusters for data splits [138], identifying activity cliffs, and, in federated learning, weighting each site's vote on a molecule by that molecule's distance to the site's own data [139]. If the distance

Table 4.5: Agreement between representations, per dataset. Overlap: mean fraction of shared five-nearest-neighbor molecules, neighbors from other scaffold groups. MCS: Spearman correlation between ECFP4 Tanimoto distance and maximum-common-substructure distance on 500 random pairs (seed 0). Pairs that hit the 2-second limit are left out; the last two columns count the pairs used and left out.

		Five-neighbor overlap			ECFP4 against		
Dataset	Desc.	ECFP4–Desc.	ECFP4–SMI-TED	Desc.–SMI-TED	Spearman	Pairs	T
hERG K_i	Mordred	0.52	0.50	0.47	0.20	495	
LogD	Mordred	0.38	0.41	0.41	0.51	494	
Caco-2 efflux	PDV	0.38	0.41	0.38	0.51	492	
RLM clearance	Mordred	0.15	0.21	0.18	0.28	500	
MDR1–MDCK efflux	Mordred	0.15	0.20	0.17	0.26	500	
HLM clearance	Mordred	0.15	0.21	0.18	0.20	500	
QM9	Mordred	0.13	0.18	0.21	0.46	500	

is flawed, then every application of distance will suffer the consequences. When I was exploring LNL methods, I noticed that the ones that relied on the concept of molecular distance were unsuccessful, prompting me to look into the distance itself, as desribed in Section 4.4.1.

Section 4.4.1 reports the exploratory LNL study, in which every configuration that used a distance ranked among the worst. Section 4.4.2 turns to noise detection by neighbors, the most direct use of similarity in this thesis.

4.4.1 The exploratory learning-with-noisy-labels study

This section reuses an exploratory study of learning with noisy labels (LNL) carried out earlier in this project. In Chapter 3, the choice of representation explained more of the variance in clean performance than the choice of model. The relationship flipped for performance lost under noise: there, the model had the stronger effect. We therefore asked whether training methods built for noisy labels improve noise robustness, and whether the representation still matters under them.

LNL methods, and the five groups of Song et al. [100], are described in Section 2.5.2. They have been studied across machine learning, including QSAR [17]. Cortes-Ciriano et al. [17] examined 12 algorithms on 12 bioactivity datasets. They added Gaussian noise ranging from zero up to each dataset's range of pIC50 values. Gradient boosting tolerated noise worst. SVM and GP models with polynomial kernels were sensitive at high noise in some cases. The other algorithms degraded smoothly and linearly with the noise level. Most LNL methods were built for im-

age classification with flipped labels [100]. The labels here are continuous, so these methods do not transfer automatically. Cheminformatics data adds further challenges: high-dimensional molecular representations and a wide variety of chemical domains.

The exploratory study tested 12 LNL methods on the HOMO-LUMO gap from QM9, using 10,000 molecules and scaffold splits. Gaussian noise with standard deviation σ from 0 to 1.0, in steps of 0.1, was added to the training labels only. Test labels stayed clean. The methods were paired with five representations: ECFP4 (2,048 bits), Sort & Slice [125], PDV, canonical SMILES and randomized SMILES. Depending on the method, it could take up to four optional add-ons. These were the model's own uncertainty, a distance between molecules, an alternative GP kernel, and an alternative loss function. The distances were Tanimoto on ECFP4, Euclidean on PDV, and Mahalanobis. The GP kernels were Tanimoto, RBF and Matérn. In total we tested 221 configurations over method, loss function, distance and representation. The report grouped the 12 methods into five categories of its own: meta-learning, sample selection, curriculum learning, iterative refinement and GP extensions. These are not the five groups of Song et al. [100]. Four of the 12 remove suspected molecules and then retrain: small-loss selection, confident learning, distance-based selection and multistage cleaning. These four are noise mitigation. The other eight are noise-robust training.

We measured noise robustness by retention: R^2 under noise divided by R^2 with clean labels, as a percentage. Methods that used a distance did worse than those that did not. All 20 configurations with the lowest retention used a distance. None of the configurations with a distance reached both a clean R^2 above 0.6 and a retention above 80%. The failure sits in the distance step. The same methods without the distance add-on did not fail this way. Not all use of similarity failed, however. The heteroscedastic GP compares molecules through its kernel and still held up. The failure is specific to judging one label by its neighbors' labels.

The methods that held up estimate noise from the model's own loss or uncertainty. DivideMix [140] kept the most, with 86.7% retention (standard deviation 0.4%). Small-loss selection [141] kept 84.2% (standard deviation 2.1%). The uncertainty curriculum [142] kept 82.7% (standard deviation 3.5%). The heteroscedastic GP [143] kept 79.0%, averaged over 15 configurations (3 kernels and 5 representations). Its R^2 was 0.71 with clean labels and 0.56 at $\sigma = 0.6$, the noise level its retention was measured at. The report does not state the noise level for the other

methods' retention. Regularization alone kept less: 73.0% for Mixup and 74.0% for sharpness-aware minimization. None of the methods that held up compares a label with its neighbors' labels. Under these methods, the choice of representation explained even less variance than in Chapter 3. Chapter 6 compares twelve LNL methods on the current datasets.

The study used one dataset, one property and one noise type: Gaussian noise on every training label. Chapter 6 repeats the question on more datasets and noise types. The distance step was also added to methods designed without one. A poor adaptation is therefore an alternative explanation. The recommendation of noise-robust training over noise mitigation (Chapter 7, Limitations) starts here.

There are two possible reasons for the poor performance of the distances. First, a single molecular representation may lack sufficient information for distance calculations, with and possibly even without noise. PDVs encode global molecular properties, ECFPs contain substructure information, and MHG-GNNs encode the molecular graph [8]. These single sources of data may not be sufficient for robust similarity estimation. Second, some of these representations are sparse. They contain many features that are not relevant to the molecules of a given dataset. When a large proportion of features is uninformative, similarity measures become dominated by them. Each uninformative feature that varies adds one more term to the distance, so their share of it grows linearly with their number. The evidence here does not separate the two reasons. The second is the one we sought to address, with a representation of informative features and low dimensionality. The budget-constrained hybrid representation concatenates the top-ranked features by importance from a selection of base representations.

4.4.2 Noise detection by neighbors

Similar molecules should have similar labels. A label far from its neighbors' labels is therefore treated as probably wrong. The idea goes back to editing a nearest-neighbor training set by removing points whose label disagrees with their neighbors [103]. The wider literature on filtering noisy labels is about classification [144, 145]. Martín et al. [146] adapt fourteen classification noise filters to regression. They inject noise into the labels of 20 datasets and find that the filters improve the regression methods. Seven of the fourteen are nearest-neighbor filters: ENN, AENN, GE, BBNR, CNDC, CNN and RNN. For regression, each replaces the neighbors'

majority class with the mean of the neighbors' labels [146]. The other seven build ensembles of models and remove a sample by vote. Four of these (DF, EF, FMF and HRRF) include regression k-NN among their models [146]. Their 20 datasets are general regression benchmarks, not QSAR. This is the most direct use of the similarity assumption in this thesis. The scores, removal, weighting, smoothing and the recovery rate are defined in Chapter 6, "Methods for learning with noisy labels".

The method relies on two things at once. The distance must pick the right neighbors, and those neighbors' labels must be clean. The earlier sections of this chapter argue that the first fails. The second fails whenever noise is spread over the whole training set. A large gap between a label and its neighbors' labels can mean a noisy label, a noisy neighbor or a wrong set of neighbors. The score alone cannot tell which. Activity cliffs also look like noise to such a detector (Chapter 2, "Molecular similarity and activity cliffs").

The exploratory study also measured what happens when neighbors are used to clean labels. It used 5,000 QM9 molecules chosen at random, predicting the HOMO-LUMO gap, with a scaffold split. Gaussian noise was added to the training labels at $\sigma = 0.3$ and $\sigma = 0.6$. Each cleaning method was judged by retraining one random forest on ECFP4, on clean, noisy and cleaned labels. From these three fits we computed the recovery rate in R^2 (Chapter 6, "Recovery rate"). Neighbor-based scores compared each molecule's label with the labels of its k nearest training molecules. Euclidean, Manhattan and cosine distances were tried. The table below holds only the rows built on a single representation. Rows built on a hybrid used an allocation method that is no longer part of the thesis. This is a separate study from the one in "Sample removal methods" (Chapter 6). That one removed molecules, and used 10,000 QM9 molecules and eight distances. This one weighted and smoothed labels, and used 5,000 molecules and three distances.

In Table 4.6, one row is one representation used to find neighbors. The neighbor-based scores were used as sample weights at strength $\alpha = 0.1$. With these weights, mean recovery was negative for every representation at both noise levels. Across all 860 cleaning configurations, 8 had positive recovery at both noise levels. Six of the 8 used a single representation, and 4 of those 6 were neighbor-based. Label smoothing toward neighbor or model estimates had positive recovery in 5.0% of configurations at $\sigma = 0.3$ and 0.8% at $\sigma = 0.6$. Mean recovery was negative at every weighting and smoothing strength tried, from 0.1 to 0.7.

Judging a label by its neighbors usually made the model worse, and rarely helped

Table 4.6: Recovery rate in R^2 when neighbor-based noise scores are used as sample weights ($\alpha = 0.1$), by the representation used to find neighbors. QM9 HOMO-LUMO gap, 5,000 molecules, random forest on ECFP4 as the downstream model. "Positive" is the share of configurations with recovery above zero.

	Mean recovery (%)		Positive (% of configurations)	
Representation	$\sigma = 0.3$	$\sigma = 0.6$	$\sigma = 0.3$	$\sigma = 0.6$
ECFP4	−3.0	−7.3	33.3	16.7
PDV	−2.4	−3.4	50.0	27.8
MHG-GNN	−0.1	−7.2	38.9	5.6

at both noise levels. It did not always fail. The best single configuration at $\sigma = 0.6$ was neighbor-based. It used PDV, Euclidean distance and the local variance ratio score, and recovered +17.2%. Noise was added to every training label, however. A molecule's neighbors are then noisy too, so the neighbor mean is itself a noisy target. The report gives this as the reason label smoothing failed.

Overall, in agreement with Section 4.4.1, using a sample's neighbors to determine its ground truth is flawed here. It is likely that all labels are too uniformly noisy for a method like this. Uncertainty-based methods, which use no neighbors, show potential at lower noise, but their estimation degrades with higher noise. Sample weighting did better than smoothing but still usually hurt. It gave positive recovery in 14.2% of configurations at $\sigma = 0.3$ and 4.7% at $\sigma = 0.6$. Its mean recovery was −22.9% and −20.9%. The configuration above, with exponential sample weighting, recovered +11.4% at $\sigma = 0.3$ and +17.2% at $\sigma = 0.6$.

The hybrid in that study was PDV plus MHG-GNN at 100 features, built by greedy forward selection. Greedy allocation is no longer part of the method. The current hybrid is built as described in Chapter 5, and its search sweeps budgets from 10 to 2,000 features. For neighbor-based scores, the old hybrid's mean recovery was −0.9% at $\sigma = 0.3$ and −3.0% at $\sigma = 0.6$. Its share of positive configurations was 44.4% and 27.8%. Distance-based methods remain challenging under Gaussian noise regardless of representation. This suggests the core issue is the noise distribution rather than representation quality alone. Chapter 6 repeats the measurement with neighbors found in a hybrid, and under noise types other than Gaussian.

4.5 Conclusions: what a representation would need

This chapter asked three questions about the representations we use to compare molecules. Each was answered with a small measurement, not a new method.

The first question was how much of a standard representation varies on one dataset. Often very little does. On hERG K_i, 1,815 of ECFP4's 2,048 features take one value in at least 95% of the 1,412 molecules (Section 4.2.1). Several representations also give different molecules the same feature vector. ECFP4 gives 56 of the 1,412 hERG K_i molecules a vector shared with another molecule, all of them stereoisomers ("Molecules the representation cannot tell apart"). No model can separate those molecules, whatever their labels.

The second question was whether distance follows the property. A distance sums over every feature. When only a few features matter for the property, the rest dominate the sum (Section 4.3). Across 70 combinations of dataset and representation, the Spearman correlation between the distance of a pair and its label difference was below 0.10 in 44. We also predicted each molecule's label as the mean of its five nearest neighbors' labels. That prediction's Spearman correlation with the label was higher in all 70. The useful information sits in a few close pairs, not in the distance as a whole.

The third question was what happens to methods that rely on distance. In the exploratory LNL study, the configurations that used a distance between molecules did worst under label noise (Section 4.4.1). Judging a label by its neighbors' labels usually made the model worse, and rarely helped at both noise levels (Section 4.4.2). It did not always fail: one neighbor-based configuration gave the best recovery at the higher noise level. The noise was Gaussian and added to every training label, so each molecule's neighbors were noisy too. Chapter 6 returns to other noise types.

There are two possible reasons the distance failed. One representation may carry too little information to judge similarity. Or a sparse representation may carry many features irrelevant to the dataset's molecules. Our evidence does not separate the two. The requirements below address the second.

A representation that avoids these flaws would need five things. Its features would be chosen for the task, using only the training labels of each fold. It would be compact: features that do not vary or do not matter are removed before any distance is taken or model is fit. The number of features kept would be searched, not fixed. It would draw from several representations, since each misses things the

others capture. Plain concatenation meets this requirement but not the previous one (Chapter 5, "Combining representations and the alternatives"). It would work both as a model input and as a space in which to compute similarity. And it would be traceable: each chosen feature leads back to its representation, and can be read chemically where the base feature can.

Feature selection cannot fix everything. Molecules that are identical in every base representation stay identical in the hybrid. A selected hashed bit still merges every substructure that folds into it. Selection also needs enough labeled molecules to estimate which features matter.

This is where the thesis began. Our hypothesis was that similarity as usually computed, ECFP4 with Tanimoto and its variants, carries many features irrelevant to the task. Methods built on it, such as noise detection and noise mitigation, suffer as a result. The first version of our method selected the task-relevant features of several representations, with the number of features set by the user. By a task we mean a supervised model fit on a training set. That version was built only to compute similarity. The selected feature sets then turned out to work as ordinary model inputs, without any similarity step. The selection itself became the method: the adaptive hybrid representation.

That leaves several questions for Chapter 5. How should a feature be scored (the importance method)? How should the budget be split across representations (the allocation method)? How should a candidate hybrid be scored (the internal evaluator)? How many features should be kept (the budget), and which downstream model should they feed? Features are chosen per representation, not from one pooled ranking, because some datasets prefer features from only the top few representations. Chapter 5 makes that argument.

Table 4.7 lists each flaw found in this chapter, what a representation would need to avoid it, and where the thesis handles it. One row is one flaw.

Table 4.7: Flaws shown in this chapter, the requirement each sets, and where it is handled. The first column names sections of this chapter. The last names sections of Chapter 5 unless another chapter is given.

Flaw (where shown)	Requirement	Where handled
Features that do not vary ("Most features do not vary on a given dataset")	Features chosen for the task, within a budget	"Feature importance scoring"
Molecules merged into one vector ("Molecules the representation cannot tell apart")	Features drawn from several representations	"Combining representations and the alternatives"
Distance dominated by irrelevant features ("Flaws in similarity and other pairwise computations")	Compact, with the number of features searched	"The search space and why each axis is swept"
Neighbor-based methods doing badly ("Consequences for methods that rely on similarity")	Usable as a space for similarity	"Similarity in a hybrid" (Chapter 6)

Chapter 5

Building Adaptive Hybrid Representations

This chapter asks how to join several representations without also joining their irrelevant features. Its answer is the adaptive hybrid representation: up to a budget of features, selected from several base representations by a search for one dataset and one downstream model. The hybrid is an input, not a model. The search covers the importance method, the allocation method, the number of representations, the internal evaluator, the budget and the downstream model. Every selected feature traces back to its representation, so each finished search gives a blueprint that a chemist can read. The method needs only a supervised label and several vector representations of the same samples, and this chapter tests it on small molecules, antibodies, reactions, nucleotide sequences and tumor data. **[RESULT NEEDED: on how many datasets the hybrid beats the best single base representation, and whether the preferred search settings differ between datasets and models]**.

5.1 Introduction

Chapter 4 showed that standard representations carry many features that do not vary, or do not matter, on a given dataset. A distance computed over all of those features inherits the problem. That chapter ended by asking for features chosen for the task from several representations, with the irrelevant ones removed first. Chapter 3 gave the practical reason to search rather than fix one choice: the best representation changed with the dataset and with the model. The selection described

here was first built to compute similarity, and was then found to work as a model input.

This chapter builds the adaptive hybrid representation, the hybrid for short, and reports first results on when and why it helps. A hybrid is a set of at most a fixed number of features, selected from several base representations of the same molecules for one dataset and one downstream model.

This chapter measures prediction only. Distances computed in a hybrid, noise detection and learning with noisy labels are the subject of Chapter 6. The methods described here are final. Each dataset is split into 5 outer folds. The hybrid beats the best single base representation on **[RESULT NEEDED: number of the 16 datasets on which the hybrid beats the best single base representation]** of the 16 datasets. **[RESULT NEEDED: whether the preferred search settings vary between datasets, and which settings]**.

The rest of this introduction reviews ways of combining representations and the alternatives to combining them, then describes the hybrid, the blueprints it produces, and its use beyond small molecules. The first methods section describes how a hybrid is built, which settings the search covers and why, and augmentation. It also gives two ways to make the search cheaper, a fast tier and adaptive pruning. The second methods section gives the datasets and splitting, the representations used in each domain, and the models and metrics. It then gives the comparators, controls and ablations, and the transfer, antibody-scan and separate-site experiments. The results report when the hybrid helps, why it helps, what the blueprints show, the transfer and antibody scans, and data held at separate sites. The chapter ends with the Discussion, Limitations and Conclusions.

5.1.1 Combining representations and the alternatives

Prior research in this field shows promise, with conjoint fingerprints achieving stronger performance on binding affinity prediction than alone across a range of models [23]. Augmenting structural-based fingerprints like ECFP4 with bio-activity based fingerprints has also been successful at improving both predictive performance and scaffold-hopping [147]. A very similar fusion strategy, in which features are selected from six non-hashed fingerprints and are combined by feature importance score within individual models, consistently improves predictive accuracy over the original fingerprints [148].

The majority of the work in this space deals with combining relatively standard fingerprints and linear representations like SMILES [23, 147]. However, graph-based representations capture important structural information that is often lost in fingerprints. We explore using GNN embeddings within these hybrid, fusion methodologies. We further expand on the mechanisms used to compute feature importance, concatenate features, and use these hybrids within similarity quantification and noise mitigation tasks. We also explore adding on smaller sets of structural features to a single base representation, referred to as augmentation.

The majority of the work in this space deals with combining relatively standard fingerprints and linear representations like SMILES [23, 147]. However, graph-based representations capture important structural information that is often lost in fingerprints. We explore using GNN embeddings within these hybrid, fusion methodologies. MHG-GNN and GROVER are base representations on every small-molecule property dataset except QM9, and GraphMVP is a base representation on QM9 and LogD (Table 5.11). We further expand on the mechanisms used to compute feature importance, select and join features, and use these hybrids within similarity quantification, noise detection and learning with noisy labels. We also explore adding on smaller sets of structural features to a single base representation, referred to as augmentation.

So far, the most common strategy for combining feature sets has been concatenation, which requires only a single downstream model [149]. It works best when feature sets are homogeneous. When they are not, concatenation degrades [149], and Deng et al. [106] called the practice "misleading" when the goal is to measure the contribution of individual representation types. Late fusion trains a separate model per source and aggregates predictions [150], avoiding the dimensionality problem. But cross-source feature interactions are lost. To recover cross-source interactions requires intermediate fusion (MOGONET [151], cross-attention [**?**]), but at the cost of end-to-end training on all sources; adding a new source means retraining from scratch. No interpretable feature subsets emerge from multiple kernel learning [152], which operates entirely at the kernel level. There remains, as Steyaert et al. noted, no decisive evidence that any of these strategies is broadly superior [150].

So far, the most common strategy for combining feature sets has been concatenation, which requires only a single downstream model [149]. It works best when feature sets are homogeneous. When they are not, concatenation degrades [149]. Concatenation also hides which representation earned the score. Deng et al.

found that appending RDKit2D descriptors to GROVER lowered RMSE on 9 of 24 datasets. They still called the practice "misleading" when the goal is to measure the contribution of individual representation types [106]. Late fusion trains a separate model per source and aggregates predictions [150], avoiding the dimensionality problem. But cross-source feature interactions are lost. To recover cross-source interactions requires intermediate fusion (MOGONET [151], cross-attention), but at the cost of end-to-end training on all sources; adding a new source means retraining from scratch. No interpretable feature subsets emerge from multiple kernel learning [152], which operates entirely at the kernel level. Its example in drug bioactivity is pairwiseMKL [153]. There remains, as Steyaert et al. noted, no decisive evidence that any of these strategies is broadly superior [150].

In practice, most research groups default to one representation and ignore the rest. ECFP is common in small molecules, while deep learning groups may opt for embeddings like MoLFormer or ChemBERTa. ESM-2 embeddings are common for antibodies, and can be broken up by chain. While there is some research that compares these representations, there is not much that focuses on combining them beyond simple concatenation. Concatenating just three standard sources, however (ECFP4 at 2,048 dimensions, ESM-2 at 1,280, Mordred descriptors at ∼1,600), gives a ∼5,000-dimensional input for a dataset that may contain only a few hundred labeled samples. At these sample-to-feature ratios, more features mean worse predictions [154, 155]. Models sensitive to distance, such as kNN and SVM, are affected most severely [156].

In practice, most research groups default to one representation and ignore the rest. ECFP is common in small molecules, while deep learning groups may opt for embeddings like MoLFormer or ChemBERTa. ESM-2 embeddings are common for antibodies, and can be broken up by chain. While there is some research that compares these representations, there is not much that focuses on combining them beyond simple concatenation. Concatenating just three standard sources, however, gives about 5,000 features: ECFP4 at 2,048, ESM-2 at 1,280 and Mordred descriptors at ∼1,600. A dataset may contain only a few hundred labeled samples. At these sample-to-feature ratios, more features mean worse predictions [154, 155]. Models that compute a distance over every feature, such as kNN and SVM, are the most exposed, for the reasons given in Chapter 4. A concatenation of two representations scored between its two parts, not above the better one, on **[RESULT NEEDED: count of datasets]**. We propose a reason. Concatenation adds the irrelevant fea-

tures of both parts along with the useful ones. A method that joins representations therefore has to combine and filter at once.

The pace of new model development shows no sign of slowing, and extends well beyond small molecules. First, these new models may prefer different feature sets, and that's what this methodology can provide. But beyond that, new models can be fit and converted into frozen embeddings and used as further input into this farmework.

The pace of new model development shows no sign of slowing, and extends well beyond small molecules. First, a new downstream model may prefer a different feature set, so it gets its own search. Beyond that, a new model can be converted into frozen embeddings, which enter this method as one more base representation with no change to the method.

A comparison of 62,820 models in molecular property prediction benchmarks showed that classical RDKit2D descriptors matched or outperformed learned features from graph neural networks and pretrained language models on the majority of datasets [106]. Descriptor-based models occupied 73% of top-three positions on 11 benchmarks [157], and in 1,310 ChEMBL assays, feedforward networks on ECFP fingerprints outperformed graph convolutions and SMILES-based recurrent models [158]. The input often matters more than the model. Yet we still lack a systematic way to exploit this. No single representation wins. The same pattern holds outside small-molecule chemistry: no single protein language model consistently achieves the highest performance on all tasks [159, 160]; for antibodies, general-purpose models sometimes outperform those built for the domain [159]. In computational pathology, no individual foundation model dominates on clinical endpoints [161], and in cancer genomics, integrating additional omics layers can negatively impact performance when the data types are not properly combined [150].

For small molecules alone, the landscape now includes extended-connectivity fingerprints, physicochemical descriptor sets exceeding 1,600 dimensions, graph neural network embeddings [12, 162], language models trained on billions of SMILES strings [40], image-based encodings (molecules rendered as 2D images) [163], and stereoelectronics-infused molecular graphs [164]. The same is true for proteins (ESM-2 [165], ProtTrans [166]), antibodies (AntiBERTy [167], AbLang [168]), histopathology [161], and multi-omics cancer profiles [150]. A recent review counted over 40 molecular foundation models published between 2020 and 2024 [169]. Increasingly, new models appear faster than we can compare them [170]. The choice is not getting

simpler. Different sources presumably encode distinct aspects of the same biology. Noisy or incomplete on their own, single data modalities somehow give more sensitive and robust predictions when combined with independent signals from other modalities [150]. An ensemble of frozen embeddings from the top four pathology foundation models, El Nahhas et al. found, achieved a mean AUROC 1.2% higher than the best individual model on 31 clinical tasks [161].

Representations now exist for small molecules, reactions, antibodies, nucleotide sequences and tumor profiles; those used in this thesis are listed in Tables 5.11, 5.12, 5.13, 5.14 and 5.15. A recent review counted over 40 molecular foundation models published between 2020 and 2024 [169]. Increasingly, new models appear faster than we can compare them [170]. The choice is not getting simpler. Different sources encode distinct aspects of the same biology. Noisy or incomplete on their own, single data modalities give more sensitive and robust predictions when combined with independent signals from other modalities [150].

Concatenation, the most common method for combining feature sets, gives the full pool that KIRBy starts from, but on biological sample sizes it can degrade rather than help [149, 155]. At the kernel level, multiple kernel learning [152] (e.g., pairwiseMKL for drug bioactivity [153]) can learn source weights, but no interpretable feature-level selections result. Consensus methods [171] combine predictions, not features, and so cannot exploit cases where one source's noise is another's signal. Instead, KIRBy works at the feature level: it selects features that span source boundaries. Perhaps the closest analogy in the optimization literature is the CASH problem [172] (the joint selection of algorithm and hyperparameters), extended here to include the input encoding as an additional search dimension. Tishby and Zaslavsky [173] showed why compression helps: the information bottleneck principle states that optimal representations maximize mutual information with the target while minimizing mutual information with the input. In effect, the budget forces the model to keep only what matters. The feature count controls the trade-off between informativeness and dimensionality.

The hybrid search starts from all base representations side by side, but works at the feature level. It keeps original features, drawn from several representations, and so selects features that span representation boundaries. Perhaps the closest analogy in the optimization literature is the CASH problem [172] (the joint selection of algorithm and hyperparameters). The hybrid search adds a third choice: which features, from which representations, go in.

Ensembling combines representations after prediction rather than before it. One model is fitted per representation, and their predictions are pooled. Consensus QSAR does this: it combines the predictions of several models, not their features [171]. So it cannot exploit cases where one representation's noise is another's signal. In computational pathology, an ensemble built on the four best foundation models reached a mean AUROC 1.2% above the best single model, over 31 clinical tasks [161]. In an ensemble, no single model sees features from two representations together. Every representation and every model must also be computed for each new molecule.

Feature selection keeps a subset of the original features. It builds no new ones, which separates it from PCA. Methods are usually grouped into three families [174], reviewed for high-dimensional biological data by Saeys et al. [175]. Filter methods score features against the labels before any model is fitted. For example, mRMR ranks features by their mutual information with the label, minus their redundancy with features already chosen [176]. Wrapper methods search over subsets of features, scoring each subset by fitting the model [177]. In QSAR, descriptors were selected this way with a k-nearest-neighbor model [133]. Embedded methods select while the model is fitted. For example, the lasso's L1 penalty sets some coefficients to exactly zero [178]. Recursive feature elimination fits a model, drops the lowest-ranked features, and repeats [179]. Two pitfalls are known. The selected set can change between resamples of the same data, and the Nogueira index measures this across folds [180]. Impurity importance in random forests favors some feature types over others [181].

The hybrid ranks features by the importance scores of a fitted model, as embedded methods do. It then keeps a fixed number of top-ranked features, as a filter does. Classical methods treat all features as one flat list and ignore which representation each feature came from. The nearest classical relative is selection over predefined groups. The group lasso keeps or drops whole groups of variables together [182]. The hybrid does not keep or drop whole representations. Four of the five allocation methods in its search give each representation a share of the budget, then take that many top-ranked features from it. The fifth, the global pool, gives each representation a share of a candidate pool twice the size of the budget. It then ranks the pooled candidates together and keeps the top ones, up to the budget. Because only original features are kept, every feature in the hybrid traces back to one representation. This is what makes blueprints possible.

Dimensionality reduction replaces the original features with fewer new features, each a combination of the originals. In QSAR it turns many correlated descriptors into fewer uncorrelated ones [183]. PCA is the standard method. Each component is a linear combination of the original features, and components are ordered by the variance they explain [184]. Components are usually functions of all the original features, which makes them hard to interpret [184]. PCA also depends on the units of the features, and z-scoring each feature first removes this [184]. This matters when fingerprint bits, descriptors and embeddings are joined. PCA never sees the labels. It keeps the directions of largest variance, which need not predict the property, and components with small variance can still predict it [185]. Partial least squares is the supervised counterpart in chemometrics: its components are built to covary with the property [186]. Both methods give mixed components. A component cannot be traced to one substructure, descriptor or representation, so no blueprint can be read from it. Nonlinear methods include kernel PCA [183], t-SNE [187] and UMAP [188]. t-SNE and UMAP are used mainly for two- or three-dimensional maps, not as model inputs.

Every alternative above except ensembling also reduces the number of features. Models may want different numbers of features. Distance-based models such as kNN and SVM use every feature in every distance. Tree ensembles choose one feature at each split, so they can partly ignore irrelevant features. Models may also want different features, not only different numbers of them. Selection that uses the labels may overfit. Sort & Slice, which uses no labels, beat two label-based schemes for selecting substructure features [6]. Dablander suggests that using the labels may add a risk of overfitting [125, p. 138]. Table 5.1 sets the alternatives side by side, one row per method.

5.1.2 The adaptive hybrid representation

Here, we present KIRBy (Knowledge Integration by Representation), a framework that takes a collection of feature matrices for a given data set and outputs a customized feature set optimized for a specific prediction task. The inputs can be any type of frozen feature vector, such as fingerprints, descriptor sets, foundation model embeddings, gene expression signatures, histopathology features. Within a given domain, these individual vectors typically contain its own type of information. It's a well-established phenomenon when it comes to machine learning, there's rarely

Table 5.1: Ways of combining or shrinking representations. One row is one method. "Several representations" asks whether one model sees features from more than one representation. "Retrain to add one" asks whether adding a representation means refitting everything already fitted.

Method	Uses the labels	Keeps original features	Several representations	Retrain to add one
Concatenation [149]	no	yes	yes	yes
Late fusion and ensembling [150, 171]	only if pooling weights are learned	yes	no	no, one more model
Intermediate fusion [151]	yes	no	yes	yes
Multiple kernel learning [152, 153]	yes	no, kernels only	yes	yes
PCA and PLS [184, 186]	PCA no, PLS yes	no	yes, on a concatenation	yes
Flat feature selection [174]	yes	yes	yes, on a concatenation	yes
Group lasso [182]	yes	yes, whole groups	yes	yes
Adaptive hybrid representation	yes	yes	yes	search rerun, method unchanged

ever a one-size-fits-all approach. Take the noise robustness study above as evidence, NGBoost's predictive performance on QM9 was only ever strong with given PDV input features. However, on Caco-2, NGBoost preferred Avalon, with PDV as a close second. The model NGBoost itself produces better fits when given physicochemical properties, yet this preference can be overidden by a different prediction task. Avalon likely captures features that are pertinent to the log efflux ratio.

Here, we present the adaptive hybrid representation, or "hybrid" for short. It takes a collection of feature matrices for a given dataset and outputs a customized feature set optimized for a specific prediction task. The inputs can be any type of frozen feature vector, such as fingerprints, descriptor sets, foundation model embeddings, gene expression signatures, histopathology features. Within a given domain, these individual vectors typically contain their own type of information. It's a well-established phenomenon when it comes to machine learning, there's rarely ever a one-size-fits-all approach. Take the noise robustness study in Chapter 3 as evidence, with NGBoost on clean labels. On QM9, NGBoost's held-out R^2 was highest with PDV input features (0.865, median of 10 replicates). MHG-GNN was only 0.013 lower (0.852), and ECFP4 was lowest (0.706). On LogD, PDV was again highest

(0.673), with MHG-GNN second (0.658). On hERG K_i, MHG-GNN was highest (0.511). PDV was second (0.456), the median of the 4 of 5 folds whose clean R^2 was at least 0.3. On Caco-2, NGBoost appeared to prefer Avalon (0.402), with ChemBERTa at 0.404. Both numbers are medians of only 2 of 5 folds; the other 3 fell below a clean R^2 of 0.3 and were excluded. Over all 5 folds, PDV is highest (0.370), ahead of Avalon (0.287) and ChemBERTa (0.256). The model NGBoost itself produces better fits when given physicochemical properties, yet this preference can be overridden by a different prediction task. Accuracy and robustness also pick different representations. On QM9 under Gaussian noise, NGBoost's highest noise robustness (AUC_{norm}, the normalized area under the R^2 against noise level curve) was on Avalon, 0.982 against 0.975 for PDV.

But as important as the selection of useful features is, it is also important to filter out the useless features. We could throw the kitchen sink at the models and pool together features across a wide array of representations. Not only would that pose computational challenges, but it also introduces a huge set of irrelevant features that function as noise in downstream applications (Chapter 4). With all of that in mind, I decided to develop a method that selects the best features for a given data set and model. In theory, it should effectively filter out noisy features and isolate the feature which carry the most relevant information.

But as important as the selection of useful features is, it is also important to filter out the useless features. We could throw the kitchen sink at the models and pool together features across a wide array of representations. Not only would that pose computational challenges, but it also introduces a huge set of irrelevant features that function as noise in downstream applications (Chapter 4). In an earlier exploratory study, the noise handling methods that used a distance between molecules did worst (Section 4.4.1). That study was motivation only: it covered one dataset, the QM9 HOMO–LUMO gap, and one noise type, Gaussian. Chapter 6 measures the label-noise question over more datasets and noise types. With all of that in mind, I decided to develop a method that selects the best features for a given dataset and model. In theory, it should effectively filter out noisy features and isolate the features which carry the most relevant information.

The questions this chapter and Chapter 6 address are set out in Chapter 1.

An adaptive hybrid representation is a feature set built by selecting up to a fixed number of features, the budget, from several base representations of the same molecules. It is chosen by a search, for one dataset and one downstream model. A

different dataset, split or model gives a different hybrid. The hybrid is an input, not a model. Any standard supervised model can be trained on it. It is meant for base representations that say different things about the molecule, such as PDV descriptors, ECFP4 bits, graph embeddings and sequence embeddings. The features for the task are then picked from all of them.

The budget is split between the representations before any feature is picked. Representations differ in width by orders of magnitude, from tens to thousands of features. In one ranking over all their features, a wide representation can crowd out a narrow one. Importance scores from different types of feature are also not on a common scale. So each representation's features are ranked only against each other. The search also includes one pooled allocation method, the global pool, which ranks the features offered by every representation together. Per-representation and pooled selection are therefore compared, not assumed. The idea leaves six choices open. Three concern the features: how useful each feature is, how many features each representation gets, and how many representations take part. The other three are how a candidate hybrid is judged during the search, the total budget, and whether the answer depends on the downstream model.

A hybrid can also be used to measure similarity, in two ways. Distances can be computed in an ordinary hybrid, or the search can pick features by how well each molecule's activity is predicted from its five nearest neighbors. Both are defined in Chapter 6, "Similarity in a hybrid".

5.1.3 Blueprints and what they tell a chemist

A finished search is saved as a blueprint: the settings needed to rebuild the hybrid. Every selected feature traces back to its representation and to its importance rank within that representation. So the search returns information about the problem, not only a better score. The search keeps the best configuration for each pair of downstream model and budget. One dataset therefore gives one blueprint per downstream model and budget. Comparing them at one budget shows whether tree ensembles and neural networks draw on different features.

Some selected features can be read by a chemist. A descriptor from the PDV set carries its name, such as molecular weight or logP. A selected ECFP bit can be shown as the atom environment that sets it. A MACCS key stands for a known substructure. Outside small molecules the readable unit changes. For an antibody

it is a sequence position and the residue at it. For gene expression it is a named gene set, such as a pathway or a chromosome arm. A feature of a learned embedding says only which representation was useful, not which chemistry. Chapter 4 covers why.

Readable features can be used in two directions. The first is discovery. Substructures or descriptors that recur among the kept features point to parts of the molecule that may be involved in binding or activity. They are a starting point for a chemist, not a finding. The second is confirmation. If the kept features make chemical sense for the endpoint, that supports the claim that the selection finds real signal and is not fitting noise. Explainability is a recognized need in drug discovery [189]. Inputs that can be read directly are an alternative to explaining a black-box model after the fact [190]. Two caveats apply. First, selected means predictive, not causal. Correlated features can stand in for each other, so an unselected feature may matter just as much. Second, a blueprint rebuilt on other molecules can select other features. Features should be read from the fit on the dataset in question.

A hybrid also gives a distance chosen for one task. Anything that uses nearby molecules as evidence could use it instead of a standard fingerprint distance. Chapter 4 argues for several such uses: noise detection, nearest-neighbor prediction, applicability domain, activity-cliff detection, clustering molecules for group-aware splits, and weighting predictions from separate sites. One use is not covered there. Generative design scores each proposed molecule with a predictor [191], and that predictor could take a hybrid as its input. We leave this to future work.

5.1.4 Beyond small molecules

The search needs two things: a supervised task, and several vector representations of the same samples. Each representation is one feature matrix, with one row per sample. The default search objective scores prediction only. It reads no chemical structure. So the method applies to tabular data in general: any task where several encoders or measurements exist for the same inputs. What does not carry over is specific to chemistry. Reading substructures from a blueprint needs named substructure features. An optional search objective scores how closely distances in the hybrid track the maximum common substructure shared by pairs of molecules. It applies only to molecules. The nearest-neighbor objective of Chapter 6 needs only labels and a distance, so it does carry over. It is searched on the antibody and other non-molecular datasets as well.

One-hot sequence encodings are very sparse. At most positions the residue never changes across a dataset, so those features carry no importance. One-hot can still work. Among the four antibody datasets of this chapter, one-hot is a base representation on nanobody thermostability only. The position-wise BLOSUM encoding is a base representation on humanness and trastuzumab–HER2 binding. One-hot was among the selected representations on **[RESULT NEEDED: the antibody datasets where one-hot was among the selected representations, with the score in the dataset's metric]**. We recommend language-model sequence embeddings over raw one-hot encodings.

Antibodies received the most work after small molecules. This chapter uses four antibody datasets: Koenig expression, nanobody thermostability, humanness and trastuzumab–HER2 binding (Table 5.9). Their base representations are mostly embeddings from general protein language models and from antibody-specific language models. Some embeddings cover only the CDR loops, and some come from early or middle layers of ESM-2 (Table 5.13). Chemical reactions are a proof of concept, on two Buchwald–Hartwig yield datasets. Their base representations are computed on the reactants, on the products minus the reactants, or on the whole reaction; product fingerprints enter only as augmentation (Table 5.12). Nucleotide sequences are also a proof of concept, on the H3K4me3 histone-mark and TATA promoter tasks (Table 5.14). In cancer genomics (TCGA), each representation is a different measurement of the patient. These are gene expression, methylation (lung task only), copy number, miRNA, protein and slide images (Table 5.15). In the other domains, each representation is a different encoder of one input. The search treats both cases the same way.

A dataset needs two things for the hybrid to help. The first is enough samples. **[RESULT NEEDED: the number of samples below which the hybrid stops beating the best single representation; about 200 samples is expected]**. The second is base representations whose features carry importance. Where every representation already scores poorly, there are too few useful features to improve on.

5.2 Methods: the hybrid search

This section describes how a hybrid is built from several base representations of the same samples. The search is a grid: every combination of setting values is built and

scored inside the cross-validation folds. Adaptive pruning, when it is switched on, drops some combinations after the first two folds. A winner is kept for each pair of downstream model and budget. The budget is a maximum number of features, and a hybrid can have fewer. The comparison we report runs the whole search inside each outer training fold and scores the winner once on the untouched outer test fold.

The first subsection gives the steps from base representations to one feature matrix, which molecules each step sees, and the blueprint that stores the settings. The second subsection gives each searched setting, its values, and why it is searched rather than fixed. Its parts cover feature importance scoring, feature allocation strategies and internal evaluation. The third subsection, on augmentation, offers a finished hybrid, or one full representation, a few features from small or specialized feature sets. These extra features are added on top of the budget or swapped in for the weakest features. The last subsection gives two ways to make the search cheaper. The fast tier uses shorter lists of values, fixed in advance. Adaptive pruning drops values that lose a vote on the first two folds of the dataset being searched.

5.2.1 From base representations to a hybrid

Given K frozen representation matrices $\mathbf{R}_1, \ldots, \mathbf{R}_K$ for the same n samples, a label vector $\mathbf{y}$, and optional group assignments for cross-validation, the search finds which features from which sources best predict for a specified metric and downstream model. The output is a hybrid feature matrix of at most b columns, where b is the feature budget (plus any augmentation features added outside it), together with a Blueprint that records all parameters needed to reproduce the identical feature selection on new data. The search first scores each source independently, then scores feature importance on training data within each cross-validation fold, allocates the budget among sources according to baseline performance, and scores candidate hybrid configurations via cross-validation. We retain the best configuration for each combination of downstream model and budget. Every selected feature traces back to its source and importance rank.

The search is given K frozen representation matrices $\mathbf{X}^{(k)} \in \mathbb{R}^{n \times d_k}$, $k = 1, \ldots, K$, for the same n samples. Their widths d_k run from tens to thousands of features. The search is also given a label vector $\mathbf{y}$ and one group label per sample for cross-validation, for example its scaffold. It also needs the dataset's metric, the task type, one or more downstream models and a list of budgets. It finds which features from

which representations best predict for that metric and downstream model. The output is a hybrid feature matrix of at most B features, where B is the feature budget. The budget is a maximum, not a target. Uncapped augmentation adds its extra features outside the budget; capped augmentation stays within it. The output also includes a blueprint that records the settings needed to rebuild the hybrid. The search first scores each representation independently, then scores feature importance on training data within each cross-validation fold. It then allocates the budget among representations. Four of the five allocation methods do this according to each representation's standalone performance; the global pool instead ranks the offered features from all representations together. Candidate hybrid configurations are scored via cross-validation. We retain the best configuration for each combination of downstream model and budget. Every selected feature traces back to its representation and importance rank.

Inputs from different sources differ in scale, distribution, and missingness patterns, and naively pooling them before splitting would leak validation-set statistics into the training procedure, a well-documented source of optimistic bias in feature selection studies [192, 193]. All preprocessing therefore runs within each cross-validation fold. We impute missing values with per-feature medians computed on the training fold and drop columns that are entirely missing. Every feature of the hybrid, binary fingerprint bits included, is standardized; only when a single representation is scored on its own are binary features left as 0/1. We standardize to zero mean and unit variance using training-fold statistics only.

Inputs from different representations differ in scale, distribution, and missingness patterns. Naively pooling them before splitting would leak validation-set statistics into the training procedure, a well-documented source of optimistic bias in feature selection studies [192, 193]. All preprocessing therefore runs within each cross-validation fold. When a single representation is scored, we impute missing values with per-feature medians computed on the training fold and drop features that are entirely missing. When features are selected for the hybrid, a missing value is set to 0 before standardization. Every feature of the hybrid, binary fingerprint bits included, is standardized to zero mean and unit variance using training-fold statistics only. Binary features are left as 0/1 in one case only: when the search scores a single representation to pick the comparator. A single representation scored on the outer test fold is standardized like the hybrid. A representation wider than 5,000 features is first cut, inside each fold, to the 500 features a ranking model splits on most

often. It keeps 200 instead when the search is given fewer than 1,000 molecules. The ranking model is a LightGBM model of 50 trees of depth at most 5, fitted on the fold's training molecules. Features are ranked by how many times its trees split on them, not by gain. This cut applies only when the search scores a single representation on its own. For such a representation, the comparator is therefore picked on its top 500 (or 200) features, then scored on the outer test fold at full width. The hybrid search always sees the full width.

Each dataset is split into outer folds, 5 by default. The folds are group-aware, so no group appears on both sides of a split. Classification folds are also stratified by class. A representation learned from the labels is re-trained inside each outer training fold, on its training molecules only. The search then runs its own group-aware cross-validation on the molecules it is given, with 5 folds by default. Inside each training fold of the search, one group-aware internal split holds out about 20% of the molecules as whole groups. It is the first fold of a 5-fold grouped split, so it is deterministic. When a training fold holds fewer than five groups, both splits use as many folds as there are groups. On Koenig expression, which has five groups, each outer training fold holds four. The search inside it then uses 4 folds of three groups each, and the internal split holds out one of the three. Importance and each representation's standalone score come from this training fold only. The outer test molecules score only the finished hybrid.

For each representation k, the importance method gives every feature one score $s_j^{(k)} \geq 0$. These scores rank features within one representation only. Importance is computed on the internal training part of the fold. The standalone score r_k comes from the internal evaluator. It is fitted on representation k alone and scored on the internal held-out part, in the dataset's metric. We rescale r_k to $\tilde{r}_k$ so that 0 means "no better than a trivial predictor". This way the budget split does not depend on which metric the dataset uses. For Spearman, Pearson, MCC and R^2, $\tilde{r}_k = \max(0, r_k)$. For AUROC and the concordance index, chance is 0.5, so $\tilde{r}_k = \max(0, 2(r_k - 0.5))$. For RMSE, $\tilde{r}_k = \max(0, 1 - \mathrm{RMSE}_k/\mathrm{RMSE}_0)$, where RMSE_0 is the error of predicting the training mean. MAE is rescaled the same way, with the training median as the trivial predictor. For accuracy and weighted F1, $\tilde{r}_k = \max(0, (r_k - r_0)/(1 - r_0))$. Here r_0 is the score of always predicting the majority training class. The trivial predictor is scored on the same held-out molecules. A representation whose fit fails gets weight 0 and is recorded as a failure.

An optional representation count N keeps only the N representations with the

highest standalone scores on this fold. Each remaining representation gets a weight $w_k = g(\tilde{r}_k)$, and the allocation method sets g. The exact share of representation k is $b_k^* = B\, w_k / \sum_l w_l$ features. We round the shares by largest remainder. Each representation first gets $\min(\lfloor b_k^* \rfloor, d_k)$ features. The leftover features are then given out one at a time, largest fractional part first, skipping representations already at full width. The result satisfies $\sum_k b_k = \min(B, \sum_k d_k)$ and $b_k \leq d_k$. A representation with $w_k > 0$ keeps at least one feature. If every weight is 0, the budget is split equally and the run reports this. If no representation can be scored at all, the configuration is refused. The global pool does not use this rounding. Each representation offers candidate features, and one joint ranking keeps at most B of them.

Let S_k be the top b_k features of representation k by $s_j^{(k)}$, among those with $s_j^{(k)} > 0$. The same cut at 0 applies to the features offered to the global pool and to its final ranking. If no feature of a representation scores above 0, its full b_k is taken. The cut exists because tree importance gives exactly 0 to every feature no tree split on. Filling the budget from those features would pick them by their position in the matrix. The hybrid is $\mathbf{H} = [\mathbf{X}^{(1)}_{:,S_1}, \ldots, \mathbf{X}^{(K)}_{:,S_K}]$, with $\sum_k |S_k| \leq B$. A hybrid can therefore be narrower than its budget. $\mathbf{H}$ is then preprocessed inside the fold, as described above.

A configuration is one value on every searched setting. For each configuration, we fit the downstream model on the hybrid's training molecules in each fold of the search. We score it on that fold's held-out molecules and average over the folds. The winner is picked separately for each pair of downstream model and budget, never across models. That score chose among many configurations, so it is optimistic [194, 195]. The comparison we report is therefore nested. The whole search runs inside each outer training fold, and the winning settings are rebuilt on that fold. The result is scored once on the untouched outer test fold. Both the winner at each budget and each downstream model's best across budgets are scored this way. The inner search score exceeds the outer test score by **[RESULT NEEDED: the gap between inner search score and outer test score, per dataset, in each dataset's own metric]**.

Chapter 6 uses two variants of this search. Under label noise, every training-side quantity uses the noisy labels, and held-out scoring uses clean labels unless stated. Under the nearest-neighbor objective, each molecule's activity is predicted from its five nearest neighbors in the feature space. That score replaces both the standalone score and the downstream-model score, so no downstream model or internal evalu-

ator is used.

Blueprints. To be able to better examine the selected features and to regenerate them on new data sets, we produce blueprints. After a hybrid feature selection search is completed, it's results are stored as a blueprint containing the "recipe" required to recreate this exact hybrid representation on any data. These blueprints store the selected picks for the pre-specified budget, the full list of input representations searched and how many of them the recipe used, feature importance scoring method, allocation strategy, and internal evaluator. It contains other runtime information like the step size, number of folds, the seed, the metric being used to score, the task type (regression or classification), whether augmented features were being used, and the per-representation scores.

To be able to better examine the selected features and to rebuild the hybrid on new datasets, we produce blueprints. A blueprint is the "recipe": the search settings needed to rebuild the winning hybrid. Every scored configuration has one, and the search returns the winner's blueprint with its result. On other molecules, a rebuild re-scores the representations and re-picks the features, so it need not recreate the same hybrid. These blueprints store the budget, the full list of input representations searched, and how many of them the recipe used. They also store the feature importance scoring method, allocation strategy, internal evaluator and objective. They contain other runtime information like the number of folds, the seed, the metric being used to score and its direction, and the task type (regression or classification). They also store the augmentation settings: how many extra features to take, from which candidates, and the rule for admitting one. Some information is stored for inspection only. This is each representation's best score across the downstream models, how many features each representation got in each fold, and which augmentation features were kept in each fold. These show which representations fed the hybrid and whether that holds across folds. That stored score is not the standalone score that splits the budget. It is measured on every molecule the search saw, so a rebuild never reads it.

The blueprint does not store selected feature indices, those can only be recovered when the blueprint is refit on the data, which re-runs the feature selection process deterministically from that recipe and seed.

The blueprint does not store the selected base-representation features. It does store how many features each representation got in each fold, and which augmenta-

tion features were kept. A blueprint can be reused in two ways. The first re-runs the recipe on new labeled data. Importance and standalone scores are recomputed, so the features adapt to the new dataset. This costs a new selection and can give a different feature set. The second reuses the exact features from a labeled fit, taking the same features from the new molecules. This needs no labels, so it is the only option for unlabeled molecules. It assumes the same features matter on the new data. Re-running the recipe without labels is refused. Every importance score would be 0, and the pick would fall back to feature order. Which of the two transfers better to a smaller related dataset is **[RESULT NEEDED: re-run recipe vs. reused exact features, large to small related dataset]**. Importance and allocation can also be computed separately within k-means clusters of the molecules' Morgan fingerprints, to ask whether the budget split changes across chemical space.

It also provides a registry of selected features, stating where that feature came from. On some representations, we added an extension where features are mapped back to their function. For instance, ECFP and MACCS bits are mapped to SMARTS substructures. This blueprint will tell you which of those substructures were selected. This is not possible for learned embeddings for which the blueprint will return the bare representation name and dimension index. This mapping provides a layer of explainability. It can be used both to discover what features are useful for your task and to confirm that the features being selected are in fact lining up with the underlying known chemistry.

A separate explanation step, run on a finished hybrid, provides a registry of selected features, stating where each feature came from. The blueprint itself does not store these names. On some representations, we added an extension where features are mapped back to their function. An ECFP (Morgan) bit is named by the atom environment that sets it, written as a SMILES fragment rather than SMARTS. The fragment is taken from the first molecule, of at most 50 supplied, that has the bit set. MACCS keys are named by their SMARTS pattern and a short name. The descriptor vector's features carry the descriptor names. The explanation step will tell you which of those substructures and descriptors were selected. Outside small molecules, protein composition, descriptor and k-mer features are named too. So are antibody composition, one-hot and BLOSUM positions, structural and inverse-folding scores, gene-expression pathway sets and PCA components. MAP4 and DRFP features are returned as a bit index only. Avalon, Mordred, atom-pair and graph-kernel features, and every learned embedding, return only the bare representation name

and dimension index. This mapping provides a layer of explainability. It can be used to discover which features are useful for your task. It can also confirm that the selected features line up with the known chemistry.

5.2.2 The search space and why each axis is swept

When I began to implement this idea, I figured that there would be a fairly straightforward way to select the most informative features. However, I began to ask some questions. How do you measure feature importance? If you're starting with a handful of input representations, how many features do you take from each? How do you score these potential feature sets? Unfortunately, my one-size-fits-all hypothesis rang true on multiple fronts. As I found out, different prediction tasks prefer different answers to all of those questions. Each question maps onto a setting of the search. How to measure importance is the importance method. How many features to take from each representation is set by the allocation method, the budget and the representation count. How to score a feature set is the internal evaluator. So to figure out how to even go about selecting the best features, I realized I had to sweep a search space. The answer to each question is searched per dataset.

The search is a grid. Every combination of setting values is built and scored, and nothing is fixed in advance except the values each setting may take. The claim of this chapter is that no single setting suits every dataset. A grid measures that claim instead of assuming it. The motivation is a familiar one: no learning method is best on every problem [101]. That result motivates the grid; it does not prove the claim for hybrids. We search the full product of values, not one setting at a time, because the settings interact. For example, the best budget depends on the downstream model, as the nanobody thermostability example below shows.

Six settings are searched: downstream model, budget, representation count, importance method, allocation method and internal evaluator. Their values at the full tier and at the fast tier are listed in Table 5.7. The defaults and the number of configurations for each downstream model are in Additional files, Section S.3. The importance method is LightGBM gain, TreeSHAP or DeepLIFT. It is searched because importance is a property of a model, not of the data alone [196]. The three methods come from different model families and rank features differently, and a different ranking builds a different hybrid. The allocation method decides how the budget is split among the representations taking part; five methods are searched.

The internal evaluator scores each representation on its own. It is matched (a model of the downstream model's own family), random forest, LightGBM or DNN. Two choices are not searched inside one search. The objective is prediction by default; Chapter 6 replaces it with predicting each molecule's label from its nearest neighbors in the hybrid. Augmentation is none, uncapped or capped. Each objective and each augmentation setting is a separate search.

The downstream model is the model trained on the finished hybrid. Five are searched: random forest, XGBoost, LightGBM, SVM and DNN. A Gaussian process is added on datasets under 1,000 molecules. Three of the 16 datasets are that small, but run time rules it out on two, so it runs on humanness only. The downstream model is searched because it decides what counts as a useful feature and how many features it can use. So the best hybrid for one model need not be the best for another.

The budget is the most features the hybrid may have. Eight budgets are searched: 10, 25, 50, 100, 250, 500, 1,000 and 2,000 features. The budget is searched because too few features leave informative ones out. Too many add uninformative features, and on small datasets they add room to overfit. Budgets above the number of training molecules are kept. The best budget depends on the downstream model. In a preliminary run on nanobody thermostability with 5 folds, SVM did best at 300 features, with 0.6384 Spearman. DNN, LightGBM and random forest did best at 50 features, and XGBoost at 100. That run used an earlier list of budgets, which included 300 features. **[RESULT NEEDED: the best budget for each downstream model on nanobody thermostability, from the full runs]**

Each dataset has a fixed set of base representations, chosen before the search (Table 5.16). Fourteen of the 16 datasets have 10; H3K4me3 has 8 and TATA promoter 9. The representation count sets how many of them may contribute features to the hybrid: 2, 3, 5 or all. On each fold, the representations with the highest standalone scores go on. This is the only step that drops whole representations before allocation. Correlation between representations plays no part in it. The count is searched because datasets differ in how many representations are worth using. Pooled feature selection over all representations has no such setting. Every allocation method is paired with every count. In preliminary runs, some datasets did best with 2 or 3 representations and others with all 10. **[RESULT NEEDED: which datasets do best with 2 or 3 representations and which with all of them, from the full runs]**

Feature importance scoring

Not all features from a given base representation are equally informative, and there are a variety of techniques that can assess each feature's relative importance. Tree-based feature importance measures favor features with high split gain, while neural attributions such as DeepLIFT measure how much each feature moves a neural network's output away from its output on a reference input [181, 197]. We tested if, for the purposes of this task, there was a single feature importance method that worked across data sets and model architectures. Not only was there no clear winner, but there was evidence that certain networks preferred tree-based measures of importance while certain tree ensembles preferred gradient-based approaches. We landed on a set of three who's predictions did not correlate strongly with each other and who's selected features performed well as model input.

Not all features from a given base representation are equally informative, and there are a variety of techniques that can assess each feature's relative importance. The search ranks the features of each representation before choosing any. An importance method gives every feature of one representation one non-negative score. Each representation gets its own freshly fitted model, on its own features only; the downstream model is not used. Only the order within a representation is used. These scores are never compared across representations or across methods, so no normalization is applied. Scores are computed inside each fold, on the internal training part only: 80% of that fold's training molecules, split by group. Before fitting, missing values are set to 0 and each feature is standardized to mean 0 and variance 1 on the same molecules. Scores are computed once per fold and method. Every allocation method, budget, internal evaluator and downstream model then reuses them. Split-based tree importance is biased toward features with many possible split points [181]. This matters here, because binary fingerprint bits sit beside continuous embeddings. Neural attributions such as DeepLIFT instead measure how much each feature moves a neural network's output away from its output on a reference input [197].

We tested if, for the purposes of this task, there was a single feature importance method that worked across datasets and downstream models. We compared the methods' feature rankings on 7 datasets. We compared the hybrids those rankings build on one dataset, with two downstream models. We counted which method gave the best configuration on 5 datasets with 5 downstream models. Not only was

there no clear winner, but the preferred method changed with the dataset. With a DNN downstream model, LightGBM gain was preferred on four of five datasets. On LogD alone, the tree ensembles XGBoost and LightGBM preferred DeepLIFT. We landed on a set of three whose rankings did not correlate strongly with each other and whose selected features performed well as model input.

We score features using LightGBM gain, DeepLift, and TreeSHAP. LightGBM gain is a tree based method which.... TreeSHAP is explanation-based, meaning it... DeepLift relies on neural attribution which...

We score features using LightGBM gain importance, SHAP values for tree ensembles (TreeSHAP), and DeepLIFT (Deep Learning Important FeaTures) [198]. They come from three families: split gain, Shapley values and reference-based attribution. LightGBM gain measures how much a feature reduced training loss where trees split on it [57]. Its model is 50 gradient-boosted trees of maximum depth 6. A feature's score is the loss reduction of every split on it, summed over all trees, and 0 if no tree splits on it. TreeSHAP measures how much a feature moves each molecule's prediction, as an exact Shapley value for tree models [199]. Its model is a random forest of 50 trees with maximum depth 10 [44]. A feature's score is its mean absolute Shapley value over the training molecules. DeepLIFT measures how much a feature moves a small network's output away from its output on a reference input [198, 200]. Its model is a small neural network with two hidden layers, of 64 and 32 ReLU units. It is trained for 100 full-batch epochs of Adam at a learning rate of 0.001 [201]. A feature's score is its mean absolute attribution over the training molecules, under DeepLIFT's Rescale rule. The reference input is all zeros, which after standardizing is every feature at its training mean. For a fingerprint bit, that is the fraction of training molecules that have the bit. On binary labels, TreeSHAP and DeepLIFT use the positive class. On multiclass labels, they average absolute values over the classes. For LightGBM gain and TreeSHAP, a representation wider than 2,000 features is pre-screened. Only its 2,000 highest-variance features, with variance measured before standardizing, are fitted; the rest score 0. DeepLIFT scores every feature. On Caco-2, the median share of features scoring above 0 was 30% for LightGBM gain, 70% for TreeSHAP and 90% for DeepLIFT. That median is over 10 representations and 3 folds, and is preliminary. Table 5.2 summarizes the three methods.

One row is one importance method, as run inside the search.

When searching for the features to use in the hybrid, thse feature importance

Table 5.2: The three importance methods searched. Each is fitted separately on every representation, inside each fold, on the internal training part. Settings not listed are the library's defaults.

Method (family)	Model fitted per representation	Score of one feature	Classification labels	Representations over 2,000 features
LightGBM gain (split gain)	LightGBM, 50 trees, maximum depth 6	Loss reduction of every split on the feature, summed over all trees; 0 if never split on	One model on the class labels	Only the 2,000 highest-variance features are fitted; the rest score 0
TreeSHAP (Shapley values)	Random forest, 50 trees, maximum depth 10	Mean absolute Shapley value over the training molecules	Binary: positive class. Multiclass: mean absolute value over classes	As for LightGBM gain
DeepLIFT (reference-based attribution)	Small neural network, ReLU hidden layers of 64 and 32 units, 100 full-batch epochs of Adam, learning rate 0.001	Mean absolute attribution against an all-zero reference, Rescale rule	Binary: positive class. Multiclass: mean absolute value over classes	Every feature is scored

scoring methods are swept.

When searching for the features to use in the hybrid, these feature importance scoring methods are swept. The full search sweeps all three; the fast tier uses LightGBM gain only (Table 5.7). Adaptive pruning is off unless a search asks for it. When it is on, importance methods that rarely give the best configurations on the first two folds are dropped from the remaining folds. At least one method is always kept. Pruning is described in "Fast tier and adaptive pruning".

Two comparisons were run. The first compared how the methods rank features. The second compared the hybrids those rankings build. The ranking comparison scored 8 importance methods on 7 datasets: LogD, hERG Ki, Buchwald–Hartwig yield, nanobody thermostability, antibody FcRn retention, BRCA PAM50 subtype and LUAD vs LUSC. For each pair of methods, it measured the Spearman correlation between their scores over every feature of every representation. The three kept methods rank features differently. Across the 7 datasets, the correlation ran from 0.20 to 0.64 between LightGBM gain and TreeSHAP. It ran from 0.21 to 0.67 between LightGBM gain and DeepLIFT, and from 0.56 to 0.99 between TreeSHAP and DeepLIFT. These numbers are preliminary. Each method was fitted once on

the whole dataset, not inside folds.

The hybrid comparison used Caco-2: 2,158 molecules, 10 representations, 3 scaffold folds and a budget of 100 features. Hybrids were built from each method's ranking and scored with random forest and LightGBM downstream models. Three control rankings were added: feature variance, Laplacian score and a random order. The measure was the RMSE of the best single representation minus the RMSE of the hybrid, in $\log_{10}$ efflux ratio units. A positive value means the hybrid has the lower error. The preliminary results were +0.0144 for DeepLIFT, +0.0100 for LightGBM gain, +0.0067 for TreeSHAP and −0.0118 for the random ranking. The gap between DeepLIFT and LightGBM gain, 0.0044 RMSE, is smaller than the fold-to-fold spread. The methods also differ at the top of their rankings. Of each representation's 100 top-ranked features on Caco-2, the kept pairs of methods share a median of 28 to 54 features.

Three rules chose the methods. First, a method must score every feature of every representation, for regression, binary and multiclass labels; all three kept methods do. Second, of two methods whose rankings agree above Spearman 0.95, we keep the faster one. Under this rule DeepLIFT replaced integrated gradients, which agreed with it above 0.95 on 4 of 6 datasets. It also replaced gradient SHAP, which agreed above 0.95 on 3 of 6. Third, we keep methods that rank features differently, because a different ranking builds a different hybrid. TreeSHAP came last of the three on Caco-2 and was kept under the third rule. Only two filters act on features before allocation: the cut at a score of 0, and the pre-screen on representations wider than 2,000 features. On a 2,048-bit fingerprint, the pre-screen drops the 48 lowest-variance bits. On hERG Ki all 48 are among the 105 constant ECFP4 bits. On QM9, which has no constant ECFP4 bits, they are near-constant ones. Near-constant features otherwise stay candidates and are ranked by importance.

No single importance method suits every dataset and downstream model. A preliminary run used 5 datasets: nanobody thermostability, hERG Ki, humanness, LogD and BRCA PAM50 subtype. It used 5 folds, 10 representations, 5 downstream models and budgets of 25, 50, 100, 150 and 300 features. For each pair of downstream model and budget, 25 per dataset, we counted which importance method gave the best configuration. Table 5.3 gives the counts. LightGBM gain won the most pairs on three datasets and tied with TreeSHAP on humanness. DeepLIFT won the most on one dataset, LogD, with 17 of its 25 pairs. Outside LogD, DeepLIFT gave the best configuration at 2 of 100 pairs. With the DNN downstream model, LightGBM

gain won 17 of 20 budgets outside LogD. On LogD the DNN took DeepLIFT at 3 of 5 budgets. There, XGBoost and LightGBM took DeepLIFT at 5 of 5 budgets each, and random forest took LightGBM gain at 5 of 5. **[RESULT NEEDED: the same win counts, per dataset and importance method, from the full runs]**

One row is one dataset. A win is one pair of downstream model and budget at which that importance method gave the best configuration, out of 25 pairs. Best score is the best configuration reached with that method, in the dataset's own metric.

Table 5.3: Which importance method gave the best configuration, in a preliminary run with 5 folds, 10 representations, 5 downstream models and budgets of 25, 50, 100, 150 and 300 features. Lower is better for RMSE; higher is better otherwise.

		Wins (of 25)			
Dataset	Metric	LightGBM gain	TreeSHAP	DeepLIFT	LightGBM ga
Nanobody thermostability	Spearman	15	10	0	0.6696
hERG Ki	RMSE (pK_i)	14	11	0	0.5589
Humanness	AUROC	12	12	1	0.9723
LogD	RMSE (log units)	8	0	17	0.4250
BRCA PAM50 subtype	weighted F1	21	3	1	0.9097

Feature allocation strategies

How to divide the total feature budget among sources is the role of the allocation strategy. We search five. Four divide the budget in fixed shares from transformed standalone scores (linear, squared, softmax, rank decay). The fifth, a global pool, lets features from all sources compete in one importance ranking. Once we have the allocation, we select the top features from each source by importance score and concatenate them to form the hybrid.

How to divide the total feature budget among representations is the role of the allocation method. It decides how many of the budget's B features each representation contributes; the importance method then decides which ones. We search five. Four give each representation a fixed share of the budget, computed from its rescaled standalone score (linear, squared, softmax, rank decay). The fifth, a global pool, lets the candidate features offered by every representation compete in one importance ranking. Under the global pool only those offered candidates compete, not every feature. Under the four fixed-share methods, we select the top features from each

representation by importance score and join them to form the hybrid.

The four fixed-share methods are variants of performance-weighted allocation, where the distribution of features from each base representation is proportional to that representation's standalone predictive performance on the training set. This works under the assumption that representations with stronger predictive performance contribute more valuable features. It is worth noting that this is not always the case, as high-quality, large representations may rely on a small subset of valuable features.

The four fixed-share methods set each representation's share from its standalone score. That score is measured on the internal validation part of the training fold, not on the molecules the internal evaluator was fitted on. Only linear allocation makes the share proportional to that score; the other three transform it first. All four work under the assumption that representations with stronger predictive performance contribute more valuable features.

1. Fit the internal evaluator on each base representation independently, on the internal training part of the fold.

2. Score each representation on the internal validation part in the dataset's own metric, rescaled so 0 means no better than a trivial predictor.

3. Allocate features proportionally:

$$b_k^* = B \frac{w_k}{\sum_l w_l}, \quad w_k = g(\tilde{r}_k)$$

4. For each representation, rank features by the configuration's importance method and select the top b_k, after rounding b_k^* as in "From base representations to a hybrid".

5. Concatenate selected features to form the hybrid representation.

Each fixed-share method follows the steps in "From base representations to a hybrid". It rescales each standalone score to $\tilde{r}_k$, sets b_k from $w_k = g(\tilde{r}_k)$, and takes the top b_k features of representation k. Allocation runs only over the representations the representation count keeps. The standalone scores cost one fit per representation per fold and internal evaluator. Every budget, allocation method and importance method reuses them.

This method cannot adapt allocation based on feature complementarity, and it relies on the assumption that many features in a well-performing representation are valuable.

Fixed shares rely on the assumption that many features in a well-performing representation are valuable. This is not always the case: a strong representation may owe its score to a few features, so a large share can fill with weak ones. Fixed shares also cannot adapt allocation to feature complementarity. Two representations carrying the same information both get a share. Of the five methods, only the global pool ranks features from different representations against each other.

The four fixed-share methods differ only in the weight w_k computed from the rescaled score $\tilde{r}_k$ (Table 5.4). Their settings are fixed for every dataset, not searched. Linear sets $w_k = \tilde{r}_k$, and squared sets $w_k = \tilde{r}_k^2$. Under both, a representation gets 0 features when its rescaled score is 0. Softmax sets $w_k = \exp\big((\tilde{r}_k - \max_l \tilde{r}_l)/T\big)$, with temperature $T = 1$. Rank decay sets $w_k = e^{-0.3\rho_k}$, where ρ_k is the representation's rank by score, 0 for the best. Under these two, no representation gets 0 features because of its score. Every rescaled score lies between 0 and 1, so softmax cannot give one representation most of the budget. With 10 representations, the best takes at most 23.2% of it. Under rank decay only the order counts. With 10 representations, the best gets 27.3% and the tenth 1.8%, whatever the gaps between their scores. Table 5.5 works one example: a budget of 100 features and three representations with rescaled scores 0.6, 0.5 and 0.2.

One row is one allocation method. $\tilde{r}_k$ is representation k's rescaled standalone score and B is the budget in features.

One row is one fixed-share method. Each column is one representation, headed by its rescaled score; each entry is the number of features it gets from a budget of 100.

The global pool sizes each representation's offer with squared weights $\tilde{r}_k^2$, as shares of a pool of $2B$ candidate features. An offer is capped at the representation's width in features. Only features scoring above 0 are offered. The offered features are joined into one matrix. The same importance method is then fitted once on that matrix, on the internal training part. The hybrid keeps the top B features of this joint ranking, again only those scoring above 0. Each representation's final count is an outcome of the joint ranking, not set in advance, and there is no one-feature floor. A representation gets 0 features when its score is 0 or when none of its candidates reach the joint top B. Here a feature's rank depends on which features

Table 5.4: The five allocation methods searched. Under every method, a representation also gets no features when the representation count leaves it out or its standalone fit fails.

Method	Weight w_k	Fixed setting	How features are chosen	When a representation gets 0 features
Linear	$\tilde{r}_k$	none	Top b_k of its own ranking	$\tilde{r}_k = 0$
Squared	$\tilde{r}_k^2$	power 2	Top b_k of its own ranking	$\tilde{r}_k = 0$
Softmax	$\exp\big((\tilde{r}_k - \max_l \tilde{r}_l)/T\big)$	$T = 1$	Top b_k of its own ranking	never from its score
Rank decay	$e^{-0.3\rho_k}$, ρ_k = rank by score, 0 for the best	decay 0.3	Top b_k of its own ranking	never from its score
Global pool	$\tilde{r}_k^2$, sizing its offer to a pool of $2B$ candidates	power 2; pool $2B$	Top B of one joint ranking over all offered candidates	$\tilde{r}_k = 0$, or none of its candidates in the joint top B

Table 5.5: Worked example of the four fixed-share methods: a budget of 100 features split among three representations. Shares are rounded by largest remainder, as in "From base representations to a hybrid".

Method	$\tilde{r}_1 = 0.6$	$\tilde{r}_2 = 0.5$	$\tilde{r}_3 = 0.2$
Linear	46	39	15
Squared	55	39	6
Softmax	39	35	26
Rank decay	44	32	24

the other representations offered. Under fixed shares it does not. The cost is one extra importance fit per configuration, on the pooled candidates.

Under all five methods, a representation also gets no features when the representation count leaves it out, or when its standalone fit fails.

We first tried greedy forward selection. It picks features for how well they work together, which fixed shares cannot do [174, 177]. Each round, it added the next block of features from whichever representation most improved the internal validation score. It stopped when the budget was spent or when the score had not improved for a set number of rounds. With blocks of s features and K representations, it needs up to $(B/s) \times K$ fits per configuration and fold. With blocks of 10 features, that is 500 fits at 500 features and 10 representations, against 10 for the score-based methods. In preliminary runs it scored 0.3600 Spearman on Jain antibody FcRn retention, against 0.4526 for linear allocation, and placed 16th of 17 methods. On breast cancer PAM50 subtype it scored 0.8551 accuracy against 0.8938 for linear, also 16th of 17. It was removed from the search.

The five methods were chosen in a comparison run before the main experiments. It compared 8 candidate allocation methods, each at 4 representation counts, on 9 regression datasets. It used 3 downstream models (LightGBM, random forest and CatBoost) and budgets of 500 and 1,000 features. That gives 54 combinations of dataset, downstream model and budget (preliminary). A fixed-share method gave the single best strategy in 38 of the 54 combinations, and the global pool in 16. All five stay because no single method was best in most combinations. This comparison ran before the current score rescaling existed. It sized the weights with an inner 3-fold group split, not the 80/20 internal split used now.

Search configuration and internal evaluation

Features that improve predictions under RF may not necessarily help a kNN model, and vice versa. This is because variable importance depends fundamentally on the model, as Dong and Rudin showed [196], and as we found in our feature importance scoring analysis. To address this, we use an internal evaluator which scores each base representation on its own with a model whose inductive bias best fits the downstream model. We use the following evaluators: RF, DNN, LightGBM, and the special case of matched, which matches the internal evaluator to the final downstream evalutors. We fit each on the internal training part of the training fold and score it on the

internal validation part.

Features that improve predictions under RF may not necessarily help an SVM model, and vice versa. This is because variable importance depends fundamentally on the model, as Dong and Rudin showed [196]. To address this, we use an internal evaluator, a model fitted inside the search on one representation at a time. It is separate from the downstream model scored at the end. We search four evaluators: random forest, DNN, LightGBM, and the special case of matched, which matches the internal evaluator to the downstream model. No evaluator is chosen in advance to fit the downstream model; all four are searched and the best configuration wins. Features are standardized on the internal training part of the training fold. We fit each evaluator there and score it on the internal validation part, in the dataset's own metric.

This internal evaluator is used to score each representation, and those scores set how the budget is split and which representations are kept within the larger hybrid search. Thus, it is another axes we sweep while searching for the best set of features.

This internal evaluator is used to score each representation, and those scores set how the budget is split and which representations are kept within the larger hybrid search. It does not rank features inside a representation; the importance method does that. Judging a set of features by the score of a model fitted on it is the wrapper idea of Kohavi and John [177]. Thus, it is another setting we sweep while searching for the best set of features.

The full search tests all combinations of budgets, allocations, importance methods, representation counts, downstream models, and internal evaluators under 5-fold cross-validation: 8,160 configurations per dataset when using 5 downstream models. We approach this problem combinatorially as it allows us to measure which factors contribute most to performance variance (model choice, budget, allocation, or their interactions) rather than assuming any single axis dominates.

The full search tests all combinations of budgets, allocation methods, importance methods, representation counts, downstream models, and internal evaluators under 5-fold cross-validation. That gives 8,160 configurations per dataset where all five downstream models run. The count is lower where a dataset leaves a downstream model out, and 10,080 on humanness, where a Gaussian process is added. A fast-tier search has 816 configurations. The breakdown by downstream model and the number of model fits are in Additional file 14. We search every combination rather than assume one setting dominates. This lets us measure which settings, or which

interactions between them, account for most of the spread in scores.

The complete search result is cached to disk. The search runs once; everything after that is a lookup.

The complete search result is cached to disk. Importance scores are computed once per fold and importance method, and each representation's own score once per fold and internal evaluator. Every configuration reuses them, so the cost grows more slowly than the number of configurations. Each configuration's score is stored by its settings, so a fast search can later be extended to a full one. The chosen configuration for each pair of downstream model and budget is saved as a blueprint. The search runs once; everything after that is a lookup.

When I began to implement this idea, I figured that there would be a fairly straightforward way to select the most informative features. However, I began to ask some questions. How do you measure feature importance? If you're starting with a handful of input representations, how many features do you take from each? How do you score these potential feature sets? Unfortunately, my one-size-fits-all hypothesis rang true on multiple fronts. As I found out, different prediction tasks prefer different answers to all of those questions. So to figure out how to even go about selecting the best features, I realized I had to sweep a search space.

We ran a series of experiments to determine the best variables to include in the search space. When making a selection, we took into account how many potential hybrid representations would be gained by including that variable, as well as the compute time

We ran a series of design experiments before the full runs to determine the values each setting of the search may take. They used 1 to 9 datasets and 1 to 5 folds (preliminary). Each dataset was scored in its own metric, and scores were never averaged across datasets. When making a selection, we took into account whether a value gave the best score for some dataset, downstream model and budget, as well as the compute time in hours. A value that won somewhere but cost many hours stayed in the full search and left the fast tier. Each setting's values were then either kept, because each won for some dataset, model and budget, or fixed at one value. A setting was fixed when the spread between its values was smaller than one value's spread across folds. As an example of a kept value, nine downstream models were fitted on each of 10 base representations over 5 folds. The datasets were Buchwald–Hartwig yield, Caco-2 efflux, Koenig expression, LogD and breast cancer PAM50 subtype. The DNN scored best on 4 of the 5 datasets and third on Caco-2 (0.2936

against 0.2851 RMSE, $\log_{10}$ efflux ratio), so it was added. As an example of a fixed rule, we tested five ways of rescaling each representation's score before the budget is split. On hERG K_i, with linear allocation at 300 features, the five rescalings gave 0.5870 to 0.5924 RMSE ($\mathrm{p}K_i$). One of them alone ran from 0.5475 to 0.6294 RMSE across the 5 folds. So rescaling is one fixed rule per metric. Every design experiment is listed in Additional file 15.

We search four values of the internal evaluator: matched, random forest, LightGBM and DNN. The fast tier searches the same four. Matched means an evaluator of the same model family as the downstream model. For a random forest, LightGBM or DNN downstream model, matched is already in the list, so those three are searched with 3 evaluators. For an XGBoost or SVM downstream model, matched adds an XGBoost or SVM evaluator, so those two are searched with 4. On humanness, where a Gaussian process is also a downstream model, matched adds a Gaussian process evaluator, so it is searched with 4 as well. Same family is not the same settings. Only the DNN evaluator shares the downstream DNN's architecture and training settings. One difference remains even there. The evaluator draws the 20% of molecules it uses for early stopping at random. The downstream DNN splits its 20% by the same groups as the outer split. The tree evaluators are smaller than the downstream trees: 50 trees against 100. Their depth is limited to 10 for the random forest and to 4 for LightGBM and XGBoost. This keeps scoring every representation cheap. The SVM and Gaussian process evaluators are fitted on at most 500 training molecules, drawn with a fixed seed. Their cost grows with the square to the cube of the number of molecules. The random forest evaluator tries all features at each split on regression datasets, and the square root of the number of features on classification datasets. The downstream random forest tries the square root on both. If the DNN evaluator fails to fit on a fold, a random forest scores that fold and the run records a warning. Every evaluator setting, beside the same model's settings as a downstream model, is in Additional file 12.

A user can also pass their own downstream model, under any name, as long as it can be fitted and can predict. Matched knows only a fixed list of model families. A model outside that list gets no matched evaluator, and no warning is given. That model is then scored only by the other evaluators, by default random forest, LightGBM and DNN. If matched was the only evaluator requested, a random forest scores the representations. A blueprint written by hand that says matched, rebuilt with a model the search does not know, is scored by LightGBM instead. This is a

limit of the method: a user with their own model should name an evaluator.

Variable importance depends on the model [196]. The representation a random forest scores highest need not be the one an SVM uses best. Matching the evaluator to the downstream model is the obvious default. But the evaluator only has to rank representations, and another model may rank them better for the downstream model. Matched is one of the four searched values, so the search already covers it. A mismatched evaluator winning is informative, not a negative result. We first counted wins in the search itself (preliminary), on 5 datasets: nanobody thermostability, hERG K_i, humanness, LogD and breast cancer PAM50 subtype. These runs used 10 representations, budgets of 25 to 300 features and 5 folds. Their scores are the search's own fold scores, not held out. The matched evaluator gave the best score for its downstream model in 9 of 25 (dataset, downstream model) pairs. Counted per budget, it was best in 43 of 125 (dataset, downstream model, budget) combinations. With 3 or 4 evaluators per downstream model and no effect of matching, about 38 of 125 would be expected. On nanobody thermostability with a DNN downstream, matched scored 0.6696 Spearman against 0.6506 for the LightGBM evaluator, a gap of 0.0190. Matched's 5 fold scores ran from 0.5095 to 0.7920. On LogD with an SVM downstream, matched scored 0.4322 RMSE (log units) against 0.4217 for the LightGBM evaluator, where lower is better. That is a gap of 0.0105; matched lost at all 5 budgets. Its 5 fold scores ran from 0.4118 to 0.4634. We then scored the evaluators on held-out folds (preliminary): 3 outer folds, a budget of 250 features, 5 representations and LightGBM gain only. The downstream models were random forest and SVM. On nanobody thermostability with a random forest downstream, matched scored 0.6098 Spearman, the LightGBM evaluator 0.6220 and the SVM evaluator 0.6172. Matched's fold scores ran from 0.5659 to 0.6734. With an SVM downstream, matched scored 0.6189 Spearman, the Gaussian process evaluator 0.6265 and the LightGBM evaluator 0.6317. Matched's fold scores ran from 0.5530 to 0.6703. On hERG K_i with a random forest downstream, matched scored 0.6046 RMSE ($\mathrm{p}K_i$), the SVM evaluator 0.6059 and the LightGBM evaluator 0.6075. Matched's fold scores ran from 0.5957 to 0.6178, and it beat the SVM evaluator on 2 of 3 folds. With an SVM downstream, matched scored 0.5723 RMSE ($\mathrm{p}K_i$), the LightGBM evaluator 0.5724 and the Gaussian process evaluator 0.5765. Matched's fold scores ran from 0.5605 to 0.5862. Every gap above, held out and in the search, is smaller than the range of matched's own fold scores. **[RESULT NEEDED: per dataset and downstream model, how often the winning**

configuration used the matched evaluator, and its gap to the best other evaluator in the dataset's metric]

5.2.3 Augmentation

Some features are too low-dimensional or too specialized to participate in the budget allocation stage: MACCS fingerprints (167 features), for instance, or log-likelihood ratios (3 features). Excluding them from the allocation would lose potentially useful signal; including them would distort the budget distribution because they cannot contribute many features. We handle these as augmentation: after the base hybrid is constructed, we pool the augmentation features, rank them by how strongly each correlates with the out-of-fold errors of a small random forest fitted on the base features, and keep the top 0, 5, 10, 20 or 40 of them, whichever scores best with the downstream model on the internal validation set.

Some features are too low-dimensional or too specialized to participate in the budget allocation stage: MACCS fingerprints (167 features), for instance, or log-likelihood ratios (3 features). Others, such as mutation features that record differences from a parent antibody, only mean something next to a full representation. Excluding them from the allocation would lose potentially useful signal; including them would distort the budget distribution because they cannot contribute many features. We handle these as augmentation. We call the finished hybrid **H** the base, and the feature sets the extra features are drawn from the donors. After the base is constructed, we pool the donor features. We rank them by how strongly each correlates with the out-of-fold errors of a small random forest fitted on the base features. We keep the top 0, 5, 10, 20 or 40 of them, whichever scores best with the downstream model on the internal validation set. In the uncapped mode, the kept features are appended outside the budget. In the capped mode, they replace the weakest base features, so the hybrid stays within the budget.

Augmentation features range from smaller fingerprints like MACCS or Coloumb matrices, sets of structural information like 3D pharmacophores, and features describing only the differences between the sames like mutation features for antibodies.

Donors are set per dataset, not per domain, and are listed in the Augmentation column of Table 5.16. Augmentation features range from smaller fingerprints like MACCS to sets of structural information like 3D pharmacophores and Coulomb matrix eigenvalues. They also include features describing only the differences between

variants, like mutation features for antibodies. The Coulomb matrix donor is not a fingerprint. It is the 50 largest eigenvalues of a matrix built from atomic numbers and 3D distances between atoms. MACCS is usually a donor: it is one on four of the six small-molecule datasets, and a base representation on MDR1–MDCK efflux. QM9 is the exception. There MACCS is not offered, and the topological torsion fingerprint (2,048 features) is a donor, the one wide donor. Two datasets have no donors, humanness and trastuzumab–HER2 binding. Nanobody thermostability has one, mutation features. Each donor is small, at most 167 features with the torsions aside, or describes one aspect only: shape, charge, the mutated positions, or one tumor property. So each is a plausible supplement and a weak representation on its own. 3D donors are worth offering. Bahia et al. [202] modeled six protein–ligand series (461 structures) with 2D, 3D and combined 2D and 3D descriptors, using k-nearest neighbors, random forest and lasso. Combining 2D and 3D descriptors gave many more significant models, which the authors attribute to the two coding for complementary properties. Their 3D descriptors came from bioactive conformations in the PDB, while our 3D donors use one generated conformer per molecule.

Donor features are ranked by what the base gets wrong. Inside each outer training fold, an 80/20 internal split is made, with each scaffold or sequence group kept on one side. The outer test fold is never used. A small random forest (50 trees, depth at most 10) is fitted on the base features. Its predictions for the internal-training molecules come from 3-fold group-aware cross-validation. Its out-of-fold residuals are $r_i = y_i - \hat{y}_i^{\text{oof}}$. For binary tasks, $\hat{y}$ is the predicted probability of the positive class. We use out-of-fold residuals because a forest's in-sample residuals shrink toward zero and mostly measure fit noise. All donor features are pooled, and each is scored as $c_j = |\rho_{\text{P}}(\mathbf{x}_j, \mathbf{r})|$, its absolute Pearson correlation with the residuals. The top $M = 50$ features go on to the next step. Ranking against the labels $\mathbf{y}$ would favor features that repeat what the base already knows. Ranking against the residuals $\mathbf{r}$ favors features that explain its errors. The same idea fits each new stage to the current residual in gradient boosting [203]. Least angle regression likewise adds the variable most correlated with the residual [204]. The residual model is always this cheap forest, never the downstream model. The ranking then costs the same for every downstream model.

Block sizes $k \in \{0, 5, 10, 20, 40\}$ features are tried, capped at the number of candidates. Block k is the top k ranked features. Each block is added to the base and scored with the downstream model on the internal-validation molecules. The

chosen size k^* has the best score, and it must beat $k = 0$. No extras, $k = 0$, is scored first and wins ties. So "no extras" is the default outcome, not a failure. Five whole-block choices replace about 50 one-by-one decisions to accept or reject a feature. Choosing among many options on a small validation set overfits the choice [195], and five options leave less room for that. The chosen donors and feature positions are stored with the rest of the search settings. They are re-extracted for the outer fold like the rest of the hybrid. **[RESULT NEEDED: per dataset and downstream model, how often $k^* = 0$ is chosen, and per-fold scores for no augmentation, uncapped and capped]**

There are two ways to fit the extras against the budget B. In uncapped mode, extras are appended outside the budget. The final width is $|\mathbf{H}| + k^*$, at most $B + 40$ features. In capped mode, extras must displace base features. The base features are ranked once by the importance of a random forest (50 trees, depth at most 10), and the k^* weakest are dropped. The final width stays at most B. In capped mode, any unspent budget is filled first, and at least one base feature is always kept. Only in capped mode does an extra feature have to beat a base feature to earn its place. We run both because uncapped augmentation and no augmentation differ in width at the same nominal B. That comparison mixes "better features" with "more features". Capped mode removes the width difference.

Augmentation is a separate condition, not a searched setting. For each dataset with donors, the whole search runs three times: no augmentation, uncapped and capped. Every other setting (downstream model, budget, representation count, allocation method, importance method, internal evaluator) is searched inside each condition. Fourteen of the 16 datasets run all three conditions. Humanness and trastuzumab–HER2 binding have no donors, so they run no augmentation only. With no augmentation the width is at most B and nothing is offered. Uncapped allows at most $B + 40$ features, and an extra feature must beat keeping no extras. Capped keeps the width at most B, and an extra must beat both keeping no extras and the weakest base feature it replaces. The three conditions are also run under the nearest-neighbor search objective of Chapter 6. Two datasets never run that objective. Breast cancer PAM50 subtype has 4 classes, and the objective is scored by AUROC, which needs 2. On trastuzumab–HER2 binding, a held-out block can hold one group only. Augmentation has a second use outside the main search. A user can keep one strong representation at full width as the base and add a few extras. There B is that representation's width, so only uncapped mode makes sense. We test this

use in a separate experiment that adds a few features to one strong representation. Decoy features are planted in it to check that the rule takes none. **[RESULT NEEDED: how many decoy features are admitted in the decoy check]** This use matters for adoption. A chemist can keep the representation they already trust and learn which few outside features help, and how many. Augmentation also asks directly whether another representation holds signal the base lacks (Chapter 4). On Koenig expression, the mutation features and the ESM-2 log-likelihood ratios reach the model only through augmentation. **[RESULT NEEDED: which donors are kept most often, per domain]**

The ranking has four limits. $|\rho_{\mathrm{P}}|$ sees only linear relations with the residual, one feature at a time. It does not see redundancy among the extras, so two near-identical donor features can both rank high. Donor features are pooled with no limit per donor. A 2,048-feature donor therefore gets more chances in the ranking than a 49-feature one. The margin over $k = 0$ is zero, so a near-tie on one internal split can admit extras.

Table 5.6 summarizes the three augmentation conditions. One row is one condition.

Table 5.6: The three augmentation conditions. B is the budget and $|\mathbf{H}|$ the width of the base hybrid, both in features. An extra feature is one drawn from a donor.

Condition	Final width (features)	What an extra feature must beat	Datasets that run it
No augmentation	at most B	nothing is offered	all 16
Uncapped	$\lvert\mathbf{H}\rvert + k^*$, at most $B + 40$	keeping no extras	the 14 with donors
Capped	at most B	keeping no extras, and the weakest base feature it replaces	the 14 with donors

5.2.4 Fast tier and adaptive pruning

There are two ways to make the search cheaper, and they differ in when they decide. The fast tier uses shorter lists of values on some settings. The lists are fixed before the search and are the same on every fast-tier dataset. Adaptive pruning decides during the search, from this dataset's own fold scores. The fast tier knowingly accepts a possibly worse answer in exchange for a much smaller grid. Pruning aims

to find the same best configuration while no longer scoring configurations that have already clearly lost. Without either, the full search does not finish on the largest datasets.

Six of the 16 datasets run the fast tier: QM9, LogD, Koenig expression, trastuzumab–HER2 binding, H3K4me3 and TATA promoter. They are marked in Table 5.16, and all others run the full search. A dataset runs the fast tier when the full search would take longer than about 40 hours, what four 12-hour jobs in a row can cover. It also runs the fast tier when it has more than 10,000 molecules or sequences. The estimated hours for the full search were about 103 for QM9, 64 for Koenig expression, 62 for LogD and 39 for TATA promoter. H3K4me3 has 18,244 sequences. The values the fast tier drops are still searched on every other dataset. The fast tier is a compute compromise, not a claim that those values are useless. A fast run can later be extended to a full one without redoing work. Each configuration's score is stored by its settings, not by the tier. A full search then scores only the configurations the fast one skipped.

The fast lists were chosen by one rule (preliminary). A value stays in the fast lists if it is cheap and adds to the best score. An expensive value that wins often stays in the full search only. Importance methods and downstream models were measured on five datasets, 5 folds each, against the unpruned full search. The five were breast cancer PAM50 subtype, nanobody thermostability, humanness, hERG K_i and LogD. DeepLIFT took about half of the fast run's hours on all five datasets (49.9% to 51.6%). Dropping it changed only LogD's best score, from 0.4905 to 0.4930 RMSE (log units). Adding the DNN as a downstream model moved LogD from 0.4930 to 0.4273 RMSE, against a full-search best of 0.4170. Adding the SVM moved hERG K_i from 0.5776 to 0.5599 RMSE (pK_i). Adding XGBoost changed no dataset's best score. All five downstream models are now searched at both tiers. The global pool won 60 of the 125 combinations of dataset, downstream model and budget on those five datasets. It is expensive, so it is in the full search only. The three fast allocation methods are linear, squared and rank decay. **[RESULT NEEDED: the share of run time the fast tier and adaptive pruning saved on the full runs]**

Search tier settings

Table 5.7 lists, for each search setting, the values tried at the full tier and at the fast tier.

Table 5.7: Values tried for each search setting at the two search tiers. Both tiers use five folds; configurations removed by adaptive pruning are scored on the first two folds only.

Setting	Full tier	Fast tier
Downstream models	Random forest, XGBoost, LightGBM, SVM[a], DNN; Gaussian process[b]	Random forest, XGBoost, LightGBM, SVM[a], DNN
Internal evaluators	The downstream model itself, random forest, LightGBM, DNN	Same as full
Feature budgets (features)	10, 25, 50, 100, 250, 500, 1,000, 2,000	Same as full
Allocation methods	Linear, squared, softmax, rank decay, global pool	Linear, squared, rank decay
Importance methods	LightGBM gain, DeepLIFT, TreeSHAP	LightGBM gain
Representation counts	2, 3, 5, all representations	3, all representations

[a]SVM is not fitted above 10,000 molecules (5,000 when augmentation features are added outside the budget). [b]Added only on Humanness. Of the 16 datasets, it is the only one that is under 1,000 molecules, has its size recorded and is not excluded for run time.

Adaptive pruning

Adaptive pruning: every configuration is first scored on the first two folds. At budget 100, within each (model, evaluator, representation count) group, (allocation, importance) pairs with no configuration in the better half are not searched at larger budgets. Each axis is then voted on within groups that fix every other axis: an allocation method is pruned if it is in the best 2 in fewer than 20% of groups, a representation count if it is best in fewer than 10%, an importance method if it is in the best 2 in fewer than 20%, and an internal evaluator if it is in the best 3 in fewer than 10%. Survivors are scored on the remaining folds.

Adaptive pruning: every configuration is first scored on the first two folds, and the rules rank configurations by their mean score over those two folds, in the dataset's own metric. Budgets are scored smallest first. Once every budget up to 100 features has been scored, a checkpoint runs within each (downstream model, internal evaluator, representation count) group. There, (allocation, importance) pairs with

no configuration in the better half are not searched at larger budgets. At least one pair per group is kept. Each setting is then voted on within groups that fix every other setting, including the budget. The best few values in each group get a point, and a value is removed if it gets a point in fewer than a cut-off share of groups. An allocation method is pruned if it is in the best 2 in fewer than 20% of groups, a representation count if it is best in fewer than 10%. An importance method is pruned if it is in the best 2 in fewer than 20%. An internal evaluator is pruned if it is in the best 3 in fewer than 10%. A vote runs only if the setting has more values than get a point, and it can never remove every value of a setting. A configuration without a score is ranked last in its group. Survivors are scored on the remaining folds. The best configuration is picked by its mean over all five folds, exactly as without pruning.

Some configurations are so far behind after two folds that the other three folds would only confirm a loss. Adaptive pruning stops scoring those. It is judged on one criterion: it must never remove a configuration that would have been the best. Time saved is secondary. Pruning is off unless a search asks for it. As submitted, the search stored once per dataset uses pruning on every dataset, fast and full tier alike. On a fast-tier dataset only one importance method is searched, so the importance vote does not run. Hyperparameter search has a related idea: drop configurations early on partial evaluations, as in successive halving [205] and Hyperband [206]. Here the partial evaluation is two of the five folds.

Table 5.8 lists the five pruning rules. One row is one rule.

The votes are grouped by every other setting, so values compete only with each other. A value that is best for a random forest at 25 features is not removed for losing to a DNN at 500 features. One fold is not enough. In the representation-count vote on breast cancer PAM50 subtype after fold 1 alone, 726 of 1,020 groups had every value at the identical score. Ties are then broken by list order, so a one-fold vote removes values by accident. After one fold, no cut-off of any vote was safe on the calibration datasets. After two folds, each vote had at least one safe cut-off.

The cut-offs were calibrated on five datasets (preliminary): LogD, hERG K_i, breast cancer PAM50 subtype, nanobody thermostability and humanness. Each dataset's grid was first scored unpruned on all five folds. Each rule was then replayed on the stored fold scores, so no rule needed new fits. A cut-off counts as safe if the best five-fold score is unchanged for each of 25 combinations of downstream model and budget, on every dataset. Where several safe cut-offs made the

Table 5.8: The adaptive pruning rules. Every rule ranks configurations by their mean score over folds 1 and 2, in the dataset's own metric. The last column is what the rule removed on the five calibration datasets (preliminary).

Rule	How groups are formed	Kept, or gets a point	Cut-off	Removed on the calibration datasets
Checkpoint at 100 features	configurations at budgets up to 100 features with the same downstream model, internal evaluator and representation count	(allocation, importance) pairs with a configuration in the better half of the group	better half of the configurations	configurations within groups only; never a whole allocation or importance method
Representation count	every other setting fixed, budget included	best 1	10% of groups	the count of 5 on LogD; the count of 2 on breast cancer PAM50 subtype
Allocation method	every other setting fixed, budget included	best 2	20% of groups	softmax on LogD
Importance method	every other setting fixed, budget included	best 2	20% of groups	DeepLIFT on nanobody thermostability and hERG K_i; TreeSHAP on LogD
Internal evaluator	every other setting fixed, budget included	best 3	10% of groups	the SVM evaluator on LogD

same cuts, we chose the one whose neighboring cut-off was also safe. The saving was not the criterion. The chosen rules removed the count of 5 representations on LogD and the count of 2 on PAM50 subtype. They removed softmax allocation on LogD, DeepLIFT on nanobody thermostability and hERG K_i, and TreeSHAP on LogD. They also removed the SVM evaluator on LogD. The checkpoint removed configurations within groups only, never a whole allocation or importance method. TreeSHAP is 71% of LogD's fitting time and was never the best on LogD, so most of the saving comes from there. We estimated run times for eight budgets, unpruned against pruned. They were 79 against 44 hours on LogD, 42 against 35 on hERG K_i and 42 against 30 on PAM50 subtype. On nanobody thermostability they were 32 against 24 hours, and on humanness 15 against 13. The calibration applied the votes one after another. The search applies all four at once, on the same scores from folds 1 and 2. So the cut-offs were checked in a different order from the one the search uses. Five datasets is a small calibration set, and this is a limit of the rules.

Pruning changes what the reported score means. With pruning on, folds 1 and 2 both choose the survivors and score them. The five-fold mean of the winner is then slightly optimistic [195]. Single representations are never pruned. A fair comparison of the hybrid against a single representation therefore leaves both voting folds out of both, or re-scores without them.

5.3 Methods: experimental setup

This section sets out how every experiment in this chapter is run. "Datasets and splitting" describes the 16 datasets from six domains and how their folds are cut. "Representations by domain" lists the base representations offered to the search in each domain. "Models and metrics" gives the downstream models and the metric used on each dataset. "Comparators, controls and ablations" describes the experiments that test whether the hybrid's gain is real. The last subsection sets up three further studies: reusing search settings on a second dataset, antibody mutational scans, and data held at separate sites.

Every experiment reuses one protocol. Each of the 16 datasets is split into five outer folds, and no group is split across folds. One analysis of the antibody expression scan uses three outer folds of unequal size instead; this is stated where that result is reported. The hybrid search sees only the training samples of each outer fold. The selected features are then rebuilt on those training samples and

applied unchanged to the held-out samples.

The hybrid is compared against the best single base representation, used at its full number of features. This asks whether the hybrid beats what a user would otherwise run. The comparator is chosen from the search's own single-representation scores on the training samples of the outer fold. It is then refit on those training samples and scored on the same held-out samples as the hybrid. It is chosen separately for each downstream model and each fold, so it can be a different representation in each fold. Choosing it on the held-out samples would give it an advantage the hybrid does not get. The hybrid is held to its budget of base features, with any augmentation features added on top. The comparator is held to no budget, so it is often many times wider than the hybrid. Both widths are recorded for every fold. A difference between the two is therefore partly a difference in width, not only in method. The main quantity per fold is the hybrid's score minus the best single representation's score. It is signed so that a positive value always means the hybrid scored better; for RMSE the sign is flipped. For each dataset and downstream model, we report the per-fold scores of the hybrid and of every single representation, and their spread across the five folds.

Chapter 6 reuses this protocol with noise added to the training labels of each outer fold. The held-out labels stay clean.

5.3.1 Datasets and splitting

The hybrid experiments use 16 datasets from six domains: five small-molecule ADME and toxicity endpoints, one quantum-chemical property, two reaction-yield sets, four antibody sets, two nucleotide-sequence tasks and two cancer classification tasks from TCGA (Table 5.9). Every score in this chapter is computed on held-out folds that keep each group whole. Each dataset is grouped to match what "unseen" means in its field. The curation of each dataset, with counts before and after each step, is given in Appendix A; the representations each dataset uses are listed in Table 5.16.

Different biological domains have established different conventions for what constitutes a good prediction, and applying a single metric everywhere would misrepresent performance in at least some of them. To this end, we assign each task a primary metric matched to its domain: RMSE for molecular property prediction, Spearman rank correlation for antibody regression and developability, AUROC for

binary classification (including antibody classification and tumor subtype), weighted F1 for multiclass classification, concordance index for drug-target affinity, and R^2 for reaction yield. Different biological domains have established different conventions for what constitutes a good prediction, and applying a single metric everywhere would misrepresent performance in at least some of them. Each dataset's metric is given in Table 5.9, and the reason for each choice is given in "Models and metrics".

No random splits are used. Random splitting inflates performance estimates for molecular data by placing structurally similar compounds on both sides of the train/test boundary [162, 207], and position-level leakage in deep mutational scanning data allows models to memorize site-specific effects rather than learning generalisable sequence-fitness relationships [208, 209]. Splits therefore match domain conventions and use 5-fold GroupKFold throughout. No split is random (Chapter 2, "Evaluation methodology"): regression datasets use five-fold group k-fold, and classification datasets use five-fold stratified group k-fold.

Pre-processing. The steps shared across datasets are listed below. The per-dataset steps and the counts after each one are in Appendix A.

1. Every molecule is parsed with RDKit [32], and molecules that fail are dropped. On hERG K_i, LogD and Caco-2 the structures are also standardized: largest fragment kept, charges neutralized, canonical tautomer, and molecules containing a metal removed.

2. Repeat measurements of one molecule are merged into their median (hERG K_i, LogD, Caco-2). hERG K_i molecules whose repeat measurements have a standard deviation above 1.0 pK_i units are removed. K_i values combined from different sources are a known source of large label noise [16].

3. Duplicate molecules are removed by InChIKey on the six small-molecule datasets, keeping the first.

4. LogD and Caco-2 labels are clipped at the 1st and 99th percentiles of the dataset.

5. Two datasets are subsampled once with a fixed seed: 10,000 of the 129,428 parsable QM9 molecules, and 5,000 of the 524,346 trastuzumab variants, drawn in proportion to each combination of label and number of mutations.

Table 5.9: The 16 datasets used in the hybrid experiments. One row is one dataset after curation. n is the number of samples; the last column gives what the folds are grouped by and the number of groups. ↓: lower is better; ↑: higher is better.

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)
Small molecules: ADME and toxicity				
hERG K_i [110]	pK_i	regression, RMSE ↓	1,412	scaffold (769)
LogD [109]	LogD (log units)	regression, RMSE ↓	5,028	scaffold (2,413)
Caco-2 efflux [109]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,158	scaffold (1,086)
Rat liver microsome clearance [210, 211]	$\log_{10}$ CL_{int} (mL/min/kg)	regression, RMSE ↓	3,053	scaffold (2,625)
MDR1–MDCK efflux [210, 211]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,641	scaffold (2,318)
Quantum chemistry				
QM9 [107, 212]	HOMO–LUMO gap (eV)	regression, RMSE ↓	9,978	scaffold (3,452)
Chemical reactions				
Buchwald–Hartwig yield [213]	yield (%)	regression, RMSE ↓	3,955	additive (22)
Buchwald–Hartwig plates [213]	yield (%)	regression, RMSE ↓	4,312	additive (24)
Antibodies				
Koenig expression [214, 215]	expression enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position modulo 5 (5)
Nanobody thermostability [215, 216]	melting temperature (°C)	regression, Spearman ↑	672	95% identity cluster (309)
Humanness [217]	human or not (198 : 366)	binary, AUROC ↑	564	95% identity cluster (461)
Trastuzumab–HER2 binding [218]	binder or not (1,639 : 3,361)	binary, AUROC ↑	5,000	mutations from wild type (10)
Nucleotide sequences				
H3K4me3 [219]	mark present or not (9,122 : 9,122)	binary, MCC ↑	18,244	chromosome (23)
TATA promoter [219]	promoter or not (2,637 : 2,637)	binary, MCC ↑	5,274	chromosome (23)
Cancer genomics (TCGA)				
Lung adenocarcinoma vs squamous [220–222]	tumor type (515 : 502)	binary, AUROC ↑	1,017	tissue source site (68)
Breast cancer PAM50 subtype [222–224]	subtype (421 : 192 : 141 : 67)	4 classes, weighted F1 ↑	821	tissue source site (19)

Table 5.10: The 16 datasets used in the hybrid experiments. One row is one dataset after curation. n is the number of samples; the fifth column gives what the folds are grouped by and the number of groups. The last column gives the best single base representation and its mean score over the five outer folds, in the dataset's own metric. ↓: lower is better; ↑: higher is better.

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)	Best single representation (score)
Small molecules: ADME and toxicity					
hERG K_i [110]	pK_i	regression, RMSE ↓	1,412	scaffold (769)	**[RESULT NEEDED: name, score]**
LogD [109]	LogD (log units)	regression, RMSE ↓	5,028	scaffold (2,413)	**[RESULT NEEDED: name, score]**
Caco-2 efflux [109]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,158	scaffold (1,086)	**[RESULT NEEDED: name, score]**
Rat liver microsome clearance [210, 211]	$\log_{10}$ CL_{int} (mL/min/kg)	regression, RMSE ↓	3,053	scaffold (2,625)	**[RESULT NEEDED: name, score]**
MDR1–MDCK efflux [210, 211]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,641	scaffold (2,318)	**[RESULT NEEDED: name, score]**
Quantum chemistry					
QM9 [107, 212]	HOMO–LUMO gap (eV)	regression, RMSE ↓	9,978	scaffold (3,452)	**[RESULT NEEDED: name, score]**
Chemical reactions					
Buchwald–Hartwig yield [213]	yield (%)	regression, RMSE ↓	3,955	additive (22)	**[RESULT NEEDED: name, score]**
Buchwald–Hartwig plates [213]	yield (%)	regression, RMSE ↓	4,312	additive (24)	**[RESULT NEEDED: name, score]**
Antibodies					
Koenig expression [214, 215]	expression enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position modulo 5 (5)	**[RESULT NEEDED: name, score]**
Nanobody thermostability [215, 216]	melting temperature (°C)	regression, Spearman ↑	672	95% identity cluster (309)	**[RESULT NEEDED: name, score]**
Humanness [217]	human or not (198 : 366)	binary, AUROC ↑	564	95% identity cluster (461)	**[RESULT NEEDED: name, score]**
Trastuzumab–HER2 binding [218]	binder or not (1,639 : 3,361)	binary, AUROC ↑	5,000	mutations from wild type (10)	**[RESULT NEEDED: name, score]**
Nucleotide sequences					
H3K4me3 [219]	mark present or not (9,122 : 9,122)	binary, MCC ↑	18,244	chromosome (23)	**[RESULT NEEDED: name, score]**
TATA promoter [219]	promoter or not (2,637 : 2,637)	binary, MCC ↑	5,274	chromosome (23)	**[RESULT NEEDED: name, score]**
Cancer genomics (TCGA)					
Lung adenocarcinoma vs squa-	tumor type (515 : 502)	binary, AUROC ↑	1,017	tissue source site (68)	**[RESULT NEEDED:**

Splitting rules. Four rules hold for every dataset.

1. Folds are formed as in Chapter 2, "Evaluation methodology": no group is split, regression datasets use group k-fold, and classification datasets use stratified group k-fold.

2. There are five outer folds, and the stratified folds use a fixed seed.

3. No split is ever random. What happens when a dataset has fewer groups than folds is stated in Chapter 2, "Evaluation methodology". Every dataset here has at least five groups, so all outer splits have five folds.

4. On a given dataset, every representation, model and hybrid is scored on the same five folds.

The hybrid construction step uses a nested internal train/validation split within each training fold to score representations for feature allocation. This internal split is group-aware, taking the first fold of a grouped 5-fold split (GroupKFold) with the same grouping variable as the outer cross-validation, to prevent the allocation evaluator from seeing related samples on both sides of its split. Inside each outer training fold, the search scores representations on one group-aware internal split, described in "From base representations to a hybrid".

Groups by domain. The small-molecule and QM9 datasets are grouped by Bemis–Murcko scaffold, defined in Chapter 2, "Evaluation methodology". In this chapter the scaffold keeps stereochemistry, and all acyclic molecules share one empty scaffold. On QM9 this acyclic group holds 1,041 of the 9,978 molecules and is the largest group. On the two Biogen datasets (rat liver microsome clearance and MDR1–MDCK efflux) there are 1.2 and 1.1 molecules per scaffold (the n and group columns of Table 5.9). On the Biogen datasets, holding out a scaffold is therefore close to holding out single molecules. Chapter 3 used a different scaffold key and different curation for its versions of LogD, Caco-2, hERG K_i and QM9; Appendix A lists the differences.

The reaction datasets are Buchwald–Hartwig C–N couplings run on high-throughput plates, in which aryl halide, ligand, base and additive are varied. They are grouped by additive, following the out-of-sample tests of Ahneman et al. [213], so each held-out fold contains four or five additives the model never saw in training. The two sets are not independent: they share 3,955 reactions (Appendix A).

On the Koenig deep mutational scan of the anti-VEGF antibody G6, every variant carries one mutation. Variants are grouped by the position of that mutation modulo 5, as in ProteinGym [208], so each fold is one residue class and holds 836 to 874 variants. All mutations at one position then fall in the same fold, and a model cannot learn a per-position effect and look it up at test time [209]. The nanobody and humanness sets are grouped by single-linkage clusters at 95% sequence identity, after Kenlay et al. [159]. The identity measure, the cluster sizes and the humanness labels are given in Appendix A. The trastuzumab variants differ from the wild type only in the ten residues of CDR-H3, and are grouped by how many of those residues are mutated. A held-out fold therefore contains variants at mutation counts not seen in training.

The two nucleotide tasks come from the revised Nucleotide Transformer benchmark [219]. Both are grouped by chromosome, so a held-out fold contains whole chromosomes the model never saw in training: between two and six of the 23 chromosome groups (Appendix A gives the task definitions and how the released files are regrouped).

The TCGA datasets use RNA-seq expression and other measurements of primary tumors, one sample per patient, obtained through UCSC Xena [222]. Samples are grouped by tissue source site, the institution that supplied the sample, which is read from the TCGA barcode. A model therefore cannot score well by recognizing the site a sample came from, and no patient spans training and test.

Consequences of the grouping. Group definitions differ between domains on purpose; the one shared rule is that no group spans training and test. Some groupings leave few or very unequal groups, and the folds inherit that. The trastuzumab held-out folds range from 13 to 3,035 variants, with binder shares from 15.5% to 92.3% against 32.8% overall. The breast cancer folds range from 3 to 403 tumors. None of the 68 lung sites supplied both tumor types, so the lung folds are unbalanced: one holds 20 adenocarcinomas and 113 squamous tumors. The Koenig folds are fixed by its five groups. Per-fold scores on these datasets spread more, so results are read per dataset, in the dataset's own metric, and never pooled across datasets.

5.3.2 Representations by domain

We combine a variety of component, or base representations to create these hybrids.

Any representation that gives one fixed-length vector per sample can serve as a base representation, as set out in the description of how the hybrid is built earlier in this chapter. The first version of the method used three molecular representations: PDV, ECFP4 and MHG-GNN. These three are now one small-molecule example among many. Every base representation is frozen: it is computed once from the raw input (a SMILES string, a sequence, a reaction or a patient profile) and is never trained on the labels.

For every data set in this study, we pre-selected ten base representations that spanned across different domains of information and showed some positive results when used on their own for the given prediction task. For every dataset in this study, we pre-selected at most ten base representations. They span different domains of information, and each showed some positive results on its own for the given prediction task.

To choose them, every candidate representation was first scored on its own with a gradient-boosted tree model [57] under the dataset's grouped folds. Candidates were then compared by the Spearman correlation between their out-of-fold predictions, since two representations whose predictions correlate highly carry much the same signal. At most ten were then chosen by hand, looking at each candidate's own score next to these correlations and favoring representations that encode the input in different ways. Two nucleotide datasets have fewer. H3K4me3 had 12 candidates, and four of them predicted almost the same as a better-scoring candidate, with a Spearman correlation of 0.95 or above between their out-of-fold predictions; eight remain. TATA promoter had 10 candidates, one was dropped by the same rule, and nine remain. Augmentation features do not count toward the ten. They are added after the hybrid is built (see the Augmentation subsection above), and the same representation can be a base representation on one dataset and augmentation on another. On each dataset, the two base representations whose out-of-fold predictions correlate most are **[RESULT NEEDED: most alike pair, with its Spearman correlation]**. The two that correlate least are **[RESULT NEEDED: least alike pair, with its Spearman correlation]**.

If an input cannot be parsed or embedded, it gets an all-zero row, so that every representation of a dataset keeps the same rows. In computed descriptors, a missing or infinite value is set to 0, and values are stored unscaled. A pretrained model gives one vector per input, the mean of its last-layer token vectors with padding excluded, unless the tables say otherwise. Scaling and filling of missing values happen inside

each fold, on the training rows only, as described in the construction of the hybrid.

Small molecules. The small-molecule representations (Table 5.11) are the fingerprint, descriptor, graph and language-model families described in Chapter 2. The only additions are four sets of 3D shape and pharmacophore features, used as augmentation only. All four are computed from a single conformer per molecule, embedded with a fixed seed and optimized with the MMFF force field. A single conformer may misdescribe flexible molecules.

Chemical reactions. Reaction datasets (Table 5.12) use three kinds of representation. The condensed graph of reaction merges reactants and products into one graph in which the bonds that change are marked [225]; here it is hashed into a circular fingerprint after atom mapping [226]. Reaction language models read the reaction SMILES directly [227, 228]. Any molecular representation can also be applied to the reactants, to the product, or to their difference. A one-hot encoding of which aryl halide, ligand, base and additive was used carries no chemistry. It is kept because on plate data such identity features can predict yield about as well as chemical descriptors [229].

Antibodies. Antibody representations (Table 5.13) start from the heavy- and light-chain amino-acid sequences. Besides one-hot and BLOSUM62 encodings [230], most are frozen language models, either trained on all proteins or on antibody repertoires only. Most models embed each chain separately and join the two vectors, so their width doubles on datasets with both chains. Several representations read one model in different ways, for instance averaging over earlier layers or over the CDR residues only, with CDRs found by IMGT numbering [231] using ANARCI [232]. Datasets of point variants of one parent antibody also use features that describe each mutation.

Nucleotide sequences. A k-mer frequency vector counts how often each DNA word of length k occurs in a sequence, divided by the total number of words [233]; the canonical variant merges each word with its reverse complement [234]. DNA language models are networks pretrained on genomes to predict masked or next tokens [219, 235, 236]. Table 5.14 lists both kinds, along with the small sequence summaries used as augmentation.

Cancer patients. For the two TCGA datasets (Table 5.15), each representation is a different measurement of the same patient's tumor, not a different encoding of one input. The measurements are RNA-seq gene expression (read through foundation models or summarized per chromosome arm), DNA methylation on the HM450 array, gene-level copy number, microRNA abundance, protein levels from reverse-phase protein arrays, and embeddings of the diagnostic whole-slide image. Data were obtained from TCGA through UCSC Xena [222, 237]. The most-variable selections, the z-scoring and the principal components for methylation, copy number, microRNA and protein were fitted once on all patients in a cohort, held-out patients included. No labels are used in these steps. A patient with no measurement for one data type gets an all-zero row for that representation.

We list all of these representations alongside their associated data sets and prediction tasks below:

Table 5.11: Representations used for small molecules. One row is one representation. Role: B = base representation, A = augmentation, B/A = base on some datasets and augmentation on others.

Representation	What it encodes	Features	Source	Role
ECFP4	hashed circular substructures, radius 2	2048	[4]	B
Atom pairs	hashed atom-type pairs with bond distance	2048	[238]	B
Topological torsions	hashed paths of four bonded atoms	2048	[239]	B/A
Avalon	hashed paths, rings and atom environments	2048	[240]	B
MACCS keys	166 predefined substructure keys	167	[241]	B/A
WL graph kernel	Weisfeiler–Lehman label counts, 5 rounds, labels present in at least 5% of molecules	140–317	[242]	B
PDV	RDKit 2D physicochemical descriptors	200	[32]	B
Mordred	2D molecular descriptors	1613	[243]	B

Representation	What it encodes	Features	Source	Role
GROVER	frozen pretrained graph transformer	4800	[244]	B
MHG-GNN	frozen pretrained graph encoder	1024	[8]	B
GraphMVP	frozen 2D graph encoder pretrained with 3D views	300	[245]	B
MoLFormer	frozen SMILES language model; public checkpoint trained on 10% of its corpus	768	[40]	B
SMI-TED	frozen SMILES encoder–decoder	768	[246]	B
ChemBERT	frozen SMILES language model; community checkpoint trained on ChEMBL following the cited method	256	[247]	B
ChemBERTa-2	frozen SMILES language model	384	[28]	B
SELFormer	frozen SELFIES language model	768	[248]	B
Mol2vec	summed substructure word vectors	300	[25]	B
3D pharmacophore	counts of 7 pharmacophore types, and mean and spread of distances for the 21 type pairs, one conformer	49	[32]	A
USR	shape moments from 4 reference points, one conformer	12	[249]	A
USRCAT	USR per pharmacophore atom type, one conformer	60	[250]	A
Coulomb matrix	sorted eigenvalues of the Coulomb matrix, one conformer	50	[251]	A

Table 5.12: Representations used for chemical reactions. One row is one representation. Role: B = base representation, A = augmentation, B/A = base on some datasets and augmentation on others.

Representation	What it encodes	Features	Source	Role
CGR fingerprint	circular fingerprint, radius 2, hashed from the condensed graph of reaction	2048	[225]	B
rxnfp	frozen reaction language model	256	[227]	B
ReactionT5	frozen reaction language model	768	[228]	B
Component one-hot	which aryl halide, ligand, base and additive	45	[213]	B
GROVER (reactants)	GROVER averaged over reactants	4800	[244]	B
Mordred (reactants)	Mordred averaged over reactants	1613	[243]	B
Mol2vec (reactants)	Mol2vec averaged over reactants	300	[25]	B
ECFP4 (reactants)	ECFP4, bitwise OR over reactants	2048	[4]	B
Avalon (difference)	product minus reactant Avalon bits	2048	[240]	B
MHG-GNN (difference)	product minus reactant embedding	1024	[8]	B
MACCS (difference)	product minus reactant MACCS keys	167	[241]	B
Mol2vec (difference)	product minus reactant Mol2vec	300	[25]	B
MACCS (reactants)	MACCS keys, bitwise OR over reactants	167	[241]	A
MACCS (product)	MACCS keys of the product	167	[241]	A
3D pharmacophore (reactants)	3D pharmacophore averaged over reactants	49	[32]	A

Representation	What it encodes	Features	Source	Role
3D pharmacophore (product)	3D pharmacophore of the product	49	[32]	A

Table 5.13: Representations used for antibodies. One row is one representation. LM = language model. Widths marked /chain double on datasets with heavy and light chains (Koenig expression, Trastuzumab–HER2) and are single on the nanobody and heavy-chain-only humanness datasets. – in Source: a standard encoding or one built for this thesis. Role: B = base representation, A = augmentation, B/A = base on some datasets and augmentation on others.

Representation	What it encodes	Features	Source	Role
One-hot	one-hot residues, both chains padded to fixed length	5600	–	B
BLOSUM62	BLOSUM62 row per residue, both chains padded	5600	[230]	B
Amino-acid composition	fraction of each amino acid, per chain	40	–	A
ESM-2	protein LM, last layer, mean over residues	1280/chain	[165]	B
ESM-2 (joined chains)	ESM-2 on heavy and light chain joined into one string	1280	[165]	B
ESM-2 early layers	ESM-2, mean over layers 0–16	1280/chain	[165]	B
ESM-2 middle layers	ESM-2, mean over layers 20–33	1280/chain	[165]	B
ESM-2 CDR	ESM-2, mean over CDR residues only	1280/chain	[165]	B
ESM-2 per CDR	ESM-2, CDR1/2/3 pooled separately then joined	3840/chain	[165]	B
ESM-2 variable positions	ESM-2, mean over the 20 highest-entropy positions	1280/chain	[165]	B
ESM-2 shift from parent	ESM-2 variant minus parent embedding	1280/chain	[165]	B

Representation	What it encodes	Features	Source	Role
ESM-2 log-likelihood ratio	masked mutant vs parent log-probability: sum, mean, min	3	[252]	A
ESM3	protein LM	1536/chain	[253]	B
ESM3 CDR	ESM3, mean over CDR residues	1536/chain	[253]	B
ESM C	protein LM	1152/chain	[254]	B
ProtT5	protein LM	1024/chain	[166]	B
ProtT5 CDR	ProtT5, mean over CDR residues	1024/chain	[166]	B
SaProt	structure-aware protein LM	1280/chain	[255]	B
AbLang-2	paired antibody LM, both chains read together	480	[256]	B
IgT5	paired antibody LM, both chains read together	1024	[257]	B
AntiBERTy	antibody LM	512/chain	[258]	B
AntiBERTy CDR	AntiBERTy, mean over CDR residues	512/chain	[258]	B
Ab-RoBERTa	antibody LM	768/chain	[259]	B
Ab-RoBERTa CDR	Ab-RoBERTa, mean over CDR residues	768/chain	[259]	B
Mutation features	wild-type and mutant residue, substitution score, changes in hydrophobicity, weight and charge, position; averaged over mutations	48	–	B/A

Table 5.14: Representations used for nucleotide sequences. One row is one representation. LM = language model. – in Source: a simple summary built for this thesis. Role: B = base representation, A = augmentation, B/A = base on some datasets and augmentation on others.

Representation	What it encodes	Features	Source	Role
4-mer counts	normalized counts of every 4-letter word	256	[233]	B
5-mer counts	normalized counts of every 5-letter word	1024	[233]	B
6-mer counts	normalized counts of every 6-letter word	4096	[233]	B
Canonical TNF	4-mer counts merged with reverse complements	136	[234]	B
DNABERT-2	DNA LM	768	[235]	B
Nucleotide Transformer v2 250M	DNA LM	768	[219]	B
Nucleotide Transformer v2 500M	DNA LM	1024	[219]	B
HyenaDNA	long-context DNA LM	256	[236]	B
GROVER (DNA)	DNA LM (not the molecular GROVER)	768	[260]	B
Mistral-DNA	mixture-of-experts DNA LM	768	[261]	B
GC content	GC fraction, GC and AT skew, mean and spread of GC over 100-base windows	5	–	A
Dinucleotide frequency	frequency of each 2-letter word	16	[233]	A
Codon usage	relative usage of each of the 61 sense codons, read in frame from the first base	61	[262]	A
Sequence complexity	entropy of 1-, 2- and 3-letter words, linguistic complexity, fraction of alphabet used	5	–	A

Representation	What it encodes	Features	Source	Role
DNA shape	minor groove width, propeller twist, helix twist and roll from a dinucleotide lookup table; mean and spread	8	–	A

Table 5.15: Representations used for cancer patients (TCGA). One row is one representation. – in Source: computed for this thesis from the patient's own data. Role: B = base representation, A = augmentation, B/A = base on some datasets and augmentation on others.

Representation	What it encodes	Features	Source	Role
Geneformer (2,048 genes)	single-cell foundation model (V2-316M) on the 2,048 top-ranked genes, [CLS] vector	1152	[263, 264]	B
Geneformer (4,096 genes)	the same model on the 4,096 top-ranked genes, [CLS] vector	1152	[263, 264]	B
BulkRNABert	bulk RNA-seq foundation model	256	[265]	B
Top-variance methylation probes	beta values of the 5,000 most variable CpG probes	5000	[222]	B
Methylation PCA	50 principal components of the standardized CpG probes above median variance	50	[222]	B
Copy number PCA	50 principal components of gene-level copy number	50	[266]	B
miRNA	200 most variable miRNAs, z-scored	200	[222]	B
Protein (RPPA)	reverse-phase protein array, z-scored	258	[267]	B
Chromosome-arm expression	mean expression of the genes on each chromosome arm	42	–	B/A

Representation	What it encodes	Features	Source	Role
TITAN	whole-slide image foundation model	768	[268]	B
Prov-GigaPath	whole-slide image foundation model	512	[269]	B
CTransPath	histopathology image encoder	512	[270]	B
Phikon	histopathology image encoder	512	[271]	B
H-optimus-0	histopathology image encoder	512	[272]	B
HRD score	homologous recombination deficiency: LOH, TAI, LST and their sum	4	[273]	A
Tumor mutation burden	non-silent, missense and truncating mutation counts, log count, rate per megabase	5	[274]	A
Tumor purity	purity, ploidy, genome doublings, subclonal genome fraction	4	[275]	A
Expression statistics	summaries of the patient's expression distribution	15	–	A
Immune signatures	10 CIBERSORT cell fractions and Th1 and Th2 signature scores	12	[276]	A
CIBERSORT	immune cell fractions deconvolved from bulk expression	21	[276, 277]	A
Stemness index	stemness indices from expression and from epigenetically regulated expression	2	[278]	A
Hypermethylation summary	fraction of all methylation probes with beta above 0.3, and mean beta; a genome-wide stand-in for CIMP status [279]	2	–	A

Table 5.16: Datasets and the representations each one uses. One row is one dataset. Base representations are the (up to) ten the hybrid search selects features from; augmentation features are added afterward. Datasets marked † are searched at the fast tier (Table 5.7). Sample counts, metrics and folds are given in the Datasets and splitting subsection.

Dataset	Task	Base representations	Augmentation
Small molecules			
Caco-2 efflux [109]	efflux ratio (log), regression	WL graph kernel, MHG-GNN, ECFP4, PDV, GROVER, Avalon, Topological torsions, SMI-TED, Atom pairs, MoLFormer	MACCS keys, 3D pharmacophore, USRCAT, Coulomb matrix, USR
LogD† [109]	LogD, regression	MHG-GNN, Mordred, GROVER, ECFP4, SMI-TED, MoLFormer, Avalon, Topological torsions, ChemBERT, GraphMVP	MACCS keys, 3D pharmacophore, USRCAT
hERG Ki [110]	hERG inhibition, pKi, regression	MHG-GNN, Mordred, WL graph kernel, Avalon, GROVER, PDV, ECFP4, Atom pairs, ChemBERT, SMI-TED	MACCS keys, 3D pharmacophore, Coulomb matrix, USR, USRCAT
MDR1-MDCK efflux [210]	efflux ratio (log), regression	Mordred, PDV, MHG-GNN, Avalon, GROVER, SMI-TED, Atom pairs, WL graph kernel, MACCS keys, ECFP4	3D pharmacophore, Coulomb matrix
Rat liver microsome clearance [210]	intrinsic clearance (log), regression	PDV, MHG-GNN, SMI-TED, Mordred, MoLFormer, GROVER, WL graph kernel, Atom pairs, Avalon, ECFP4	3D pharmacophore, MACCS keys, USRCAT, Coulomb matrix, USR

Dataset	Task	Base representations	Augmentation
QM9† [107]	HOMO–LUMO gap, regression	Mordred, Avalon, SMI-TED, ECFP4, SELFormer, ChemBERTa-2, Mol2vec, ChemBERT, GraphMVP, MoLFormer	Topological torsions, 3D pharmacophore, USRCAT, Coulomb matrix
Chemical reactions			
Buchwald–Hartwig yield [213]	yield (%), regression	CGR fingerprint, GROVER (reactants), Avalon (difference), Mordred (reactants), MHG-GNN (difference), ReactionT5, Component one-hot, Mol2vec (reactants), rxnfp, MACCS (difference)	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)
Buchwald–Hartwig plates [213]	yield (%), regression	ECFP4 (reactants), GROVER (reactants), MHG-GNN (difference), CGR fingerprint, Component one-hot, Avalon (difference), ReactionT5, Mol2vec (difference), Mordred (reactants), rxnfp	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)
Antibodies			
Koenig expression† [214]	expression enrichment of variants, regression	ProtT5, ESM3 CDR, SaProt, IgT5, ESM-2 early layers, ESM C, AbLang-2, ESM-2 (joined chains), ESM-2, ESM-2 middle layers	Mutation features, ESM-2 log-likelihood ratio, Amino-acid composition
Nanobody thermostability [216]	melting temperature, regression	SaProt, ESM3, ESM C, ESM-2 variable positions, ESM-2 CDR, ESM-2 early layers, ESM-2 shift from parent, ProtT5 CDR, IgT5, One-hot	Mutation features

Dataset	Task	Base representations	Augmentation
Humanness [217]	human vs non-human, binary	IgT5, AntiBERTy CDR, AbRoBERTa, AbLang-2, BLOSUM62, ESM C, ESM-2 per CDR, ESM-2 early layers, ProtT5, ESM-2 middle layers	none
Trastuzumab–HER2 binding† [218]	binder vs non-binder, binary	BLOSUM62, ESM-2 per CDR, ESM-2 middle layers, ProtT5, ESM3, ESM C, AbRoBERTa CDR, AntiBERTy, Mutation features, AbLang-2	none
Nucleotide sequences			
H3K4me3† [219]	histone mark present vs absent, binary	GROVER (DNA), Mistral-DNA, Canonical TNF, Nucleotide Transformer v2 250M, Nucleotide Transformer v2 500M, HyenaDNA, 6-mer counts, DNABERT-2	Codon usage, Sequence complexity, Dinucleotide frequency, DNA shape, GC content
TATA promoter† [219]	promoter vs non-promoter, binary	Mistral-DNA, GROVER (DNA), HyenaDNA, 6-mer counts, 5-mer counts, Canonical TNF, 4-mer counts, Nucleotide Transformer v2 500M, Nucleotide Transformer v2 250M	Codon usage, Sequence complexity, Dinucleotide frequency, DNA shape, GC content
Cancer patients (TCGA)			
BRCA PAM50 subtype [223]	breast cancer subtype, 4 classes	Geneformer (4,096 genes), BulkRNABert, Protein (RPPA), Copy number PCA, TITAN, H-optimus-0, Prov-GigaPath, CTransPath, Phikon, miRNA	HRD score, Tumor mutation burden, Tumor purity, Expression statistics, Chromosome-arm expression, Immune signatures

Dataset	Task	Base representations	Augmentation
LUAD vs LUSC [220]	lung cancer type, binary	Geneformer (2,048 genes), Top-variance methylation probes, BulkRNABert, TITAN, Methylation PCA, Chromosome-arm expression, Copy number PCA, miRNA, CTransPath, Protein (RPPA)	Immune signatures, Expression statistics, Stemness index, CIBERSORT, Hypermethylation summary, Tumor mutation burden, HRD score, Tumor purity

5.3.3 Models and metrics

For a fair comparison, the downstream model pool must cover a range of inductive biases, from methods that degrade in high dimensions (kNN, SVM, GP) to tree ensembles that tolerate irrelevant features. Eleven models are available: tree ensembles (random forest, XGBoost, LightGBM, CatBoost, NGBoost), kernel methods (SVM, Gaussian process), nearest-neighbor (kNN), and neural networks (three-layer DNN, MC-Dropout BNN for uncertainty quantification, and a multi-scale 1D CNN). Standard experiments use five of these: random forest, XGBoost, LightGBM, SVM and DNN, with a Gaussian process added on datasets under 1,000 molecules. We use the full set for model-matching and noise experiments, and skip models with cubic or higher scaling for large datasets: Gaussian processes above 1,000 samples, SVMs above 10,000. No per-dataset tuning is performed: all models use fixed hyperparameters throughout, so performance differences reflect the input, not the model configuration. For a fair comparison, the downstream models must cover a range of inductive biases. They run from methods that degrade in high dimensions (SVM, GP) to tree ensembles that tolerate irrelevant features. Standard experiments use five downstream models: random forest, XGBoost, LightGBM, SVM and DNN, with a Gaussian process added on humanness. An experiment that uses a model outside these five names it in its own methods. We skip the two models whose fitting cost grows fastest with the number of molecules on large datasets: Gaussian processes above 1,000 samples, SVMs above 10,000. No per-dataset tuning is performed: all

models use fixed hyperparameters throughout, so performance differences reflect the input, not the model configuration.

The five downstream models cover bagged trees (random forest), boosted trees (XGBoost and LightGBM), a kernel method (SVM) and a neural network (DNN). We use five because the best set of features depends on the model that uses it. A hybrid that helped only one model family would be a property of that model, not a better input. Tree ensembles are the baseline to beat. A hybrid is a tabular feature matrix, and tree-based models still lead on medium-sized tabular data of about 10,000 samples [280]. A Gaussian process [281] is added as a sixth model on datasets under 1,000 molecules. Three of the 16 datasets are that small: humanness (564 sequences), nanobody thermostability (672 sequences) and breast cancer PAM50 subtype (821 tumors). The Gaussian process runs on humanness only. Humanness is a classification task, so the Gaussian process there has an RBF kernel. The regression version, with an RBF kernel plus a white-noise term, runs on none of the 16 datasets. PAM50 subtype is left out under every augmentation setting for run time. In one search, the one with augmentation features added on top of the budget, the Gaussian process took 44 of its 60 hours. Nanobody thermostability is left out under augmentation on top of the budget, where its searches did not finish in the time limit. A dataset that drops the Gaussian process under any setting gets it under none, so nanobody thermostability is left out everywhere.

Every model uses the same settings on every dataset, so differences between scores come from the input. There is no per-dataset tuning by default. An optional per-fold random search exists. It tries 20 draws of a model's settings, scored in the dataset's own metric on 3 group-aware inner folds of the outer training fold only. The DNN is never tuned this way; it relies on early stopping. **[RESULT NEEDED: whether tuning pays on base representations, tuned against untuned models, in each dataset's metric]**. Two models are skipped above a cap on the number of molecules. The Gaussian process is skipped above 1,000 molecules, because it builds and factorizes the full molecule-by-molecule kernel. The SVM is skipped above 10,000 molecules. Above that, its kernel cache no longer holds the support vectors, and a fit stops finishing in useful time. Under augmentation on top of the budget, the SVM is skipped above 5,000 molecules. The caps count the molecules the search is given. The search run once per dataset is given the whole dataset. A search inside one outer fold is given that fold's training molecules, about 80% of the dataset. On two datasets the two counts give different downstream

models, both under augmentation on top of the budget. LogD has 5,028 molecules and TATA promoter 5,274. The SVM is dropped when the whole dataset is counted, and kept when about 4,000 training molecules are counted. Some default models are also not run on named datasets where a single fit was too slow. They are listed in Additional file 13. Every downstream model's settings, library, molecule cap and citation are in Table S.12 (Additional files).

k-nearest neighbors is not a default downstream model. Where it is used, its default is $k = 5$ neighbors, lowered to the square root of the number of training molecules when a fold has fewer than 25. It is the model in the comparison of importance-weighted against unweighted distances, which reads 5, 10 and 20 neighbors. We state its settings once, here, because Chapter 7, "Simple is often better", relies on that result.

Each dataset is scored in the metric its field normally uses, so its numbers can be compared with published work on the same dataset. Small-molecule, quantum-chemistry and reaction-yield regression use RMSE, which is in the label's own units. Antibody regression uses Spearman ρ. These assays are used to rank candidates, and their scales differ between labs. Binary classification uses AUROC, which does not depend on a decision threshold. The two nucleotide sets are the exception: they use MCC. MCC is high only when all four entries of the confusion matrix are good [282]. The four PAM50 subtypes, whose class sizes run from 67 to 421 tumors, use support-weighted F1. Table 5.9 gives the metric of each dataset. The definitions are in Chapter 2, "Evaluation methodology". Metrics are never averaged across datasets or domains.

Every score is computed on an outer test fold the search never saw. We report the per-fold scores and their spread. Each hybrid is paired by fold with the best single representation on the same fold. Each comparison is reported as its per-fold differences and their spread across the 5 folds, with no threshold and no test. We use no significance tests and no confidence intervals, within or across datasets. With 5 folds, a p-value or an interval would rest on 5 correlated values. The fold scores are correlated because the folds share training molecules. No unbiased estimator of the variance of k-fold cross-validation exists, and estimates that ignore the overlap underestimate it [283].

A full search takes **[RESULT NEEDED: hours per dataset for the full search]**. The searches ran on **[RESULT NEEDED: hardware, as CPU cores, memory, and a GPU where the DNN or the embeddings used one]**.

5.3.4 Comparators, controls and ablations

The hybrid picks the best of many configurations: an allocation method, an importance method, an internal evaluator and a budget. A reader will ask whether its gain is only the reward for searching hard. The experiments in this subsection answer that, one question each. The first five ask whether the search overfits, whether a larger search helps, and whether fusing features beats fusing predictions. They also compare the hybrid with standard feature reduction, and ask whether the internal evaluator must match the final model. The next three measure how stable the selected features are across folds, how the gain changes with dataset size, and whether importance helps during training. The last ones add a few features to one representation, test robustness to label noise, and join two whole representations. All use the group folds of "Datasets and splitting".

The hybrid's score is the best of many configurations, while each single representation gives one score. The best of many cross-validated scores is biased upward [194]. Several experiments therefore use leave-one-fold-out reselection. For held-out fold f, we pick the hybrid configuration with the best mean score on the other folds,

$$c^*_{-f} = \arg\max_c \frac{1}{F-1} \sum_{g \neq f} s_{c,g},$$

where $s_{c,g}$ is the score of configuration c on fold g and F is the number of folds (arg min for RMSE). We pick the best single representation, k^*_{-f}, the same way. We then report

$$\Delta_f = s_{c^*_{-f},f} - s_{k^*_{-f},f},$$

signed so that $\Delta_f > 0$ means the hybrid is better; for RMSE the sign is flipped. This is arithmetic on stored per-fold scores, so it needs no extra model fits. Importance scores and the feature selection are always recomputed on each fold's training molecules only. Some experiments choose the configuration by a search over all molecules, and redo only the feature selection per fold. Their scores are not fully held out, and we say so where it applies.

The first experiment asks whether the search overfits as the budget grows. It uses nested cross-validation. For every budget B, that budget's winning configuration is rebuilt on the outer training molecules and scored on the outer held-out fold. The selection bias at a budget is the score the inner search reported minus the held-out score. It is signed so that a positive value means the search promised more than

the held-out fold gave. The budgets are the standard list, 10 to 2,000 features, plus 4,000 and 8,000 features. We add budgets at 0.5, 0.75, 1, 1.25, 1.5 and 2 times the smallest outer-fold training-set size, in molecules. These place a budget where the number of features equals the number of molecules. For the five downstream models, the budget does not set the number of fitted parameters. The DNN and the random forest have more fitted parameters than training molecules at every budget. The SVM is fitted with at most one coefficient per training molecule, so it never can. The capacity of the boosted trees is set by the number and depth of their trees. For these five, the curves therefore say whether the search's choice of budget overfits. This experiment adds one model outside the five, a linear model with almost no penalty. It is ridge regression with penalty 10^{-6}, or logistic regression with $C = 10^{6}$ on classification. It has one weight per selected feature, so its parameter count equals the budget. We report two numbers per model: the budget each outer fold's search picked, and the budget with the best held-out score. A pick far above the best held-out budget means the search overfits. **[RESULT NEEDED: held-out score and selection bias against budget, per model and dataset]**.

The size of a search is the number of configurations it scores: budgets times allocation methods, importance methods, internal evaluators and representation counts. At the full tier that is 480 configurations for each pair of downstream model and internal evaluator, and at the fast tier 48 (Additional file 14). The question is whether the hybrid's lead over each comparator grows with the number of configurations. The comparators are the ensemble, the best single representation, PCA, and feature reduction at the same width. They get no search. So part of any gap that grows with search size is the hybrid taking the best of more draws. Leave-one-fold-out reselection charges the hybrid for this, and we report the reselected numbers. In the full runs each dataset is searched at one tier only, full or fast. A search-size comparison therefore needs the same dataset searched at two sizes. **[RESULT NEEDED: hybrid minus each comparator, per dataset, at each search size]**.

The ensemble fits one model per representation and pools their predictions. The hybrid fits one model on one fused feature vector. Joining features before one model is early fusion; pooling the predictions of one model per feature set is late fusion [149]. The hybrid runs its own search on each outer fold's training molecules. Every ensemble choice is made on an inner 3-fold group split of the same training molecules, and the held-out fold is used once. The ensemble either fits the same model on every

representation, or lets each representation use its own best model. Its members are either the top 3 representations, or every representation with at least 50 features. The pooling is one of seven: the mean, the median, a 20% trimmed mean, and two means after removing outliers. The outliers are found by interquartile range or by median absolute deviation. The last two are a mean weighted by each member's inner score, and the mean of the top 3 members. Classification ensembles pool predicted probabilities. Where the metric reads class labels, a majority vote is also offered. There are three controls. The first is the best single representation. The second joins all representations and reduces them by PCA to the hybrid's width. The third is the best pair of model and representation, read off the held-out fold. It is an upper bound, not a fair comparator. Beside each score we report the cost: representations computed at prediction time, models deployed, and fit and prediction time in seconds. The hybrid only saves cost if it draws on fewer representations than the ensemble. An ensemble is the obvious simpler alternative. If it matches the hybrid at lower cost, that result belongs in Chapter 7, "Simple is often better". In QSAR, Turkina et al. [148] combined six non-hashed fingerprints by low-, mid- and high-level fusion, on six regression and classification datasets. Mid-level fusion, which keeps fingerprint bits by their importance in the per-fingerprint models, gave the best accuracy. **[RESULT NEEDED: held-out score and cost per method]**.

The next experiment asks whether the gain comes from drawing on several representations, or only from having fewer features. Every method gets the same number of features, B, drawn from all base representations and fitted on the fold's training molecules. It runs on regression datasets only. The methods are the hybrid, PCA, mRMR [176], recursive feature elimination [179], and B features drawn at random. The hybrid uses its stored configuration at budget B, with the features re-selected on each fold. For PCA, all base representations are joined, and each feature is z-scored on the fold's training molecules. PCA is then fitted on those molecules. The number of components is B, capped at the total number of features and at one fewer than the number of training molecules. Held-out molecules are projected with the fitted PCA. The PCA control of the ensemble comparison is built the same way. The mRMR ranking stops at 200 features for run time. When the representations hold more than 2,000 features in total, mRMR first keeps the 2,000 with the highest mutual information with the label. Recursive feature elimination uses a random forest of 50 trees of depth at most 5. One downstream model scores every method,

a random forest of 100 trees, so only the features differ. This comparison is not fully held out. The hybrid's configuration is chosen using all molecules, and only its feature selection is redone per fold. **[RESULT NEEDED: held-out score against B (features) for each method]**.

The search scores each representation with an internal evaluator to split the budget. The winning hybrid then goes to a downstream model. This experiment asks whether the two must match. It uses nested cross-validation as in the overfitting experiment, with one search per outer fold for each evaluator setting. Each setting is compared with the matched evaluator. The matched evaluator is the same model family as the downstream model, at the lighter evaluator settings of Table S.12. The near-family setting scores the random forest with LightGBM and the reverse, and XGBoost with LightGBM. It scores the SVM with a Gaussian process, and the DNN with a narrower DNN (64 and 32 units). The far-family setting scores the three tree models with an SVM, the SVM with LightGBM, and the DNN with k-nearest neighbors. A fourth setting scores the DNN with a wider DNN (512, 256 and 128 units). The last uses a fixed set of three evaluators, random forest, LightGBM and XGBoost, and keeps the best. We report each setting minus matched, paired per outer fold, at the same number of features. Evaluators with the exact settings of the downstream random forest and LightGBM are implemented. No search uses them, so the full runs do not report them. The answer sets whether one cheap evaluator is enough, or whether the search must be re-run for each model. **[RESULT NEEDED: each setting minus matched, per downstream model]**.

Selection on each of the M folds gives a binary vector $\mathbf{z}_m \in \{0,1\}^d$, where d is the number of features in all base representations joined. We measure how stable the selection is with the index of Nogueira et al. [180],

$$\hat{\Phi} = 1 - \frac{\frac{1}{d}\sum_{f=1}^{d} s_f^2}{\frac{\bar{k}}{d}\left(1 - \frac{\bar{k}}{d}\right)}, \qquad s_f^2 = \frac{M}{M-1}\hat{p}_f(1-\hat{p}_f).$$

Here $\hat{p}_f$ is the fraction of folds that selected feature f, and $\bar{k}$ is the mean number of features selected. $\hat{\Phi} = 1$ means the same features were chosen on every fold. $\hat{\Phi} \approx 0$ is what random selection gives. d must be that full width, not the union of the selected features. Unstable feature sets would weaken any chemical reading of the blueprints. **[RESULT NEEDED: $\hat{\Phi}$ per dataset and model]**.

The learning curves draw subsamples of 10, 25, 50, 75 and 100% of the molecules,

as whole groups, and never fewer than 50 molecules. Above 5,000 molecules we use absolute sizes of 250, 500, 1,000 and 2,000 molecules instead. Each size is drawn 3 times. The 100% point is drawn once, since every draw is the same. A full search runs on each subsample, with budgets above the number of training molecules dropped. The gain at each size is the hybrid minus the best single representation, with leave-one-fold-out reselection. Small subsamples search fewer budgets. So part of any rise in gain with size may come from searching more configurations. **[RESULT NEEDED: gain per dataset, model and size]**.

This experiment asks whether importance helps during training, not only during selection. It uses the hybrid at 100 features with random forest, LightGBM and XGBoost, the three models that accept per-molecule weights. Importance is computed on the training molecules with the importance method the search picked for that hybrid. There are five conditions:

1. unweighted;
2. features multiplied by the square root of their importance, after standardizing, with the weights normalized to average 1;
3. molecules weighted by how well their label agrees with their 10 nearest neighbors in the rescaled space;
4. both together;
5. only the features strictly above the median importance.

The weight of molecule i is $1-|y_i-\bar{y}_{N(i)}|/(\max y-\min y)$, clipped to between 0.1 and 1. Here $\bar{y}_{N(i)}$ is the mean label of its 10 nearest neighbors, by Manhattan distance. This comparison is not fully held out. The configuration comes from a search over all molecules, and only the feature selection is redone per fold. The molecule weighting is a noise-mitigation method: it down-weights suspect labels. Tree ensembles are unchanged when a feature is multiplied by a positive constant, because their splits depend only on the order of values [284]. So the second condition differs from the unweighted one in one way only. Features with zero importance get weight zero, become constant, and drop out. Every other feature is only rescaled, which the three tree models ignore. The rescaling does matter for the third condition, because it changes which molecules are nearest neighbors. Checking a label against its nearest neighbors goes back to editing a training set by its nearest neighbors [103].

Here the molecule is down-weighted, not removed. We found no QSAR study that down-weights training molecules whose labels disagree with their nearest neighbors. The nearest prior work is outside QSAR. Nearest-neighbor noise filters adapted to regression labels remove such samples rather than down-weight them [146]. Learned per-sample training weights for noisy labels come from general machine learning [285]. **[RESULT NEEDED: one result line]**.

This experiment asks whether a few extra features help one strong single representation. Two decoys are planted among the candidate features. The first is a block of pure Gaussian noise, as wide as the real candidate features combined. The second is near-copies of 64 base features, with added noise at 5% of each feature's standard deviation. We count how many decoy features are taken on each fold; a working rule takes none. The same extra features are also offered to the hybrid built from several representations. This asks whether extra features do more on one representation or on the hybrid. **[RESULT NEEDED: one result line, with the decoy features taken per fold]**.

This experiment asks whether hybrids are more robust to label noise than their base representations. Robustness is the normalized area under the retention curve of Chapter 3, $\mathrm{AUC}_{\mathrm{norm}}$(Equation 2.7). There are two differences from Chapter 3. First, the retention curve uses each dataset's own metric, not R^2. For an error metric the ratio is inverted, so 1 still means no loss. Second, there are seven noise levels: 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5. Each is a fraction of the spread of the fold's clean training labels. Censoring has its own axis, the fraction of labels clipped: 0, 0.10, 0.20, 0.25, 0.30 and 0.40. We compare the hybrid at each budget with each single representation. We also compare it with the best single representation, re-picked in each fold. One downstream model, a random forest, is used under each regression noise type. The hybrid's settings are searched again at every noise type and level, on the noisy training labels. Held-out labels stay clean. The search uses the first fold's scores to drop configurations, so that fold is left out of both sides and four folds are reported. In Chapter 3 the model, not the representation, explained most of the variance in $\mathrm{AUC}_{\mathrm{norm}}$. Here the model is fixed, so only the representation changes.

The last experiment asks whether joining two whole representations scores above the better of the two, or between them. For each dataset we take its top three single representations and form each of the three pairs. We score each representation alone and each concatenation, with the same model and folds. A concatenation that lands between its two parts supports the case for combining and filtering at

once. **[RESULT NEEDED: for each pair, the score of each representation alone and of their concatenation]**.

5.3.5 Transfer across datasets, antibody mutational scans and data held at separate sites

This subsection sets up three studies outside the single-dataset setting. They reuse search settings on a second dataset, build hybrids on antibody mutational scans, and select features with data held at separate sites. We describe how each setup was built and scored, with a short background for each; the outcomes are in the results sections. One requirement covers all three. Each needs several numeric representations of the same objects, one supervised target, and a grouped split that keeps near-duplicates on one side.

The first study asks whether search settings found on a large dataset can be reused on a small one, instead of running a new search. The search settings are the importance method, the allocation method, the budget B and the internal evaluator. They are decisions, not a set of features. The source dataset is LogD (lipophilicity), with 5,028 molecules. The target dataset is hERG binding affinity ($\mathrm{p}K_i$), with 1,412 molecules. The candidate representations are those listed for both datasets, kept only where the width (number of features) matches. Both searches then range over the same features. The current lists share seven representations: Avalon, ECFP4, Mordred, and four pretrained embeddings (GROVER, MHG-GNN, SMI-TED and ChemBERT). The width check can still drop some of them. **[RESULT NEEDED: the shared representation set the full run uses]**. The search is the expensive step. Settings that transfer would make it a one-off cost, paid on the largest dataset. The closest prior work is Meta-QSAR [286]. It chose the learning method and molecular representation for a new QSAR dataset from results on over 2,700 other QSAR datasets. It beat random forest on a fingerprint by up to 13% on average. Like the source search settings here, it reuses decisions, not a set of selected features. We have not found a paper that applies feature indices selected on one QSAR dataset unchanged to another, but our search of the literature was not complete.

All four comparisons run inside every held-out fold of the target. The first is the best single representation, re-chosen on that fold's training molecules. The second is a hybrid searched on that fold's training molecules, which is the upper reference. The third applies the source search settings, with the features re-selected

on the target's training molecules. The fourth applies the source's own selected feature indices unchanged, so no target label is used to pick features. The last two separate two questions: whether the search decisions transfer, and whether the chosen features transfer. We use five scaffold-group folds, RMSE in pK_i units, and the five downstream models (random forest, LightGBM, XGBoost, SVM and DNN). **[RESULT NEEDED: hybrid minus best single representation for each transfer comparison, per downstream model]**.

A deep mutational scan measures a property, here expression or binding, for many single- or few-point mutants of one antibody. It is used to map which positions tolerate change. We use four scans with a single wild type, taken from FLAb [215]. Two are the expression and binding scans of the anti-VEGF antibody G6 [214]; the expression scan is one of the 16 datasets of this chapter. The other two are the expression and binding scans of the anti-fluorescein antibody 4-4-20. The candidate representations are the protein language-model embeddings and cheaper sequence features of "Representations by domain". We add two representations of the change from the wild type. The first has 5 features: the change in hydrophobicity, charge, isoelectric point, molecular weight and aromaticity, summed over the mutations. Each property is z-scored over the 20 amino acids first. The second places the same five features at the mutated position, so it has five features for every sequence position. We use nested cross-validation. The search sees the outer training variants only, and the hybrid and every single representation are scored on the untouched outer fold. The continuous scans are scored by Spearman correlation. The 4-4-20 binding scan is binarized, binder if its pK_d is above 5.0, and scored by AUROC. We report hybrid minus best single representation, per scan, on the outer folds. The two wild-type-difference representations are also scored as single representations, and each run reports their rank among all representations. The number of features each representation got is stored per fold with every result, but this comparison does not report it. So hybrid minus best single says nothing about the wild-type-difference representations unless that count is also reported. **[RESULT NEEDED: features the hybrid took from each of the two wild-type-difference representations, per scan, per fold]**. **[RESULT NEEDED: hybrid minus best single representation per scan, per downstream model]**.

Federated learning trains one model across data held by several parties without moving the data [287]. It has been run across pharmaceutical companies on shared property-prediction targets [288]. In this field the motivation is commercial. Struc-

tures and assay results are the asset, so pooling them is not possible [18, 19]. Our collaborator, Lhasa Limited, uses the teacher-student process described in Chapter 2, "Noise in molecular data" [139]. We use the collaborator's code for the fingerprints, the site models and the site clustering, so the numbers stay comparable to theirs. The step that combines the site models' predictions is re-implemented from that code.

The site models' predictions are combined in one of two ways. Under distance weighting, each site model's vote on a molecule is weighted by how close that molecule is to the site's own training molecules. A site counts most on chemistry it has seen. Closeness is the mean distance between fingerprints, from the query molecule to its 8 nearest neighbors among a random 20% of the site's training molecules. The distance is the one set for the run, Manhattan by default; the collaborator's code uses Euclidean. For each site, these distances are rescaled to 0–1 over the query molecules. Each is turned into a similarity as $(1 - \text{distance})^2$, and the similarities are normalized to sum to 1 over the sites. Under score weighting, each site instead gets one weight, from its own cross-validated MCC. Only predictions on shared molecules cross a site boundary. In the published process, a student model is then trained on the most confident combined labels, the same number from each class. In the hERG comparison below no student is trained. The combined predictions score the held-out molecules directly.

The addition in this thesis is that feature selection, not only model fitting, happens at the sites. Each site runs the hybrid search on its own molecules. Features are chosen on 80% of its molecules and scored on the other 20%, split by scaffold group. Each site reports three things: feature indices, importance values and one score. No molecule and no feature value leaves the site. A coordinator splits the total budget B across sites in proportion to their scores. Each share b_s is rounded down, with at least one feature per site, so $\sum_s b_s \approx B$. If every site scores 0, the budget is split equally. Only the proportional split is run. With no equal-split comparison, we cannot say what the site scores buy. A feature another site has already taken is skipped, and the site takes its next-ranked feature instead, so the features in the hybrid are all distinct. Indices and importances are smaller, and harder to invert, than model weights or gradients. Splitting the budget between sites is also the same question the allocation method answers between representations. In related work, each client runs a mutual-information feature selection on its own data, and a server aggregates the results, with no raw data exchanged [289].

hERG channel blockade is a cardiotoxicity liability screened early in drug discovery, and its public data comes from many labs [290]. Three real sources act as sites. The first is the public ChEMBL hERG K_i set used elsewhere in this thesis. The other two are the collaborator's training set (7,772 molecules) and test set (4,383 molecules). The two collaborator sets were labeled by different curation rules, so the sites differ in curation, not only in chemistry. This is source heterogeneity that exists in the world. The process is usually simulated with sites made by clustering one dataset. Molecules held by more than one source are deduplicated on InChIKey, keeping the public copy. The public set's target is binary: a molecule is a blocker if $\mathrm{p}K_i \geq 6$, that is $K_i \leq 1\,\mu$M. We use five scaffold-group folds over the pooled molecules. The search is scored by AUROC. The reported number is MCC, which does not reward predicting the majority class.

The comparison crosses two choices: molecules pooled or kept at their sites, and all features or a selected set. Pooled molecules with all features is the upper reference, not possible in practice. Pooled molecules with selected features measure what the selection is worth on its own. Molecules at their sites with all features is federated training as usually done. Molecules at their sites with selected features is the only setting where nothing but indices, importances, one score and predictions moves. Selected minus all features, at fixed site structure, is what selection buys. Pooled minus site-held, at fixed features, is what pooling buys. We add four context comparisons. They are the single best source, picked on training molecules; both ways of weighting the site models; the same search on one fingerprint alone; and one site model per representation. The budget B is swept over 50, 100, 250, 500 and 1,000 features, so no comparison rests on one width. Both all-features settings fit one 2,048-bit Morgan fingerprint of radius 2, with pharmacophoric atom types. Both selected settings fit features drawn from the hERG K_i dataset's candidate representations. So selected minus all features differs in two ways at once, width and source. The budget sweep varies the width of the selected set only. It cannot separate the source difference. The single-fingerprint search is the closest control for the source difference: the same search, on ECFP4 only. Two curators disagreeing on one molecule is label noise with a known source (Chapter 6).

Four other separate-site experiments are run. In the first, the sites carry different amounts of label noise. Selection at the sites is compared with the site-model ensembles and with one model trained on all sites' molecules pooled, at several noise levels. One version has each site first remove the molecules it flags as noisy. In the

second, every site holds the same molecules as a different representation. Fusing features is compared with pooling the site models' predictions. In the third, the sites hold different families of representations (substructure, physicochemical and pretrained) of shared molecules. In the fourth, the sites are antibody sources: a binding-energy-change set, a deep mutational scan of binding, and a thermostability set. Within a site, sub-sites are made by clustering language-model embeddings, not fingerprints. The antibody version skips the two ensemble comparisons. Each site's features are reduced separately, so one site's model cannot predict on another site's features. The labels are also in incompatible units (kcal/mol, a normalized score and degrees Celsius). **[RESULT NEEDED: one line on the outcome of each of the four]**.

The separate-site work has four limits. One source is proprietary and cannot be redistributed, so the comparison cannot be reproduced from public data alone (Appendix B). The comparison depends on the collaborator's implementation of the process. The published process labels a large separate public set and trains a student on it. Where a student is trained here, the training molecules play that role, and held-out molecules are excluded from it. This is a compromise. Finally, the sites differ in size and in how their labels were derived, so a site's score reflects its curation as well as its chemistry.

Three external benchmark tasks, not among the current datasets, are possible later additions and are not run here. The first is activity-cliff prediction on the dopamine D2 receptor, factor Xa and SARS-CoV-2 main protease, following a published comparison of representations and models [73]. The second is antibody developability, on the biophysical properties of a public competition set, using that competition's cluster splits [291]. The third is antibody–antigen binding free-energy change, asking how close sequence representations get to a structure-based graph network [292].

5.4 Results: when the hybrid helps

5.5 Results: why the hybrid helps

5.6 Results: blueprints on specific datasets

5.7 Results: transfer across datasets and antibody mutational scans

5.8 Results: data held at separate sites

5.9 Discussion

Ambroise and McLachlan [192] showed the extreme case of feature selection overfitting: an SVM achieved near-zero cross-validated error on random labels when gene selection was performed on the full dataset before cross-validation, but when selection was nested inside each fold the median error was 11 out of 40, close to the 50% expected by chance. Vabalas et al. [193] showed that the bias introduced by pre-selection can exceed the bias from hyperparameter tuning, and Cawley and Talbot [195] argued that feature selection is itself a form of model selection whose complexity cost must be accounted for. KIRBy follows the standard remedy: we compute all importance scores within each cross-validation fold, on training data only. But the problem of "search degrees of freedom" [293] remains. Searching over many possible feature subsets adds complexity even when each individual selection is unbiased. The budget acts as a hard capacity constraint: selecting 100 features from a pool of 5,000 limits the space of possible selections far more tightly than L_1 regularization over the same pool, because the feasible set grows polynomially in the budget while the continuous parameter space of a regularized model is unbounded [294]. Important here is a distinction: the features being selected are frozen embeddings, not parameters being fitted. Choosing which pre-existing knowledge to trust is, we think, a less expressive operation than learning new features, and Kumar et al. [295] showed that frozen embeddings fed through a linear probe can outperform full fine-tuning by up to 10% in out-of-distribution accuracy precisely because fine-tuning distorts pretrained structure. And stability can be measured directly: the

Nogueira index [180] quantifies the consistency of selected feature sets between folds. If selection were overfit, feature sets would differ wildly from fold to fold.

5.10 Limitations

We want to be clear about KIRBy's limitations. For small datasets (below about 200 samples), the hybrid search cannot reliably outperform the best single source. We suspect this is because the internal validation set within each fold shrinks to roughly 20 samples, and at that size the evaluator cannot distinguish genuine improvements from noise. Tuning cannot fix this. And at the other extreme, frozen embeddings have a ceiling. No signal can be added beyond what the inputs already contain, and on tasks where the true signal requires end-to-end learning from raw inputs (certain 3D quantum-mechanical properties, for example, where SchNet outperforms any frozen embedding by a wide margin), this ceiling binds. Although the methodology naturally accommodates embeddings from sequence-trained models (ESM-2, ProtBERT, etc.), encoding raw sequences directly as one-hot or k-mer features disperses the positional and compositional information that makes sequences useful in the first place. Integrating 20 to 50 foundation models into a single pipeline, each with its own dependencies, GPU requirements, and version constraints, is genuinely difficult engineering. Embeddings need only be computed once per dataset and everything is cached. Yet the initial integration burden is real and scales with the number of models in the pool. Finally, on endpoints with high experimental noise, even a well-constructed hybrid may not improve on simpler baselines. Kramer et al. [15] documented that experimental uncertainty in heterogeneous public K_i data is approximately 0.54 log units. This sets a natural accuracy ceiling of $R^2 \approx 0.81$ regardless of model sophistication; the budget constraint plausibly guards against overshoot, but it cannot push past the noise floor.

5.11 Conclusions

Chapter 6

Hybrid Representations for Similarity and Label Noise

The hybrid began as a fix for similarity. This chapter tests whether features chosen for the task give better neighbors than the single representations they come from. It also asks whether better neighbors help to find noisy labels, estimate their size, and train on them. These are three different tasks, and a method can do the first and fail the third. The type of noise decides what can be fixed, so we treat censoring, Gaussian noise and real labels separately.

6.1 Introduction

Chapter 4 showed where the similarity assumption fails on standard representations. ECFP4 bits compared by Tanimoto similarity are one example. They mix a few features relevant to the task with many that are constant or irrelevant on the dataset. Any method that trusts a molecule's neighbors inherits that error. This includes finding noisy labels from the labels of their neighbors. Chapter 5 built the adaptive hybrid representation. It is a budget of features picked from several base representations for one task and one downstream model. It is an input, not a model.

The hybrid started as a fix for similarity: select the features relevant to the task first, then compute distances on those features only. Hybrids then turned out to work as ordinary model inputs (Chapter 5), and this chapter returns to the original motivation.

We test three methods that rely on neighbors or on held-out predictions to handle noisy labels. These are nearest-neighbor editing [103], filtering by classifiers trained

on other folds [144], and confident learning [52]. Each depends on the feature space it works in, so we ask whether a hybrid changes how well they work. The research questions, one per results section, are:

1. Distances and nearest-neighbor prediction: is predicting each molecule's activity from its five nearest neighbors more accurate in a hybrid than in the single representations it is built from?

2. Noise detection and estimation: are noisy labels easier to find, or their size easier to estimate, in a hybrid?

3. Learning with noisy labels: does a hybrid change how well noise mitigation (label corrections and cleaning) and noise-robust training work?

This chapter is planned as the second hybrid paper; its methods are final and its results are preliminary. **[RESULT NEEDED: one-sentence preview of the pattern found for each of the three research questions]**

6.2 Methods

This section has three parts. The first describes how similarity is computed in a hybrid. The second gives the noise conditions used. The third describes the three experiments on learning with noisy labels.

The three experiments share their datasets, scaffold-group-aware folds and metrics with Chapter 5. Noise touches the training labels only. Each downstream model is compared against its own top two single representations, ranked by that model's own score. We never compare against a fixed list or one chosen fingerprint.

In all three experiments, the hybrid's features are rebuilt within each fold from that fold's training labels. Those labels are noisy, clipped or uncleaned, depending on the experiment. The search settings (representations, budget and allocation method) are chosen differently in each experiment. In the survey of twelve methods for learning with noisy labels, the search runs again in each fold and at each noise level, on the noisy training labels. It is scored by the same neural network the methods use. It searches budgets of 50, 100 and 250 features, with 2 to 3 inner folds. Chapter 5 searches eight budgets, from 10 to 2000 features. If this search returns no score in a fold, that fold uses the neural network's settings from the search on clean labels. The results record which folds did this. In the experiment

with labels clipped at a detection limit, the settings come from the search on the clean, unclipped labels. Only the feature ranking and the budget split inside each fold see the clipped labels. In the experiment that cleans real labels, the settings come from the search on the same real labels the experiment trains on, since no clean labels exist. Settings searched on clean labels give the hybrid information a real user never has. The survey avoids this; the clipped-label experiment does not.

We keep apart three tasks that are often treated as one: finding noisy labels, estimating how large the noise is, and improving the model's held-out score. A method can do the first well and fail the third. **[FIGURE NEEDED: one-panel flow. Left: dataset with injected noise, or none. Middle: the hybrid built per fold on the training labels, beside the top two single representations. Right: similarity score, and the held-out score after each method for learning with noisy labels.]**

6.2.1 Similarity in a hybrid

We use a hybrid for similarity in two ways. The first is an ordinary hybrid: the search of Chapter 5 picks features that help a downstream model, and distances are then computed in that hybrid. The second is a similarity hybrid. It runs the same search, but ranks each candidate by how well each molecule's activity is predicted from its five nearest neighbors in the hybrid. No downstream model is fitted while searching for a similarity hybrid. Importance methods, allocation methods, budgets and folds are all shared; only the score that ranks candidates changes. The neighbor score also sets each representation's share of the budget, so no internal evaluator is used. For the budget split, the score is computed on a validation part carved from each fold's training molecules. The score that ranks whole candidates uses the fold's held-out molecules. Every representation in the dataset's set is a candidate, and the number of representations (2, 3, 5 or all) is not searched. Chapter 4 showed that the representation decides which molecules count as neighbors. The similarity hybrid tests whether picking features for neighbor quality fixes that more directly than picking them for prediction. **[FIGURE NEEDED: D6.1, two pipelines sharing a first half (base representations, importance, allocation, hybrid). Top: hybrid, downstream model, dataset metric. Bottom: hybrid, 5 nearest neighbors from other scaffold groups, distance-weighted mean activity, Spearman or rescaled AUROC. Caption: same search, different score.]**

Each feature is z-scored with the mean and standard deviation of the training molecules only, as in Chapter 5. The distance is Manhattan distance on the z-scored features, as defined in Chapter 4. In a hybrid, width is weight. Every z-scored feature adds one term of similar size to the distance. A representation's pull on the distance is therefore roughly its number of features divided by the budget. So in a hybrid, the allocation method also weights the representations inside the distance. The neighbor score used by the search takes the z-scored features with no importance weighting. **[RESULT NEEDED: confirm Manhattan distance and one-over-distance neighbor weighting once the design experiment crossing distance, neighbor weighting and number of neighbors has run in full]**

To rank whole candidates, the score is computed on the held-out molecules of each outer fold. The neighbors are other held-out molecules, not training molecules. The held-out molecules' scaffold groups are split into up to 5 parts, or fewer if there are fewer groups. Each molecule's 5 nearest neighbors come from the other parts, and so from other scaffold groups. A neighbor with the same scaffold is an easy match and would inflate the score. Each neighbor is weighted by one over its distance:

$$\hat{y}_i = \frac{\sum_{j \in N_5(i)} y_j / d_{ij}}{\sum_{j \in N_5(i)} 1 / d_{ij}}. \tag{6.1}$$

A neighbor at distance 0 takes all the weight. On a regression dataset the score is the Spearman correlation $\rho(\hat{\mathbf{y}}, \mathbf{y})$. On a binary classification dataset, $\hat{y}_i$ is the distance-weighted fraction of neighbors in class 1, and the score is $2(\text{AUROC} - 0.5)$. The rescaling puts chance at 0 under both scores. This matters because the budget split divides by this score. The modelability index of Golbraikh et al. [118] (Chapter 4) is a related published measure; it uses one nearest neighbor and a hard class match, while this score keeps the graded neighbor fraction. The score measures how far the similar property principle (Chapter 2) holds in the hybrid.

A fold is not scored when it has fewer than 10 held-out molecules, fewer than 2 scaffold groups, or constant labels or predictions. Such a fold is left out of the mean rather than counted as 0. AUROC is defined for two classes, so multiclass labels are also not scored. This removes the PAM50 breast cancer subtypes (4 classes) and the trastuzumab HER2 multiclass dataset (3 classes). The folds of the trastuzumab HER2 binary dataset are cut on edit-distance shells around the wild-type CDRH3. A held-out block can hold one shell, so no molecule has a neighbor from another group.

The similarity hybrid is not searched on these three datasets. It is searched on every other dataset in the thesis, antibody and other non-molecule datasets included.

Feature importance weighting. Beyond simply using the feature importance scores for allocation, we investigate whether feature weighting by importance during distance computation improves noise detection. For a feature importance vector w, weighted distances are computed by scaling the standardized features: $x' = x \odot w$ under Manhattan distance ($x \odot \sqrt{w}$ under Euclidean), with negative importances set to 0 and w rescaled to average 1.

6.2.2 Noise conditions

The noise conditions are defined in Section 2.5.1 (Figure 2.3). Three of them drive the three experiments on learning with noisy labels: censoring, Gaussian noise, and real labels with nothing injected. Outlier and grouped-wider noise are used to grade detection. Grouped-shifted noise grades estimation only, because every training label moves under it. Noise is added to the training labels of each outer fold only, and held-out labels are never changed. The noise size, the censoring limit, and which scaffold families grouped-wider noise widens are all computed from that fold's clean training labels. One exception is the simulation of several sources per molecule with correlated errors. It sizes its noise from the standard deviation of all molecules' labels, held-out molecules included, although its held-out labels stay clean.

Under censoring, the level is the fraction of training labels clipped at a detection limit: 0, 0.10, 0.20, 0.25, 0.30 and 0.40. The survey and the detection results run 0, 0.10, 0.25 and 0.40. The 0 level is the negative control. The clip involves no random draw, so we run one draw per level. The error is one-sided, a bias rather than a scatter, and both its direction and its location (the limit) are known. It stands for assay range limits, such as "$> 10\,\mu$M" recorded as $10\,\mu$M [81]. Censoring applies to regression only, because a class label has no value to clip.

Under Gaussian noise, the levels are multiples of the standard deviation of the fold's clean training labels: 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5. We run three noise draws per level. On a classification dataset, the level is instead the fraction of training labels flipped to the wrong class: 0, 0.05, 0.10, 0.20, 0.30 and 0.40. Gaussian noise stands for the random error of repeating a measurement. It is also the standard assumption in studies of noise robustness. The survey of twelve methods for learning

with noisy labels uses it.

Outlier, grouped-wider and grouped-shifted noise are used for detection and estimation only; their parameters are in Section 2.5.1. Outlier noise stands for annotation and transcription errors [14, 98]. It is run with 1%, 5% and 10% of training molecules affected, not only the 10% of Chapter 3. Grouped-wider and grouped-shifted noise stand for differences between laboratories or sources [80]. Both grouped conditions need a scaffold per molecule, so they run only on datasets with SMILES.

In the last condition, labels are used as they come and nothing is injected. No list of wrong labels exists. Success is therefore only a better held-out score after cleaning the training labels. A better score shows that cleaning helped the model, not that it found errors, since it may have removed hard molecules instead. QM9 is the control. Its labels come from a quantum-chemical calculation, with no experimental error [107]. A cleaning method that helps on QM9 is therefore not removing measurement error. Table 6.1 gives, for each condition, what it stands for, its levels, which experiments use it and which datasets it covers. **[RESULT NEEDED: the datasets each condition covers in the full runs, for Table 6.1]**

6.2.3 Noise mitigation methods

In addition to evaluating hybrid representation quality for distance-based reasoning, our investigation evolved to explore alternative noise handling strategies. Initial experiments with sample removal methods revealed limitations when applied to Gaussian noise affecting all samples. This motivated a pivot toward sample weighting and label smoothing approaches, which do not require accurate binary classification of noisy versus clean samples but instead leverage continuous noise magnitude estimates.

Sample removal methods

Distance-based approaches rely on the assumption that similar molecules have similar properties [31]. Presumably, noisy data would break this pattern, making it a method for detecting anomalous data. k-Nearest neighbor filtering examines the variance of a sample's neighborhood. For sample x_i with k nearest neighbors $\mathcal{N}_k(x_i)$,

we compute the standard deviation of neighbor labels:

$$\sigma_i = \text{std}(\{y_j : x_j \in \mathcal{N}_k(x_i)\}) \tag{6.2}$$

Samples with σ_i exceeding the 90th percentile are marked as potentially noisy. We test $k \in \{5, 10, 20\}$.

Local Outlier Factor (LOF) identifies anomalous samples by detecting local density estimates between a sample and its neighbors [296]. These density estimates are based on the distance to its k nearest neighbors. If a point's density is lower than that of its neighbors, its LOF is high. We flag the 10% of training molecules with the highest LOF, using 20 neighbors.

We also tested the following noise removal methods: distance-weighted prediction filtering, neighborhood consensus filtering, AC detection, Mahalanobis outlier detection, cross-validation disagreement, bootstrap ensemble filtering, co-teaching [141], and DivideMix [297].

Noise magnitude estimation

We now examine the core of the noise mitigation research: estimating the magnitude of noise. This enables proportional handling rather than complete removal. We used the following neighbor-based estimators:

- Neighbor residual: Estimates noise as deviation from k-NN mean, normalized by global label standard deviation:

$$\text{score}_i = \frac{|y_i - \bar{y}_{\mathcal{N}_k(x_i)}|}{\sigma_y} \tag{6.3}$$

- Local variance ratio: Measures deviation relative to local label variance, identifying samples that deviate from neighbors more than neighbors deviate from each other:

$$\text{score}_i = \frac{|y_i - \bar{y}_{\mathcal{N}_k(x_i)}|}{\max\big(\text{std}(y_{\mathcal{N}_k(x_i)}),\ c\big)}, \qquad c = \max(0.01\,\sigma_y,\ 10^{-3}) \tag{6.4}$$

  The floor c keeps a molecule whose neighbors all share one label from receiving an unbounded score.

- AC-aware: Distinguishes noise from activity cliffs by weighting deviation by

neighborhood consistency. High deviation with high neighbor agreement suggests noise while high deviation with neighbor disagreement suggests an AC [73]:

$$\text{score}_i = \frac{|y_i - \bar{y}_{\mathcal{N}_k(x_i)}|}{\sigma_y} \cdot \frac{1}{1 + \text{std}(y_{\mathcal{N}_k(x_i)})/\sigma_y} \tag{6.5}$$

And we used the following model-based estimators using prediction disagreement:

- Cross-validation residual: Uses out-of-fold predictions; high residuals indicate labels inconsistent with the model learned from other samples.
- Ensemble disagreement: Trains multiple models on out-of-fold splits. A molecule's score is the standard deviation of their predictions plus the absolute residual of their mean prediction.

We used probabilistic methods that directly predict noise magnitude using uncertainty estimates:

- Gaussian Process: Provides natural uncertainty estimates via posterior variance, with learned noise level σ^2 from the kernel [66].
- Heteroscedastic Neural Network: Predicts both $\mu(x)$ and $\sigma^2(x)$ using β-NLL loss [298]:

$$\mathcal{L} = \frac{1}{2}\left(\frac{(y-\mu)^2}{\sigma^2} + \log \sigma^2\right) \cdot \sigma^{2\beta} \tag{6.6}$$

  where $\beta = 0.5$ prevents the variance from hiding the mean errors.
- Conformal prediction: Uses split conformal prediction to obtain prediction intervals. Wider intervals should indicate higher uncertainty [299].
- Deep ensemble: Trains M independent networks and uses the variance of means as epistemic uncertainty plus the mean of variances as aleatoric uncertainty [69].
- Bayesian Random Forest: A molecule's score is the standard deviation of the trees' predictions plus half the absolute out-of-fold residual [300].
- Quantile regression: Fits models for multiple quantiles and uses interval width as noise estimate.
- NGBoost: Natural gradient boosting that predicts distributional parameters (μ, σ) directly [58].

NGBoost in particular was selected due to its promising uncertainty values with respect to the correlation between noise and prediction error, as discussed in Section 3.7.

Label smoothing and sample weighting

Given the fact that all labels contain some magnitude of noise, we focused strongly on label smoothing over removal using the following methods:

- Proportional: $y_{\text{smooth}} = y + \alpha \cdot \text{score}_{\text{norm}} \cdot (\text{target} - y)$
- Threshold: Only molecules whose normalized score lies above the $100(1-\alpha)$th percentile are corrected, to $(1-\alpha)\, y + \alpha \cdot \text{target}$. The same α therefore sets both how many labels move and how far.
- Soft threshold: Applies sigmoid-like correction strength:

$$\text{strength} = \alpha \cdot \sigma\left(5 \cdot (\text{score}_{\text{norm}} - 0.5)\right) \tag{6.7}$$

- Confidence: For methods that output raw uncertainty σ (e.g., NGBoost):

$$\text{correction} = \frac{\alpha \cdot \sigma}{1 + \alpha \cdot \sigma}, \quad y_{\text{smooth}} = (1 - \text{correction}) \cdot y + \text{correction} \cdot \mu \tag{6.8}$$

Rather than directly modifying labels, we weighted more trustworthy samples during training:

- Inverse: $w = \frac{1}{1+\alpha \cdot \text{score}_{\text{norm}}}$
- Exponential: $w = \exp(-\alpha \cdot \text{score}_{\text{norm}})$
- Soft: $w = 1 - \alpha \cdot \text{score}_{\text{norm}}^2$, clipped to $[0.1, 1.0]$

All weights are normalized to mean 1 to preserve effective sample size.

Recovery rate

For noise detection accuracy, we compute precision, recall, and F1 score, treating noise injection as ground truth.

Each downstream model m is trained and tested three times: on the clean training labels, giving the score s^m_{clean}; on the noisy labels, giving s^m_{noisy}; and on the labels

after noise estimation and correction, giving s^m_{denoised}. Scores are in the dataset's own metric, averaged over folds. The recovery rate is the share of the loss that correction wins back,

$$\text{Recovery}_m = 100 \times \frac{s^m_{\text{denoised}} - s^m_{\text{noisy}}}{s^m_{\text{clean}} - s^m_{\text{noisy}}} \ \%, \tag{6.9}$$

which reads the same way whether the metric is higher- or lower-is-better. Each model is divided by its own clean-to-noisy gap. When that gap is no more than 2% of the clean score, the noise did too little damage to recover, and the recovery rate is left undefined rather than reported. A recovery rate of 100% indicates complete restoration to clean performance; negative values indicate that correction made performance worse than the noisy labels. The exploratory study reused in Chapter 4 computed the same ratio in R^2, with one random forest on ECFP4 as the only downstream model.

6.3 Results: similarity

6.4 Results: noise detection and estimation

6.5 Results: noise mitigation by noise type

6.5.1 Censoring: correcting labels clipped at a detection limit

6.5.2 Gaussian noise: learning-with-noisy-labels methods

6.5.3 Real labels, nothing injected: cleaning

6.5.4 By noise type

6.6 Conclusions

Chapter 7

Discussion and Conclusions

Real assay labels are not noisy in the way most studies assume. Measurements pooled across laboratories carry an offset shared by each laboratory [80]. Labels near an assay's detection limit are clipped to it [81] ("Noise in molecular data", Chapter 2). Gaussian noise, the usual assumption, is also the hardest case to correct. Every label is off by an unknown amount in an unknown direction ("By noise type", Chapter 6). Cleaning labels can even leave a model worse than the uncorrected labels would. So before choosing a method, we ask where the labels came from: many laboratories, one controlled experiment, or an unknown source ("By noise type").

This chapter first summarizes each chapter's question and its answer. It then makes two arguments: no single method fits every dataset, and simple is often better. It then sets out the limitations of this work and directions for future work, and ends with the conclusions of the thesis.

7.1 Summary

QSAR models are trained on measured labels, and those labels carry experimental noise. Most work on label noise in QSAR assumes that this noise is Gaussian. Chapter 2 defined the representations, models and noise types used throughout, the NoiseInject package that adds the noise, and how models were evaluated.

Chapter 3 asked four questions about label noise: how the representation, the model and the type of noise shape a QSAR model, and whether its uncertainty finds the noisy labels. For accuracy, model and representation explained comparable shares of the variance in R^2. On the QM9 HOMO–LUMO gap, with Gaussian noise as wide as the clean training labels, the model explained 29.1% and the represen-

tation 26.6%. For robustness, measured as $\mathrm{AUC}_{\mathrm{norm}}$, the model dominated. Under Gaussian noise it explained 49.5% of the variance, against 9.2% for the representation. Changing the shape of the noise barely changed performance; grouped-shifted and censored noise cost the most. Median $\mathrm{AUC}_{\mathrm{norm}}$ was 0.813 under censoring and 0.951 under Gaussian noise, over the same five pairings of model and representation. Censoring was run on those five pairings only. Mean predicted uncertainty rose with the amount of noise, but no model's per-molecule uncertainty found the corrupted labels. Under six of the seven noise conditions, the Spearman correlation between a molecule's predicted uncertainty and its injected noise stayed between -0.16 and 0.16. Under censoring, the models were most confident on the clipped labels.

Chapter 4 asked whether standard representations, and the distances computed over them, fail on a given dataset. On one dataset, many features of a general-purpose fingerprint do not vary, or do not relate to the property. A distance sums over all of them. Some representations also give different molecules identical feature vectors. In the exploratory study of learning with noisy labels, the configurations that used a distance between molecules did worst. Judging a label by its neighbors' labels usually made the model worse. A representation that avoids these flaws would have features chosen for the task, few of them, drawn from several representations and traceable to their source.

Chapter 5 asked how to select the best set of features for a given prediction task. The adaptive hybrid representation chooses features from several representations for one task, under a budget in features. Every choice is made inside each fold, from that fold's training labels only. The hybrid beat the best single representation on **[RESULT NEEDED: which datasets, with the score pair in each dataset's own metric]**, and did not on **[RESULT NEEDED: which datasets, with the score pair]**. Datasets **[RESULT NEEDED: whether different datasets preferred different search settings]**. **[RESULT NEEDED: transfer result, one line.]** **[RESULT NEEDED: antibody result, one line.]** **[RESULT NEEDED: separate-site result, one line.]** Importance weighting **[RESULT NEEDED: whether it helped during hybrid construction and during model training, in each dataset's own metric]**. The limits of the method are set out in "Limitations", Chapter 5.

Chapter 6 asked whether a representation chosen for the task repairs three things: nearest-neighbor prediction, noise detection, and learning with noisy labels. For nearest-neighbor prediction, **[RESULT NEEDED: answer]**. For noise detec-

tion, **[RESULT NEEDED: answer]**. For learning with noisy labels, **[RESULT NEEDED: answer, and under which noise types any LNL method helped]**.

Chapter 3 tells a reader which model to choose when labels are noisy. Chapters 4 to 6 tell them which features to give it. What works depends on the dataset, as the next section argues. The contributions are the NoiseInject package and its paper, the adaptive hybrid representation and its blueprints, and evidence that distance-based noise handling on standard representations usually made models worse. The papers and software are listed in the Summary of Research Contributions.

7.2 No single method fits every dataset

No single setting of the hybrid search was best on every dataset **[RESULT NEEDED: confirm on the full runs]**. The search settings are the downstream model, the budget, the number of base representations, the importance method, the allocation method and the internal evaluator. The same holds for label noise: which LNL method helps, if any, depends on the data **[RESULT NEEDED: confirm on the full runs]**. Methods for learning with noisy labels exist. Noise mitigation, the subset of them that changes the labels, is the harder part (Chapter 6). We therefore report no overall winner averaged across datasets.

The best choice depends on at least five things. The first is the type of label noise ("By noise type", Chapter 6). The second is the amount of label noise. The third is the number of molecules. A small dataset leaves few molecules inside a fold to score each setting on, so the chosen setting is less stable. The size threshold is given in "Limitations", Chapter 5. The fourth is whether any feature describes the property at all. The search cannot add signal that its inputs do not carry. The fifth is the downstream model. The best hybrid for a tree ensemble need not be the best for a neural network, so the internal evaluator is searched per downstream model.

The evidence is spread across the thesis. In Chapter 3, the representation with the highest $\mathrm{AUC}_{\mathrm{norm}}$ for a model changed between Gaussian and grouped-shifted noise for five of the thirteen base models on QM9. Model ranks also moved with the amount of noise. On ECFP4 under Gaussian noise, random forest went from mid-ranking on clean labels to the top at higher noise levels. Tree and kernel models kept a similar $\mathrm{AUC}_{\mathrm{norm}}$ on every representation. The plain and Bayesian networks and NGBoost depended on the representation. On the three assay datasets, XGBoost and LightGBM ranked lower for robustness than on QM9. In Chapter 5, the winning

allocation method, number of base representations, importance method and budget were **[RESULT NEEDED: per dataset]**. The winning internal evaluator was **[RESULT NEEDED: per dataset and downstream model]**. A fixed share of features per representation won on **[RESULT NEEDED: which datasets]**, and one pooled ranking across all representations won on **[RESULT NEEDED: which datasets]**. In Chapter 6, under censoring, **[RESULT NEEDED: whether raising the clipped labels helped, per dataset and downstream model]**. Under Gaussian noise, the number of the twelve LNL methods that beat the plain network **[RESULT NEEDED: whether it changed with the amount of noise and with the input]**. The LNL methods that helped, including the noise-mitigation ones, were **[RESULT NEEDED: per noise type]**.

The thesis therefore does not offer a fixed recipe, or a new architecture tuned to one problem. It offers a set of methods that might work, and a procedure that gives one supervised task its best chance by choosing its features. For a practitioner, this means there is no default to trust. The useful question is which settings to try, and how to compare them fairly.

7.2.1 Simple is often better

On QM9 under Gaussian noise, the forests, boosted trees, SVM and the GP were both accurate on clean labels and robust to noise. The four plain and Bayesian networks were among the most accurate on clean labels, and the four least robust. On ECFP4 their $\mathrm{AUC}_{\mathrm{norm}}$ ran from 0.881 to 0.897. Their $\mathrm{AUC}_{\mathrm{norm}}$ also varied more across the six representations than any other model's. The simpler choice here is tree ensembles and SVM, not plain neural networks. The variational networks sat in the middle, at $\mathrm{AUC}_{\mathrm{norm}}$ 0.924 and 0.926. Robust is not the same as accurate. NGBoost had the highest $\mathrm{AUC}_{\mathrm{norm}}$ on all six representations under Gaussian noise on QM9, and the lowest clean R^2 on four of the six. Rankings also did not carry over whole to the assay datasets. NGBoost ranked first on QM9 on every representation, but sixth to ninth on hERG K_i. Random forest ranked between first and seventh on every assay dataset and representation, and rose as noise grew.

The hybrid search does not assume that more features are better; it sweeps the budget in features. The best hybrid used fewer features than the largest budget in **[RESULT NEEDED: how often, per dataset and downstream model]**. An ensemble of single-representation models **[RESULT NEEDED: whether it**

matched the hybrid at lower cost, per dataset]. We also predicted each molecule's activity from its five nearest neighbors. We compared a small hybrid chosen for the task with each downstream model's top two single representations. The hybrid scored **[RESULT NEEDED: per dataset, the score pair in the dataset's own metric]**. The model stays simple, and any gain comes from the input.

Several kinds of LNL method did not help. In the exploratory study, the methods that used a distance between molecules did worst (Section 4.4.1). Neighbor-based label cleaning usually made the model worse (Section 4.4.2). Chapter 6 compared twelve LNL methods with one plain neural network under added Gaussian noise ("Gaussian noise: learning-with-noisy-labels methods"). At 0.5 and 1.5 times the labels' standard deviation, **[RESULT NEEDED: how many of the twelve beat the plain network, per dataset, on each model's top two single representations and on the hybrid]**. On real labels with nothing injected, **[RESULT NEEDED: per dataset, whether any cleaning method scored better than leaving the labels as they came]** ("Real labels, nothing injected: cleaning"). Labels clipped at a detection limit are a separate case, where correcting the labels helped **[RESULT NEEDED: confirm on the full runs]**. At separate sites with different noise levels, removing flagged molecules **[RESULT NEEDED: whether it improved on pooling the sites]**.

LNL methods from other fields carry over poorly to assay data, for several reasons. Most were built and tested on large image datasets with flipped class labels [100]. Assay datasets are small: the three in Chapter 3 hold 1,415 to 5,039 molecules. Their labels are continuous, and every label carries some error. Many LNL methods rely on a sample's neighbors, or on how easily the sample is learned. Both need a space where close molecules have similar activity, and Chapter 4 shows that standard representations do not give one. On medium-sized tabular data, tree models still beat deep networks, partly because uninformative features hurt networks more [280]. Financial price data give a contrast. Ma et al. [301] treat labels built from price movements as noisy throughout, because one cannot tell whether a price rose from signal or noise. They replace those labels with labels from a denoising autoencoder, and report better downstream classification. Their labels are still classes, though: the price went up or down.

A method tuned closely to one dataset or task loses generality. The tasks in this thesis span binding affinity, ADME, quantum properties and antibodies. That

range is why learned models are used at all. Simple methods do not generalize automatically either. Feature selection needs tuning per dataset, which is why the search sweeps its settings. The search can also over-fit ("Discussion", Chapter 5). The balance is to fix the simple, robust parts, such as a tree ensemble or neighbors on a small feature set. Only the choices that vary by dataset are searched.

7.3 Limitations

The limits of the hybrid search itself, including run time, repeating the search and reproducibility, are set out in "Limitations", Chapter 5. This thesis set out to mitigate label noise. Our conclusion is that correcting label noise without ground truth is hard, with no clear general solution. Every experimental label carries an unknown amount of noise, so there is no clean copy to check a correction against. Finding a noisy label is easier than correcting it ("By noise type", Chapter 6). Two things are possible: outlier detection, and detection of systematic errors with a known structure. Examples are a shift shared by each source, and clipping at a detection limit. In the assay experiments, "clean" means "no added noise", not "true". Only QM9 is close to noise-free. The injected noise conditions are also models of noise. Real data mix them in unknown amounts, so a method that works under one condition may fail on real data. **[RESULT NEEDED: per noise condition, how many LNL methods, and how many noise-mitigation methods, improved on the uncorrected model, and where recovery went negative.]**

Noise mitigation estimates each label's error and writes the correction into the data before training. A wrong guess then stays in the data for every later model. LNL methods that change training instead adapt throughout training and use all the data [100]. Noise mitigation is cheaper, since it is done once and reused by any model. Correcting labels directly is justified in two cases. One is a known direction, as in censoring, where the limit and the direction are both known. The other is a clearly separate subset, such as gross outliers or an offset tied to one source. Otherwise we recommend LNL methods that change training. Either approach can be paired with uncertainty estimation where it is reliable. The overall level of predicted uncertainty rises with the amount of noise (Chapter 3). Per-molecule uncertainty does not pick out which labels are noisy. Uncertainty should be used to judge how noisy a dataset is, not which labels are wrong.

Other limits concern scope. The label-noise experiments are almost all regression;

only the survey of twelve LNL methods flips class labels, uniformly at random. Molecule-dependent flips are the first next step.

7.4 Future directions

The first next step is classification. The label-noise experiments in this thesis are almost all regression, where a noisy label is a shifted value. The exception is the survey of twelve methods for learning with noisy labels in Chapter 6, which flips a fraction of class labels uniformly at random on its classification datasets. In classification, a noisy label is a flipped class. Many QSAR class labels come from cutting a continuous value at a threshold. Flips then concentrate near the cut-off, so the noise depends on the molecule. NoiseInject already provides six flipping strategies (Section 2.6.1), and the hybrid search already handles classification. What is missing is a study of the molecule-dependent flipping strategies at the scale of the regression study. Most LNL methods that change training were designed for flipped labels, so whether the regression findings carry over is an open question. One limit applies: nearest-neighbor prediction is scored by AUROC, which is defined for two classes. It gives no score with more than two classes, or on a held-out block that holds only one class.

Some inputs and importance methods are not yet searched. The inputs are transformed features, such as continuous features normalized or binarized, and features derived from distances in the molecular graph. The importance methods are those that treat the model as a black box. The software already computes KernelSHAP and permutation importance, but the search sweeps neither. Dropping one feature and refitting is not implemented. The search objective could also reward robustness to label noise as well as accuracy. No such objective is built.

Chapter 6 compares LNL methods under censoring, Gaussian noise and real labels. Outliers and grouped offsets, where a whole laboratory or source document is shifted together, are used there only to grade noise detection. Two questions remain for them. The first is whether cleaning labels by each molecule's nearest neighbors, which fails under Gaussian noise, does better on outliers. The second is whether label smoothing and sample weighting still do best under those noise types.

Hybrid features could also serve as inputs to generative models. The model that scores molecules for a molecule generator could take the hybrid as its input. It is not clear what the search should optimize for that task. Held-out accuracy on known

molecules may not reward the right thing for new ones. There is also a cost: every generated molecule needs its hybrid features computed, including a pass through any embedding model in the hybrid.

As it stands, the full search is too heavy for routine use. Two ways to cut its cost exist. The fast tier searches shorter lists of values (Table 5.7), and adaptive pruning drops configurations early. The next step is a warm start: begin from the blueprint of a similar dataset and search near it. Chapter 5 already reuses one dataset's search settings on a second dataset with no search. A warm start would search near those settings instead. A faster search must still be able to reach every configuration; it should only visit fewer of them. **[RESULT NEEDED: hours and number of model fits for the full search against the fast tier, per dataset.]**

Three further directions are smaller. The first is whether the hybrid improves a weak new embedding, not only absorbs a strong one. That experiment would score the hybrid that includes the embedding against the embedding alone with its best downstream model. It would also score it against a hybrid in which a random matrix of the same width replaces the embedding. The second is blueprints across domains. Individual features cannot transfer between domains, because the representations differ, so transfer would work at the level of representation families. The third is data held at separate sites. Chapter 5 used the hybrid indirectly with such data. The next step is a fuller federated-learning deployment with Lhasa, with real sites running the selection on their own data [287].

7.5 Conclusions

This thesis set out to predict molecular properties from labels that carry experimental noise. The first half asked what makes a QSAR model robust to that noise. Model architecture matters more than representation for how much performance a model keeps as noise rises. The shape of the noise distribution matters little. Noise shared within a scaffold family, and labels clipped at an assay limit, do the most damage. Predicted uncertainty rises with the amount of noise, but it does not pick out which labels are corrupted.

The second half began from a failure. Methods that judge a label by its neighbors usually made models worse on standard representations. On a given dataset, most features of a general-purpose representation do not vary or do not relate to the property, and a distance counts them all (Chapter 4). The adaptive hybrid rep-

resentation instead selects features for the task, from several representations, under a budget in features. **[RESULT NEEDED: one sentence, what Chapter 5 found.] [RESULT NEEDED: one sentence, what Chapter 6 found.]**

Both halves lead to the same advice. Before choosing a model, a representation or a method for learning with noisy labels, ask where the labels came from and what shape their noise is likely to have. Then choose the features for the task, rather than taking one representation whole. No single choice was best on every dataset, so the choices that vary by dataset should be searched and compared fairly.

This matters in practice because representations keep changing. A new foundation model appears every few months in molecular, protein and antibody modeling. With the hybrid search, a researcher does not need to decide whether to replace their current representation or retrain their pipeline. They compute the new model's embeddings, add them to the representations searched, and re-run the search. The search then decides whether the new embedding contributes features, and if so, how many and which. The choice of representation is often set by familiarity or lab convention. The search replaces that choice with one measured on each task, and it can be repeated as new representations appear.

Summary of Research Contributions

Research Papers

In prepration: Punt, A., Morris, G., Werner, S., & Hanser, T. (2026). Improving noise prediction robustness and mitigation with feature-importance hybrid representations in multi-modal contexts
I am in the early stages of writing a draft and extending the experiments in Chapters 5 and 6 within the next few months.

Conference Presentations

Punt, A., Morris, G., Werner, S., & Hanser, T. (2024). Investigating the impact and mitigation of noise in QSAR models. Posted presented at the Autumn UK-QSAR 2024 Meeting.

Blogposts

I have contributed to the *Blopig* blog of the Oxford Protein Informatics Group (OPIG):

- Are you addicted to dopamine?
- Demystifying Git and merge conflicts
- AI in Academic Writing: Ethical Considerations and Best Practices
- Memory-mapped files for efficient data processing
- Exploring multilingual programming

Appendix A

Datasets and curation

This appendix records where each dataset came from, what its label measures, and every curation step applied to it, in the order the steps run. It covers the four datasets of Chapter 3, the 16 datasets of Chapter 5, and the further sets used only for the separate-site experiments of Chapter 5 and in Chapter 6. The label, metric and fold grouping of each of the 16 datasets are summarized in Table 5.9, and the splitting rules are given in the Datasets and splitting subsection of Chapter 5; neither is repeated here. Table A.1 gives the number of samples in each release and after curation. All counts were obtained by running the curation code on the downloaded files.

A.1 Small molecules

Three steps are shared by several small-molecule datasets. The first is structure standardization with RDKit [32]. The steps run in this order: cleanup, keep the largest fragment (removing salts and solvents), neutralize charges, and choose the canonical tautomer. Any molecule that then contains a metal atom is rejected, and the result is written as canonical SMILES. Standardization is applied to hERG K_i, LogD and Caco-2. The Biogen sets, QM9 and the hERG blocker sets are only checked to parse in RDKit.

The second shared step removes duplicate molecules by InChIKey [302], keeping the first occurrence. Two different SMILES strings can describe the same molecule, and without this step one molecule could sit in both a training fold and a test fold. The step is applied to hERG K_i, LogD, Caco-2, the two Biogen sets and QM9. The hERG blocker sets are deduplicated by InChIKey only against the other sites, as

Table A.1: Sample counts before and after curation. One row is one dataset. "Released" is the number of rows in the downloaded file; "Used" is the number after every step in this appendix. The last column names the steps that change the count.

Dataset	Released	Used	Steps that remove samples
Small molecules			
hERG K_i	1,815	1,412	no pChEMBL value (3); not a binding assay (75); repeats merged into one median per compound (322); InChIKey duplicates (3)
LogD	5,326	5,028	no value (287); repeats merged after standardization (11)
Caco-2 efflux	5,326	2,158	no value (3,165); repeats merged after standardization (3)
Rat liver microsome clearance	3,521	3,053	no value (467); InChIKey duplicates (1)
MDR1–MDCK efflux	3,521	2,641	no value (879); InChIKey duplicates (1)
QM9	130,831	9,978	not parsable by RDKit (1,403); random draw of 10,000; InChIKey duplicates (22)
hERG blockers, training set	7,772	7,606	also in the hERG K_i set (166)
hERG blockers, test set	4,383	4,213	also in the hERG K_i set (170)
hERG IC_{50}	12,010	12,008	missing or non-positive value (2)
Chemical reactions			
Buchwald–Hartwig yield	3,955	3,955	none
Buchwald–Hartwig plates	4,599	4,312	no aryl halide (287)
Antibodies			
Koenig expression	4,275	4,275	none
Nanobody thermostability	672	672	none
Humanness	564	564	none
Trastuzumab–HER2 binding	524,346	5,000	stratified random draw of 5,000
Nucleotide sequences			
H3K4me3	18,244	18,244	none (training and test files merged)
TATA promoter	5,274	5,274	none (training and test files merged)
Cancer genomics (TCGA)			
Lung adenocarcinoma vs squamous	1,129	1,017	not a primary tumor (112)
Breast cancer PAM50 subtype	1,097	821	no PAM50 call, or called normal-like (276); count is of primary tumors

described below. The step is skipped on purpose for hERG IC_{50}, which keeps one row per measurement. The third shared step is the fold grouping by Bemis–Murcko scaffold [303], whose form in the hybrid experiments is given in the Datasets and splitting subsection of Chapter 5.

hERG K_i. Records for the hERG potassium channel (ChEMBL target CHEMBL240) were retrieved from ChEMBL [110] through its web interface. The query keeps records with standard type K_i, a pChEMBL value present, an exact relation ("="), and no curator data-validity comment. Records reported as "<" or ">" a value are therefore never included. Only binding assays (ChEMBL assay type B) are kept. The label is the pChEMBL value, $\mathrm{p}K_i = -\log_{10} K_i$ with K_i in molar units. Repeat measurements of one compound are merged into their median, and compounds whose repeats have a standard deviation above 1.0 $\mathrm{p}K_i$ unit are removed, following Landrum and Riniker [16]. The structures are then standardized and deduplicated by InChIKey. Because repeats are merged on ChEMBL's own SMILES before standardization, two ChEMBL entries that standardize to the same molecule are not merged by median; the InChIKey step keeps the first of them. On a pull of September 2026, 1,815 K_i records reduce to 1,812 with a pChEMBL value and 1,737 from binding assays. These give 1,415 compounds, 226 of them measured more than once and none removed by the standard-deviation rule. The InChIKey step removes 3 more, leaving 1,412 compounds with $\mathrm{p}K_i$ from 4.07 to 9.41. The file read by the experiments was pulled on 1 February 2026; the ChEMBL release was not recorded at the time. Re-running the same query in September 2026 against ChEMBL release 37 gives a byte-identical file of 1,415 compounds.

LogD and Caco-2 efflux. Both endpoints come from the training split of the OpenADMET–ExpansionRx challenge data, released on Hugging Face under a CC-BY-4.0 license [109]. The file holds 5,326 molecules; 5,039 have a LogD value and 2,161 a Caco-2 efflux ratio. For each endpoint, rows without a value are dropped and the structures are standardized. Rows that standardize to the same SMILES are merged into their median label. This merges 11 LogD pairs and 3 Caco-2 pairs. The efflux ratio, which has no unit, is then transformed to $\log_{10}$. Labels are clipped at the dataset's own 1st and 99th percentiles: -1.0 and 4.5 for LogD, and -0.319 and 1.679 for $\log_{10}$ efflux ratio. Clipping sets 116 LogD labels (54 low, 62 high) and 45 Caco-2 labels (23 low, 22 high) to a limit. The extreme labels of both sets

are therefore flattened before any noise is added in Chapter 6. The InChIKey step removes nothing on either set, leaving 5,028 and 2,158 molecules.

Rat liver microsome clearance and MDR1–MDCK efflux. These are two endpoints of the Biogen ADME set of 3,521 compounds [210], downloaded from Polaris as `biogen/adme-fang-v1` [211]. Rat liver microsome clearance is the intrinsic clearance $\mathrm{CL}_{\mathrm{int}}$ in mL/min/kg, and the MDR1–MDCK endpoint is an efflux ratio. Both are released already as $\log_{10}$ values and are used as released. Rows without a value, or whose SMILES RDKit cannot parse, are dropped, and duplicates are removed by InChIKey. There is no standardization, no median merge and no clipping. Clearance has 3,054 values and efflux 2,642; the InChIKey step removes one molecule from each, leaving 3,053 and 2,641.

QM9. QM9 [107, 212] was read through PyTorch Geometric [112], which distributes 130,831 molecules. Of these, 129,428 parse in RDKit; the other 1,403 carry an over-valent carbon atom in the released structure file. This list of parsable molecules is identical to the one used in Chapter 3. From it, 10,000 molecules were drawn once at random with seed 42, so the subsample is a random draw of molecules, not a random split. The InChIKey step removes 22 duplicates, leaving 9,978. The label is the HOMO–LUMO gap in eV, from 1.88 to 11.37 eV in this sample.

hERG blockers held at separate sites. The separate-site experiments of Chapter 5 use two further hERG sets from Lhasa Limited: a training set of 7,772 molecules (3,630 blockers, 46.7%) and a test set of 4,383 molecules (1,084 blockers). Both were provided through the Effiris and FLuID collaboration [18, 19]. Each label is Lhasa's own binary blocker call, and the two sets were labeled under different rules. The molecules are stored as RDKit molecule objects and converted to SMILES. Entries that give no valid SMILES are dropped, which removes none. Neither set is standardized. The sets are proprietary and cannot be redistributed (Appendix B). In the same experiments the hERG K_i set acts as a third site. Its labels are made binary, with a compound counted as a blocker when $\mathrm{p}K_i \geq 6.0$ ($K_i \leq 1\,\mu\mathrm{M}$), which gives 467 blockers and 945 non-blockers. Molecules are then matched by InChIKey across the three sites, taken in the order hERG K_i, training set, test set, and only the first copy is kept. This removes 166 molecules from the training set and 170 from the test set, all of them also in the hERG K_i set, leaving 7,606 and 4,213. The two

Lhasa sets share no molecule. For 56 of the 166 and 57 of the 170 shared molecules, the Lhasa blocker call disagrees with the call from pK_i. The training and test sets fall into 4,002 and 2,298 scaffold groups before this step, and 3,903 and 2,222 after it.

hERG IC_{50}. This set is used only in Chapter 6, where the disagreement between repeat measurements and the curator flags serve as real evidence of label error. It keeps every measurement as its own row. The query takes ChEMBL records for CHEMBL240 with standard type IC_{50} in nM and an exact relation [110]. Records with a missing or non-positive value are dropped, and the label is $pIC_{50} = 9 - \log_{10}(IC_{50}/\text{nM})$. No assay-type filter is applied. Of the 12,010 records, 10,219 come from binding assays, 1,543 from toxicity assays, 217 from functional assays and 31 from ADME assays, as ChEMBL classifies them. Records with a curator data-validity comment are kept and marked; there are 301 of them, on 215 molecules. There is no standardization, no median merge and no InChIKey step. The result is 12,008 measurements of 9,802 molecules. Folds are grouped by scaffold, and every measurement of a molecule shares that molecule's scaffold, so no molecule is split between training and test.

A.2 The Chapter 3 versions of LogD, Caco-2, hERG K_i and QM9

The datasets used in this study are publicly available. The QM9 dataset is available on Figshare [212] and is also redistributed through the OpenADMET benchmark suite [107, 109]. The LogD and Caco-2 datasets are available in the OpenADMET repository [109]. The hERG-Ki dataset was derived from ChEMBL [110] following the curation protocol described in Section 2, and can be regenerated from ChEMBL using that protocol.

Chapter 3 used the same sources as Chapter 5 for LogD, Caco-2 and hERG K_i, but a lighter curation. The differences are listed in Table A.2. They explain why the sample counts differ between the two chapters. The QM9 experiments of Chapter 3 draw ten replicates of 10,000 molecules from the same 129,428 parsable molecules. Each replicate has its own seed and its own 80/10/10 scaffold split into 8,000, 1,000 and 1,000 molecules. Chapter 3 also merges hERG K_i repeats a second time, on its

own standardized SMILES; on this file that merges nothing, so its set keeps 1,415 compounds.

Table A.2: Curation of the three assay datasets in Chapters 3 and 5. One row is one curation step or one count.

	Chapter 3	Chapter 5
Structure standardization	largest fragment and canonical SMILES only	cleanup, largest fragment, neutralized charges, canonical tautomer, metals rejected
Label clipping (LogD, Caco-2)	none	1st and 99th percentile
InChIKey deduplication	none	first occurrence kept
Scaffold key	stereochemistry removed; each acyclic molecule its own group	stereochemistry kept; all acyclic molecules one group
Molecules: LogD, Caco-2, hERG K_i	5,039; 2,161; 1,415	5,028; 2,158; 1,412
Scaffold groups: LogD, Caco-2, hERG K_i	2,245; 1,038; 749	2,413; 1,086; 769

A.3 Chemical reactions

Both reaction datasets come from the high-throughput Buchwald–Hartwig C–N coupling experiments of Ahneman et al. [213], in which aryl halide, palladium ligand, base and additive were varied on reaction plates. The label is the reaction yield in %, from 0 to 100. Here one sample is a whole reaction, written as its reactants and product, so the representations and the grouping differ from those of the single-molecule sets.

Buchwald–Hartwig yield. This is the version released with Schwaller et al. [304]: the first cross-validation sheet of the input workbook in that paper's public repository. All 16 sheets of the workbook hold the same 3,955 reactions. Each reaction is written as aryl halide, 4-methylaniline, ligand, base and additive going to a product. The product is generated by applying a C–N coupling template to the aryl halide. A reaction would be dropped if the template failed, which it never does, so all 3,955 reactions are kept. They cover 15 aryl halides, 4 ligands, 3 bases and 22 additives, and fall into 22 additive groups of 178 to 180 reactions.

Buchwald–Hartwig plates. This is the data table of the public code repository of Ahneman et al. [213], which holds 4,599 rows over three plates. Its product SMILES are taken from the file. The 287 rows without an aryl halide are dropped. The 180 remaining rows without an additive are kept and form their own group, so the set has 24 additive groups of 177 to 180 reactions. The 4,312 reactions kept are not independent of the first set. Every one of the 3,955 reactions in the first set appears here, with the same aryl halide, ligand, base, additive and yield. The plates set adds 357 reactions: the 180 without an additive and 177 with 5-phenyl-1,2,4-oxadiazole as the additive.

A.4 Antibodies

The Koenig and nanobody sets were downloaded as CSV files from the FLAb benchmark repository [215]. In both, rows with a missing heavy chain, a heavy chain shorter than 11 residues, or a missing or non-numeric label are dropped; no row is removed by this rule. The humanness set was downloaded from the BioPhi publication repository and the trastuzumab set from its own repository. The FLAb and BioPhi files are taken from each repository's main branch, with no commit pinned. Each file has been changed only once: the Koenig file in FLAb commit `c41e56e` and the nanobody file in commit `005e6c1`, both on 6 August 2025, and the two humanness files in BioPhi commit `c0797f2` on 24 August 2021. The local copies of the Koenig and humanness files are identical to those commits.

The nanobody and humanness sets are grouped by sequence identity. The identity of two sequences a and b is

$$\mathrm{id}(a, b) = 1 - \frac{\mathrm{lev}(a, b)}{\max(|a|, |b|)}, \tag{A.1}$$

where lev is the Levenshtein (edit) distance and $|a|$ is the length in residues. Sequences are clustered by single linkage at id ≥ 0.95: two sequences share a cluster if a chain of pairs, each at identity 0.95 or more, joins them. For a set with paired chains, heavy and light chain would be joined into one string before clustering, as in Kenlay et al. [159]; both clustered sets here carry one chain per antibody. The clusters are not one antibody each. The 672 nanobodies fall into 309 clusters, of which 185 hold one nanobody and the largest holds 32. The 564 humanness antibodies fall into 461 clusters, of which 404 hold one antibody and the largest holds 13.

Koenig expression. This is a deep mutational scan of the anti-VEGF antibody G6 [214], taken from FLAb. The label is the expression enrichment ratio, which has no unit and runs from 0.00098 to 3.178. All 4,275 variants carry both chains, with heavy chains of 120 residues and light chains of 108. Each variant carries exactly one mutation: 2,261 in the heavy chain and 2,014 in the light chain, at 225 distinct positions. No two variants have the same pair of sequences.

Nanobody thermostability. This is the NbThermo set [216], taken from FLAb. The label is the melting temperature T_m in °C, from 39.0 to 98.2 °C, for 672 single-domain antibodies with no light chain. The melting temperatures were measured with four main methods and pooled into one label. The methods are circular dichroism (307), differential scanning fluorimetry with SYPRO dye (254), nanoDSF (76), differential scanning calorimetry (30), and other methods (5). The domains come from llama (395), camel (189), alpaca (36), shark (30), human (2) and unrecorded species (20). The 672 rows hold 556 distinct sequences. Each of 116 sequences appears twice, and 106 of those pairs carry two different melting temperatures, differing by up to 8.8 °C. These repeats are kept, so this set carries disagreement between measurements in its own labels before any noise is added.

Humanness. This is the list of therapeutic antibodies from IMGT/mAb-DB released with BioPhi [217], 564 antibodies with one sequence each. The sequence used is the first one listed for each antibody in the released sequence file. An antibody is labeled human (1) only when its species field is exactly "Human", which gives 198 human antibodies. The other 366 are labeled non-human (0): 229 humanized, 63 chimeric, 41 humanized/chimeric, 13 mouse, 7 human/humanized, 6 caninized, 4 human/chimeric and 3 felinized. The 564 antibodies hold 506 distinct sequences, mostly because an antibody and its drug conjugate share a sequence. One sequence is shared by three antibodies with different labels, two humanized and one human. All three sit in one identity cluster, so they never fall on both sides of a split.

Trastuzumab–HER2 binding. This is the library of variants of the ten-residue CDR-H3 (the third complementarity-determining region of the heavy chain) of trastuzumab, screened for binding to HER2 by Chinery et al. [218]. The released training, validation and test files are joined and re-split. Variants whose CDR-H3 is not exactly 10 residues would be dropped; there are none, so all 524,346 are kept:

172,149 binders (32.8%) and 352,197 non-binders. From these, 5,000 variants were drawn once with seed 42. The draw is stratified on each combination of label and number of mutated CDR-H3 residues, so each combination keeps its share of the full set. The sample holds 1,639 binders (32.8%). Each full heavy chain of 120 residues is rebuilt by inserting the variant CDR-H3 into the trastuzumab framework. The light chain is the fixed trastuzumab light chain, the same on every row. The variants carry 7.3 of the 10 CDR-H3 residues mutated on average.

A.5 Nucleotide sequences

Both nucleotide tasks come from the revised downstream tasks of the Nucleotide Transformer [219], downloaded from Hugging Face (`InstaDeepAI/nucleotide_transformer_downst`
The download is not pinned to a revision; the files used are those of revision `851f994`, dated 30 June 2025, the latest at the time of writing. The released training and test files are merged. Each sequence's chromosome is read from its name, and the folds are grouped by chromosome. Nothing else is filtered, and no sequence contains a letter other than A, C, G or T. Both tasks cover chromosomes 1 to 21, X and Y; chromosome 22 is absent.

H3K4me3. Each sample is a 1,000-base sequence, labeled by whether it carries the H3K4me3 histone mark. The released files hold 17,468 training and 776 test sequences, 18,244 in all, with 9,122 in each class. The dataset card lists 30,000 training sequences; the count here is that of the files.

TATA promoter. Each sample is a 300-base sequence, labeled by whether it is a promoter with a TATA box. The released files hold 5,062 training and 212 test sequences, 5,274 in all, with 2,637 in each class.

A.6 Cancer genomics (TCGA)

The two cancer datasets use data from The Cancer Genome Atlas [305], downloaded through UCSC Xena [222]. Expression is read from each cohort's Illumina HiSeq RNA-seq matrix on the Xena TCGA hub (version 2017-10-13). It holds 20,530 genes as $\log_2(x+1)$ of the RSEM-normalized count [306] from the UNC RNA-seq pipeline. The same 20,530 genes appear in every cohort, and no gene filter is applied. Only

primary tumors are kept (sample-type code 01 in the TCGA barcode). Barcodes are cut to the sample level, which leaves one sample per patient. Missing expression values are set to 0. There is no batch correction. Every fold holds out whole tissue source sites. A correction fitted on the training sites has no parameter for a site it never saw. A correction fitted on all sites has already seen the test samples.

Lung adenocarcinoma against lung squamous cell carcinoma. The adenocarcinoma [220] and squamous cell carcinoma [221] cohorts are stacked, and the cohort is the label. The two matrices hold 576 and 553 samples. Dropping 110 normal-tissue and 2 recurrent-tumor samples leaves 515 adenocarcinomas and 502 squamous tumors, 1,017 in all, from 68 tissue source sites.

Breast cancer PAM50 subtype. The label is the PAM50 subtype call [224] from the breast cancer clinical matrix on Xena [223]. Four classes are kept: luminal A, luminal B, basal-like and HER2-enriched. Tumors called normal-like, and tumors without a call, are dropped. Of 1,097 primary tumors with expression data, 821 remain: 421 luminal A, 192 luminal B, 141 basal-like and 67 HER2-enriched, from 19 tissue source sites.

Appendix B

Data and code availability

Three pieces of software lie behind this thesis. NoiseInject is the label-noise package released with Chapter 3. A separate pipeline ran the QM9 experiments of that chapter. The KIRBy repository implements the adaptive hybrid representation and holds the experiments of Chapters 4, 5 and 6, together with the assay-dataset runs of Chapter 3. The three are separate, so each can be reused on its own. For example, the hybrid search runs on feature matrices a reader has computed elsewhere, without the Rust part of the QM9 pipeline. The source and curation of each dataset are given in Appendix A.

The NoiseInject benchmarking framework, implementing all six regression and six classification noise injection strategies together with the robustness metrics (noise degradation slope and retention) and uncertainty-calibration metrics (ECE, coverage at $1\sigma/2\sigma$, mean interval width, and uncertainty–error / uncertainty–noise correlation) described in this work, is available as an open-source Python package under an MIT license [104]. It also contains optional reference wrappers (a Gaussian process, split-conformal prediction, and Monte Carlo dropout) that produce per-sample uncertainty for use with these metrics. NoiseInject is model-agnostic, operates on label arrays (NumPy arrays or pandas Series), and is designed to integrate into existing cheminformatics or other data pipelines. Worked examples covering molecular-descriptor, fingerprint, and GNN-embedding workflows are provided in the repository.

Its noise conditions and metrics are described in Section 2.5.1.

- Project name: NoiseInject
- Project home page: `https://github.com/adpunt/noise_inject`

- Archived version: https://doi.org/10.5281/zenodo.20532710
- Version: 1.0.0
- Operating system(s): Platform independent
- Programming language: Python 3.7 or later
- Other requirements: NumPy, pandas, SciPy and scikit-learn; see the repository's `requirements.txt`
- License: MIT
- Any restrictions to use by non-academics: None

The QM9 results of Chapter 3 were produced by a separate repository, written in Python and Rust, which also holds the hyperparameter tuning, the variance decomposition and the figure scripts for that chapter. It is described in Chapter 3, under "The QM9 precompute pipeline". The three assay datasets of that chapter were run from the KIRBy repository, which calls NoiseInject directly.

- Project name: qsar_qm_models
- Project home page: https://github.com/adpunt/qsar_qm_models
- Archived version: not yet archived
- Operating system(s): Linux and macOS
- Programming language: Python and Rust (2021 edition)
- Other requirements: RDKit installed through conda, to which the Rust program links; PyTorch and PyTorch Geometric
- License: GNU GPL v3.0
- Any restrictions to use by non-academics: those of the GNU GPL v3.0

The adaptive hybrid representation is implemented in the KIRBy repository. It is written in Python and works on feature matrices computed beforehand, so the hybrid search can be applied to representations from any domain. The repository contains the hybrid search itself: feature importance scoring, the allocation of a feature budget across representations, internal evaluation, and a store of every

configuration evaluated for each dataset. It also contains loaders for the datasets in Appendix A and generators for the molecular, antibody, nucleotide, reaction and gene-expression representations. The neighbor-based similarity analyzes, the learning-with-noisy-labels methods of Chapter 6 and every experiment script are included as well. Label noise is added through NoiseInject, so the noise definitions are the same as in Chapter 3. A GPU is not needed for the hybrid search, but it speeds up the pretrained encoders.

- Project name: KIRBy
- Project home page: `https://github.com/adpunt/KIRBy`
- Archived version: [Zenodo DOI]
- Operating system(s): Linux and macOS
- Programming language: Python 3.8 or later; several pretrained encoders need Python 3.10 or later
- Other requirements: NumPy, pandas, SciPy, scikit-learn, and RDKit installed through conda; each application area has an optional install
- License: [license]
- Any restrictions to use by non-academics: [depends on the license]

All datasets but one are publicly available. QM9 [107] is deposited on Figshare [212]. The LogD and Caco-2 efflux data come from the OpenADMET–ExpansionRx challenge release, under a CC-BY-4.0 license [109]. The hERG K_i data were extracted from ChEMBL [110], whose data are released under CC BY-SA 3.0. They can be regenerated with the curation protocol in Appendix A. The rat liver microsome clearance and MDR1–MDCK efflux data are the Biogen ADME set [210], released on Polaris [211] under CC-BY-4.0. The reaction-yield, antibody, nucleotide and tumor gene-expression datasets were downloaded from their public releases, which are cited in Appendix A. Pretrained model weights were downloaded from their original sources and are not redistributed.

The exception is the hERG blocker data used for feature selection with data held at separate sites (Chapter 5). These are Lhasa Limited's curated hERG training set of 7,772 molecules and test set of 4,383 molecules [139]. They are proprietary

and are not redistributed. FLuID's separate public set of molecules is not used; the training molecules take its place, as stated in Chapter 5. The comparison with FLuID also calls Lhasa's FLuID code and cannot be run without it. Every other result in this thesis can be reproduced from public data and the code above.

The adaptive hybrid representation, the KIRBy repository, and the experiments of Chapters 4, 5 and 6 were designed, implemented, run and written up by A.P. alone. Lhasa Limited supplied the curated hERG data and the FLuID code [139]. The use of generative AI is described in the declaration in the front matter.

Bibliography

[1] Markus A. Lill. Multi-dimensional QSAR in drug discovery. *Drug Discovery Today*, 12(23):1013–1017, 2007. ISSN 1359-6446.

[2] Bruno J. Neves, Rodolpho C. Braga, and Cleber C. Melo-Filho et al. QSAR-based virtual screening: Advances and applications in drug discovery. *Frontiers in Pharmacology*, 9:1275, 2018. ISSN 1663-9812.

[3] María Virginia Sabando, Ignacio Ponzoni, Evangelos E Milios, and Axel J Soto. Using molecular embeddings in QSAR modeling: Does it make a difference? *Briefings in Bioinformatics*, 23(1):bbab365, 2022. ISSN 1477-4054.

[4] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t. PMID: 20426451.

[5] Luís David, Abhijit Thakkar, Raul Mercado, et al. Molecular representations in AI-driven drug discovery: a review and practical guide. *Journal of Cheminformatics*, 12(1):56, 2020.

[6] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Sort & slice: a simple and superior alternative to hash-based folding for extended-connectivity fingerprints. *Journal of Cheminformatics*, 16(1), December 2024. ISSN 1758-2946. doi: 10.1186/s13321-024-00932-y.

[7] Peter Gedeck, Bernhard Rohde, and Christian Bartels. QSAR — how good is it in practice? Comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5): 1924–1936, 2006. doi: 10.1021/ci050413p.

[8] Akihiro Kishimoto, Hiroshi Kajino, Masataka Hirose, Junta Fuchiwaki, Indra Priyadarsini, Lisa Hamada, Hajime Shinohara, Daiju Nakano, and Seiji

Takeda. Mhg-gnn: Combination of molecular hypergraph grammar with graph neural network, 2023.

[9] Scott S. Kolmar and Christopher M. Grulke. The effect of noise on the predictive limit of QSAR models. *Journal of Cheminformatics*, 13(1):92, 2021.

[10] Kjell Jorner, Tore Brinck, Per-Ola Norrby, and David Buttar. Machine learning meets mechanistic modelling for accurate prediction of experimental activation energies. *Chem. Sci.*, 12:1163–1175, 2021. doi: 10.1039/D0SC04896H.

[11] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and qsar modeling research. *Journal of Chemical Information and Modeling*, 50 (7):1189–1204, 2010. doi: 10.1021/ci100176x. PMID: 20572635.

[12] Xiuwen Fang and Mang Ye. Robust federated learning with noisy and heterogeneous clients. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10062–10071, 2022.

[13] W. Patrick Walters. We need better benchmarks for machine learning in drug discovery. Practical Cheminformatics Blog, 2023. URL http://practicalcheminformatics.blogspot.com/2023/08/we-need-better-benchmarks-for-machine.html. Accessed: 2026-04-12.

[14] Tuomo Kalliokoski, Christian Kramer, Anna Vulpetti, and Peter Gedeck. Comparability of mixed IC50 data – a statistical analysis. *PLoS ONE*, 8 (4):e61007, 2013. doi: 10.1371/journal.pone.0061007.

[15] Christian Kramer, Tuomo Kalliokoski, Peter Gedeck, and Anna Vulpetti. The experimental uncertainty of heterogeneous public Ki data. *Journal of Medicinal Chemistry*, 55(11):5165–5173, 2012. doi: 10.1021/jm300131x.

[16] Gregory A. Landrum and Sereina Riniker. Combining IC50 or Ki values from different sources is a source of significant noise. *Journal of Chemical Information and Modeling*, 64(5):1560–1567, 2024. doi: 10.1021/acs.jcim.4c00049. PMID: 38394344.

[17] Isidro Cortes-Ciriano, Andreas Bender, and Thérèse E. Malliavin. Comparing the influence of simulated experimental errors on 12 machine learning algorithms in bioactivity modeling using 12 diverse data sets. *Journal of Chemical Information and Modeling*, 55(7):1413–1425, 2015. doi: 10.1021/acs.jcim.5b00101. PMID: 26038978.

[18] Davide Bassani, Alessandro Brigo, and Andrea Andrews-Morger. Federated learning in computational toxicology: An industrial perspective on the effiris hackathon. *Chemical Research in Toxicology*, 36(9):1503–1517, 2023.

[19] Thierry Hanser. Federated learning for molecular discovery. *Current Opinion in Structural Biology*, 79:102545, 2023. ISSN 0959-440X.

[20] David Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988.

[21] Michael Backenköhler, Paula Linh Kramer, and Joschka et al. Groß. TeachOpenCADD goes Deep Learning: Open-source Teaching Platform Exploring Molecular DL Applications. *ChemRxiv preprint*, 2023. doi: 10.26434/chemrxiv-2023-kz1pb.

[22] Alexandre V. Fassio, Laura Shub, and Luca Ponzoni et al. Prioritizing virtual screening with interpretable interaction fingerprints. *Journal of Chemical Information and Modeling*, 62(18):4300–4318, 2022.

[23] Liangxu Xie, Lei Xu, Ren Kong, Shan Chang, and Xiaojun Xu. Improvement of prediction performance with conjoint molecular fingerprint in deep learning. *Frontiers in Pharmacology*, 11, 2020. doi: 10.3389/fphar.2020.606668.

[24] Isidro Cortés-Ciriano, Ctibor Škuta, Andreas Bender, and Daniel Svozil. QSAR-derived affinity fingerprints (part 2): Modeling performance for potency prediction. *Journal of Cheminformatics*, 12(1):41, 2020.

[25] Sabrina Jaeger, Simone Fulle, and Samo Turk. Mol2vec: Unsupervised machine learning approach with chemical intuition. *Journal of Chemical Information and Modeling*, 58(1):27–35, 2018. doi: 10.1021/acs.jcim.7b00616.

[26] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta: Large-scale self-supervised pretraining for molecular property prediction. *arXiv preprint arXiv:2010.09885*, 2020.

[27] Benedek Fabian, Thomas Edlich, and Héléna et al. Gaspar. Molecular representation learning with language models and domain-relevant auxiliary tasks. *arXiv preprint arXiv:2011.13230*, 2020.

[28] Walid Ahmad, Elana Simon, Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta-2: Towards chemical foundation models. *arXiv preprint arXiv:2209.01712*, 2022.

[29] Roberto Todeschini and Viviana Consonni. *Molecular Descriptors for Chemoinformatics*. Wiley-VCH, Weinheim, 2nd edition, 2009. doi: 10.1002/9783527628766.

[30] Andrea Mauri, Viviana Consonni, Manuela Pavan, and Roberto Todeschini. DRAGON software: An easy approach to molecular descriptor calculations. *MATCH Communications in Mathematical and in Computer Chemistry*, 56 (2):237–248, 2006.

[31] Artem Cherkasov, Eugene N. Muratov, and Denis et al. Fourches. QSAR modeling: Where have you been? where are you going to? *Journal of Medicinal Chemistry*, 57(12):4977–5010, 2014. doi: 10.1021/jm4004285.

[32] Greg Landrum. Rdkit: Open-source cheminformatics. https://www.rdkit.org. Accessed: 2026-01-09.

[33] Zhichun Guo, Kehan Guo, and Bozhao Nan et al. Graph-based molecular representation learning, 2023. Accessed: 2024-11-30.

[34] Stefan M.N. Hönig, Carsten Lemmen, and Matthias Rarey. Small molecule superposition: A comprehensive overview on pose scoring of the latest methods. *WIREs Computational Molecular Science*, 13(2):e1640, 2023.

[35] Ge Yan, Huaijin Wu, and Junchi Yan. Quantum 3D graph learning with applications to molecule embedding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 39126–39137. PMLR, 2023.

[36] Ziduo Yang, Weihe Zhong, Qiujie Lv, Tiejun Dong, and Calvin Yu-Chian Chen. Geometric interaction graph neural network for predicting protein–ligand binding affinities from 3D structures (GIGN). *The Journal of Physical Chemistry Letters*, 14(8):2020–2033, 2023. doi: 10.1021/acs.jpclett.2c03906.

[37] Wenjie Du, Xiaoting Yang, and Di et al. Wu. Fusing 2D and 3D molecular graphs as unambiguous molecular descriptors for conformational and chiral stereoisomers. *Briefings in Bioinformatics*, 24(1):bbac560, 2022. ISSN 1477-4054.

[38] Denis Kuzminykh, Daniil Polykovskiy, and Artur Kadurin et al. 3D molecular representations based on the wave transform for convolutional neural networks. *Molecular Pharmaceutics*, 15(10):4378–4385, 2018.

[39] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1):657–668, 2023.

[40] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties, 2022.

[41] Esther Heid, Charles J. McGill, Florence H. Vermeire, and William H. Green. Characterizing uncertainty in machine learning for chemistry. *Journal of Chemical Information and Modeling*, 63(13):4012–4029, 2023. doi: 10.1021/acs.jcim.3c00373. PMID: 37338239.

[42] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Monterey, CA, 1984. ISBN 978-0-412-04841-8.

[43] Vladimir Svetnik, Andy Liaw, Christopher Tong, John Culberson, Robert Sheridan, and Bradley Feuston. Random forest: A classification and regression tool for compound classification and qsar modeling. *Journal of chemical information and computer sciences*, 43:1947–58, 11 2003. doi: 10.1021/ci034160g.

[44] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[45] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[46] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

[47] Junshui Ma, Robert P Sheridan, Andy Liaw, George E Dahl, and Vladimir Svetnik. Deep neural nets as a method for quantitative structure-activity relationships. *Journal of Chemical Information and Modeling*, 55(2):263–274, 2015. doi: 10.1021/ci500747n.

[48] Zhenqin Wu, Bharath Ramsundar, Evan N. Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S. Pappu, Karl Leswing, and Vijay Pande. Moleculenet: A benchmark for molecular machine learning, 2018.

[49] Igor I. Baskin, Vladimir A. Palyulin, and Nikolai S. Zefirov. Neural networks in building QSAR models. In *Methods in Molecular Biology*, volume 458, pages 137–158. Humana Press, 2008. doi: 10.1007/978-1-60327-101-1_8.

[50] Alexios Koutsoukas, Keith J. Monaghan, Xiaoli Li, and Jun Huan. Deep-learning: investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data. *Journal of Cheminformatics*, 9:42, 2017. doi: 10.1186/s13321-017-0226-y.

[51] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

[52] Curtis G Northcutt, Lu Jiang, and Isaac L Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021. doi: 10.1613/jair.1.12125.

[53] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks?, 2019.

[54] Nicolai Meinshausen. Quantile regression forests. *Journal of Machine Learning Research*, 7:983–999, 06 2006.

[55] Ismail Babajide Mustapha and Faisal Saeed. Bioactive molecule prediction using extreme gradient boosting. *Molecules*, 21(8), 2016. ISSN 1420-3049. doi: 10.3390/molecules21080983.

[56] Hao Tian, Rajas Ketkar, and Peng Tao. Accurate admet prediction with xgboost, 2022.

[57] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154, 2017.

[58] Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. Ngboost: Natural gradient boosting for probabilistic prediction, 2020.

[59] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995. doi: 10.1007/978-1-4757-2440-0.

[60] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004. doi: 10.1023/B:STCO.0000035301.49549.88.

[61] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi: 10.1007/BF00994018.

[62] Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3): 1–27, 2011. doi: 10.1145/1961189.1961199.

[63] Olga Obrezanova, Joelle M.R. Gola, and Matthew D. Segall. Gaussian processes: A method for automatic qsar and adme modelling. Presentation at ACS Boston, 22 August 2007, 2007.

[64] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 2005. ISBN 9780262256834.

[65] Joaquin Quiñonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate gaussian process regression. *Journal of Machine Learning Research*, 6(65):1939–1959, 2005.

[66] Ryan-Rhys Griffiths and Leo Klarner et al. Gauche: A library for gaussian processes in chemistry, 2022. Accessed: 2024-08-21.

[67] Swamidass S. Joshua Ralaivola, L., Hiroto Saigo, and Pierre Baldi. Graph kernels for chemical informatics. *Neural networks : the official journal of the International Neural Network Society*, 18:1093–110, 11 2005. doi: 10.1016/j.neunet.2005.07.009.

[68] Henry Moss and Ryan-Rhys Griffiths. Gaussian process molecule property prediction with flowmo, 10 2020.

[69] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles, 2017.

[70] Glenn Palmer, Siqi Du, Alexander Politowicz, Joshua Emory, Xiyu Yang, Anupraas Gautam, Grishma Gupta, Zhelong Li, Ryan Jacobs, and Dane Morgan. Calibration after bootstrap for accurate uncertainty quantification in regression models. *npj Computational Materials*, 8:115, 05 2022. doi: 10.1038/s41524-022-00794-8.

[71] Dávid Bajusz, Anita Rácz, and Károly Héberger. Why is tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7(1), May 2015. ISSN 1758-2946. doi: 10.1186/s13321-015-0069-3.

[72] Dagmar Stumpfe and Jürgen Bajorath. Exploring activity cliffs in medicinal chemistry. *Journal of Medicinal Chemistry*, 55(7):2932–2942, 2012. doi: 10.1021/jm201706b.

[73] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Exploring QSAR models for activity-cliff prediction. *Journal of Cheminformatics*, 15(1):47, 2023. doi: 10.1186/s13321-023-00708-w.

[74] Mark A. Johnson and Gerald M. Maggiora, editors. *Concepts and Applications of Molecular Similarity*. Wiley, New York, 1990.

[75] Derek van Tilborg, Alisa Alenicheva, and Francesca Grisoni. Exposing the limitations of molecular machine learning with activity cliffs. *Journal of Chemical Information and Modeling*, 62(23):5938–5951, 2022. doi: 10.1021/acs.jcim.2c01073.

[76] Gerald Maggiora, Martin Vogt, Dagmar Stumpfe, and Jürgen Bajorath. Molecular similarity in medicinal chemistry. *Journal of Medicinal Chemistry*, 57(8):3186–3204, 2014. doi: 10.1021/jm401411z.

[77] Gerald M. Maggiora. On outliers and activity cliffs—why QSAR often disappoints. *Journal of Chemical Information and Modeling*, 46(4):1535, 2006. doi: 10.1021/ci060117s.

[78] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434, Berlin, Heidelberg, 2001. Springer. doi: 10.1007/3-540-44503-X_27.

[79] Felix A. Krüger and John P. Overington. Global analysis of small molecule binding to related protein targets. *PLoS Computational Biology*, 8(1): e1002333, 2012. doi: 10.1371/journal.pcbi.1002333.

[80] Joe Bentz, Michael P. O'Connor, Dallas Bednarczyk, JoAnn Coleman, Caroline Lee, Johan Palm, Y. Anne Pak, Elke S. Perloff, Eric Reyner, Praveen Balimane, et al. Variability in P-glycoprotein inhibitory potency (IC50) using various in vitro experimental systems: Implications for universal digoxin drug-drug interaction risk assessment decision criteria. *Drug Metabolism and Disposition*, 41(7):1347–1366, 2013. doi: 10.1124/dmd.112.050500.

[81] Emma Svensson, Hannah Rosa Friesacher, Susanne Winiwarter, Lewis Mervin, Adam Arany, and Ola Engkvist. Enhancing uncertainty quantification in drug discovery with censored regression labels. *Artificial Intelligence in the Life Sciences*, 7:100128, 2025. doi: 10.1016/j.ailsci.2025.100128.

[82] Ricardo Macarron, Martyn N. Banks, and Dejan et al. Bojanic. Impact of high-throughput screening in biomedical research. *Nature Reviews Drug Discovery*, 10(3):188–195, 2011. doi: 10.1038/nrd3368.

[83] James Inglese, Douglas S. Auld, and Ajit et a.. Jadhav. Quantitative high-throughput screening: A titration-based approach that efficiently identifies biological activities in large chemical libraries. *Proceedings of the National Academy of Sciences*, 103(31):11473–11478, 2006. doi: 10.1073/pnas.0604348103.

[84] Christine Brideau, Bert Gunter, Bill Pikounis, and Andy Liaw. Improved statistical methods for hit selection in high-throughput screening. *Journal of Biomolecular Screening*, 8(6):634–647, 2003. doi: 10.1177/1087057103258285.

[85] Ji-Hu Zhang, Thomas D. Y. Chung, and Kevin R. Oldenburg. A simple statistical parameter for use in evaluation and validation of high throughput screening assays. *Journal of Biomolecular Screening*, 4(2):67–73, 1999. doi: 10.1177/108705719900400206.

[86] Anton Simeonov, Ajit Jadhav, and Craig J. et al. Thomas. Fluorescence spectroscopic profiling of compound libraries. *Journal of Medicinal Chemistry*, 51(8):2363–2371, 2008. doi: 10.1021/jm701301m.

[87] Christos Hatzis, Philippe L. Bedard, Nicolai J. Birkbak, Andrew H. Beck, Hugo J.W.L. Aerts, David F. Stern, Leming Shi, Robert Clarke, John Quackenbush, and Benjamin Haibe-Kains. Enhancing reproducibility in cancer drug screening: How do we move forward? *Cancer Research*, 74(15):4016–4023, July 2014. ISSN 1538-7445. doi: 10.1158/0008-5472.can-14-0725.

[88] Elena Postnikova, Yu Cong, Lisa Evans DeWald, and Julie et al. Dyall. Testing therapeutics in cell-based assays: Factors that influence the apparent potency of drugs. *PLOS ONE*, 13(3):e0194880, March 2018. ISSN 1932-6203. doi: 10.1371/journal.pone.0194880.

[89] Anna Gaulton, Anne Hersey, and Michał et al. Nowotka. The ChEMBL database in 2017. *Nucleic Acids Research*, 45(D1):D945–D954, 2017. doi: 10.1093/nar/gkw1074.

[90] Michael K Gilson, Tiqing Liu, Michael Baitaluk, and et al. BindingDB in 2015: A public database for medicinal chemistry, computational chemistry and systems pharmacology. *Nucleic Acids Research*, 44(D1):D1045–D1053, 2016. doi: 10.1093/nar/gkv1072.

[91] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify II: A practical guide to chemogenomics data curation. *Journal of Chemical Information and Modeling*, 56(7):1243–1252, 2016. doi: 10.1021/acs.jcim.6b00129.

[92] Antony J. Williams, Christopher M. Grulke, and Jeff et al. Edwards. The comptox chemistry dashboard: a community data resource for environmental chemistry. *Journal of Cheminformatics*, 9:61, 2017. doi: 10.1186/s13321-017-0247-6.

[93] Douglas Young, Todd Martin, Raghuraman Venkatapathy, and Phil Harten. Are the chemical structures in your QSAR correct? *QSAR & Combinatorial Science*, 27(11-12):1337–1345, 2008. doi: 10.1002/qsar.200810084.

[94] A Patrícia Bento, Anne Hersey, Eloy Félix, and Greg et al. Landrum. An open source chemical structure curation pipeline using RDKit. *Journal of Cheminformatics*, 12(1):51, 2020. doi: 10.1186/s13321-020-00456-1.

[95] Insha Ullah and A. H. Welsh. On the effect of noise on fitting linear regression models, 2024.

[96] Alexander Camuto, Matthew Willetts, Umut Şimşekli, Stephen Roberts, and Chris Holmes. Explicit regularisation in gaussian noise injections, 2021.

[97] Alex Avdeef. Multi-lab intrinsic solubility measurement reproducibility in CheqSol and shake-flask methods. *ADMET and DMPK*, 7(3):210–219, 2019. doi: 10.5599/admet.698.

[98] Frank R. Hampel. Robust statistics: a brief introduction and overview. Research Report 94, Seminar für Statistik, Eidgenössische Technische Hochschule (ETH) Zürich, 2001.

[99] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.

[100] Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8135–8153, 2023. doi: 10.1109/TNNLS.2022.3152527.

[101] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.

[102] D.H. Wolpert and W.G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi: 10.1109/4235.585893.

[103] Dennis L. Wilson. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-2(3):408–421, 1972. doi: 10.1109/TSMC.1972.4309137.

[104] Adelaide Punt. NoiseInject: A Python package for label-noise robustness benchmarking in QSAR models. https://github.com/adpunt/noise_inject, 2026. Accessed: 2026-06-01.

[105] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning, 2016.

[106] Jianyuan Deng, Zhibo Yang, and Hehe Wang et al. Unraveling key elements underlying molecular property prediction: A systematic study, 2023.

[107] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, et al. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014.

[108] Artem Fediai et al. Accurate GW frontier orbital energies of 134 kilo molecules. *Scientific Data*, 10:581, 2023. doi: 10.1038/s41597-023-02486-4.

[109] Open ADMET Consortium. OpenADMET-ExpansionRx challenge dataset, 2026. URL https://huggingface.co/datasets/openadmet/openadmet-expansionrx-challenge-data. CC-BY-4.0. ADMET data from drug discovery campaigns by Expansion Therapeutics targeting RNA-mediated diseases. Challenge closed January 19, 2026.

[110] Barbara Zdrazil, Eloy Felix, Fiona Hunter, et al. The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192, 11 2023. ISSN 0305-1048. doi: 10.1093/nar/gkad1004.

[111] Jacob R. Gardner, Geoff Pleiss, Kilian Q. Weinberger, David Bindel, and Andrew Gordon Wilson. GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[112] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[113] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? *CoRR*, abs/1703.04977, 2017.

[114] James Harrison, John Willes, and Jasper Snoek. Variational bayesian last layers, 2024.

[115] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 2623–2631, 2019.

[116] Markus Orsi and Jean-Louis Reymond. One chiral fingerprint to find them all. *Journal of Cheminformatics*, 16:53, 2024. doi: 10.1186/s13321-024-00849-6.

[117] Lien Ai Nguyen, Hua He, and Chuong Pham-Huy. Chiral drugs: an overview. *International Journal of Biomedical Science*, 2(2):85–100, 2006. PMID: 23674971; PMCID: PMC3614593.

[118] Alexander Golbraikh, Eugene Muratov, Denis Fourches, and Alexander Tropsha. Data set modelability by QSAR. *Journal of Chemical Information and Modeling*, 54(1):1–4, 2014. doi: 10.1021/ci400572x.

[119] Robert P. Sheridan, Prabha Karnachi, Matthew Tudor, Yuting Xu, Andy Liaw, Falgun Shah, Alan C. Cheng, Elizabeth Joshi, Meir Glick, and Juan Alvarez. Experimental error, kurtosis, activity cliffs, and methodology: What limits the predictivity of quantitative structure–activity relationship models? *Journal of Chemical Information and Modeling*, 60(4):1969–1982, 2020. doi: 10.1021/acs.jcim.9b01067.

[120] José L. Medina-Franco. Activity cliffs: Facts or artifacts? *Chemical Biology & Drug Design*, 81(5):553–556, 2013. doi: 10.1111/cbdd.12115.

[121] Sereina Riniker and Gregory A. Landrum. Open-source platform to benchmark fingerprints for ligand-based virtual screening. *Journal of Cheminformatics*, 5:26, 2013. doi: 10.1186/1758-2946-5-26.

[122] Mateusz Praski, Jakub Adamczyk, and Wojciech Czech. Benchmarking pretrained molecular embedding models for molecular representation learning, 2025.

[123] Florian Huber and Julian Pollmann. Count your bits: fingerprint benchmarking to assess broad chemical space representation. *Journal of Cheminformatics*, 18:83, 2026. doi: 10.1186/s13321-026-01245-y.

[124] Pawel Dabrowski-Tumanski, Bartosz Topolski, Dariusz Plewczynski, and Tomasz Jetka. The geometry of activity cliffs: Representation dependence and multi-scale characterization of activity landscapes, 2026.

[125] Michael F. Dablander. *Investigating Graph Neural Networks and Classical Feature-Extraction Techniques in Activity-Cliff and Molecular Property Prediction*. PhD thesis, University of Oxford, 2023.

[126] Rajarshi Guha and John H. Van Drie. Structure–activity landscape index: Identifying and quantifying activity cliffs. *Journal of Chemical Information and Modeling*, 48(3):646–658, 2008. doi: 10.1021/ci7004093.

[127] Kenneth López-Pérez, Juan F. Avellaneda-Tamayo, Lexin Chen, Edgar López-López, K. Eurídice Juárez-Mercado, José L. Medina-Franco, and Ramón Alain Miranda-Quintana. Molecular similarity: Theory, applications, and perspectives. *Artificial Intelligence Chemistry*, 2(2):100077, 2024. doi: 10.1016/j.aichem.2024.100077.

[128] Alice Capecchi, Daniel Probst, and Jean-Louis Reymond. One molecular fingerprint to rule them all: drugs, biomolecules, and the metabolome. *Journal of Cheminformatics*, 12(1):43, 2020. doi: 10.1186/s13321-020-00445-4.

[129] Noel M. O'Boyle and Roger A. Sayle. Comparing structural fingerprints using a literature-based similarity benchmark. *Journal of Cheminformatics*, 8:36, 2016. doi: 10.1186/s13321-016-0148-0.

[130] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of*

Machine Learning Research, 11:2487–2531, 2010. URL https://jmlr.org/papers/v11/radovanovic10a.html.

[131] David E. Patterson, Richard D. Cramer, Allen M. Ferguson, Robert D. Clark, and Laurence E. Weinberger. Neighborhood behavior: A useful concept for validation of "molecular diversity" descriptors. *Journal of Medicinal Chemistry*, 39(16):3049–3059, 1996. doi: 10.1021/jm960290n.

[132] David E. Coupry and Péter Pogány. Application of deep metric learning to molecular graph similarity. *Journal of Cheminformatics*, 14(1):11, 2022.

[133] Weifan Zheng and Alexander Tropsha. Novel variable selection quantitative structure–property relationship approach based on the k-nearest-neighbor principle. *Journal of Chemical Information and Computer Sciences*, 40(1): 185–194, 2000. doi: 10.1021/ci980033m.

[134] Peter Itskowitz and Alexander Tropsha. k nearest neighbors QSAR modeling as a variational problem: Theory and applications. *Journal of Chemical Information and Modeling*, 45(3):777–785, 2005. doi: 10.1021/ci049628+.

[135] J. Roger Bray and J. T. Curtis. An ordination of the upland forest communities of southern Wisconsin. *Ecological Monographs*, 27(4):325–349, 1957. doi: 10.2307/1942268.

[136] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

[137] Alexander Tropsha. Best practices for QSAR model development, validation, and exploitation. *Molecular Informatics*, 29(6-7):476–488, 2010. doi: 10.1002/minf.201000061.

[138] Darko Butina. Unsupervised data base clustering based on daylight's fingerprint and tanimoto similarity: A fast and automated way to cluster small and large data sets. *Journal of Chemical Information and Computer Sciences*, 39 (4):747–750, 1999. doi: 10.1021/ci9803381.

[139] Thierry Hanser, Ernst Ahlberg, Alexander Amberg, Lennart T. Anger, Chris Barber, Richard J. Brennan, Alessandro Brigo, Annie Delaunois, Susanne Glowienke, Nigel Greene, Laura Johnston, Daniel Kuhn, Lara Kuhnke, Jean-François Marchaland, Wolfgang Muster, Jeffrey Plante, Friedrich Rippmann,

Yogesh Sabnis, Friedemann Schmidt, Ruud van Deursen, Stéphane Werner, Angela White, Joerg Wichard, and Tomoya Yukawa. Data-driven federated learning in drug discovery with knowledge distillation. *Nature Machine Intelligence*, 7(3):423–436, 2025. doi: 10.1038/s42256-025-00991-2.

[140] Junnan Li, Richard Socher, and Steven C.H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning. In *International Conference on Learning Representations*, 2020.

[141] Bo Han, Quanming Yao, and Xingrui Yu et al. Co-teaching: Robust training of deep neural networks with extremely noisy labels. 2018. Accessed: 2024-09-01.

[142] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, pages 41–48. ACM, 2009.

[143] Haitao Liu, Yew-Soon Ong, and Jianfei Cai. Large-scale heteroscedastic regression via gaussian process, 2020.

[144] Carla E. Brodley and Mark A. Friedl. Identifying mislabeled training data. *Journal of Artificial Intelligence Research*, 11:131–167, 1999. doi: 10.1613/jair.606.

[145] Benoît Frénay and Michel Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5):845–869, 2014. doi: 10.1109/TNNLS.2013.2292894.

[146] Juan Martín, José A. Sáez, and Emilio Corchado. On the regressand noise problem: Model robustness and synergy with regression-adapted noise filters. *IEEE Access*, 9:145800–145816, 2021. doi: 10.1109/ACCESS.2021.3123151.

[147] Oliver Laufkötter, Noé Sturm, Jürgen Bajorath, Hongming Chen, and Ola Engkvist. Combining structural and bioactivity-based fingerprints improves prediction performance and scaffold hopping capability. *Journal of Cheminformatics*, 11(1), August 2019. ISSN 1758-2946. doi: 10.1186/s13321-019-0376-1.

[148] Viktoriia Turkina, Melanie R.W. Messih, Etienne Kant, and Jelle T. et al. Gringhuis. Enhancing predictive modeling with molecular fingerprint fusion strategies. July 2025. doi: 10.26434/chemrxiv-2025-b3lfq.

[149] Sören Richard Stahlschmidt, Benjamin Ulfenborg, and Jane Synnergren. Multimodal deep learning for biomedical data fusion: a review. *Briefings in Bioinformatics*, 23(2):bbab569, 2022. doi: 10.1093/bib/bbab569.

[150] Sandra Steyaert, Marija Pizurica, Divya Nagaraj, Priya Khandelwal, Tina Hernandez-Boussard, Andrew J. Gentles, and Olivier Gevaert. Multimodal data fusion for cancer biomarker discovery with deep learning. *Nature Machine Intelligence*, 5(4):351–362, 2023. doi: 10.1038/s42256-023-00633-5.

[151] Tongxin Wang, Wei Shao, Zhi Huang, Haixu Tang, Jie Zhang, Zhengming Ding, and Kun Huang. MOGONET integrates multi-omics data using graph convolutional networks allowing patient classification and biomarker identification. *Nature Communications*, 12(1):3445, 2021. doi: 10.1038/s41467-021-23774-w.

[152] Mehmet Gönen and Ethem Alpaydın. Multiple kernel learning algorithms. *Journal of Machine Learning Research*, 12:2211–2268, 2011. URL https://www.jmlr.org/papers/v12/gonen11a.html.

[153] Anna Cichonska, Tapio Pahikkala, Sandor Szedmak, Heli Julkunen, Antti Airola, Markus Heinonen, Tero Aittokallio, and Juho Rousu. Learning with multiple pairwise kernels for drug bioactivity prediction. *Bioinformatics*, 34 (13):i509–i518, 2018. doi: 10.1093/bioinformatics/bty277.

[154] Gordon F. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968. doi: 10.1109/TIT.1968.1054102.

[155] Richard E. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, Princeton, NJ, 1961. doi: 10.1515/9781400874668.

[156] Manuel Fernández-Delgado, Eva Cernadas, Senén Barro, and Dinani Amorim. Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research*, 15(90):3133–3181, 2014. URL https://jmlr.org/papers/v15/delgado14a.html.

[157] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. Could graph neural networks learn better molecular representation for drug discovery? A

comparison study of descriptor-based and graph-based models. *Journal of Cheminformatics*, 13(1):12, 2021. doi: 10.1186/s13321-020-00479-8.

[158] Andreas Mayr, Günter Klambauer, Thomas Unterthiner, Marvin Steijaert, Jörg K. Wegner, Hugo Ceulemans, Djork-Arné Clevert, and Sepp Hochreiter. Large-scale comparison of machine learning methods for drug target prediction on ChEMBL. *Chemical Science*, 9(24):5441–5451, 2018. doi: 10.1039/C8SC00148K.

[159] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024. doi: 10.1371/journal.pcbi.1012646.

[160] Robert Schmirler, Michael Heinzinger, and Burkhard Rost. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 15(1):7407, 2024. doi: 10.1038/s41467-024-51844-2.

[161] Peter Neidlinger, Omar S. M. El Nahhas, Hannah Sophie Muti, Tim Lenz, Michael Hoffmeister, Hermann Brenner, Marko van Treeck, Rupert Langer, Bastian Dislich, Hans Michael Behrens, Christoph Röcken, Sebastian Foersch, Daniel Truhn, Antonio Marra, Oliver Lester Saldanha, and Jakob Nikolas Kather. Benchmarking foundation models as feature extractors for weakly supervised computational pathology. *Nature Biomedical Engineering*, 10(6): 1113–1123, 2025. doi: 10.1038/s41551-025-01516-3.

[162] Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, Andrew Palmer, Volker Settels, et al. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019. doi: 10.1021/acs.jcim.9b00237.

[163] Xiangxiang Zeng, Hongxin Xiang, Linhui Yu, Jianmin Wang, Kenli Li, Ruth Nussinov, and Feixiong Cheng. Accurate prediction of molecular properties and drug targets using a self-supervised image representation learning framework. *Nature Machine Intelligence*, 4(11):1004–1016, 2022. doi: 10.1038/s42256-022-00557-6.

[164] Daniil A. Boiko, Thiago Reschützegger, Benjamin Sanchez-Lengeling, Samuel M. Blau, and Gabe Gomes. Advancing molecular machine learning representations with stereoelectronics-infused molecular graphs. *Nature Machine Intelligence*, 7(5):771–781, 2025. doi: 10.1038/s42256-025-01031-9.

[165] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

[166] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. Prottrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022. doi: 10.1109/TPAMI.2021.3095381.

[167] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. Deciphering antibody affinity maturation with language models and weakly supervised learning, 2021. URL https://arxiv.org/abs/2112.07782. Machine Learning for Structural Biology Workshop, NeurIPS 2021.

[168] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. AbLang: an antibody language model for completing antibody sequences. *Bioinformatics Advances*, 2(1):vbac046, 2022. doi: 10.1093/bioadv/vbac046.

[169] Bosheng Song, Jiayi Zhang, Ying Liu, Yuansheng Liu, Jing Jiang, Sisi Yuan, Xia Zhen, and Yiping Liu. A systematic review of molecular representation learning foundation models. *Briefings in Bioinformatics*, 27(1):bbaf703, 2026. doi: 10.1093/bib/bbaf703.

[170] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano

Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models, 2021.

[171] Cecile Valsecchi, Francesca Grisoni, Viviana Consonni, and Davide Ballabio. Consensus versus individual QSARs in classification: Comparison on a large-scale case study. *Journal of Chemical Information and Modeling*, 60(3):1215–1223, 2020. doi: 10.1021/acs.jcim.9b01057.

[172] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 847–855. ACM, 2013. doi: 10.1145/2487575.2487629.

[173] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. In *2015 IEEE Information Theory Workshop (ITW)*, pages 1–5. IEEE, 2015. doi: 10.1109/ITW.2015.7133169.

[174] Isabelle Guyon and André Elisseeff. An introduction to variable and feature

selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003. URL https://www.jmlr.org/papers/v3/guyon03a.html.

[175] Yvan Saeys, Iñaki Inza, and Pedro Larrañaga. A review of feature selection techniques in bioinformatics. *Bioinformatics*, 23(19):2507–2517, 2007. doi: 10.1093/bioinformatics/btm344.

[176] Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005. doi: 10.1109/TPAMI.2005.159.

[177] Ron Kohavi and George H. John. Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2):273–324, 1997. doi: 10.1016/S0004-3702(97)00043-X.

[178] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.

[179] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46:389–422, 2002. doi: 10.1023/A:1012487302797.

[180] Sarah Nogueira, Konstantinos Sechidis, and Gavin Brown. On the stability of feature selection algorithms. *Journal of Machine Learning Research*, 18(174): 1–54, 2018. URL https://jmlr.org/papers/v18/17-514.html.

[181] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8:25, 2007. doi: 10.1186/1471-2105-8-25.

[182] Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 68(1):49–67, 2006. doi: 10.1111/j.1467-9868.2005.00532.x.

[183] Sarfaraz K. Niazi and Zamara Mariam. Recent advances in machine-learning-based chemoinformatics: A comprehensive review. *International Journal of Molecular Sciences*, 24(14):11488, 2023. doi: 10.3390/ijms241411488.

[184] Ian T. Jolliffe and Jorge Cadima. Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016. doi: 10.1098/rsta.2015.0202.

[185] Ian T. Jolliffe. A note on the use of principal components in regression. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 31(3):300–303, 1982. doi: 10.2307/2348005.

[186] Svante Wold, Michael Sjöström, and Lennart Eriksson. PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58 (2):109–130, 2001. doi: 10.1016/S0169-7439(01)00155-1.

[187] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

[188] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. doi: 10.48550/arXiv.1802.03426.

[189] José Jiménez-Luna, Francesca Grisoni, and Gisbert Schneider. Drug discovery with explainable artificial intelligence. *Nature Machine Intelligence*, 2(10): 573–584, 2020. doi: 10.1038/s42256-020-00236-4.

[190] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019. doi: 10.1038/s42256-019-0048-x.

[191] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, and Hongming Chen. Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1):48, 2017. doi: 10.1186/s13321-017-0235-x.

[192] Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002. doi: 10.1073/pnas.102102699.

[193] Andrius Vabalas, Emma Gowen, Ellen Poliakoff, and Alexander J. Casson. Machine learning algorithm validation with a limited sample size. *PLOS ONE*, 14(11):e0224365, 2019. doi: 10.1371/journal.pone.0224365.

[194] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006. doi: 10.1186/1471-2105-7-91.

[195] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[196] Jiayun Dong and Cynthia Rudin. Exploring the cloud of variable importance for the set of all good models. *Nature Machine Intelligence*, 2(12):810–824, 2020. doi: 10.1038/s42256-020-00264-0.

[197] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. Towards better understanding of gradient-based attribution methods for deep neural networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1711.06104.

[198] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3145–3153, 2017. URL https://proceedings.mlr.press/v70/shrikumar17a.html.

[199] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020. doi: 10.1038/s42256-019-0138-9.

[200] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, and Orion Reblitz-Richardson. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896*, 2020. URL https://arxiv.org/abs/2009.07896.

[201] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980.

[202] Malkeet Singh Bahia, Omer Kaspi, Meir Touitou, Idan Binayev, Seema Dhail, Jacob Spiegel, Netaly Khazanov, Abraham Yosipof, and Hanoch Senderowitz. A comparison between 2D and 3D descriptors in QSAR modeling based on bio-active conformations. *Molecular Informatics*, 42(4):e2200186, 2023. doi: 10.1002/minf.202200186.

[203] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[204] Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani. Least angle regression. *The Annals of Statistics*, 32(2):407–499, 2004. doi: 10.1214/009053604000000067.

[205] Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, volume 51 of *Proceedings of Machine Learning Research*, pages 240–248, 2016. URL https://proceedings.mlr.press/v51/jamieson16.html.

[206] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018. URL https://jmlr.org/papers/v18/16-558.html.

[207] Izhar Wallach and Abraham Heifets. Most ligand-based classification benchmarks reward memorization rather than generalization. *Journal of Chemical Information and Modeling*, 58(5):916–932, 2018. doi: 10.1021/acs.jcim.7b00403.

[208] Pascal Notin, Aaron W. Kollasch, Daniel Ritter, Lood van Niekerk, Steffanie Paul, Hansen Spinner, Nathan Rollins, Ada Shaw, Ruben Weitzman, Jonathan Frazer, Mafalda Dias, Dinko Franceschi, et al. ProteinGym: Large-scale benchmarks for protein design and fitness prediction. *bioRxiv*, 2023. doi: 10.1101/2023.12.07.570727.

[209] Luiz C. Vieira, Sophia Lin, and Claus O. Wilke. Intrinsic dataset features drive mutational effect prediction by protein language models. *bioRxiv*, 2026. doi: 10.64898/2026.03.08.710389. Preprint.

[210] Cheng Fang, Ye Wang, and Richard et al. Grater. Prospective validation of machine learning algorithms for absorption, distribution, metabolism, and excretion prediction: An industrial perspective. *Journal of Chemical Information and Modeling*, 63(11):3263–3274, 2023. doi: 10.1021/acs.jcim.3c00160. PMID: 37216672.

[211] Cas Wognum et al. Polaris: A benchmarking platform for drug discovery, 2024. URL https://doi.org/10.5281/zenodo.13652587.

[212] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules (QM9 dataset). https://doi.org/10.6084/m9.figshare.978904, 2014. Accessed: 2026-06-02.

[213] Derek T. Ahneman, Jesús G. Estrada, Shishi Lin, Spencer D. Dreher, and Abigail G. Doyle. Predicting reaction performance in C–N cross-coupling using machine learning. *Science*, 360(6385):186–190, 2018. doi: 10.1126/science.aar5169.

[214] Patrick Koenig, Chingwei V. Lee, Benjamin T. Walters, Vasantharajan Janakiraman, Jeremy Stinson, Thomas W. Patapoff, and Germaine Fuh. Mutational landscape of antibody variable domains reveals a switch modulating the interdomain conformational dynamics and antigen binding. *Proceedings of the National Academy of Sciences*, 114(4):E486–E495, 2017. doi: 10.1073/pnas.1613231114.

[215] Michael Chungyoun, Jeffrey Ruffolo, and Jeffrey Gray. FLAb: Benchmarking deep learning methods for antibody fitness prediction. *bioRxiv*, 2024. doi: 10.1101/2024.01.13.575504.

[216] Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. *Database*, 2023:baad021, 2023. doi: 10.1093/database/baad021.

[217] David Prihoda, Jad Maamary, Andrew Waight, Veronica Juan, Laurence Fayadat-Dilman, Daniel Svozil, and Danny A. Bitton. BioPhi: A platform for antibody design, humanization, and humanness evaluation based on natural antibody repertoires and deep learning. *mAbs*, 14(1):2020203, 2022. doi: 10.1080/19420862.2021.2020203.

[218] Lewis Chinery, Alissa M. Hummer, Brij Bhushan Mehta, Rahmad Akbar, Puneet Rawat, Andrei Slabodkin, Khang Le Quy, Fridtjof Lund-Johansen, Victor Greiff, Jeliazko R. Jeliazkov, and Charlotte M. Deane. Baselining the buzz. Trastuzumab-HER2 affinity, and beyond. *bioRxiv*, 2024. doi: 10.1101/2024.03.26.586756. Version 1; later versions retitled "Simple computational methods can outperform deep learning in designing diverse, binder-enriched antibody libraries".

[219] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, Hassan Sirelkhatim, Guillaume Richard, Marcin Skwark, Karim Beguir, Marie Lopez, and Thomas Pierrot. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, 2025. doi: 10.1038/s41592-024-02523-z.

[220] The Cancer Genome Atlas Research Network. Comprehensive molecular profiling of lung adenocarcinoma. *Nature*, 511(7511):543–550, 2014. doi: 10.1038/nature13385.

[221] The Cancer Genome Atlas Research Network. Comprehensive genomic characterization of squamous cell lung cancers. *Nature*, 489(7417):519–525, 2012. doi: 10.1038/nature11404.

[222] Mary J. Goldman, Brian Craft, Mim Hastie, Kristupas Repečka, Fran McDade, Akhil Kamath, Ayan Banerjee, Yunhai Luo, Dave Rogers, Angela N. Brooks, Jingchun Zhu, and David Haussler. Visualizing and interpreting cancer genomics data via the Xena platform. *Nature Biotechnology*, 38(6):675–678, 2020. doi: 10.1038/s41587-020-0546-8.

[223] The Cancer Genome Atlas Network. Comprehensive molecular portraits of human breast tumours. *Nature*, 490(7418):61–70, 2012. doi: 10.1038/nature11412.

[224] Joel S. Parker, Michael Mullins, Maggie C. U. Cheang, Samuel Leung, David Voduc, Tammi Vickery, Sherri Davies, Christiane Fauron, Xiaping He, Zhiyuan Hu, John F. Quackenbush, Inge J. Stijleman, Juan Palazzo, J. S. Marron, Andrew B. Nobel, Elaine Mardis, Torsten O. Nielsen, Matthew J.

Ellis, Charles M. Perou, and Philip S. Bernard. Supervised risk predictor of breast cancer based on intrinsic subtypes. *Journal of Clinical Oncology*, 27(8): 1160–1167, 2009. doi: 10.1200/JCO.2008.18.1370.

[225] Frank Hoonakker, Nicolas Lachiche, Alexandre Varnek, and Alain Wagner. A representation to apply usual data mining techniques to chemical reactions – illustration on the rate constant of SN2 reactions in water. *International Journal on Artificial Intelligence Tools*, 20(2):253–270, 2011. doi: 10.1142/S0218213011000140.

[226] Philippe Schwaller, Benjamin Hoover, Jean-Louis Reymond, Hendrik Strobelt, and Teodoro Laino. Extraction of organic chemistry grammar from unsupervised learning of chemical reactions. *Science Advances*, 7(15):eabe4166, 2021. doi: 10.1126/sciadv.abe4166.

[227] Philippe Schwaller, Daniel Probst, Alain C. Vaucher, Vishnu H. Nair, David Kreutter, Teodoro Laino, and Jean-Louis Reymond. Mapping the space of chemical reactions using attention-based neural networks. *Nature Machine Intelligence*, 3(2):144–152, 2021. doi: 10.1038/s42256-020-00284-w.

[228] Tatsuya Sagawa and Ryosuke Kojima. Reactiont5: a large-scale pre-trained model towards application of limited reaction data, 2023. URL https://arxiv.org/abs/2311.06708.

[229] Kangway V. Chuang and Michael J. Keiser. Comment on "predicting reaction performance in C–N cross-coupling using machine learning". *Science*, 362 (6416):eaat8603, 2018. doi: 10.1126/science.aat8603.

[230] S. Henikoff and J. G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992. doi: 10.1073/pnas.89.22.10915.

[231] Marie-Paule Lefranc, Christelle Pommié, Manuel Ruiz, Véronique Giudicelli, Elodie Foulquier, Lisa Truong, Valérie Thouvenin-Contet, and Gérard Lefranc. Imgt unique numbering for immunoglobulin and t cell receptor variable domains and ig superfamily v-like domains. *Developmental & Comparative Immunology*, 27(1):55–77, 2003. doi: 10.1016/S0145-305X(02)00039-3.

[232] James Dunbar and Charlotte M. Deane. Anarci: antigen receptor numbering and receptor classification. *Bioinformatics*, 32(2):298–300, 2016. doi: 10.1093/bioinformatics/btv552.

[233] Susana Vinga and Jonas Almeida. Alignment-free sequence comparison—a review. *Bioinformatics*, 19(4):513–523, 2003. doi: 10.1093/bioinformatics/btg005.

[234] Hanno Teeling, Anke Meyerdierks, Margarete Bauer, Rudolf Amann, and Frank Oliver Glöckner. Application of tetranucleotide frequencies for the assignment of genomic fragments. *Environmental Microbiology*, 6(9):938–947, 2004. doi: 10.1111/j.1462-2920.2004.00624.x.

[235] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genome, 2023. URL https://arxiv.org/abs/2306.15006.

[236] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Callum Birch-Sykes, Michael Wornow, Aman Patel, Clayton Rabideau, Stefano Massaroli, Yoshua Bengio, Stefano Ermon, Stephen A. Baccus, and Chris Ré. Hyenadna: Long-range genomic sequence modeling at single nucleotide resolution, 2023. URL https://arxiv.org/abs/2306.15794.

[237] The Cancer Genome Atlas Research Network, John N Weinstein, Eric A Collisson, Gordon B Mills, Kenna R Mills Shaw, Brad A Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M Stuart. The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013. doi: 10.1038/ng.2764.

[238] Raymond E. Carhart, Dennis H. Smith, and R. Venkataraghavan. Atom pairs as molecular features in structure-activity studies: definition and applications. *Journal of Chemical Information and Computer Sciences*, 25(2):64–73, 1985. doi: 10.1021/ci00046a002.

[239] Ramaswamy Nilakantan, Norman Bauman, J. Scott Dixon, and R. Venkataraghavan. Topological torsion: a new molecular descriptor for sar applications. comparison with other descriptors. *Journal of Chemical Information and Computer Sciences*, 27(2):82–85, 1987. doi: 10.1021/ci00054a008.

[240] Peter Gedeck, Bernhard Rohde, and Christian Bartels. Qsar – how good is it in practice? comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5): 1924–1936, 2006. doi: 10.1021/ci050413p.

[241] Joseph L. Durant, Burton A. Leland, Douglas R. Henry, and James G. Nourse. Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical Information and Computer Sciences*, 42(6):1273–1280, 2002. doi: 10.1021/ci010132r.

[242] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(77):2539–2561, 2011. URL https://www.jmlr.org/papers/v12/shervashidze11a.html.

[243] Hirotomo Moriwaki, Yu-Shi Tian, Norihito Kawashita, and Tatsuya Takagi. Mordred: a molecular descriptor calculator. *Journal of Cheminformatics*, 10 (1):4, 2018. doi: 10.1186/s13321-018-0258-y.

[244] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, and Junzhou Huang. Self-supervised graph transformer on large-scale molecular data, 2020. URL https://arxiv.org/abs/2007.02835.

[245] Shengchao Liu, Hanchen Wang, Weiyang Liu, Joan Lasenby, Hongyu Guo, and Jian Tang. Pre-training molecular graph representation with 3d geometry, 2022. URL https://arxiv.org/abs/2110.07728.

[246] Eduardo Soares, Emilio Vital Brazil, Victor Shirasuna, Dmitry Zubarev, Renato Cerqueira, and Kristin Schmidt. An open-source family of large encoder-decoder foundation models for chemistry. *Communications Chemistry*, 8, 2025. doi: 10.1038/s42004-025-01585-0.

[247] Xiao-Chen Zhang, Cheng-Kun Wu, Jia-Cai Yi, Xiang-Xiang Zeng, Can-Qun Yang, Ai-Ping Lu, Ting-Jun Hou, and Dong-Sheng Cao. Pushing the boundaries of molecular property prediction for drug discovery with multitask learning bert enhanced by smiles enumeration. *Research*, 2022, 2022. doi: 10.34133/research.0004.

[248] Atakan Yüksel, Erva Ulusoy, Atabey Ünlü, and Tunca Doğan. Selformer: molecular representation learning via selfies language models. *Machine Learning: Science and Technology*, 4(2):025035, 2023. doi: 10.1088/2632-2153/acdb30.

[249] Pedro J. Ballester and W. Graham Richards. Ultrafast shape recognition to search compound databases for similar molecular shapes. *Journal of Computational Chemistry*, 28(10):1711–1723, 2007. doi: 10.1002/jcc.20681.

[250] Adrian M. Schreyer and Tom Blundell. Usrcat: real-time ultrafast shape recognition with pharmacophoric constraints. *Journal of Cheminformatics*, 4 (1):27, 2012. doi: 10.1186/1758-2946-4-27.

[251] Matthias Rupp, Alexandre Tkatchenko, Klaus-Robert Müller, and O. Anatole von Lilienfeld. Fast and accurate modeling of molecular atomization energies with machine learning. *Physical Review Letters*, 108(5):058301, 2012. doi: 10.1103/PhysRevLett.108.058301.

[252] Joshua Meier, Roshan Rao, Robert Verkuil, Jason Liu, Tom Sercu, and Alexander Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. *bioRxiv*, 2021. doi: 10.1101/2021.07.09.450648.

[253] Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025. doi: 10.1126/science.ads0018.

[254] Salvatore Candido, Thomas Hayes, Alexander Derry, Roshan Rao, Zeming Lin, Robert Verkuil, Bryan Z. Wu, Jin Sub Lee, Elise S. Bruguera, Jehan A. Keval, Mykhailo Kopylov, John E. Pak, Wesley Wu, Neil Thomas, Samson Mataraso, Alvin Hsu, Ashton C. Trotman-Grant, Kilian Fatras, Allan dos Santos Costa, Rohil Badkundri, Halil Akın, Deniz Oktay, Jonathan Deaton, Elizabeth Montabana, Hrishita Sitwala, Yue Yu, Marius Wiggert, Dylan Alexander Carlin, Anthony W. Goering, Tomasz Blazejewski, McCullen Sandora, Michael Hla, Tina Z. Jia, Leon H. Kloker, Nicholas J. Sofroniew, Masatoshi Uehara,

Jassi Pannu, Sharrol Bachas, Daniel S. Liu, Tom Sercu, and Alexander Rives. Language modeling materializes a world model of protein biology. *bioRxiv*, 2026. doi: 10.64898/2026.06.03.729735. Preprint, posted 4 June 2026.

[255] Jin Su, Chenchen Han, Yuyang Zhou, Junjie Shan, Xibin Zhou, and Fajie Yuan. Saprot: Protein language modeling with structure-aware vocabulary. *bioRxiv*, 2023. doi: 10.1101/2023.10.01.560349.

[256] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. Addressing the antibody germline bias and its effect on language models for improved antibody design. *Bioinformatics*, 40(11), 2024. doi: 10.1093/bioinformatics/btae618.

[257] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024. doi: 10.1371/journal.pcbi.1012646.

[258] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. Deciphering antibody affinity maturation with language models and weakly supervised learning, 2021. URL https://arxiv.org/abs/2112.07782.

[259] Eunna Huh, Hyeonsu Lee, and Hyunjin Shin. Antibody foundational model: Ab-RoBERTa, 2025.

[260] Melissa Sanabria, Jonas Hirsch, Pierre M. Joubert, and Anna R. Poetsch. Dna language model grover learns sequence context in the human genome. *Nature Machine Intelligence*, 6(8):911–923, 2024. doi: 10.1038/s42256-024-00872-0.

[261] Raphael Mourad. Mistral-DNA: Large language model Mistral for DNA. https://github.com/raphaelmourad/Mistral-DNA, 2024. Code and model release, no paper; model RaphaelMourad/Mistral-DNA-v1-422M-hg38 on Hugging Face.

[262] Paul M. Sharp, Therese M. F. Tuohy, and Krzysztof R. Mosurski. Codon usage in yeast: cluster analysis clearly differentiates highly and lowly expressed genes. *Nucleic Acids Research*, 14(13):5125–5143, 1986. doi: 10.1093/nar/14.13.5125.

[263] Christina V. Theodoris, Ling Xiao, Anant Chopra, Mark D. Chaffin, Zeina R. Al Sayed, Matthew C. Hill, Helene Mantineo, Elizabeth M. Brydon, Zexian Zeng, X. Shirley Liu, and Patrick T. Ellinor. Transfer learning enables predictions in network biology. *Nature*, 618(7965):616–624, 2023. doi: 10.1038/s41586-023-06139-9.

[264] Christina V. Theodoris. Geneformer model repository: Geneformer-V2-316M. https://huggingface.co/ctheodoris/Geneformer, 2024. Model card: V2 trained December 2024 on about 104 million human single-cell transcriptomes, input size 4096; default model in the main directory is Geneformer-V2-316M (18 layers, 1152 hidden units).

[265] Maxence Gélard, Guillaume Richard, Thomas Pierrot, and Paul-Henry Cournède. Bulkrnabert: Cancer prognosis from bulk rna-seq based language models. *bioRxiv*, 2024. doi: 10.1101/2024.06.18.599483.

[266] Craig H Mermel, Steven E Schumacher, Barbara Hill, Matthew L Meyerson, Rameen Beroukhim, and Gad Getz. Gistic2.0 facilitates sensitive and confident localization of the targets of focal somatic copy-number alteration in human cancers. *Genome Biology*, 12(4):R41, 2011. doi: 10.1186/gb-2011-12-4-r41.

[267] Jun Li, Yiling Lu, Rehan Akbani, Zhenlin Ju, Paul L Roebuck, Wenbin Liu, Ji-Yeon Yang, Bradley M Broom, Roeland G W Verhaak, David W Kane, Chris Wakefield, John N Weinstein, Gordon B Mills, and Han Liang. Tcpa: a resource for cancer functional proteomics data. *Nature Methods*, 10(11): 1046–1047, 2013. doi: 10.1038/nmeth.2650.

[268] Tong Ding, Sophia J. Wagner, Andrew H. Song, Richard J. Chen, Ming Y. Lu, Andrew Zhang, Anurag J. Vaidya, Guillaume Jaume, Muhammad Shaban, Ahrong Kim, Drew F. K. Williamson, Bowen Chen, Cristina Almagro-Perez, Paul Doucet, Sharifa Sahai, Chengkuan Chen, Daisuke Komura, Akihiro Kawabe, Shumpei Ishikawa, Georg Gerber, Tingying Peng, Long Phi Le, and Faisal Mahmood. Multimodal whole slide foundation model for pathology, 2024. URL https://arxiv.org/abs/2411.19666.

[269] Hanwen Xu, Naoto Usuyama, Jaspreet Bagga, Sheng Zhang, Rajesh Rao, Tristan Naumann, Cliff Wong, Zelalem Gero, Javier González, Yu Gu, Yanbo

Xu, Mu Wei, Wenhui Wang, Shuming Ma, Furu Wei, Jianwei Yang, Chunyuan Li, Jianfeng Gao, Jaylen Rosemon, Tucker Bower, Soohee Lee, Roshanthi Weerasinghe, Bill J. Wright, Ari Robicsek, Brian Piening, Carlo Bifulco, Sheng Wang, and Hoifung Poon. A whole-slide foundation model for digital pathology from real-world data. *Nature*, 630(8015):181–188, 2024. doi: 10.1038/s41586-024-07441-w.

[270] Xiyue Wang, Sen Yang, Jun Zhang, Minghui Wang, Jing Zhang, Wei Yang, Junzhou Huang, and Xiao Han. Transformer-based unsupervised contrastive learning for histopathological image classification. *Medical Image Analysis*, 81: 102559, 2022. doi: 10.1016/j.media.2022.102559.

[271] Alexandre Filiot, Ridouane Ghermi, Antoine Olivier, Paul Jacob, Lucas Fidon, Alice Mac Kain, Charlie Saillard, and Jean-Baptiste Schiratti. Scaling self-supervised learning for histopathology with masked image modeling. *medRxiv*, 2023. doi: 10.1101/2023.07.21.23292757.

[272] Charlie Saillard, Rodolphe Jenatton, Felipe Llinares-López, Zelda Mariet, David Cahané, Eric Durand, and Jean-Philippe Vert. H-optimus-0, 2024. URL https://github.com/bioptimus/releases/tree/main/models/h-optimus/v0.

[273] Theo A. Knijnenburg, Linghua Wang, Michael T. Zimmermann, Nyasha Chambwe, Galen F. Gao, Andrew D. Cherniack, et al. Genomic and molecular landscape of DNA damage repair deficiency across The Cancer Genome Atlas. *Cell Reports*, 23(1):239–254.e6, 2018. doi: 10.1016/j.celrep.2018.03.076.

[274] Kyle Ellrott, Matthew H. Bailey, Gordon Saksena, Kyle R. Covington, Cyriac Kandoth, Chip Stewart, et al. Scalable open science approach for mutation calling of tumor exomes using multiple genomic pipelines. *Cell Systems*, 6(3): 271–281.e7, 2018. doi: 10.1016/j.cels.2018.03.002.

[275] Scott L. Carter, Kristian Cibulskis, Elena Helman, Aaron McKenna, Hui Shen, Travis Zack, Peter W. Laird, Robert C. Onofrio, Wendy Winckler, Barbara A. Weir, Rameen Beroukhim, David Pellman, Douglas A. Levine, Eric S. Lander, Matthew Meyerson, and Gad Getz. Absolute quantification of somatic DNA alterations in human cancer. *Nature Biotechnology*, 30(5):413–421, 2012. doi: 10.1038/nbt.2203.

[276] Vésteinn Thorsson, David L. Gibbs, Scott D. Brown, Denise Wolf, Dante S. Bortone, Tai-Hsien Ou Yang, et al. The immune landscape of cancer. *Immunity*, 48(4):812–830.e14, 2018. doi: 10.1016/j.immuni.2018.03.023.

[277] Aaron M. Newman, Chih Long Liu, Michael R. Green, Andrew J. Gentles, Weiguo Feng, Yue Xu, Chuong D. Hoang, Maximilian Diehn, and Ash A. Alizadeh. Robust enumeration of cell subsets from tissue expression profiles. *Nature Methods*, 12(5):453–457, 2015. doi: 10.1038/nmeth.3337.

[278] Tathiane M. Malta, Artem Sokolov, Andrew J. Gentles, Tomasz Burzykowski, Laila Poisson, John N. Weinstein, et al. Machine learning identifies stemness features associated with oncogenic dedifferentiation. *Cell*, 173(2):338–354.e15, 2018. doi: 10.1016/j.cell.2018.03.034.

[279] Minoru Toyota, Nita Ahuja, Mutsumi Ohe-Toyota, James G. Herman, Stephen B. Baylin, and Jean-Pierre J. Issa. CpG island methylator phenotype in colorectal cancer. *Proceedings of the National Academy of Sciences*, 96(15):8681–8686, 1999. doi: 10.1073/pnas.96.15.8681.

[280] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022. URL https://arxiv.org/abs/2207.08815.

[281] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006. ISBN 0-262-18253-X.

[282] Davide Chicco and Giuseppe Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020. doi: 10.1186/s12864-019-6413-7.

[283] Yoshua Bengio and Yves Grandvalet. No unbiased estimator of the variance of K-fold cross-validation. *Journal of Machine Learning Research*, 5:1089–1105, 2004. URL https://www.jmlr.org/papers/v5/grandvalet04a.html.

[284] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer

Series in Statistics. Springer, New York, 2 edition, 2009. doi: 10.1007/978-0-387-84858-7.

[285] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-weight-net: Learning an explicit mapping for sample weighting. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[286] Ivan Olier, Noureddin Sadawi, G. Richard Bickerton, Joaquin Vanschoren, Crina Grosan, Larisa Soldatova, and Ross D. King. Meta-QSAR: a large-scale application of meta-learning to drug design and discovery. *Machine Learning*, 107(1):285–311, 2018. doi: 10.1007/s10994-017-5685-x.

[287] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282, 2017. URL https://proceedings.mlr.press/v54/mcmahan17a.html.

[288] Wouter Heyndrickx, Lewis Mervin, Tobias Morawietz, Noé Sturm, Lukas Friedrich, Adam Zalewski, Anastasia Pentina, Lina Humbeck, Martijn Oldenhof, Ritsuya Niwayama, Peter Schmidtke, Nikolas Fechner, Jaak Simm, Adam Arany, Nicolas Drizard, Rama Jabal, Arina Afanasyeva, Regis Loeb, Shlok Verma, Simon Harnqvist, Matthew Holmes, Balazs Pejo, Maria Telenczuk, Nicholas Holway, Arne Dieckmann, Nicola Rieke, Friederike Zumsande, Djork-Arné Clevert, Michael Krug, Christopher Luscombe, Darren Green, Peter Ertl, Peter Antal, David Marcus, Nicolas Do Huu, Hideyoshi Fuji, Stephen Pickett, Gergely Acs, Eric Boniface, Bernd Beck, Yax Sun, Arnaud Gohier, Friedrich Rippmann, Ola Engkvist, Andreas H. Göller, Yves Moreau, Mathieu N. Galtier, Ansgar Schuffenhauer, and Hugo Ceulemans. MELLODDY: Cross-pharma federated learning at unprecedented scale unlocks benefits in QSAR without compromising proprietary information. *Journal of Chemical Information and Modeling*, 64(7):2331–2344, 2024. doi: 10.1021/acs.jcim.3c00799.

[289] Pietro Cassarà, Alberto Gotta, and Lorenzo Valerio. Federated feature selection for cyber-physical systems of systems. *IEEE Transactions on Vehicular Technology*, 71(9):9937–9950, 2022. doi: 10.1109/TVT.2022.3178612.

[290] Paul Czodrowski. herg me out. *Journal of Chemical Information and Modeling*, 53(9):2240–2251, 2013.

[291] Lood van Niekerk, Joshua Moller, Seth Ritter, Porfirio Quintero-Cadena, Ariel Cohen, Alexander Channing, Michael Chungyoun, Ulfert Rand, Ian Smith, Nikhil Bhatt, Emmanuel Pierre, Casey Harris, Yi Ao, Vincent Grippo, Jasmin Schwenk, Amos Rosenbaum, Lucas Allen, Naif Asi, Jiaqi Zhu, Ritika Singh, Sonia Sammi, Vikram Jadhav, Jan Dušek, Sarath Chandra, Andreea Badea, Nels Thorsteinson, Marissa Blalock, Jared Kim, Oliver Turnbull, Shreyas Kulkarni, Vivek Kohar, Abiy Gebremedhin, Charlotte M. Deane, Peter M. Tessier, and Ammar Arsiwala. 2025 Ginkgo Datapoints antibody developability competition outcomes: limited model performance and a call for data standardization. *mAbs*, 18(1):2634216, 2026. doi: 10.1080/19420862.2026.2634216.

[292] Alissa M. Hummer, Constantin Schneider, Lewis Chinery, and Charlotte M. Deane. Investigating the volume and diversity of data needed for generalizable antibody–antigen $\delta\delta g$ prediction. *Nature Computational Science*, 5(8):635–647, 2025. doi: 10.1038/s43588-025-00823-8.

[293] Ryan J. Tibshirani. Degrees of freedom and model search. *Statistica Sinica*, 25(3):1265–1296, 2015. doi: 10.5705/ss.2014.147.

[294] Andrew Y. Ng. Feature selection, L1 vs. L2 regularization, and rotational invariance. In *Proceedings of the Twenty-First International Conference on Machine Learning (ICML '04)*, page 78, 2004. doi: 10.1145/1015330.1015435.

[295] Ananya Kumar, Aditi Raghunathan, Robbie Matthew Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=UYneFzXSJWh.

[296] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00, page 93–104, New York, NY, USA, 2000. Association for Computing Machinery. ISBN 1581132174. doi: 10.1145/342009.335388.

[297] Junnan Li, Richard Socher, and Steven C. H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning, 2020.

[298] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks, 2022.

[299] Glenn Shafer and Vladimir Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9(12):371–421, 2008.

[300] Matt Taddy, Chun-Sheng Chen, Jun Yu, and Mitch Wyle. Bayesian and empirical bayesian forests, 2015.

[301] Yanqing Ma, Carmine Ventre, and Maria Polukarov. Denoised labels for financial time series data via self-supervised learning. In *Proceedings of the Third ACM International Conference on AI in Finance*, pages 471–479. ACM, 2022. doi: 10.1145/3533271.3561687.

[302] Stephen R. Heller, Alan McNaught, Igor Pletnev, Stephen Stein, and Dmitrii Tchekhovskoi. InChI, the IUPAC international chemical identifier. *Journal of Cheminformatics*, 7(1):23, 2015. doi: 10.1186/s13321-015-0068-4.

[303] Guy W. Bemis and Mark A. Murcko. The properties of known drugs. 1. molecular frameworks. *Journal of Medicinal Chemistry*, 39(15):2887–2893, 1996. doi: 10.1021/jm9602928.

[304] Philippe Schwaller, Alain C. Vaucher, Teodoro Laino, and Jean-Louis Reymond. Prediction of chemical reaction yields using deep learning. *Machine Learning: Science and Technology*, 2(1):015016, 2021. doi: 10.1088/2632-2153/abc81d.

[305] The Cancer Genome Atlas Research Network, John N. Weinstein, Eric A. Collisson, Gordon B. Mills, Kenna R. Mills Shaw, Brad A. Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M. Stuart. The Cancer Genome Atlas pan-cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013. doi: 10.1038/ng.2764.

[306] Bo Li and Colin N. Dewey. RSEM: accurate transcript quantification from RNA-Seq data with or without a reference genome. *BMC Bioinformatics*, 12:323, 2011. doi: 10.1186/1471-2105-12-323.